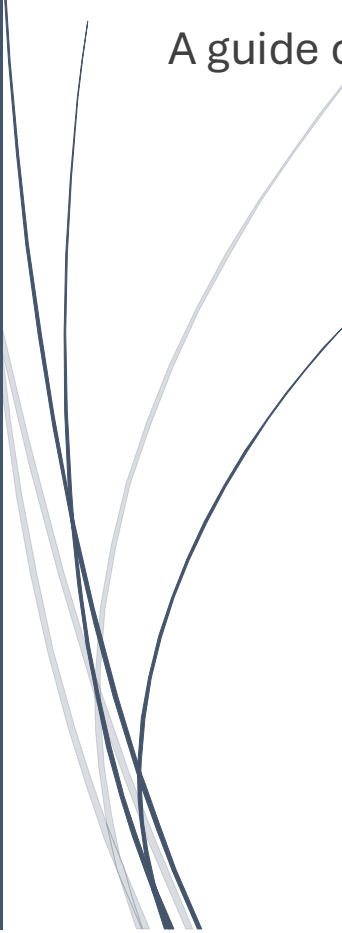




PL-300 Power BI Exam Documentation (from Microsoft Learn)

A guide covering all 4 major areas of the exam in detail



Contents

Prepare Data for Analysis with Power BI	5
Get Data in Power BI	6
Get Data from Files	6
Get Data from Relational Data Sources	6
Create Dynamic Reports with Parameters	6
Get Data from a NoSQL Database	7
Get Data from Online Services	7
Select a Storage Mode	7
Get Data from Azure Analysis Services	8
Fix Performance Issues	9
Resolve Data Import Errors	10
Summary	11
Clean, Transform, and Load Data in Power BI	12
Shape the Initial Data	12
Simplify the Data Structure	13
Evaluate and Change Column Data Types	14
Combine Multiple Tables into a Single Table	16
Profile Data in Power BI	16
Use Advanced Editor to Modify M Code	17
Choose a Power BI Model Framework	18
Describe Power BI Model Fundamentals	18
Determine When to Develop an Import Model	21
Determine When to Develop a DirectQuery Model	22
Determine When to Develop a Composite Model	23
Choose a Model Framework	26
Model Data with Power BI	27
Design a Semantic Model in Power BI	28
Work with Tables	31
Create a Date Table	31
Work with Dimensions	39
Define Data Granularity	43
Work with Relationships and Cardinality	47
Resolve Modelling Challenges	51
Write DAX Formulas for Power BI Desktop Models	52
Introduction	52
Write DAX Formulas	56

DAX Data Types.....	58
Work with DAX Functions.....	59
Use DAX Operators.....	61
Use DAX Variables	63
Add Measures to Power BI Models.....	64
Introduction.....	64
Create Simple Measures.....	66
Create Compound Measures.....	66
Create Quick Measures	67
Compare Calculated Columns with Measures	67
Summary.....	67
Add Calculated Tables and Columns to Power BI Desktop Models.....	68
Introduction.....	68
Create Calculated Columns	69
Learn About Row Context	70
Choose a Technique to Add a Column	71
Modify DAX Filter Context in Power BI Desktop Models	72
Introduction.....	72
Modify Filter Context.....	72
Use Filter Modifier Functions.....	74
Examine Filter Context.....	75
Perform Context Transition	75
Use DAX Time Intelligence Functions in Power BI Desktop Models.....	78
Introduction.....	78
Use DAX Time Intelligence Functions.....	79
Additional Time Intelligence Calculations.....	80
Optimise a Model for Performance in Power BI.....	82
Introduction.....	82
Review Performance of Measures, Relationships, and Visuals	82
Use Variables to Improve Performance and Troubleshooting.....	90
Reduce Cardinality	91
Optimize DirectQuery Models with Table Level Storage	94
Create and Manage Aggregations	98
Build Power BI Visuals and Reports.....	100
Scope Report Design Requirements.....	101
Introduction.....	101
Identify the Audience.....	101
Determine Report Types	101

Define User Interface Requirements.....	103
Define User Experience Requirements	105
Design Power BI Reports	106
Introduction.....	106
Design the Analytical Report Layout	107
Design Visually Appealing Reports.....	109
Report Objects.....	112
Create Visual Calculations in Power BI Desktop.....	113
Introduction.....	113
Understand Visual Calculations	113
Create Visual Calculations	114
Use Parameters in Visual Calculations	117
Configure Power BI Report Filters	119
Introduction to Designing Reports for Filtering.....	119
Apply Filters to the Report Structure	120
Apply Filters with Slicers	121
Design Reports with Advanced Filtering.....	123
Consumption-Time Filtering.....	124
Select Report Filter Techniques	127
Enhance Power BI Report Designs for the User Experience	129
Design Reports to Show Details.....	129
Design Reports to Highlight Values	131
Design Reports That Behave Like Apps	134
Work with Bookmarks.....	135
Design Reports with Navigation	138
Work with Visual Headers.....	140
Designing Reports with Built-In Assistance.....	142
Tune Report Performance	143
Perform Analytics in Power BI.....	146
Introduction.....	146
Explore Statistical Summary.....	146
Identify Outliers with Power BI Visuals	149
Group and Bin Data for Analysis.....	152
Apply Clustering Techniques.....	154
Conduct Time Series Analysis.....	157
Use the Analyse Feature	159
Create What-If Parameters	159
Use Specialised Visuals.....	160

Manage Workspaces and Semantic Models in Power BI	163
Create and Manage Workspaces in Power BI	164
Introduction.....	164
Distribute a Report or Dashboard	164
Monitor Usage and Performance.....	171
Recommend a Development Lifecycle Strategy.....	173
Troubleshoot Data by Viewing Its Lineage	179
Configure Data Protection	186
Manage Semantic Models in Power BI	187
Introduction.....	187
Use Power BI Gateway to Connect to On-Premises Data Sources.....	187
Configure a Semantic Model Scheduled Refresh	190
Configure Incremental Refresh Settings	193
Manage and Promote Semantic Models.....	196
Troubleshoot Service Connectivity	199
Boost Performance with Query Caching (Premium)	199
Create Dashboards in Power BI	201
Introduction to Dashboards.....	201
Configure Data Alerts.....	203
Explore Data by Asking Questions.....	204
Review Quick Insights.....	205
Add a Dashboard Theme.....	206
Pin a Live Report Page to a Dashboard	208
Configure a Real-Time Dashboard	209
Set Mobile View	211
Implement Row Level Security	212
Introduction.....	212
Configure Row-Level Security with the Static Method	214
Configure Row Level Security with the Dynamic Method.....	219

Prepare Data for Analysis with Power BI

Get Data in Power BI

Get Data from Files

Organizations often export and store data in files. One possible file format is a flat file. **A flat file is a type of file that has only one data table and every row of data is in the same structure. The file doesn't contain hierarchies.** Likely, you're familiar with the most common types of flat files, which are comma-separated values (.csv) files, delimited text (.txt) files, and fixed width files. Another type of file would be the output files from different applications, like Microsoft Excel workbooks (.xlsx).

Your Excel files might exist in one of the following locations:

- **Local** - You can import data from a local file into Power BI. The file isn't moved into Power BI, and a link doesn't remain to it. Instead, a new semantic model is created in Power BI, and data from the Excel file is loaded into it. Accordingly, changes to the original Excel file aren't reflected in your Power BI semantic model. You can use local data import for data that doesn't change.
- **OneDrive for Business** - You can pull data from OneDrive for Business into Power BI. This method is effective in keeping an Excel file and your semantic model, reports, and dashboards in Power BI synchronized. Power BI connects regularly to your file on OneDrive. If any changes are found, your semantic model, reports, and dashboards are automatically updated in Power BI.
- **OneDrive - Personal** - You can use data from files on a personal OneDrive account, and get many of the same benefits that you would with OneDrive for Business. However, you'll need to sign in with your personal OneDrive account, and select the **Keep me signed in** option. Check with your system administrator to determine whether this type of connection is allowed in your organization.
- **SharePoint - Team Sites** - Saving your Power BI Desktop files to SharePoint Team Sites is similar to saving to OneDrive for Business. The main difference is how you connect to the file from Power BI. You can specify a URL or connect to the root folder.

Using a cloud option such as OneDrive or SharePoint Team Sites is the most effective way to keep your file and your semantic model, reports, and dashboards in Power BI in-sync. However, if your data doesn't change regularly, saving files on a local computer is a suitable option.

Get Data from Relational Data Sources

If your organization uses a relational database for sales, you can use Power BI Desktop to connect directly to the database instead of using exported flat files.

Connecting Power BI to your database will help you to monitor the progress of your business and identify trends, so you can forecast sales figures, plan budgets and set performance indicators and targets.

Power BI Desktop can connect to many relational databases that are either in the cloud or on-premises.

Create Dynamic Reports with Parameters

Dynamic reports are reports in which the data can be changed by a developer according to user specifications. Dynamic reports are valuable because a single report can be used for multiple

purposes. If you use dynamic reports, you'll have fewer individual reports to create, which will save organizational time and resources.

You can use parameters by determining the values that you want to see data for in the report, and the report updates accordingly by filtering the data for you.

Creating dynamic reports allows you to give users more power over the data that is displayed in your reports; they can change the data source and filter the data by themselves.

Get Data from a NoSQL Database

Some organizations don't use a relational database but instead use a *NoSQL* database. A NoSQL database (also referred to as non-SQL, not only SQL or *non-relational*) is a **flexible type of database that doesn't use tables to store data**.

If you're working with data stored in JSON format, it's often necessary to **extract and normalize the data first**. This is because **JSON data is often stored in a nested or unstructured format, which makes it difficult to analyze or report on directly**.

Get Data from Online Services

To support their daily operations, organizations frequently use a range of software applications, such as SharePoint, OneDrive, Dynamics 365, Google Analytics and so on. These applications produce their own data. Power BI can combine the data from multiple applications to produce more meaningful insights and reports.

Select a Storage Mode

The most popular way to use data in Power BI is to **import it into a Power BI semantic model**. Importing the data means that **the data is stored in the Power BI file and gets published along with the Power BI reports**. This process helps make it easier for you to interact directly with your data. However, this approach might not work for all organizations.

However, sometimes there may be security requirements around your data that make it impossible to directly import a copy. Or your semantic models may simply be too large and would take too long to load into Power BI, and you want to avoid creating a performance bottleneck. Power BI solves these problems by using the **DirectQuery storage mode, which allows you to query the data in the data source directly and not import a copy into Power BI**. DirectQuery is useful because it ensures you're always viewing the most recent version of the data.

The three different types of storage modes you can choose from:

- Import – allows you to **create a local Power BI copy of your semantic models from your data source**. You can **use all Power BI service features with this storage mode, including Q&A and Quick Insights**. Data refreshes can **be scheduled or on-demand**. Import mode is the **default for creating new Power BI reports**.
- DirectQuery – The DirectQuery option is useful **when you don't want to save local copies of your data because your data won't be cached**. Instead, you can **query the specific tables that you'll need by using native Power BI queries, and the required data will be retrieved from the underlying data source**.
 - Essentially, you're creating a direct connection to the data source. Using this model **ensures that you're always viewing the most up-to-date data, and that all security requirements are satisfied**.
 - Additionally, this mode **is suited for when you have large semantic models to pull data from**.

- Instead of slowing down performance by having to load large amounts of data into Power BI, you can **use DirectQuery to create a connection to the source, solving data latency issues as well.**
- Dual (Composite), **used in both PBI Desktop and PBI Service** – In Dual mode, you can identify some data to be directly imported and other data that must be queried. Any table that is brought in to your report is a product of both Import and DirectQuery modes. **Using the Dual mode allows Power BI to choose the most efficient form of data retrieval.**

The storage mode setting in Power BI Desktop is one of three related features:

- **Composite models:** Allows a report to have two or more data connections, including DirectQuery connections or Import, in any combination.
- **Many-to-many relationships:** With composite models, you can establish *many-to-many relationships* between tables. In a many-to-many relationship, requirements are removed for unique values in tables. It also removes prior workarounds, such as introducing new tables only to establish relationships.
- **Storage mode:** With storage mode, you can now specify which visuals require a query to back-end data sources. Visuals that don't require a query are imported even if they're based on DirectQuery. This feature helps improve performance and reduce back-end load. Previously, even simple visuals, such as slicers, initiated queries that were sent to back-end sources.

Get Data from Azure Analysis Services

Azure Analysis Services is a fully managed platform as a service (PaaS) that provides enterprise-grade semantic models in the cloud. You can use advanced mashup and modelling features to combine data from multiple data sources, define metrics, and secure your data in a single, trusted tabular semantic model. The semantic model provides an easier and faster way for users to perform ad hoc data analysis using tools like Power BI.

Connect live is an option for Azure Analysis Services. Azure Analysis Services uses the tabular model and DAX to build calculations, similar to Power BI.

- These models are compatible with one another. Using the Connect live option **helps you keep the data and DAX calculations in their original location, without having to import them all into Power BI.**
- Azure Analysis Services can have **a fast refresh schedule**, which means that when data is refreshed in the service, Power BI reports will immediately be updated, without the need to initiate a Power BI refresh schedule. This process can improve the timeliness of the data in your report.
- Similar to a relational database, you can choose the tables that you want to use. If you want to directly query the Azure Analysis Services model, you can use DAX or MDX.
- You'll likely import the data directly into Power BI. An acceptable alternative is to import all other data that you want (from Excel, SQL Server, and so on) into the Azure Analysis Services model and then use a live connection.
- This approach **simplifies your solution by keeping the semantic modelling and DAX measures in one place.**

Fix Performance Issues

Occasionally, organizations will need to address performance issues when running reports. Power BI provides the Performance Analyzer tool to help fix problems and streamline the process.

Optimize performance in Power Query

- **The performance in Power Query depends on the performance at the data source level.**
- The variety of data sources that Power Query offers is wide, and the performance tuning techniques for each source are equally wide.
 - For instance, if you extract data from a Microsoft SQL Server, you should follow the performance tuning guidelines for the product.
 - Good SQL Server performance tuning techniques include index creation, hardware upgrades, execution plan tuning, and data compression. These topics are beyond the scope here, and are covered only as an example to build familiarity with your data source and reap the benefits when using Power BI and Power Query.

Power Query takes advantage of good performance at the data source through a technique called Query Folding.

Query folding

- The query folding within Power Query Editor helps you increase the performance of your Power BI reports.
- *Query folding* is the process by which **the transformations and edits that you make in Power Query Editor are simultaneously tracked as native queries, or simple Select SQL statements, while you're actively making transformations.**
- The reason for implementing this process is to ensure that these transformations can take place in the original data source server and don't overwhelm Power BI computing resources.

You can use Power Query to load data into Power BI. Then use Power Query Editor to transform your data, such as renaming or deleting columns, appending, parsing, filtering, or grouping your data.

The benefits to query folding include:

- **More efficiency in data refreshes and incremental refreshes** – When you import data tables by using query folding, Power BI is better able to allocate resources and refresh the data faster because Power BI doesn't have to run through each transformation locally.
- **Automatic compatibility with DirectQuery and Dual storage modes** – All DirectQuery and Dual storage mode data sources must have the back-end server processing abilities to create a direct connection, which means that query folding is an automatic capability that you can use. If all transformations can be reduced to a single **Select** statement, then query folding can occur.

Query diagnostics

- Another tool that you can use to study query performance is query diagnostics.
- You can determine what bottlenecks may exist while loading and transforming your data, refreshing your data in Power Query, running SQL statements in Query Editor, and so on.

Other techniques to optimize performance

Other ways to optimize query performance in Power BI include:

- **Process as much data as possible in the original data source** – Power Query and Power Query Editor allow you to process the data; however, the processing power that is required to complete this task might lower performance in other areas of your reports. Generally, a good practice is to process, as much as possible, in the native data source.
- **Use native SQL queries.** When using DirectQuery for SQL databases, such as the case for our scenario, make sure that you aren't pulling data from stored procedures or common table expressions (CTEs).
- **Separate date and time, if bound together.** If any of your tables have columns that combine date and time, make sure that you separate them into distinct columns before importing them into Power BI. This approach will increase compression abilities.

Resolve Data Import Errors

While importing data into Power BI, you may encounter errors resulting from factors such as:

- Power BI imports from numerous data sources.
- Each data source might have dozens (and sometimes hundreds) of different error messages.
- Other components can cause errors, such as hard drives, networks, software services, and operating systems.
- Data often can't comply with any specific schema.

Query timeout expired

- **Relational source systems often have many people who are concurrently using the same data in the same database.**
- Some relational systems and their administrators seek to limit a user from monopolizing all hardware resources by setting a query timeout.
- These timeouts can be configured for any timespan, from as little as five seconds to as much as 30 minutes or more.

Power BI Query Error: Timeout expired

- This error indicates that **you've pulled too much data according to your organization's policies.** Administrators incorporate this policy to avoid slowing down a different application or suite of applications that might also be using that database.
- You can resolve this error by **pulling fewer columns or rows from a single table.** While you're writing SQL statements, **it might be a common practice to include groupings and aggregations.** You can **also join multiple tables in a single SQL statement.**
- Additionally, you can **perform complicated subqueries and nested queries in a single statement. These complexities add to the query processing requirements of the relational system and can greatly elongate the time of implementation.**
- If you need the rows, columns, and complexity, consider **taking small chunks of data and then bringing them back together by using Power Query. For instance, you can combine half the columns in one query and the other half in a different query. Power Query can merge those two queries back together after you're finished.**

Data type errors

- Sometimes, when you import data into Power BI, the columns appear blank. This situation happens because of an **error in interpreting the data type in Power BI.**
- **The resolution to this error is unique to the data source.** For instance, if you're importing data from SQL Server and see blank columns, you could try to convert to the correct data type in the query.
- Instead of using this query:

- `SELECT CustomerPostalCode FROM Sales.Customers`
- Use this query:
 - `SELECT CAST(CustomerPostalCode as varchar(10)) FROM Sales.Customers`
- **By specifying the correct type at the data source, you eliminate many of these common data source errors.**
- You may encounter different types of errors in Power BI that are caused by the diverse data source systems where your data resides.

Summary

- You can pull data from files, relational databases, Azure Analysis Services, cloud-based applications, websites, and more.
- Retrieving data from different data sources requires treating each data source differently. For instance, Microsoft Excel data should be pulled in from an Excel table. Relational databases often have query timeouts.
- You can connect to SQL Server Analysis Services with Connect live, which allows you to see data changes in real-time.
- **It's important to select the correct storage mode for your data. Do you require that visuals interact quickly but don't mind possibly refreshing the data when the underlying data source changes? If so, select Import to import data into Power BI. If you prefer to see updates to data as soon as they happen at the cost of interactivity performance, then choose Direct Query for your data instead.**
- In addition, you learned how to solve performance problems and data import errors. You learned that Power BI gives you tooling to identify where performance problems may exist.
- Data import errors can be alarming at first, but you can see that the resolution is easily implemented.

Clean, Transform, and Load Data in Power BI

Shape the Initial Data

- Power Query Editor in Power BI Desktop allows you to shape (transform) your imported data.
- You can accomplish actions such as renaming columns or tables, changing text to numbers, removing rows, setting the first row as headers, and much more.
- It is important to shape your data to ensure that it meets your needs and is suitable for use in reports.

Identify column headers and names

- The first step in shaping your initial data is to identify the column headers and names within the data and then evaluate where they are located to ensure that they are in the right place.
- When you have identified where the column headers and names are located, you can make changes to reorganize the data.

Promote headers

- When a table is created in Power BI Desktop, Power Query Editor assumes that all data belongs in table rows.
- However, a data source might have a first row that contains column names
- To correct this inaccuracy, you need to promote the first table row into column headers.
 - You can promote headers in two ways: by selecting the **Use First Row as Headers** option on the **Home** tab or by selecting the drop-down button next to **Column1** and then selecting **Use First Row as Headers**.

Rename columns

- The next step in shaping your data is to examine the column headers. You might discover that one or more columns have the wrong headers, a header has a spelling error, or the header naming convention is not consistent or user-friendly.
- You can rename column headers in two ways. One approach is to right-click the header, **select Rename**, edit the name, and then **press Enter**. Alternatively, you can double-click the column header and overwrite the name with the correct name.
- **You can also work around this issue by removing (skipping) the first two rows and then renaming the columns to the correct name.**

Remove top rows

- When shaping your data, you might need to remove some of the top rows, for example, if they are blank or if they contain data that you do not need in your reports.
- To remove these excess rows, select **Remove Rows > Remove Top Rows** on the **Home** tab.

Remove columns

- A key step in the data shaping process is to remove unnecessary columns. It is much better to remove columns as early as possible.
- One way to remove columns would be to limit the column when you get data from data source.
- For instance, if you are extracting data from a relational database by using SQL, you would want to limit the column that you extract by using a column list in the **SELECT** statement.

- Removing columns at an early stage in the process rather than later is best, especially when you have established relationships between your tables.
 - Removing unnecessary columns will help you to focus on the data that you need and help improve the overall performance of your Power BI Desktop semantic models and reports.
- Examine each column and ask yourself if you really need the data that it contains. If you don't plan on using that data in a report, the column adds no value to your semantic model.
 - Therefore, the column should be removed. You can always add the column later, if your requirements change over time.
- You can remove columns in two ways –
 - The first method is to select the columns that you want to remove and then, on the **Home** tab, select **Remove Columns**.
 - Alternatively, you can select the columns that you want to keep and then, on the **Home** tab, select **Remove Columns > Remove Other Columns**.

Unpivot columns

- Unpivoting is a useful feature of Power BI.
- You can use this feature with data from any data source, but you would most often use it when importing data from Excel.
- Unpivoting streamlines the process of creating DAX measures on the data later.

Pivot columns

- If the data that you are shaping is flat (in other words, it has a lot of detail but is not organized or grouped in any way), the lack of structure can complicate your ability to identify patterns in the data.
- You can use the **Pivot Column** feature to convert your flat data into a table that contains an aggregate value for each unique value in a column.
 - For example, you might want to use this feature to summarize data by using different math functions such as **Count**, **Minimum**, **Maximum**, **Median**, **Average**, or **Sum**.

Power Query Editor records all steps that you take to shape your data, and the list of steps are shown in the **Query Settings** pane.

- If you have made all the required changes, select **Close & Apply** to close Power Query Editor and apply your changes to your semantic model.
- However, before you select **Close & Apply**, you can take further steps to clean up and transform your data in Power Query Editor.

Simplify the Data Structure

- When you import data from multiple sources into Power BI Desktop, the data retains its predefined table and column names.
- You might want to change some of these names so that they are in a consistent format, easier to work with, and more meaningful to a user.
- You can use Power Query Editor in Power BI Desktop to make these name changes and simplify your data structure.

Rename a query

- It's good practice to change uncommon or unhelpful query names to names that are more obvious or that the user is more familiar with.

- For instance, if you import a product fact table into Power BI Desktop and the query name displays as *FactProductTable*, you might want to change it to a more user-friendly name, such as *Products*.
- Similarly, if you import a view, the view might have a name that contains a prefix of *v*, such as *product*. People might find this name unclear and confusing, so you might want to remove the prefix.

Replace values and Replace null values

- You can use the **Replace Values** feature in Power Query Editor to replace any value with another value in a selected column.
 - Select the column that contains the value that you want to replace and then select **Replace Values** on the **Transform** tab.
 - In the **Value to Find** box, enter the name of the value that you want to replace, and then in the **Replace With** box, enter the correct value name and then select **OK**.
 - In Power Query, you can't select one cell and change one value, like you might have done in Excel.
- Occasionally, you might find that your data sources contain null values.
 - For example, a freight amount on a sales order might have a null value if it's synonymous with zero.
 - If the value stays null, the averages will not calculate correctly.
- One solution would be to change the nulls to zero, which will produce the more accurate freight average.
 - In this instance, using the same steps that you followed previously will help you replace the null values with zero.

Remove duplicates

- You can also remove duplicates from columns to only keep unique names in a selected column by using the **Remove Duplicates** feature in Power Query.

Best practices for naming tables, columns, and values

- Naming conventions for tables, columns, and values have no fixed rules
 - However, we recommend that you use the language and abbreviations that are commonly used within your organization and that everyone agrees on and considers them as common terminology.
- A best practice is to give your tables, columns, and measures descriptive business terms and replace underscores (" _ ") with spaces.
 - Be consistent with abbreviations, prefixes, and words like "number" and "ID."
 - Excessively short abbreviations can cause confusion if they are not commonly used within the organization.
- Also, by removing prefixes or suffixes that you might use in table names and instead naming them in a simple format, you will help avoid confusion.
- When replacing values, try to imagine how those values will appear on the report. Values that are too long might be difficult to read and fit on a visual.
 - Values that are too short might be difficult to interpret.
 - Avoiding acronyms in values is also a good idea, provided that the text will fit on the visual.

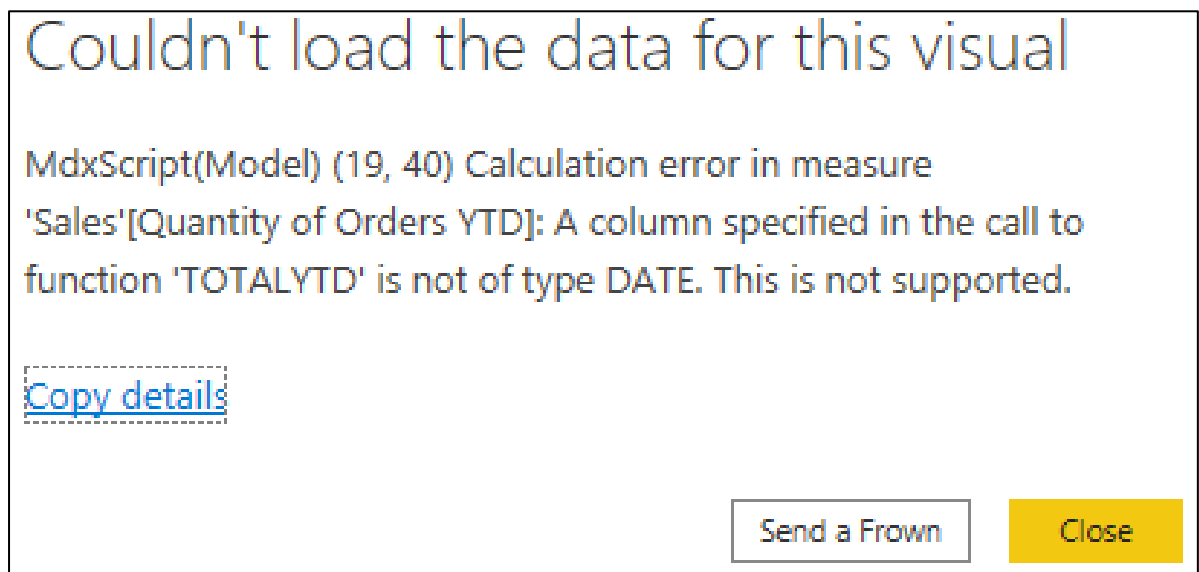
Evaluate and Change Column Data Types

- When you import a table from any data source, Power BI Desktop automatically starts scanning the first 1,000 rows (default setting) and tries to detect the type of data in the columns.

- Some situations might occur where Power BI Desktop doesn't detect the correct data type. Where incorrect data types occur, you'll experience performance issues.
- You have a higher chance of getting data type errors when you're dealing with flat files, such as comma-separated values (.CSV) files and Excel workbooks (.XLSX), because data was entered manually into the worksheets and mistakes were made.
 - Conversely, in databases, the data types are predefined when tables or views are created.
- A best practice is to evaluate the column data types in Power Query Editor before you load the data into a Power BI semantic model.
 - If you determine that a data type is incorrect, you can change it.
 - You might also want to apply a format to the values in a column and change the summarization default for a column.

Implications of incorrect data types

- The following information provides insight into problems that can arise when Power BI doesn't detect the correct data type.
- Incorrect data types will prevent you from creating certain calculations, deriving hierarchies, or creating proper relationships with other tables.
 - For example, if you try to calculate the Quantity of Orders YTD, you'll get the following error stating that the **OrderDate** column data type isn't Date, which is required in time-based calculations.
 - Quantity of Orders YTD = TOTALYTD(SUM('Sales'[OrderQty]), 'Sales'[OrderDate])



- Another issue with having an incorrect data type applied on a date field is the inability to create a date hierarchy, which would allow you to analyze your data on a yearly, monthly, or weekly basis.

Change the column data type

- You can change the data type of a column in two places: in Power Query Editor and in the Power BI Desktop Report view by using the column tools.
- It is best to change the data type in the Power Query Editor before you load the data.
- As with any other changes that you make in Power Query Editor, the change that you make to the column data type is saved as a programmed step. This step is called Changed Type and it will be iterated every time the data is refreshed.

- After you have completed all steps to clean and transform your data, select Close & Apply to close Power Query Editor and apply your changes to your semantic model. At this stage, your data should be in great shape for analysis and reporting.

Combine Multiple Tables into a Single Table

The ability to combine queries is powerful because it allows you to append or merge different tables or queries together. You can combine tables into a single table in the following circumstances:

- Too many tables exist, making it difficult to navigate an overly complicated semantic model.
- Several tables have a similar role.
- A table has only a column or two that can fit into a different table.
- You want to use several columns from different tables in a custom column.

You can combine the tables in two different ways: merging and appending.

Append queries

- When you append queries, you'll be adding rows of data to another table or query.
- For example, you could have two tables, one with 300 rows and another with 100 rows, and when you append queries, you'll end up with 400 rows.
- When you merge queries, you'll be adding columns from one table (or query) into another.
- To merge two tables, you must have a column that is the key between the two tables.

Merge queries

- When you merge queries, you're combining the data from multiple tables into one based on a column that is common between the tables.
- This process is similar to the JOIN clause in SQL.
- Consider a scenario where the Sales team now wants you to consolidate orders and their corresponding details (which are currently in two tables) into a single table.
- You can also choose how to join the two tables together, a process that is also similar to JOIN statements in SQL. These join options include:
 - Left Outer - Displays all rows from the first table and only the matching rows from the second.
 - Full Outer - Displays all rows from both tables.
 - Inner - Displays the matched rows between the two tables.

Profile Data in Power BI

- Profiling data is about studying the nuances of the data: determining anomalies, examining and developing the underlying data structures, and querying data statistics such as row counts, value distributions, minimum and maximum values, averages, and so on.
 - This concept is important because it allows you to shape and organize the data so that interacting with the data and identifying the distribution of the data is uncomplicated, therefore helping to make your task of working with the data on the front end to develop report elements near effortless.
- Assume that you are developing reports for the Sales team at your organization. You are uncertain how the data is structured and contained within the tables, so you want to profile the data behind the scenes before you begin developing the visuals.
 - Power BI has inherent functionality that makes these tasks user-friendly and straightforward.

Examine data structures

- Before you begin examining the data in Power Query Editor, you should first learn about the underlying data structures that data is organized in. You can view the current semantic model under the **Model** tab on Power BI Desktop.
- On the **Model** tab, you can edit specific column and table properties by selecting a table or columns, and you can transform the data by using the **Transform Data** button, which takes you to Power Query Editor. Additionally, you can manage, create, edit, and delete relationships between different tables by using **Manage Relationships**, which is located on the ribbon.

Find data anomalies and data statistics

- After you have created a connection to a data source and have selected **Transform Data**, you are brought to Power Query Editor, where you can determine if anomalies exist within your data.
- Data anomalies are outliers within your data. Determining what those anomalies are can help you identify what the normal distribution of your data looks like and whether specific data points exist that you need to investigate further.
 - Power Query Editor determines data anomalies by using the **Column Distribution** feature.
 - Select **View** on the ribbon, and under **Data Preview**, you can choose from a few options. To understand data anomalies and statistics, select the **Column Distribution**, **Column Quality**, and **Column Profile** options. The following figure shows the statistics that appear.
 - **Column quality** and **Column distribution** are shown in the graphs above the columns of data. **Column quality** shows you the percentages of data that is valid, in error, and empty. In an ideal situation, you want 100 percent of the data to be valid.
- **Column distribution** shows you the distribution of the data within the column and the counts of distinct and unique values, both of which can tell you details about the data counts.
 - Distinct values are all the different values in a column, including duplicates and null values, while unique values do not include duplicates or nulls. Therefore, **distinct** in this table tells you the total count of how many values are present, while **unique** tells you how many of those values only appear once.
- **Column profile** gives you a more in-depth look into the statistics within the columns for the first 1,000 rows of data.
 - This column provides several different values, including the count of rows, which is important when verifying whether the importing of your data was
- Additionally, the **Value distribution** graph tells you the counts for each distinct value in that specific column.
- On a numeric column, **Column Statistics** will also include how many zeroes and null values exist, along with the average value in the column, the standard deviation of the values in the column, and how many even and odd values are in the column.
 - These statistics give you an idea of the distribution of data within the column, and are important because they summarize the data in the column and serve as a starting point to determine what the outliers are.

Use Advanced Editor to Modify M Code

- Each time you shape data in Power Query, you create a step in the Power Query process.
- Those steps can be reordered, deleted, and modified where it makes sense.

- Each cleaning step that you made was likely created by using the graphical interface, but Power Query uses the M language behind the scenes.
- The combined steps are available to read by using the Power Query Advanced Editor.
 - The M language is always available to be read and modified directly. It is not required that you use M code to take advantage of Power Query.
 - You will rarely need to write M code, but it can still prove useful. Because each step in Power Query is written in M code, even if the UI created it for you, you can use those steps to learn M code and customize it to suit your needs.
- Each Power Query step will roughly align with one or two lines of M code. You don't have to be an expert in M code to be able to read it. You can even experiment with changing it. For instance, if you need to change the name of a database, you could do it right in the code and then select **Done**.
- You might notice that M code is written top-down. Later steps in the process can refer to previous steps by the variable name to the left of the equal sign. Be careful about reordering these steps because it could ruin the statement dependencies. Write to a query formula step by using the **in** statement. Generally, the last query step is used as the **in final data set** result.

Choose a Power BI Model Framework

Describe Power BI Model Fundamentals

Data model

- A Power BI data model is a query-able data resource that's optimized for analytics.
- Reports can query data models by using one of two analytic languages: Data Analysis Expressions (DAX) or Multidimensional Expressions (MDX).
 - Power BI uses DAX, while paginated reports can use either DAX or MDX.
 - The Analyze in Excel features use MDX.
- A data model is also **described as a semantic model, especially in enterprise scenarios**. Commonly, in the context of a data discussion and in this module, a data model is simply referred to as a model.

Power BI dataset

- You develop a Power BI model in Power BI Desktop, and once published to a workspace in the Power BI service, it's then known as a dataset.
- A dataset is a Power BI artifact that's a source of data for visualizations in Power BI reports and dashboards.

Analytic query

- Power BI reports and dashboards must query a dataset.
- When Power BI visualizes dataset data, it prepares and sends an analytic query.
- An analytic query produces a query result from a model that's easy for a person to understand, especially when visualized.
- An analytic query has three phases that are executed in this order:
 1. Filter
 2. Group
 3. Summarize

Filtering

- Sometimes known as slicing, narrows down on a subset of the model data.

- Filter values aren't visible in the query result.
- Most analytic queries apply filters because it's common to filter by a time period, and usually other attributes.
- Filtering happens in different ways. In a Power BI report, you can set filters at report, page, or visual level.
- Report layouts often include slicer visuals to filter visuals on the report page.
- When the model enforces row-level security (RLS), it applies filters to model tables to restrict access to specific data.
- Measures, which summarize model data, can also apply filters.

Grouping

- Sometimes known as dicing, divides query result into groups.
- Each group is also a filter, but unlike the filtering phase, filter values are visible in the query result.
- For example, grouping by customer filters each group by customer.

Summarization

- Produces a single value result.
- Typically, a report visual summarizes a numeric field by using an aggregate function. Aggregate functions include sum, count, minimum, maximum, and others.
- You can achieve simple summarization by aggregating a column, or you can achieve complex summarization by creating a measure using a DAX formula.

Tabular model

- A Power BI model is a tabular model. A tabular model comprises one or more tables of columns. It can also include relationships, hierarchies, and calculations.

Star schema design

- To produce an optimized and easy-to-use tabular model, we recommend you produce a star schema design.
- Star schema design is a mature modelling approach widely adopted by relational data warehouses.
- It requires you to classify model tables as either dimension or fact.
- Dimension tables describe business entities; the things you model.
- Entities can include products, people, places, and concepts including time itself.
- Fact tables store observations or events, and can be, for example, sales orders, stock balances, exchange rates, or temperature readings.
- A fact table contains dimension key columns that relate to dimension tables, and numeric measure columns.
- A fact table forms the centre of a star, and the related dimension tables form the points of the star.
- In an analytic query, dimensions table columns filter or group. Fact table columns are summarized.

Normalization vs. denormalization

- To understand some star schema concepts, it's important to know two terms: **normalization and denormalization**.
- Normalization is the term used to describe data that's stored in a way that reduces repetitious data.

- Consider a table of products that has a unique key value column, like the product key, and other columns that describe product characteristics, like product name, category, colour, and size.
 - A sales table is considered normalized when it stores only keys, like the product key.
- If, however, the sales table stores product details beyond the key, it's considered **denormalized**
- When you source data from an export file or data extract, it's likely that it represents a denormalized set of data.
 - In this case, use Power Query to transform and shape the source data into multiple normalized tables.
- You should strive to develop optimized Power BI semantic models with tables that represent normalized fact and dimension data.
- However, there's one exception where a snowflake dimension might be denormalized in order to produce a single model table.

Star schema relevance to Power BI semantic models

- Star schema design and many related concepts are highly relevant to developing Power BI models that are optimized for performance and usability.
- Consider that each Power BI report visual generates a query that's sent to the Power BI semantic model.
 - Generally, queries filter, group, and summarize model data. A well-designed model, then, is one that provides tables for filtering and grouping, and tables for summarizing.
- This design fits well with star schema principles:
 - Dimension tables enable *filtering* and *grouping*.
 - Fact tables enable *summarization*.
- There's no table property that modelers set to set the table type as dimension or fact. It's in fact determined by the model relationships.
- A model relationship establishes a filter propagation path between two tables, and it's the cardinality property of the relationship that determines the table type.
 - A common relationship cardinality is *one-to-many* or its inverse *many-to-one*. The "one" side is always a dimension table while the "many" side is always a fact table.
- A well-structured model design includes tables that are either dimension tables or fact tables. Avoid mixing the two types together for a single table.
 - We also recommend that you strive to deliver the right number of tables with the right relationships in place.
 - It's also important that fact tables always load data at a consistent grain.
- Lastly, it's important to understand that optimal model design is part science and part art. Sometimes you can break with good guidance when it makes sense to do so.

Table storage mode

Each Power BI model table (except calculated tables) has a storage mode property. The storage mode property can be either **Import**, **DirectQuery**, or **Dual**, and it determines whether table data is stored in the model.

- **Import** – Queries retrieve data that's stored, or cached, in the model.
- **DirectQuery** – Queries pass through to the data source.
- **Dual** – Queries retrieve stored data or pass through to the data source. Power BI determines the most efficient plan, striving to use cached data whenever possible.

Model framework

- Table storage mode settings determine the model framework, which can be either import, DirectQuery, or composite.
- The following units describe each of these frameworks and provides guidance on their use.
 - An import model comprises tables that have their storage mode property set to **Import**.
 - A DirectQuery model comprises tables that have their storage mode property set to **DirectQuery**, and they belong to the same source group.
 - A composite model comprises more than one source group.

Determine When to Develop an Import Model

An import model comprises tables that have their storage mode property set to **Import**. It includes calculated tables, which you define with a DAX formula, too.

Import model benefits

Import models are the most frequently developed model framework because there are many benefits. Import models:

- Support all Power BI data source types, including databases, files, feeds, web pages, dataflows, and more.
- Can integrate source data. For example, one table sources its data from a relational database while a related table sources its data from a web page.
- Support all DAX and Power Query (M) functionality.
- Support calculated tables.
- Deliver the best query performance. That's because the data cached in the model is optimized for analytic queries (filter, group, and summarize) and the model is stored entirely in memory.

In short, import models offer you the most options and design flexibility, and they deliver fast performance. For this reason, Power BI Desktop defaults to use import storage mode when you "Get data."

Import model limitations

Despite the many compelling benefits, there are limitations of import models that you must bear in mind. Limitations are related to **model size** and **data refresh**.

Model size

- Power BI imposes dataset size restrictions, which limit the size of a model. When you publish the model to a shared capacity, there's a 1-GB limit per dataset.
- When this size limit is exceeded, the dataset will fail to refresh.
- When you publish the model to a dedicated capacity (also known as Premium capacities), it can grow beyond 10-GB, providing you enable the Large dataset storage format setting for the capacity.
- You should always strive to reduce the amount of data stored in tables. This strategy helps to reduce the duration of model refreshes and speed up model queries. There are numerous data reduction techniques that you can apply, including:
 - Remove unnecessary columns
 - Remove unnecessary rows
 - Group by and summarize to raise the grain of fact tables
 - Optimize column data types with a preference for numeric data
 - Preference for custom columns in Power Query instead of calculated columns in the model

- Disable Power Query load
- Disable auto date/time
- Use DirectQuery table storage

Data refresh

- Imported data must be periodically refreshed. Dataset data is only as current as the last successful data refresh.
 - To keep data current, you set up scheduled data refresh, or report consumers can perform an on-demand refresh.
- Power BI imposes limits on how often scheduled refresh operations can occur. **It's up to eight times per day in a shared capacity, and up to 48 times per day in a dedicated capacity.**
- You should determine whether this degree of latency is tolerable. It often depends on the velocity (or volatility) of the data, and the urgency to keep users informed about the current state of data.
 - When scheduled refresh limits aren't acceptable, consider using DirectQuery storage tables, or creating a hybrid table. Or take a different approach, and create a real-time dataset instead.
- You must also consider refresh workload and duration. By default, to refresh a table, Power BI removes all data and reloads it again.
 - These operations can place an unacceptable burden on source systems, especially for large fact tables.
 - To reduce this burden, you can set up the incremental refresh feature. Incremental refresh automates the creation and management of time-period partitions, and intelligently update only those partitions that require refresh.
- When your data source supports incremental refresh, it can result in faster and more reliable refreshes, and reduced resource consumption of Power BI and source systems.

Determine When to Develop a DirectQuery Model

A DirectQuery model comprises tables that have their storage mode property set to **DirectQuery**, and they belong to the same source group.

A source group is a set of model tables that relate to a data source. There are two types:

- **Import** – Represents all import storage mode tables including calculated tables. There can only be one import source group in a model.
- **DirectQuery** – Represents all DirectQuery storage mode tables that relate to a specific data source.

DirectQuery model benefits

There are several benefits to developing a DirectQuery model.

- **Model large or fast-changing data sources**
 - A DirectQuery model is a great framework choice when your source data exhibits volume and/or velocity characteristics. **Because DirectQuery tables don't require refresh, they're well suited to large data stores, like a data warehouse.** It's impractical and inefficient, if not impossible, to import an entire data warehouse into a model. **When the source data changes rapidly and users need to see current data, a DirectQuery model can deliver near real-time query results.**
 - When a report queries a DirectQuery model, Power BI passes those queries to the underlying data source.

- **Enforce source RLS**
 - DirectQuery is also useful when the source database enforces row-level security (RLS). Instead of replicating RLS rules in your Power BI model, the source data base can enforce its rules. This approach works only for some relational databases, and it involves setting up single sign-on for the dataset data source.
- **Data sovereignty restrictions**
 - If your organization has security policies that restrict data leaving their premises, then it isn't possible to import data. A DirectQuery model that connects to an on-premises data source may be appropriate. (You can also consider installing Power BI Report Server for on-premises reporting.)
- **Create specialized datasets**
 - Typically, DirectQuery mode supports relational database sources. That's because Power BI must translate analytic queries to native queries understood by the data source.
 - However, there's one powerful exception. You can connect to a Power BI dataset (or Azure Analysis Services model) and convert it to a DirectQuery local model. **A local model is a relative term that describes a model's relationship to another model. In this case, the original dataset is a remote model, and the new dataset is the local model. These models are chained, which is term used to describe related models. You can chain up to three models in this way.**
 - This capability to chain models supports the potential to personalize and/or extend a remote model. The simplest thing you can do is rename objects, like tables or columns, or add measures to the local model. You can also extend the model with calculated columns or calculated tables, or add new import or DirectQuery tables. However, these extensions result in the creation of new source groups, which means the model becomes a composite model.

Boost DirectQuery model performance

When there's a justification to develop a DirectQuery model, you can mitigate some limitations in two ways.

- **Data source optimizations**
 - You can optimize the source database to ensure the expected analytic query workload performs well. Specifically, you can create indexes and materialized views, and ensure the database has sufficient resources for all workloads.
 - We recommend that you always work in collaboration with the database owner. It's important that they understand the additional workload a DirectQuery model can place on their database.
- **DirectQuery user-defined aggregation tables**
 - You can add user-defined aggregation tables to a DirectQuery model. User-defined aggregation tables are special model tables that are hidden (from users, calculations, and RLS). They work best when they satisfy higher-grain analytic queries over large fact tables. When you set the aggregation table to use DirectQuery storage mode, it can query a materialized view in the data source. You can also set an aggregation table to use import storage mode or enable automatic aggregations

Determine When to Develop a Composite Model

A composite model comprises more than one source group. Typically, there's always the import source group and a DirectQuery source group.

Composite model benefits

- There are several benefits to developing a composite model.
 - Composite models provide you with design flexibility. You can choose to integrate data using different storage modes, striking the right balance between imported data and pass-through data. Commonly, enterprise models benefit from using DirectQuery tables on large data sources and by boosting query performance with imported tables. Power BI features that support this scenario are described later in this unit.
 - Composite models can also boost the performance of a DirectQuery model by providing Power BI with opportunity to satisfy some analytic queries from imported data. Querying cached data almost always performs better than pass-through queries.
 - Lastly, when your model includes DirectQuery tables to a remote model, like a Power BI dataset, you can extend your model with new calculated columns and tables. It results in a specialized model based on a core model.

Composite model limitations

- There are several limitations related to composite models.
 - Import (or dual, as described later) storage mode tables still require periodic refresh. Imported data can become out of sync with DirectQuery sourced data, so it's important to refresh it periodically.
 - When an analytic query must combine imported and DirectQuery data, Power BI must consolidate source group query results, which can impact performance. To help avoid this situation for higher-grain queries, you can add import aggregation tables to your model (or enable automatic aggregations) and set related dimension tables to use dual storage mode. This scenario is described later in this unit.
 - When chaining models (DirectQuery to Power BI datasets), modifications made to upstream models can break downstream models. Be sure to assess the impact of modifications by performing dataset impact analysis first.
 - Relationships between tables from different source groups are known as limited relationships. A model relationship is considered limited when there's no guaranteed "one" side (The "one" side means the column contains unique values; the "many" side means the column can contain duplicate values.)
 - This can occur in two situations. The first is when the relationship uses a many-to-many cardinality type, even if one or both columns contain unique values.
 - The second is when the relationship spans different source groups, which only happens in composite models. Limited relationships may result in different evaluations of model queries and calculations.

Boost DirectQuery model performance with import data

When there's a justification to develop a DirectQuery model, you can mitigate some limitations by using specific Power BI features that involve import tables.

Import aggregation tables

- You can add import storage mode user-defined aggregation tables or enable automatic aggregations.
- This way, Power BI directs higher-grain fact queries to a cached aggregation.
- To boost query performance further, ensure that related dimension tables are set to use dual storage mode.

Dual storage mode

- A dual storage mode table is set to use both import and DirectQuery storage modes. At query time, Power BI determines the most efficient mode to use.
- Whenever possible, Power BI attempts to satisfy analytic queries by using cached data.
- Dual storage mode tables work well with import aggregation tables. They allow Power BI to satisfy higher-grain queries entirely from cached data.
- Slicer visuals and filter card lists, which are often based on dimension table columns, render more quickly because they're queried from cached data.

Deliver real-time data from an import model

- When you set up an import table with incremental refresh, you can enable the **Get the latest data in real-time with DirectQuery** option.

Incremental refresh and real-time data

Refresh large tables faster with incremental refresh. Plus, get the latest data in real time with DirectQuery (Premium only). [Learn more](#)

 These settings will apply when you publish the dataset to the Power BI service. Once you do that, you won't be able to download it back to Power BI Desktop. [Learn more](#)

1. Select table

FactInternetSales 

2. Set import and refresh ranges

 Incrementally refresh this table

Archive data starting

Years 

 before refresh date

Data imported from 12/21/2016 to 12/18/2021.

Incrementally refresh data starting

Days 

 before refresh date

Data will be incrementally refreshed from 12/18/2021 to 12/21/2021.

3. Choose optional settings

 Get the latest data in real time with DirectQuery (Premium only) [Learn more](#)

- By enabling this option, Power BI automatically creates a table partition that uses DirectQuery storage mode. In this case, the table becomes a hybrid table, meaning it has import partitions to store older data, and a single DirectQuery partition for current data.
- When Power BI queries a hybrid table, the query uses the cache for older data, and passes through to the data source to retrieve current data.
- This option is only available with a Premium license.

Choose a Model Framework

- Here's some general guidance about choosing the appropriate model framework for your project. It especially applies to enterprise solutions, where data volumes are large, query throughput is high, and fast responsiveness is important.
 - Most importantly, choose the import model framework whenever possible. This framework offers you the most options, design flexibility, and delivers fast performance.
 - Be sure to apply data reduction techniques to ensure that Power BI loads the least amount of data possible.
 - Choose the DirectQuery model framework when your data source stores large volumes of data and/or your report needs to deliver near real-time data.
 - Choose the composite model framework to:
 - Boost the query performance of a DirectQuery model.
 - Deliver near real-time query results from an import model.
 - Extend a Power BI dataset (or AAS model) with additional data.
 - You can boost the query performance of a DirectQuery model by using aggregation tables, which can use import or DirectQuery storage mode.
 - When using import aggregation tables, be sure to set related dimension tables to use dual storage mode. That way, Power BI can satisfy higher-grain queries entirely from cache.
 - You can deliver near real-time query results from in import model by creating a hybrid table.
 - In this case, Power BI adds a DirectQuery partition for the current period.
 - Lastly, you can create specialized models by chaining to a core model by using DirectQuery. This type of development is typically done by a business analyst who extends core models, which IT delivers and supports.

Model Data with Power BI

Design a Semantic Model in Power BI

Creating a great semantic model is one of the most important tasks that a data analyst can perform in Microsoft Power BI.

By doing this job well, you help make it easier for people to understand your data, which will make building valuable Power BI reports easier for them and for you.

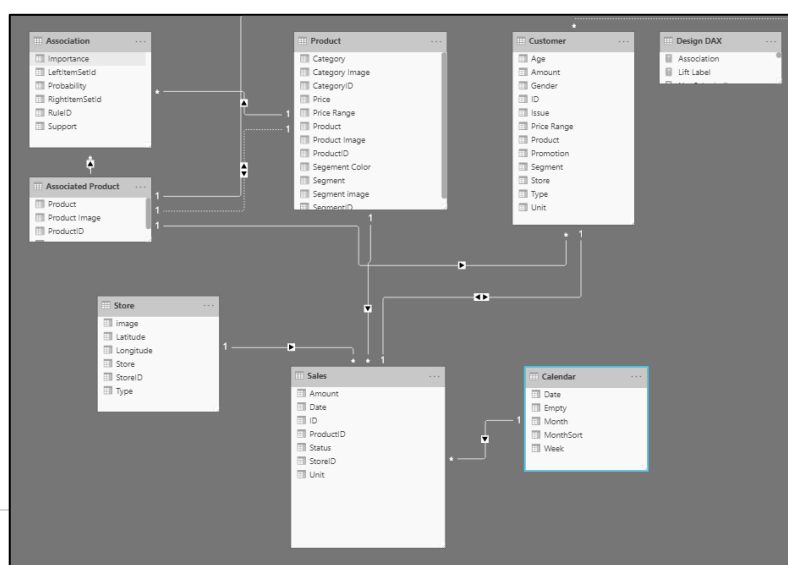
A good semantic model offers the following benefits:

- Data exploration is faster.
- Aggregations are simpler to build.
- Reports are more accurate.
- Writing reports takes less time.
- Reports are easier to maintain in the future.

Providing set rules for what makes a good semantic model is difficult because all data is different, and the usage of that data varies.

Generally, a smaller semantic model is better because it performs faster and will be simpler to use. However, defining what a smaller semantic model entails is equally as problematic because it's a heuristic and subjective concept.

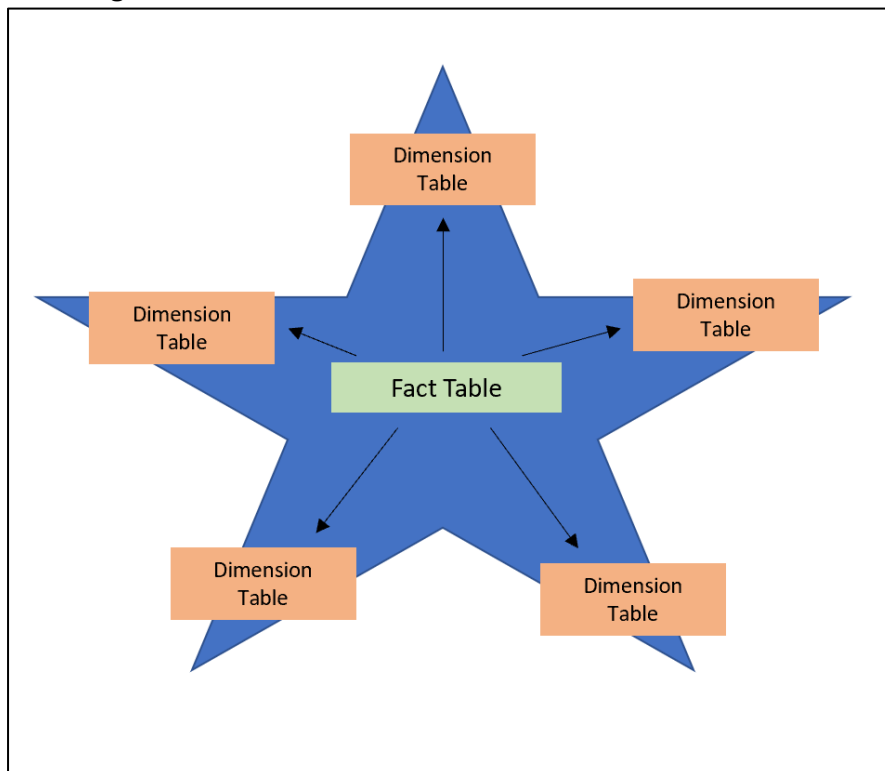
- Typically, a smaller semantic model is composed of fewer tables and fewer columns in each table that the user can see.
- If you import all necessary tables from a sales database, but the total table count is 30 tables, the user will not find that intuitive.
- Collapsing those tables into five tables make the semantic model more intuitive to the user, whereas if the user opens a table and finds 100 columns, they might find it overwhelming.
- Removing unneeded columns to provide a more manageable number increases the likelihood that the user reads all column names.
- To summarize, you should aim for simplicity when designing your semantic models.
- The following image is an example semantic model.
 - The boxes contain tables of data, where each line item within the box is a column. The lines that connect the boxes represent relationships between the tables.
 - These relationships can be complex, even in such a simplistic model.
 - The semantic model can become easily disorganized, and the total table count in the model can gradually increase.
 - Keeping your semantic model simple, comprehensive, and accurate requires constant effort.



- Relationships are defined between tables through primary and foreign keys. Primary keys are column(s) that identify each unique, non-null data row.
 - Each row is assigned a unique value, which can be referred to by this simple value: the primary key.
 - This process becomes important when you are referencing rows in a different table, which is what foreign keys do.
 - Relationships between tables are formed when you have primary and foreign keys in common between different tables.
- Power BI allows relationships to be built from tables with different data sources, a powerful function that enables you to pull one table from Microsoft Excel and another from a relational database.
- You would then create the relationship between those two tables and treat them as a unified semantic model.

Star schemas

You can design a star schema to simplify your data. It's not the only way to simplify your data, but it is a popular method; therefore, every Power BI data analyst should understand it. In a star schema, each table within your semantic model is defined as a dimension or a fact table, as shown in the following visual.



Fact tables contain observational or event data values: sales orders, product counts, prices, transactional dates and times, and quantities.

- Fact tables can contain several repeated values. For example, one product can appear multiple times in multiple rows, for different customers on different dates. These values can be aggregated to create visuals.
- For instance, a visual of the total sales orders is an aggregation of all sales orders in the fact table. With fact tables, it is common to see columns that are filled with numbers and dates.

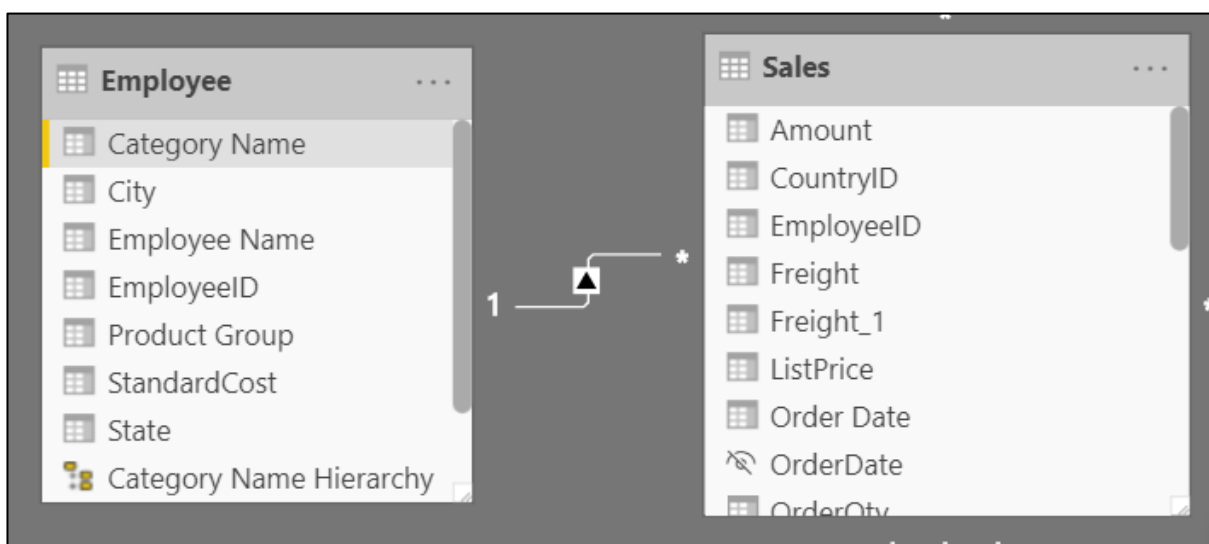
- The numbers can be units of measurement, such as sale amount, or they can be keys, such as a customer ID. The dates represent time that is being recorded, like order date or shipped date.

Dimension tables contain the details about the data in fact tables: products, locations, employees, and order types.

- These tables are connected to the fact table through key columns. Dimension tables are used to filter and group the data in fact tables.
- The fact tables, on the other hand, contain the measurable data, such as sales and revenue, and each row represents a unique combination of values from the dimension tables.
- For the total sales orders visual, you could group the data so that you see total sales orders by product, in which product is data in the dimension table.

Fact tables are much larger than dimension tables because numerous events occur in fact tables, such as individual sales. Dimension tables are typically smaller because you are limited to the number of items that you can filter and group on. For instance, a year contains only so many months, and the United States are composed of only a certain number of states.

- Considering this information about fact tables and dimension tables, you might wonder how you can build this visual in Power BI.
- The pertinent data resides in two tables, Employee and Sales, as shown in the following semantic model. Because the Sales table contains the sales order values, which can be aggregated, it is considered a fact table.
- The Employee table contains the specific employee name, which filters the sales orders, so it would be a dimension table. The common column between the two tables, which is the primary key in the Employee table, is **EmployeeID**, so you can establish a relationship between the two tables based on this column.



Star schemas and the underlying semantic model are the foundation of organized reports; the more time you spend creating these connections and design, the easier it will be to create and maintain reports.

Work with Tables

When users see fewer tables, they will enjoy using your semantic model considerably more. For example, suppose you've imported dozens of tables from many data sources and now the visual appears disorderly. In this case, you need to ensure that, before you begin working on building reports, your semantic model and table structure are simplified.

A simple table structure will:

- Be simple to navigate because of column and table properties that are specific and user-friendly.
- Have merged or appended tables to simplify the tables within your data structure.
- Have good-quality relationships between tables that make sense.

Create a Date Table

- During report creation in Power BI, a common business requirement is to make calculations based on date and time.
- Organizations want to know how their business is doing over months, quarters, fiscal years, and so on.
- For this reason, it is crucial that these time-oriented values are formatted correctly.
- Power BI autodetects for date columns and tables; however, situations can occur where you will need to take extra steps to get the dates in the format that your organization requires.

Create a common date table

Ways that you can build a common date table are:

- Source data
- DAX
- Power Query

Source data

Occasionally, source databases and data warehouses already have their own date tables. If the administrator who designed the database did a thorough job, these tables can be used to perform the following tasks:

- Identify company holidays
- Separate calendar and fiscal year
- Identify weekends versus weekdays

Source data tables are mature and ready for immediate use.

If you have a table as such, bring it into your semantic model and don't use any other methods that are outlined in this section.

We recommend that you use a source date table because it is likely shared with other tools that you might be using in addition to Power BI.

If you do not have a source data table, you can use other ways to build a common date table.

DAX

You can use the Data Analysis Expression (DAX) functions `CALENDARAUTO()` or `CALENDAR()` to build your common date table. The `CALENDAR()` function returns a contiguous range of dates based on a start and end date that are entered as arguments in the function.

Alternatively, the `CALENDARAUTO()` function returns a contiguous, complete range of dates that are automatically determined from your semantic model.

The starting date is chosen as the earliest date that exists in your semantic model, and the ending date is the latest date that exists in your semantic model plus data that has been populated to the fiscal month that you can choose to include as an argument in the `CALENDARAUTO()` function.

For the purposes of this example, the `CALENDAR()` function is used because you only want to see the data from May 31, 2011 (the first day that Sales began its tracking of this data) and forward for the next 10 years.

In Power BI Desktop, select **New Table**, and then enter in the following DAX formula:

DAXCopy

```
1 Dates = CALENDAR(DATE(2011, 5, 31), DATE(2022, 12, 31))
```

Dates = CALENDAR(DATE(2011, 5, 31), DATE(2022, 12, 31))

Now, you have a column of dates that you can use. However, this column is slightly sparse. You also want to see columns for just the year, the month number, the week of the year, and the day of the week. You can accomplish this task by selecting **New Column** on the ribbon and entering the following DAX equation, which will retrieve the year from your Date table.

✕ ✓		1 Year = YEAR(Dates[Date])	
Date		Year	
Tuesday, May 31, 2011		2011	
Wednesday, June 1, 2011		2011	
Thursday, June 2, 2011		2011	
Friday, June 3, 2011		2011	
Saturday, June 4, 2011		2011	

```
Year = YEAR(Dates[Date])
```

You can perform the same process to retrieve the month number, week number, and day of the week:

```
MonthNum = MONTH(Dates[Date])
```

```
WeekNum = WEEKNUM(Dates[Date])
```

```
DayoftheWeek = FORMAT(Dates[Date], "DDDD")
```

When you have finished, your table will contain the columns that are shown in the following figure. You have now created a common date table by using DAX. This process only adds your new table

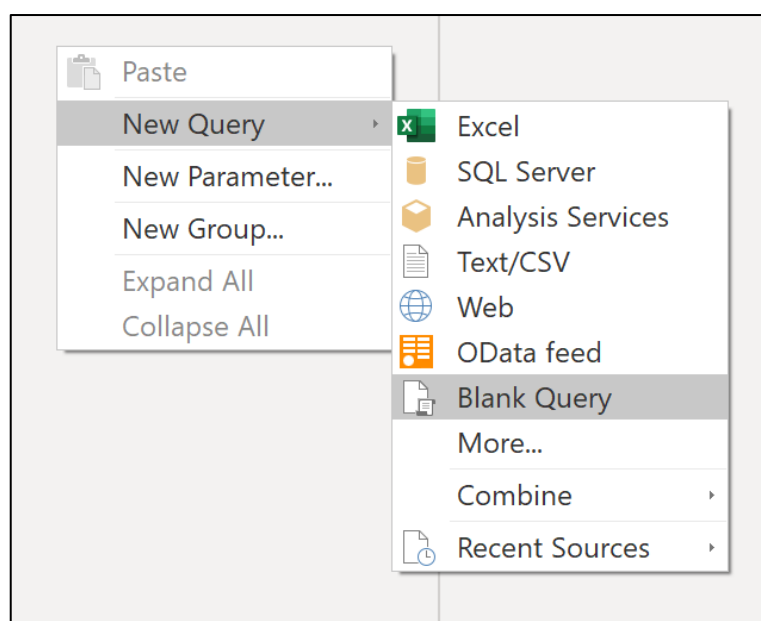
Date	Year	MonthNum	WeekNum	DayoftheWeek
Tuesday, May 31, 2011	2011	5	23	Tuesday
Wednesday, June 1, 2011	2011	6	23	Sunday
Thursday, June 2, 2011	2011	6	23	Monday
Friday, June 3, 2011	2011	6	23	Tuesday

to the semantic model; you will still need to establish relationships between your date table and the Sales and Order tables, and then mark your table as the official date table of your semantic model. However, before you complete those tasks, make sure that you consider another way of building a common date table: by using Power Query.

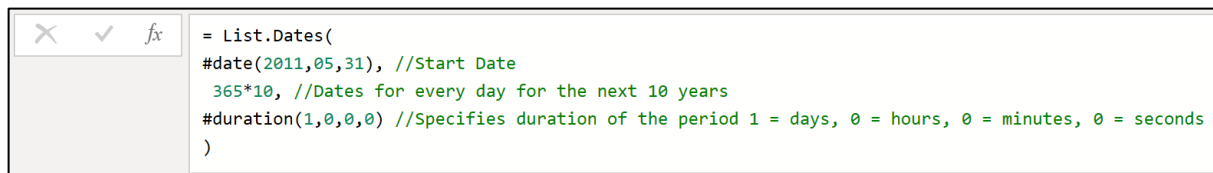
Power Query

You can use M-language, the development language that is used to build queries in Power Query, to define a common date table.

Select **Transform Data** in Power BI Desktop, which will direct you to Power Query. In the blank space of the left **Queries** pane, right-click to open the following drop-down menu, where you will select **New Query > Blank Query**.



In the resulting **New Query** view, enter the following M-formula to build a calendar table:
= List.Dates(#date(2011,05,31), 365*10, #duration(1,0,0,0))



```
= List.Dates(  
    #date(2011,05,31), //Start Date  
    365*10, //Dates for every day for the next 10 years  
    #duration(1,0,0,0) //Specifies duration of the period 1 = days, 0 = hours, 0 = minutes, 0 = seconds  
)
```

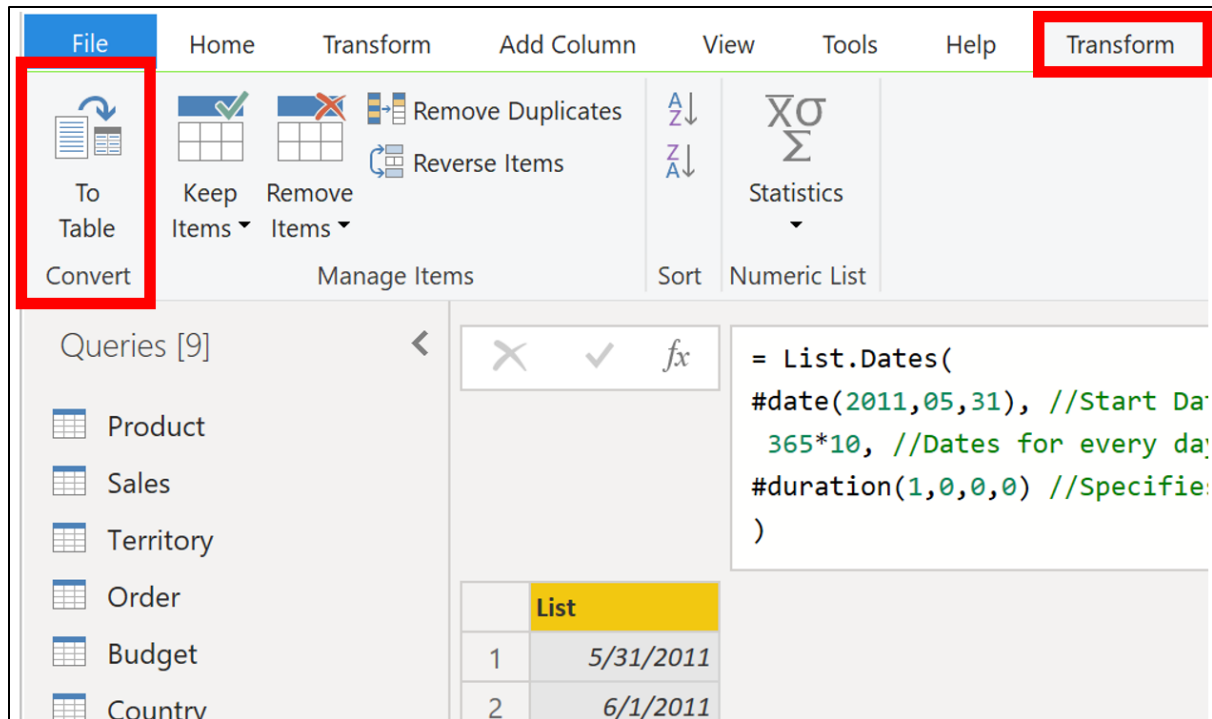
For your sales data, you want the start date to reflect the earliest date that you have in your data: May 31, 2011. Additionally, you want to see dates for the next 10 years, including dates in the future.

This approach ensures that, as new sales data flows in, you won't have to re-create this table. You can also change duration. In this case, you want a data point for every day, but you can also increment by hours, minutes, and seconds. The following figure shows the result.

	List
1	5/31/2011
2	6/1/2011
3	6/2/2011
4	6/3/2011
5	6/4/2011
6	6/5/2011
7	6/6/2011
8	6/7/2011
9	6/8/2011

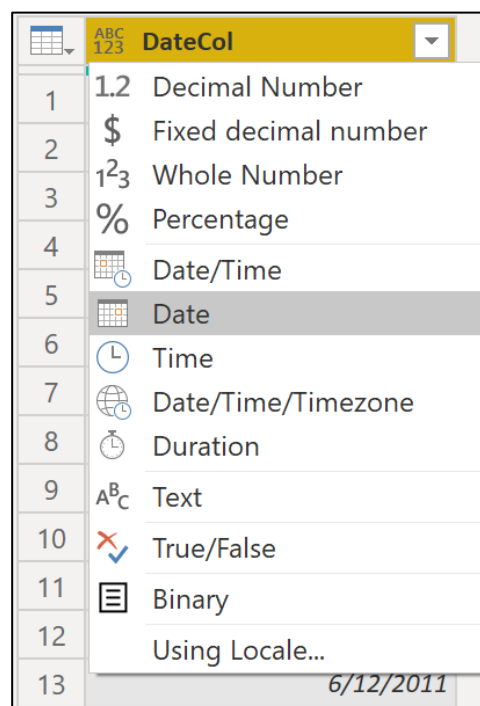
After you have realized success in the process, you notice that you have a list of dates instead of a table of dates.

To correct this error, go to the **Transform** tab on the ribbon and select **Convert > To Table**. As the name suggests, this feature will convert your list into a table. You can also rename the column to **DateCol**.

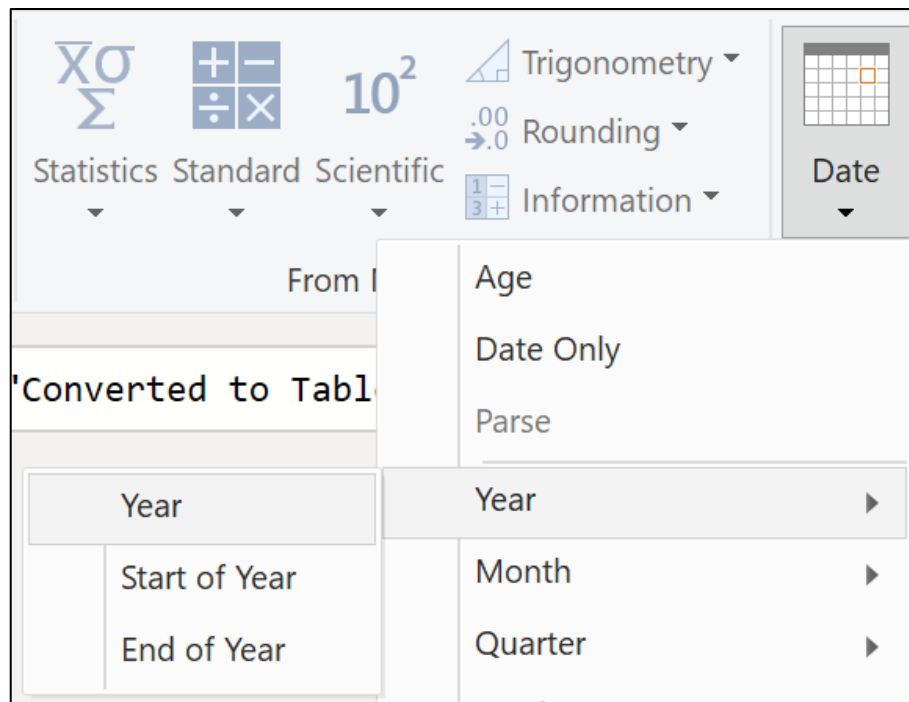


Next, you want to add columns to your new table to see dates in terms of year, month, week, and day so that you can build a hierarchy in your visual.

Your first task is to change the column type by selecting the icon next to the name of the column and, in the resulting drop-down menu, selecting the **Date** type.



After you have finished selecting the **Date** type, you can add columns for year, months, weeks, and days. Go to **Add Column**, select the drop-down menu under **Date**, and then select **Year**, as shown in the following figure.



Notice that Power BI has added a column of all years that are pulled from **DateCol**.

DateCol	1 ² ₃ Year
5/31/2011	2011
6/1/2011	2011
6/2/2011	2011

Complete the same process for months, weeks, and days. After you have finished this process, the table will contain the columns that are shown in the following figure.

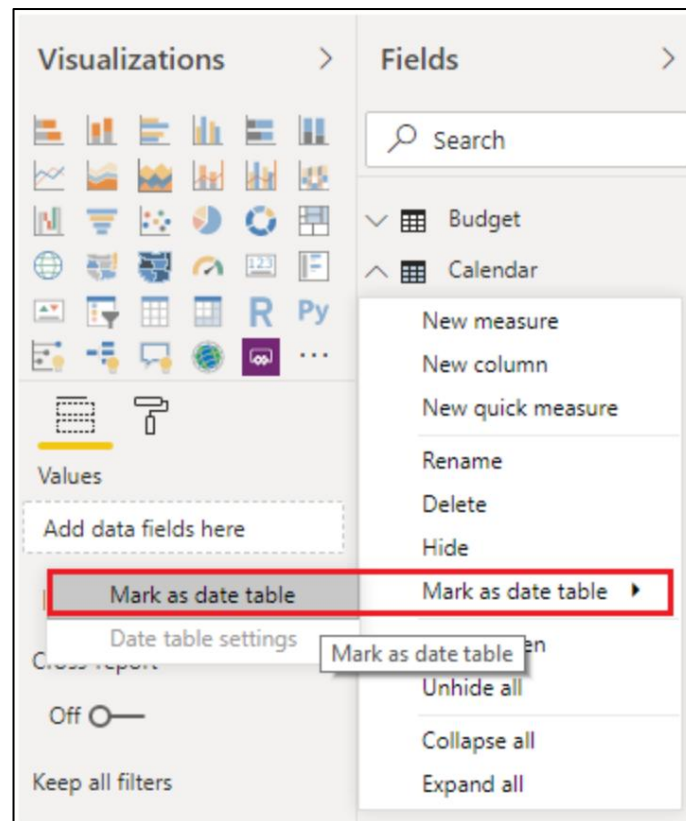
	DateCol	1 ² ₃ Year	1 ² ₃ Month	1 ² ₃ Week of Year	A ^B _C Day Name
1	5/31/2011	2011	5	23	Tuesday
2	6/1/2011	2011	6	23	Wednesday
3	6/2/2011	2011	6	23	Thursday

You have now successfully used Power Query to build a common date table.

The previous steps show how to get the table into the semantic model. Now, you need to mark your table as the official date table so that Power BI can recognize it for all future values and ensure that formatting is correct.

Mark as the official date table

Your first task in marking your table as the official date table is to find the new table on the **Fields** pane. Right-click the name of the table and then select **Mark as date table**, as shown in the following figure.



By marking your table as a date table, Power BI performs validations to ensure that the data contains zero null values, is unique, and contains continuous date values over a period.

You can also choose specific columns in your table to mark as the date, which can be useful when you have many columns within your table. Right-click the table, select **Mark as date table**, and then select **Date table settings**.

Mark as date table

Select a column to be used for the date. The column must be of the data type 'date' and must contain only unique values. [Learn more](#)

Date column

Date

Validated successfully

i When you mark this as a date table, the built-in date tables that were associated with this table are removed. Visuals or DAX expressions referring to them may break. [Learn how to fix visuals and DAX expressions](#)

OK Cancel

The following window will appear, where you can choose which column should be marked as **Date**.

Selecting **Mark as date table** will remove autogenerated hierarchies from the **Date** field in the table that you marked as a date table. For other date fields, the auto hierarchy will still be present until you establish a relationship between that field and the date table or until you turn off the **Auto Date/Time** feature.

You can manually add a hierarchy to your common date table by right-clicking the year, month, week, or day columns in the **Fields** pane and then selecting **New hierarchy**. This process is further discussed later in this module.

Build your visual

To build your visual between the Sales and Orders tables, you will need to establish a relationship between this new common date table and the Sales and Orders tables. As a result, you will be able to build visuals by using the new date table.

To complete this task, go to **Model** tab > **Manage Relationships**, where you can create relationships between the common date table and the Orders and Sales tables by using the **OrderDate** column. The following screenshot shows an example of one such relationship.

Create relationship

Select tables and columns that are related.

Sales

SalesOrderID	OrderDate	Sort_of_Sales	Freight	Freight_1	ProductID	OrderQty	SpecialOffer
52242	Sunday, July 7, 2013	Internet	0.1248	0.1248	870	1	
52592	Sunday, July 14, 2013	Internet	0.1248	0.1248	870	1	
52694	Tuesday, July 16, 2013	Internet	0.1248	0.1248	870	1	

DatesTable

DateCol	Year	Month	Week of Year	Day Name
Tuesday, May 31, 2011	2011	5	23	Tuesday
Tuesday, June 7, 2011	2011	6	24	Tuesday
Tuesday, June 14, 2011	2011	6	25	Tuesday

Cardinality: Many to one (*:1)

Cross filter direction: Both

☒ Make this relationship active

☐ Assume referential integrity

☐ Apply security filter in both directions

OK Cancel

After you have built the relationships, you can build your **Total Sales and Order Quantity by Time** visual with your common date table that you developed by using the DAX or Power Query method.

To determine the total sales, you need to add all sales because the **Amount** column in the Sales table only looks at the revenue for each sale, not the total sales revenue. You can complete this

task by using the following measure calculation, which will be explained in later discussions. The calculation that you will use when building this measure is as follows:

```
#Total Sales = SUM(Sales[Amount])
```

After you have finished, you can create a table by returning to the **Visualizations** tab and selecting the **Table** visual. You want to see the total orders and sales by year and month, so you only want to include the Year and Month columns from your date table, the **OrderQty** column, and the **#TotalSales** measure. When you learn about hierarchies, you can also build a hierarchy that will allow you drill down from years to months. For this example, you can view them side-by-side. You have now successfully created a visual with a common date table.

Year	Month	OrderQty	# TotalSales	
2011	5	825	853,422	
2011	6	141	460,085	
2011	7	2209	3,130,880	
2011	8	2904	3,917,345	
2011	9	157	503,668	
2011	10	5382	7,426,033	
2011	11	230	740,105	
2011	12	1040	1,815,966	
2012	1	3967	6,319,337	
2012	2	1442	2,106,429	
2012	3	3184	4,575,015	
Total		274914	170,964,700	

Work with Dimensions

When building a star schema, you will have dimension and fact tables. Fact tables contain information about events such as sales orders, shipping dates, resellers, and suppliers.

Dimension tables store details about business entities, such as products or time, and are connected back to fact tables through a relationship.

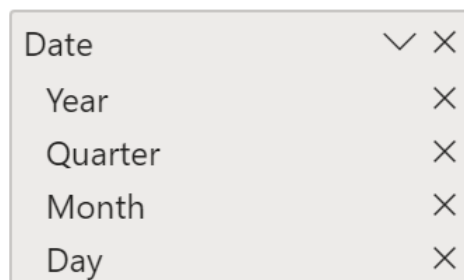
You can use hierarchies as one source to help you find detail in dimension tables. These hierarchies form through natural segments in your data.

For instance, you can have a hierarchy of dates in which your dates can be segmented into years, months, weeks, and days.

Hierarchies are useful because they allow you to drill down into the specifics of your data instead of only seeing the data at a high level.

Hierarchies

When you are building visuals, Power BI automatically enters values of the date type as a hierarchy (if the table has not been marked as a date table).



In the preceding **Date** column, the date is shown in increasingly finer detail through year, quarters, months, and days. You can also manually create hierarchies.

Parent-child hierarchy

In the following example, you have an Employee table within the database that tells you important information about the employees, their managers, and their IDs. When looking at this table, you notice that **Roy F** has been repeated multiple times in the **Manager** column. As the image shows, multiple employees can have the same manager, which indicates a hierarchy between managers and employees.

	123 Employee ID	A ^B C Employee	123 Manager ID	A ^B C Manager
1	1010	Roy F	null	
2	1011	Pam H	1010	Roy F
3	1012	Guy L	1010	Roy F
4	1013	Roger M	1011	Pam H
5	1014	Kaylie S	1011	Pam H
6	1015	Mike O	1012	Guy L
7	1016	Rudy Q	1012	Guy L

The **Manager** column determines the hierarchy and is therefore the parent, while the "children" are the employees. For this example, you want to be able to see all levels of this hierarchy. Power BI does not default to showing you all levels of the hierarchy, so it is your responsibility to ensure that you see all levels of this hierarchy or "flatten" it so that you can see more data granularity.

Flatten parent-child hierarchy

The process of viewing multiple child levels based on a top-level parent is known as *flattening the hierarchy*. In this process, you are creating multiple columns in a table to show the hierarchical path of the parent to the child in the same record. You will use `PATH()`, a simple DAX function that returns a text version of the managerial path for each employee, and `PATHITEM()` to separate this path into each level of managerial hierarchy.

While on the table, go to the **Modeling** tab and select **New Column**. In the resulting formula bar, enter the following function, which creates the text path between the employee and manager. This action creates a calculated column in DAX.

```
1 Path = PATH(Employee[Employee ID], Employee[Manager ID])
```

PATH(**ID_ColumnName**, Parent_ColumnName)
Returns a string which contains a delimited list of IDs, starting with the top/root of a hierarchy and ending with the specified ID.

Path = PATH(Employee[Employee ID], Employee[Manager ID])

The completed path between the employee and the manager appears in the new column, as shown in the following screenshot.

Employee ID	Manager ID	Employee	Manager	Path
1010		Roy F		1010
1011	1010	Pam H	Roy F	1010 1011
1012	1010	Guy L	Roy F	1010 1012
1013	1011	Roger M	Pam H	1010 1011 1013
1014	1011	Kaylie S	Pam H	1010 1011 1014
1015	1012	Mike O	Guy L	1010 1012 1015
1016	1012	Rudy Q	Guy L	1010 1012 1016

If you look at Roger M, the path of IDs is **1010 | 1011 | 1013**, which means that one level above Roger M (ID 1013) is his manager, Pam H (ID 1011), and one level above Pam H is her manager Roy F (ID 1010).

In this row, Roger M is on the bottom of the hierarchy, at the child level, and Roy F is at the top of the hierarchy and is at the parent level. This path is created for every employee.

To flatten the hierarchy, you can separate each level by using the PATHITEM function. To view all three levels of the hierarchy separately, you can create four columns in the same way that you did previously, by entering the following equations.

You will use the PATHITEM function to retrieve the value that resides in the corresponding level of your hierarchy.

- Level 1 = PATHITEM(Employee[Path],1)
- Level 2 = PATHITEM(Employee[Path],2)
- Level 3 = PATHITEM(Employee[Path],3)

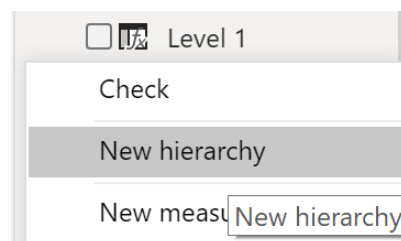
```
1 Level 1 = PATHITEM(Employee[Path], 1)
```

PATHITEM(Path, Position, [Type])
Returns the nth item in the delimited list produced by the Path function.

After you have finished, notice that you now have each level of hierarchy within your table. Roy F is at the top of the hierarchy and, as you go through **Levels 2-3**, notice that the managers and employees map with each other.

Employee ID	Name	Manager	Manager ID	Path	Level 1	Level 2	Level 3	Level 4
1000	Quincy Howard			1000	1000			
1001	Mallory Yang	Quincy Howard	1000	1000 1001	1000	1001		
1002	Donovan Maynard	Quincy Howard	1000	1000 1002	1000	1002		
1003	Giselle McClain	Mallory Yang	1001	1000 1001 1003	1000	1001	1003	
1004	Melvin Marsh	Mallory Yang	1001	1000 1001 1004	1000	1001	1004	
1005	Ria Snow	Giselle McClain	1003	1000 1001 1003 1005	1000	1001	1003	1005
1006	Callie Savage	Giselle McClain	1003	1000 1001 1003 1006	1000	1001	1003	1006

Now, you can create a hierarchy on the **Fields** pane, as you did previously. Right-click **Level 1**, because this is the first hierarchy level, and then select **New Hierarchy**. Then, drag and drop **Level 2** and **Level 3** into this hierarchy.



You have now successfully flattened a hierarchy so that you can view individual levels. Previously, you've considered dimensions that have only one relationship with a fact table. However, situations do occur where your dimension table will have multiple relationships with a fact table.



Role-playing dimensions

Role-playing dimensions have multiple valid relationships with fact tables, meaning that the same dimension can be used to filter multiple columns or tables of data. As a result, you can filter data differently depending on what information you need to retrieve.

The preceding visual shows the Calendar, Sales, and Order tables. Calendar is the dimension table, while Sales and Order are fact tables. The dimension table has two relationships: one with Sales and one with Order. This example is of a role-playing dimension because the Calendar table can be used to group data in both Sales and Order. If you wanted to build a visual in which the Calendar table references the Order and the Sales tables, the Calendar table would act as a role-playing dimension.

Define Data Granularity

Data granularity is the detail that is represented within your data, meaning that the more granularity your data has, the greater the level of detail within your data.

Data granularity is an important topic for all data analysts, regardless of the Power BI tools that you're using. Defining the correct data granularity can have a big impact on the performance and usability of your Power BI reports and visuals.

Data granularity defined

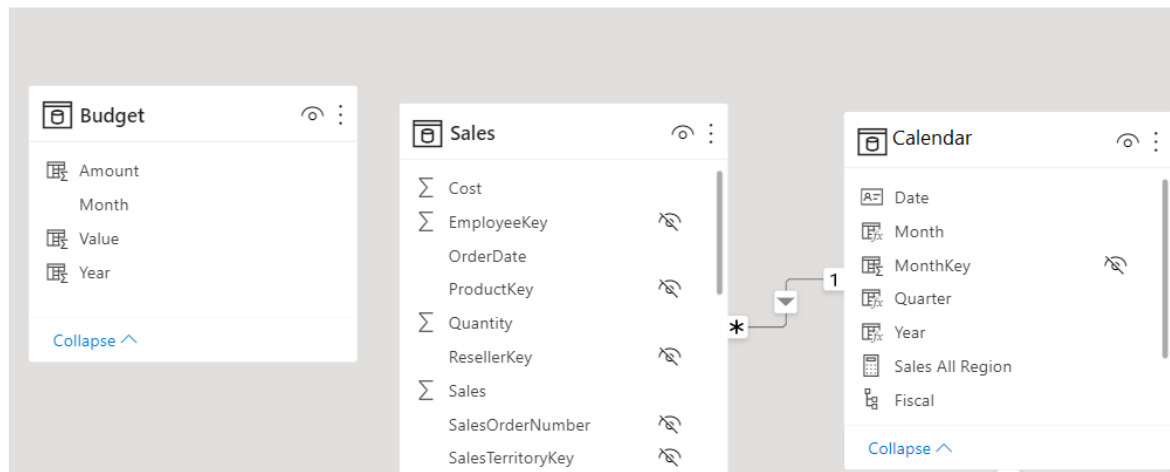
- Consider a scenario where your company manages 1,000 refrigerated semi-trucks. Every few minutes, each truck uses a Microsoft Azure IoT application to record its current temperature. This temperature is important to your organization because, if the refrigeration were to malfunction, it could spoil the entire load, costing thousands of dollars. With so many trucks and so many sensors, extensive data is generated every day. Your report users don't want to sift through numerous records to find the ones that they are particularly interested in.
- How can you change the granularity of the data to make the semantic model more usable?
 - In this scenario, you might want to import the data by using a daily average for each truck. That approach would reduce the records in the database to one record for each truck for each day. If you decide that the approach was acceptable enough for tracking costs and errors, then you could use that data granularity. Alternatively, you could select the last recorded temperature, or you could only import records that are above or below a normal range of temperatures. Any of these methods will reduce the total records that you import, while still bringing in data that is comprehensive and valuable.
- For different scenarios, you could settle on data granularity that is defined weekly, monthly, or quarterly. Generally, the fewer the records that you are working with, the faster your reports and visuals will function. This approach translates to a faster refresh rate for the entire semantic model, which might mean that you can refresh more frequently.
- However, that approach has a drawback. If your users want to drill into every single transaction, summarizing the granularity will prevent them from doing that, which can have a negative impact on the user experience. It is important to negotiate the level of data granularity with report users so they understand the implications of these choices.

Change data granularity to build a relationship between two tables

Data granularity can also have an impact when you're building relationships between tables in Power BI.

For example, consider that you're building reports for the Sales team at Tailwind Traders. You've been asked to build a matrix of total sales and budget over time by using the Calendar, Sales, and Budget tables. You notice that the lowest level of time-based detail that the Sales table goes into is by day, for instance 5/1/2020, 6/7/2020, and 6/18/2020. The Budget table only goes to the monthly level, for instance, the budget data is 5/2020 and 6/2020. These tables have different granularities that need to be reconciled before you can build a relationship between tables.

The following figure shows your current semantic model.



As shown in the preceding figure, a relationship between Budget and Calendar is missing.

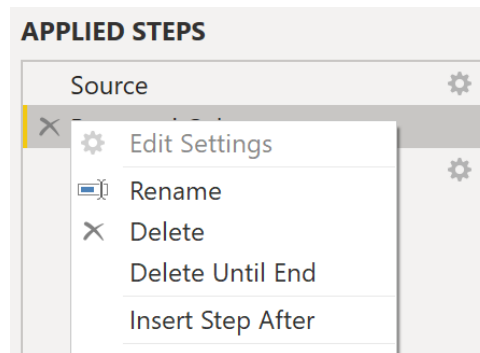
Therefore, you need to create this relationship before you can build your visual. Notice that if you transform the **Year** and **Month** columns in the Calendar table into a new column, and do the same transformation in the Budget table, you can match the format of the **Date** column in the Calendar table.

Then, you can establish a relationship between the two columns. To complete this task, you'll concatenate the **Year** and **Month** columns and then change the format.

Year	Month	Date
2013	1	01/01/2011
2013	2	01/02/2011
2013	3	01/03/2011
2013	4	01/04/2011
2013	5	01/05/2011
2013	6	01/06/2011
2013	7	01/07/2011
2013	8	01/07/2011

Budget
Calendar

Select **Transform Data** on the ribbon. On **Applied Steps**, on the right pane, right-click the last step and then select **Insert Step After**.



Under **Add Column** on the Home ribbon, select **Custom Column**. Enter the following equation, which will concatenate the **Year** and **Month** columns, and then add a dash in between the column names.

```
Column = Table.AddColumn("#Renamed Columns", "Custom", each [Year] & "-" & [Month])
```

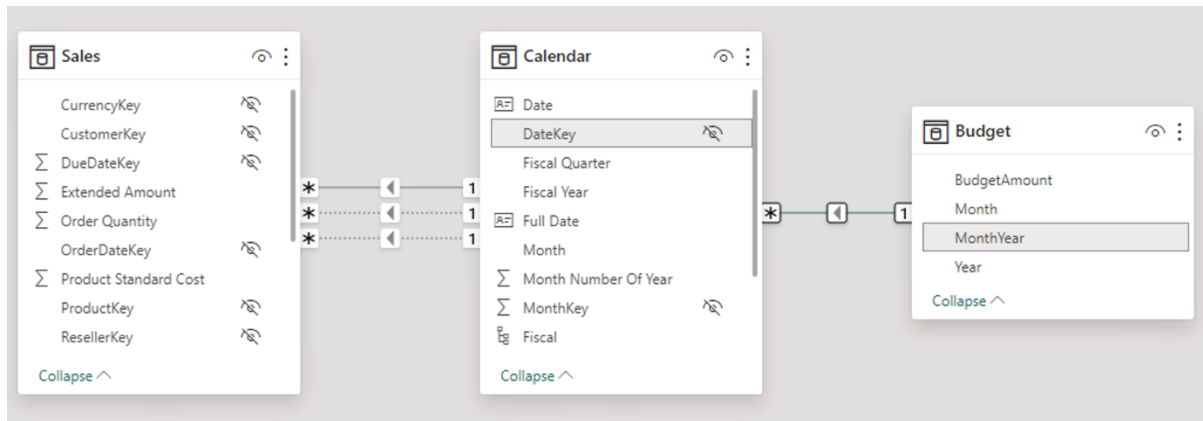
Change the data type to **Date** and then rename the column. Your Budget table should resemble the following figure.

Year	Month	BudgetAmount	MonthYear
2011	1	9493	2011-1
2011	2	2490	2011-2
2011	3	16585	2011-3
2011	4	5575	2011-4
2011	5	11611	2011-5
2011	6	14017	2011-6
2011	7	11919	2011-7
2011	8	3832	2011-8
2011	9	7094	2011-9
2011	10	6216	2011-10
2011	11	7769	2011-11
2011	12	12552	2011-12
2012	1	12781	2012-1
2012	2	7889	2012-2
2012	3	10329	2012-3
2012	4	5350	2012-4
2012	5	19216	2012-5
2012	6	5849	2012-6
2012	7	19269	2012-7
2012	8	3080	2012-8
2012	9	2381	2012-9

Now, you can create a relationship between the Budget and the Calendar tables.

Create a relationship between tables

Power BI automatically detects relationships, but you can also go to **Manage Relationships > New** and create the relationship on the **Date** column. The relationship should resemble the following figure.



By completing this task, you've ensured that the granularity is the same between your different tables. Now, you need to create DAX measures to calculate **Total Sales** and **BudgetAmount**.

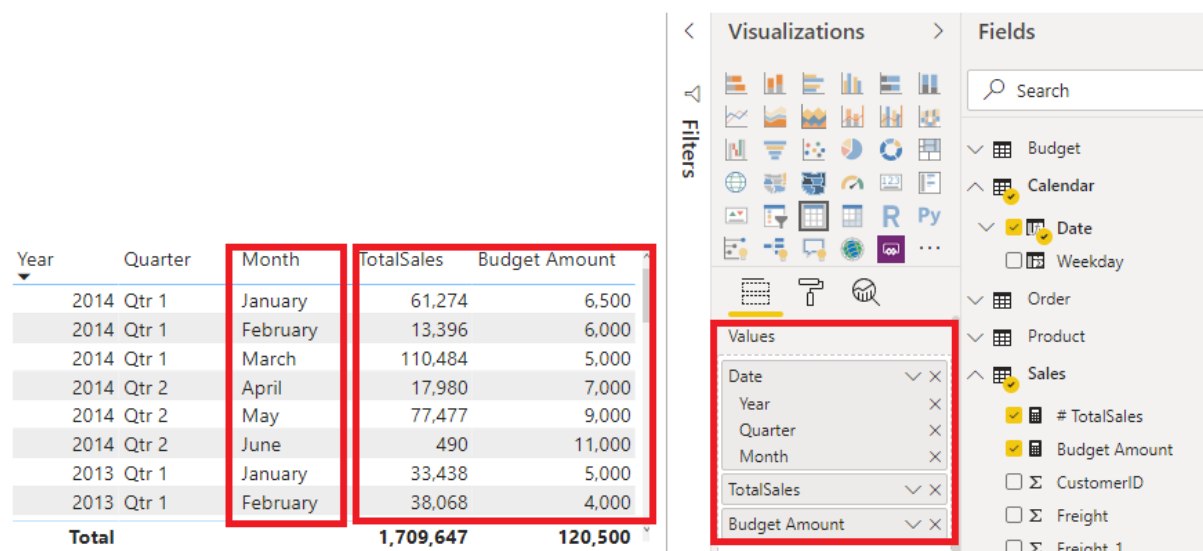
Go to the **Data** pane on Power BI Desktop, select **New Measure**, and then create two measures with the following equations:

TotalSales = SUM(Sales[Total Sales])

BudgetAmount = SUM (Budget[BudgetAmount])

Select the table visual on the **Visualization** pane, and then enter these measures and the **Date** into the **Values** field.

You've now accomplished the goal of building a matrix of the total sales and budgets over time.

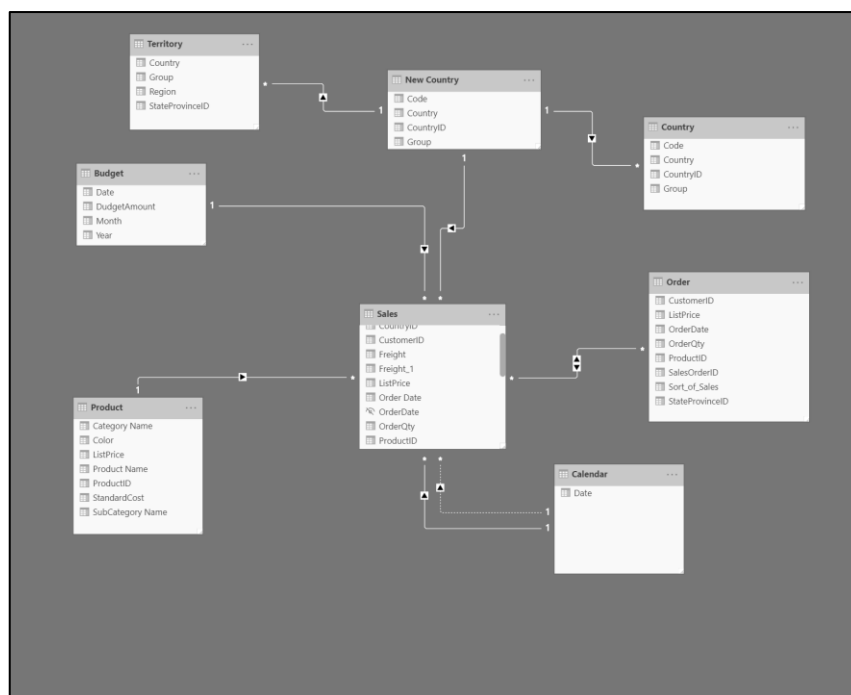


Work with Relationships and Cardinality

Unlike other database management systems, Power BI has the concept of *directionality* to a relationship. This directionality plays an important role in filtering data between multiple tables. When you load data, Power BI automatically looks for relationships that exist within the data by matching column names. You can also use **Manage Relationships** to edit these options manually.

For example, you've retrieved many tables from the Sales database, and the following image is an example of your semantic model. Power BI has autodetected several relationships, but you can't discern what they mean. You want to make sure that the relationships accurately reflect those that exist in your data.

Relationships

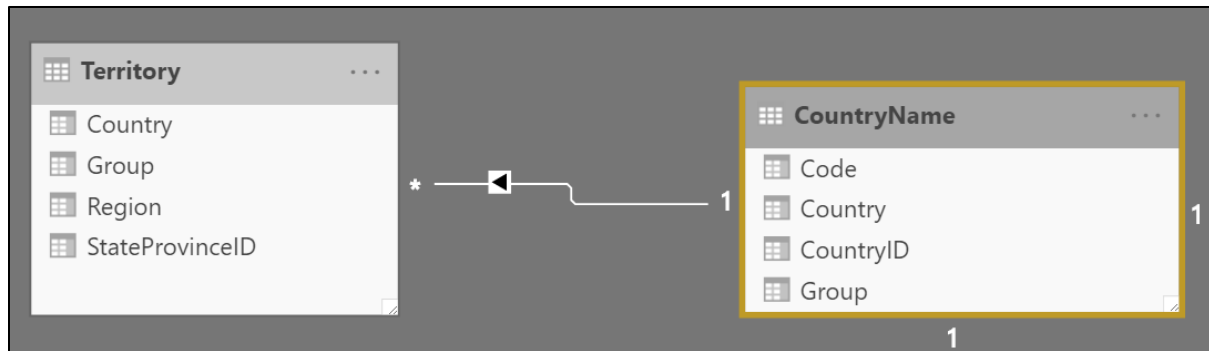


The following are different types of relationships that you'll find in Power BI.

Many-to-one (*:1) or one-to-many (1:*) relationship

- Describes a relationship in which you have many instances of a value in one column that are related to only one unique corresponding instance in another column.
- Describes the directionality between fact and dimension tables.
- Is the most common type of directionality and is the Power BI default when you are automatically creating relationships.

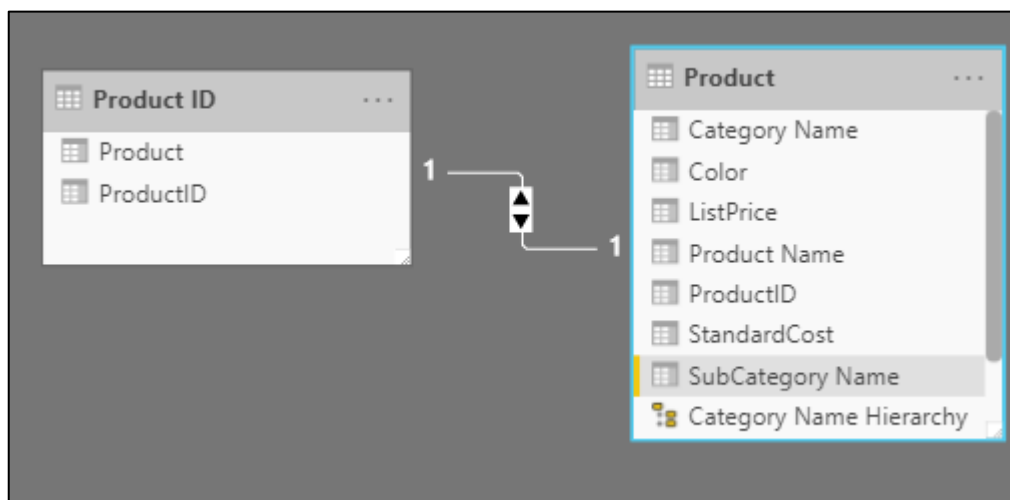
An example of a one-to-many relationship would be between the CountryName and Territory tables, where you can have many territories that are associated with one unique country/region.



One-to-one (1:1) relationship:

- Describes a relationship in which only one instance of a value is common between two tables.
- Requires unique values in both tables.
- Is not recommended because this relationship stores redundant information and suggests that the model is not designed correctly. It is better practice to combine the tables.

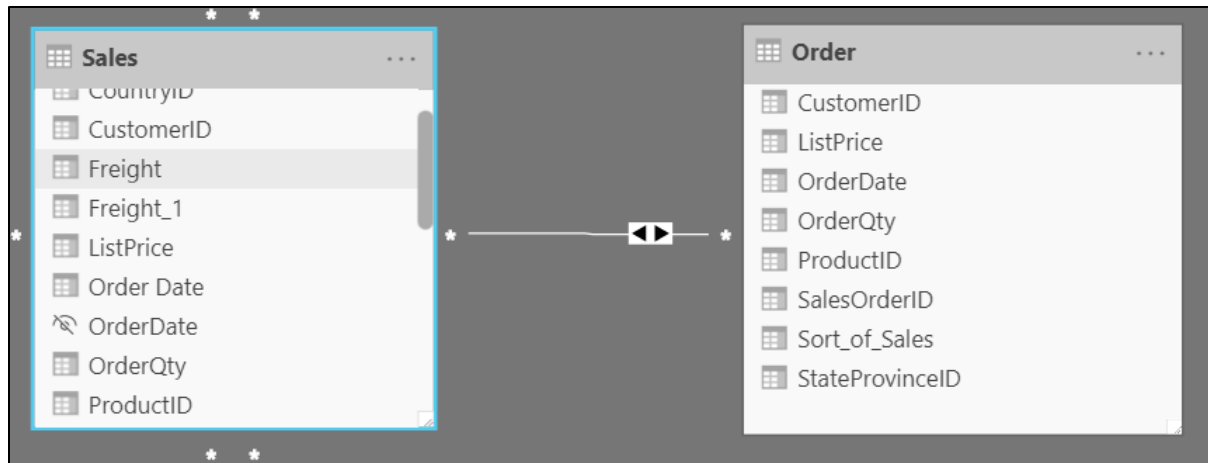
An example of a one-to-one relationship would be if you had products and product IDs in two different tables. Creating a one-to-one relationship is redundant and these two tables should be combined.



Many-to-many (.) relationship:

- Describes a relationship where many values are in common between two tables.
- Does not require unique values in either table in a relationship.
- Is not recommended; a lack of unique values introduces ambiguity and your users might not know which column of values is referring to what.

For instance, the following figure shows a many-to-many relationship between the Sales and Order tables on the **OrderDate** column because multiple sales can have multiple orders associated with them. Ambiguity is introduced because both tables can have the same order date.



Cross-filter direction

Data can be filtered on one or both sides of a relationship.

With a **single cross-filter direction**:

- Only one table in a relationship can be used to filter the data. For instance, Table 1 can be filtered by Table 2, but Table 2 cannot be filtered by Table 1.

Tip - Follow the direction of the arrow on the relationship between your tables to know which direction the filter will flow. You typically want these arrows to point to your fact table.

- For a one-to-many or many-to-one relationship, the cross-filter direction will be from the "one" side, meaning that the filtering will occur in the table that has many values.

With **both cross-filter directions** or **bi-directional cross-filtering**:

- One table in a relationship can be used to filter the other. For instance, a dimension table can be filtered through the fact table, and the fact tables can be filtered through the dimension table.
- You might have lower performance when using bi-directional cross-filtering with many-to-many relationships.

A word of caution regarding bi-directional cross-filtering:

You should not enable bi-directional cross-filtering relationships unless you fully understand the ramifications of doing so. Enabling it can lead to ambiguity, over-sampling, unexpected results, and potential performance degradation.

Cardinality and cross-filter direction

- For one-to-one relationships, the only option that is available is bi-directional cross-filtering.
- Data can be filtered on either side of this relationship and result in one distinct, unambiguous value.
- For instance, you can filter on one Product ID and be returned a single Product, and you can filter on a Product and be returned a single Product ID.
- For many-to-many relationships, you can choose to filter in a single direction or in both directions by using bi-directional cross-filtering.
 - The ambiguity that is associated with bi-directional cross-filtering is amplified in a many-to-many relationship because multiple paths will exist between different tables.

- If you create a measure, calculation, or filter, unintended consequences can occur where your data is being filtered and, depending on which relationship that the Power BI engine chooses when applying the filter, the final result might be different. This situation is also true for bi-directional relationships and why you should be cautious when using them.
- For this reason, many-to-many relationships and/or bi-directional relationships are complicated.
- Unless you are certain what your data looks like when aggregated, these types of open-ended relationships with multiple filtering directions can introduce multiple paths through the data.

Create many-to-many relationships

Consider the scenario where you are tasked with building a visual that examines budgets for customers and accounts. You can have multiple customers on the same account and multiple accounts with the same customer, so you know that you need to create a many-to-many relationship.

CustID	CustName
1022	Roy M
1023	Bob K
1024	Ellen L
1025	Mitch W
1026	Regan Q
1027	Lulu S
1028	Aliya R

CustomerTable

CustomerID	AccountID	AccountName
1022	12	BHP
1023	12	BHP
1024	13	RogerInc
1024	14	MyShip
1026	15	Holdings Unl.
1025	16	Key Biz Insiders
1028	17	Ty Inc
1022	17	Ty Inc

AccountTable

- To create this relationship, go to **Manage Relationships > New**.
- In the resulting window, create a relationship between the **Customer ID** column in CustomerTable and AccountTable.
- The relationship is set to many-to-many, and the filter type is in both directions.
- Immediately, you will be warned that you should only use this type of relationship if it is expected that neither column will have unique values because you might get unexpected values.
- Because you want to filter in both directions, choose **bi-directional cross-filtering**.
- Select **OK**. You have now successfully created a many-to-many relationship.

Resolve Modelling Challenges

Modeling data is about establishing and maintaining relationships so that you can effectively visualize the data in the form that your business requires. When you are creating these relationships, a common pitfall that you might encounter are circular relationships.

Create relationship

Select tables and columns that are related.

AccountTable

CustomerID	AccountID	AccountName
1022	12	BHP
1023	12	BHP
1024	13	RogerInc

CustomerTable

CustID	CustName
1022	Roy M
1023	Bob K
1024	Ellen L

Cardinality: Many to Many (*:*)

Cross filter direction: Both

☒ Make this relationship active

☐ Assume referential integrity

☐ Apply security filter in both directions

Warning: This relationship has cardinality Many-Many. This should only be used if it is expected that neither column (CustomerID and CustID) contains unique values, and that the significantly different behavior of Many-many relationships is understood. [Learn more](#)

OK Cancel

Relationship dependencies

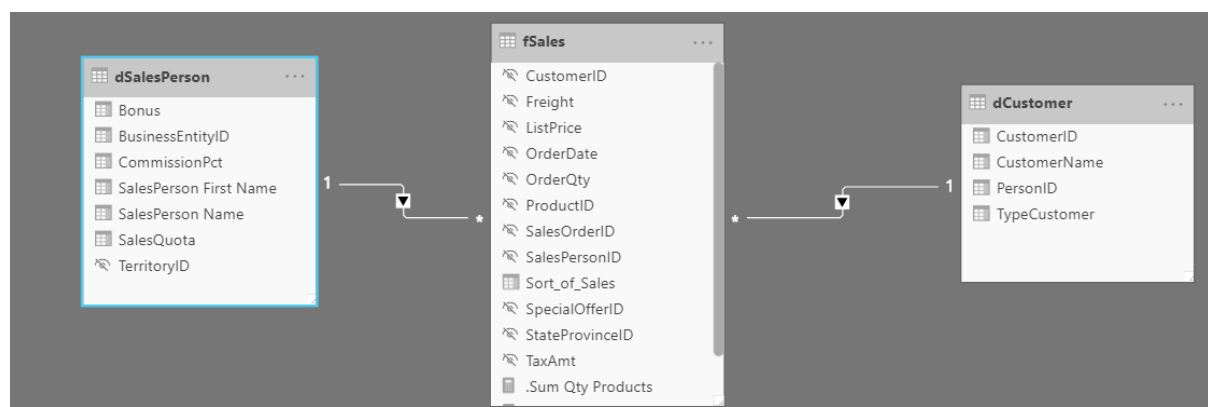
To understand circular relationships, you first need to understand dependencies.

For example, consider that you have the following calculated column **Total** in the Sales table.

$\text{Sales}[\text{'TotalCost'}] = \text{Sales}[\text{'Quantity'}] * \text{Sales}[\text{'Price'}]$

TotalCost depends on **Quantity** and **Price**, so if a change occurs in either quantity or price, a change will occur in **TotalCost** as well. This example outlines a dependency of a column on other columns, but you can also have dependencies between measures, tables, and relationships.

Consider the following relationships between **dSalesPerson**, **fSales**, and **dCustomer**. A change in **dCustomer** will result in a change in **fSales**, which results in changes in **dSalesPerson**. These types of dependencies can exist within relationships.



Write DAX Formulas for Power BI Desktop Models

Introduction

By using Data Analysis Expressions (DAX), you can add three types of calculations to your semantic model:

- Calculated tables
- Calculated columns
- Measures

DAX can also be used to define row-level security (RLS) rules, which are expressions that enforce filters over model tables. However, rules aren't considered to be model calculations so they're out of scope for this module.

Calculated tables

You can write a DAX formula to add a calculated table to your model.

The formula can duplicate or transform *existing model data*, or create a series of data, to produce a new table.

Calculated table data is always imported into your model, so it increases the model storage size and can prolong data refresh time.

A calculated table can't connect to external data; you need to use Power Query to accomplish that task.

Calculated tables can be useful in various scenarios:

- Date tables
- Role-playing dimensions
- What-if analysis

Date tables

Date tables are required to apply special time filters known as *time intelligence*.

DAX time intelligence functions only work correctly when a date table is set up.

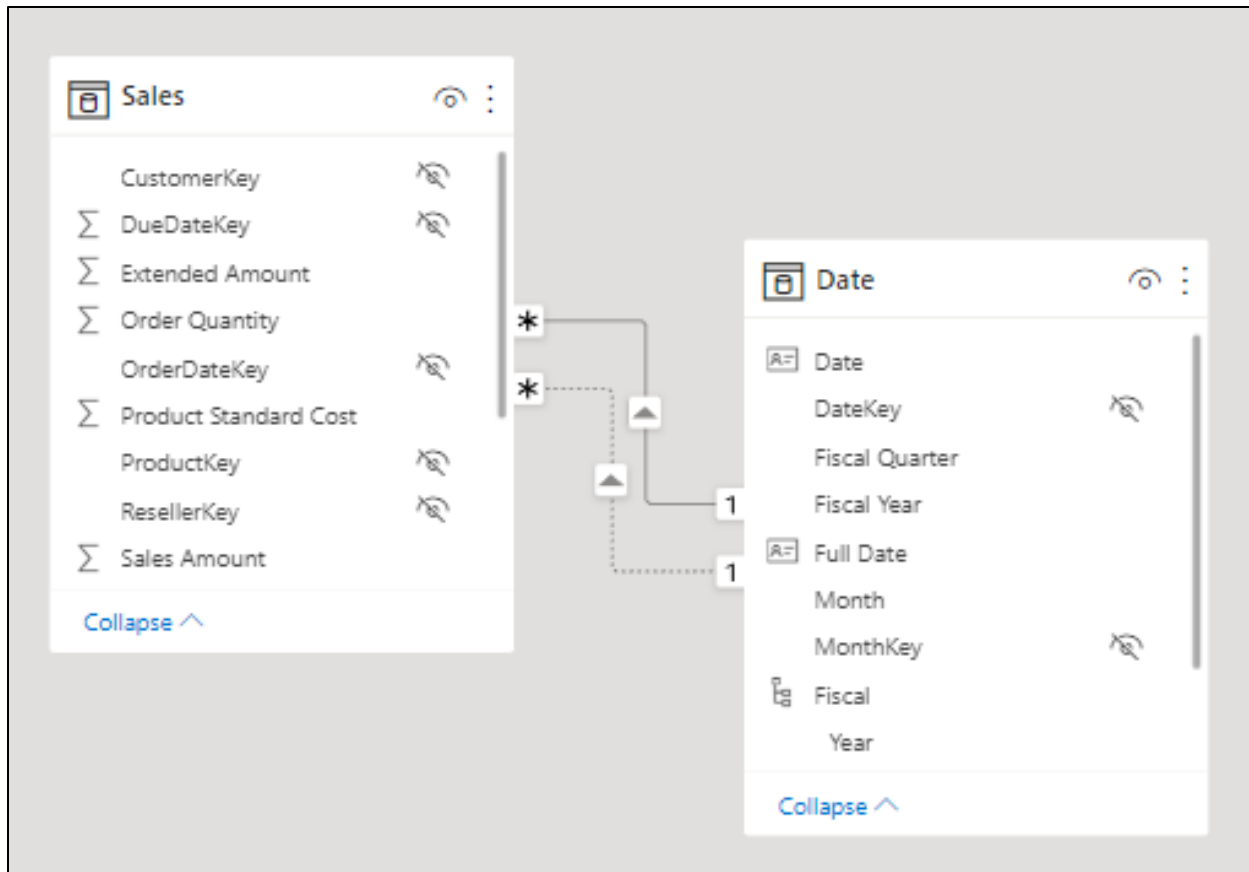
When your source data doesn't include a date table, you can create one as calculated tables by using the **CALENDAR** or **CALENDARAUTO** DAX functions.

Role-playing dimensions

When two model tables have multiple relationships, it could be because your model has a role-playing dimension.

For example, if you have a table named **Sales** that includes two date columns, **OrderDateKey** and **ShipDateKey**, both columns are related to the **Date** column in the **Date** table.

In this case, the **Date** table is described as a role-playing dimension because it could play the role of *order date* or *ship date*.



Microsoft Power BI models only allow one active relationship between tables, which in the model diagram is indicated as a solid line.

The active relationship is used by default to propagate filters, which in this case would be from the **Date** table to the **OrderDateKey** column in the **Sales** table.

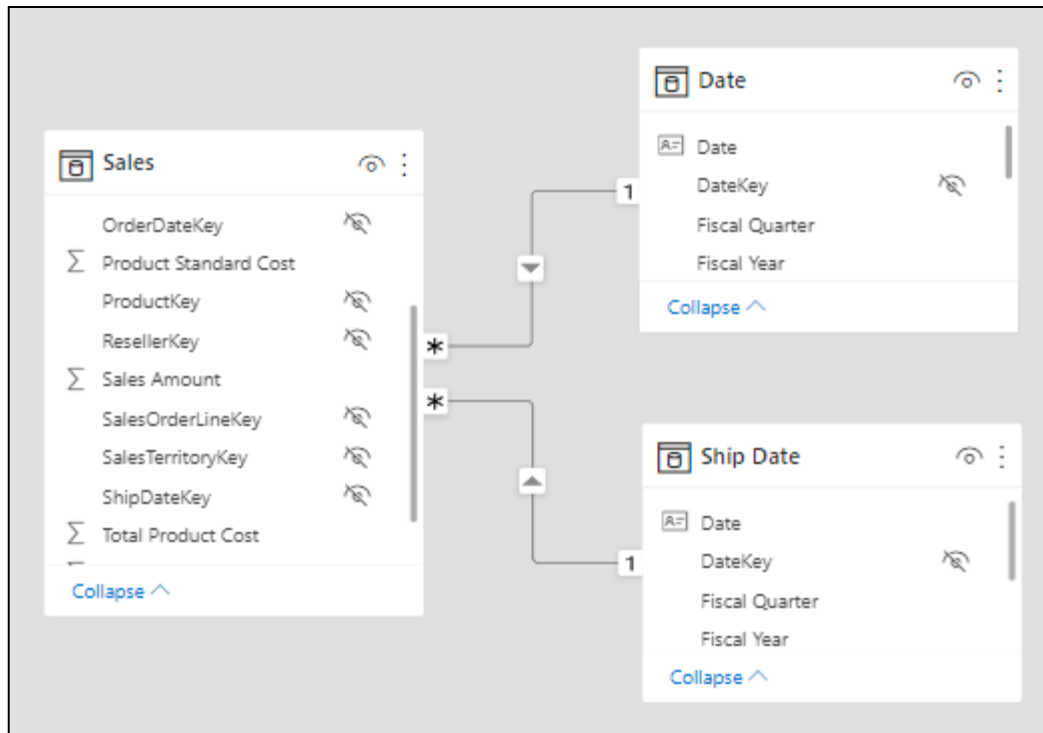
Any remaining relationships between the two tables are inactive.

In a model diagram, the relationships are represented as dashed lines.

Inactive relationships are only used when they're expressly requested in a calculated formula by using the USERELATIONSHIP DAX function.

Perhaps a better model design could have two date tables, each with an active relationship to the **Sales** table.

Thus, report users can filter by order date or ship date, or both at the same time. A calculated table can duplicate the **Date** table data to create the **Ship Date** table.



What-if analysis

Power BI Desktop supports a feature called What-if parameters. When you create a what-if parameter, a calculated table is automatically added to your model.

What-if parameters allow report users to select or filter by values that are stored in the calculated table. Measure formulas can use selected value(s) in a meaningful way.

For example, a **what-if** parameter could allow the report user to select a hypothetical currency exchange rate, and a measure could divide revenue values (in a local currency) by the selected rate.

Notably, what-if calculated tables aren't related to other model tables because they're not used to propagate filters.

For this reason, they're sometimes called *disconnected tables*.

Calculated columns

You can write a DAX formula to add a calculated column to any table in your model. The formula is evaluated for each table row and it returns a single value.

When added to an Import storage mode table, the formula is evaluated when the semantic model is refreshed, and it increases the storage size of your model.

When added to a DirectQuery storage mode table, the formula is evaluated by the underlying source database when the table is queried.

In the **Fields** pane, calculated columns are enhanced with a special icon.

The following example shows a single calculated column in the **Customer** table called **Age**.

Measures

You can write a DAX formula to add a measure to any table in your model.

The formula is concerned with achieving summarization over model data. Similar to a calculated column, the formula must return a single value.

Unlike calculated columns, which are evaluated at data refresh time, measures are evaluated at query time. Their results are never stored in the model.

In the **Fields** pane, measures are shown with the calculator icon. The following example shows three measures in the **Sales** table: **Cost**, **Profit**, and **Revenue**.

Occasionally, measures can be described as *explicit measures*. To be clear, explicit measures are model calculations that are written in DAX and are commonly referred to as simply *measures*.

Yet, the concept of *implicit measures* exists, too. Implicit measures are columns that can be summarized by visuals in simplistic ways, like count, sum, minimum, maximum, and so on.

You can identify implicit measures in the **Fields** pane because they're shown with the sigma symbol (Σ).

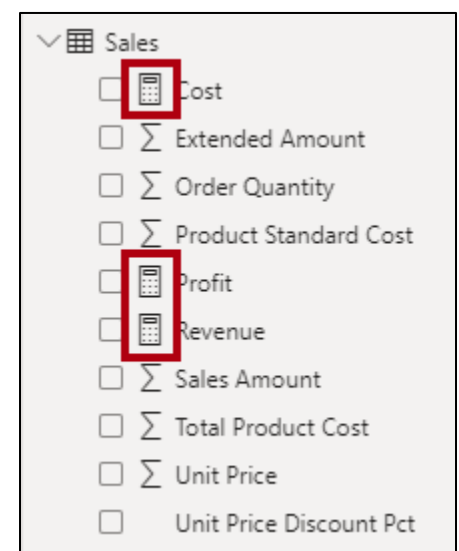
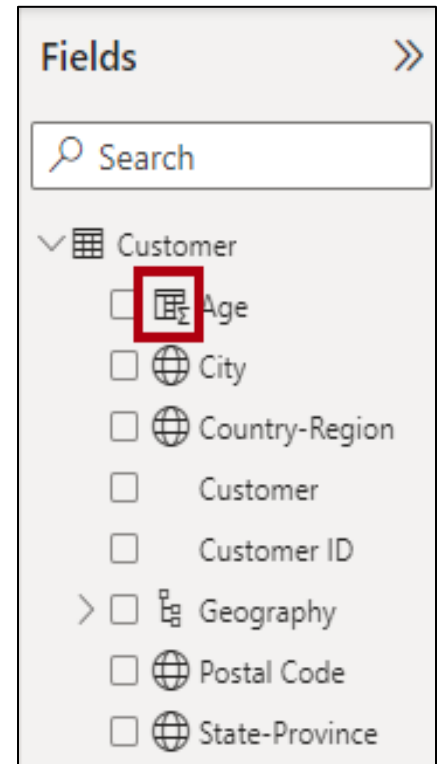
Any column can be summarized when added to a visual.

Therefore, whether they're shown with the sigma symbol or not, when they're added to a visual, they can be set up as implicit measures.

Additionally, no such concept as a *calculated measure* exists in tabular modelling.

The word *calculated* is used to describe calculated tables and calculated columns, which distinguishes them from tables and columns that originate from Power Query.

Power Query doesn't have the concept of an explicit measure.



Write DAX Formulas

Each model calculation type, calculated table, calculated column, or measure is defined by its name, followed by the equals symbol (=), which is then followed by a DAX formula. Use the following template to create a model calculation:

<Calculation name> = <DAX formula>

For example, the definition of the **Ship Date** calculated table that duplicates the **Date** table data is:

Ship Date = 'Date'

A DAX formula consists of expressions that return a result. The result is either a table object or a scalar value. Calculated table formulas must return a table object; calculated column and measure formulas must return a scalar value (single value).

Formulas are assembled by using:

- DAX functions
- DAX operators
- References to model objects
- Constant values, like the number 24 or the literal text "FY" (abbreviation for fiscal year)
- DAX variables
- Whitespace

When entering DAX formulas in Power BI Desktop, you have the benefit of *IntelliSense*. IntelliSense is a code-completion aid that lists functions and model resources. When you select a DAX function, it also provides you with a definition and description. We recommend that you use IntelliSense to help you quickly build accurate formulas.

DAX functions

Similar to Microsoft Excel, DAX is a functional language meaning that formulas rely on functions to accomplish specific goals. Typically, DAX functions have arguments that allow passing in variables. Formulas can use many function calls and will often nest functions within other functions.

In a formula, function names must be followed by parentheses. Within the parentheses, variables are passed in.

Some functions don't take arguments, or arguments might be optional.

Working with DAX functions is described later in this module.

DAX operators

Formulas also rely on operators, which can perform arithmetic calculations, compare values, work with strings, or test conditions.

References to model objects

Formulas can only refer to three types of model objects: tables, columns, or measures. A formula can't refer to a hierarchy or a hierarchy level.

(Recall that a hierarchy level is based on a column, so your formula can refer to a hierarchy level's column.)

Table references

When you reference a table in a formula, officially, the table name is enclosed within single quotation marks. In the following calculated table definition, notice that the **Date** table is enclosed within single quotation marks.

```
Ship Date = 'Date'
```

However, single quotation marks can be omitted when both of the following conditions are true:

1. The table name does not include embedded spaces.
2. The table name isn't a reserved word that's used by DAX. All DAX function names and operators are reserved words. *Date* is a DAX function name, which explains why, when you are referencing a table named **Date**, that you must enclose it within single quotation marks.

In the following calculated table definition, it's possible to omit the single quotation marks when referencing the **Airport** table:

```
Arrival Airport = Airport
```

Column references

When you reference a column in a formula, the column name must be enclosed within square brackets. Optionally, it can be preceded by its table name. For example, the following measure definition refers to the **Sales Amount** column.

```
Revenue = SUM([Sales Amount])
```

Because column names are unique within a table but not necessarily unique within the model, you can disambiguate the column reference by preceding it with its table name.

This disambiguated column is known as a *fully qualified column*. Some DAX functions require passing in fully qualified columns.

To improve the readability of your formulas, we recommend that you always precede a column reference with its table name.

The previous example measure definition can be rewritten as:

```
Revenue = SUM(Sales[Sales Amount])
```

Measure references

When you reference a measure in a formula, like column name references, the measure name must be enclosed within square brackets. For example, the following measure definition refers to the **Revenue** and **Cost** measures.

```
Profit = [Revenue] - [Cost]
```

If you're a DAX beginner, the fact that column and measure references are always enclosed within square brackets can cause confusion when you're trying to read a formula.

However, as you become proficient with DAX fundamentals, you'll be able to determine which type of object it is because, in DAX formulas, columns, and measures are used in different ways.

It's possible to precede a measure reference with its table name. However, measures are a model-level object. While they're assigned to a home table, it's only a cosmetic relationship to logically organize measures in the **Fields** pane.

Therefore, while we recommend that you always precede a column reference with its table name, the inverse is true for measures: We recommend that you never precede a measure reference with its table name.

DAX variables

Formulas can declare DAX variables to store results.

Whitespace

Whitespace refers to characters that you can use to format your formulas in a way that's quick and simple to understand. Whitespace characters include:

- Spaces
- Tabs
- Carriage returns

Whitespace is optional and it doesn't modify your formula logic or negatively impact performance. We strongly recommend that you adopt a format style and apply it consistently, and consider the following recommendations:

- Use spaces between operators.
- Use tabs to indent nested function calls.
- Use carriage returns to separate function arguments, especially when it's too long to fit on a single line. Formatting in this way makes it simpler to troubleshoot, especially when the formula is missing a parenthesis.
- Err on the side of too much whitespace than too little.

In the formula bar, to enter a carriage return, press **Shift+Enter**. Pressing **Enter** alone will commit your formula.

Consider the following measure definition that's written in a single line and that includes five DAX function calls:

```
Revenue    YoY    %    =    DIVIDE([Revenue]    -    CALCULATE([Revenue],
SAMEPERIODLASTYEAR('Date'[Date])),    CALCULATE([Revenue],
SAMEPERIODLASTYEAR('Date'[Date])))
```

DAX Data Types

Semantic model columns have a set data type, which ensures that all column values conform to that data type. Column data types are defined in Power Query, or in the case of calculated columns, it's inferred from the formula.

Measure data types, similar to calculated column data types, are inferred from the formula.

Model data types aren't the same as DAX data types, though a direct relationship exists between them.

The following table lists the model data types and DAX data types. Notice the supported range of values for each data type.

Model data type	DAX data type	Description
Whole number	64-bit integer	-2^{63} through $2^{63}-1$
Decimal number	64-bit real	Negative: -1.79×10^{308} through -2.23×10^{-308} - zero (0) - positive: 2.23×10^{-308} through 1.79×10^{308} - Limited to 17 decimal digits
Boolean	Boolean	TRUE or FALSE
Text	String	Unicode character string
Date	Date/time	Valid dates are all dates after January 1, 1900
Currency	Currency	-9.22×10^{14} through 9.22×10^{14} - limited to four decimal digits of fixed precision
N/A	BLANK	In some cases, it's the equivalent of a database (SQL) NULL

BLANK data type

The BLANK data type deserves a special mention. DAX uses BLANK for both database NULL and for blank cells in Excel. BLANK doesn't mean zero.

Perhaps it might be simpler to think of it as the *absence of a value*.

Two DAX functions are related to the BLANK data type: the BLANK DAX function returns BLANK, while the ISBLANK DAX function tests whether an expression evaluates to BLANK.

Work with DAX Functions

The DAX function library consists of hundreds of functions, each designed to accomplish a specific goal.

Because DAX originated with the Power Pivot add-in for Microsoft Excel 2010, over 80 functions are available that can also be found in Excel.

It was a deliberate design strategy by Microsoft to ensure that Excel users can quickly become productive with DAX.

However, many functions exist that you won't find in Excel because they're specific to data modelling:

- Relationship navigation functions
- Filter context modification functions
- Iterator functions
- Time intelligence functions
- Path functions

To search for documentation that is related to a DAX function, in a web search, enter the keyword *DAX* followed by the function name.

Functions that originate from Excel

The following sections consider several useful functions that you might already be familiar with because they exist in Excel.

The IF DAX function tests whether a condition that's provided as the first argument is met. It returns one value if the condition is TRUE and returns the other value if the condition is FALSE. The function's syntax is:

```
IF(<logical_test>, <value_if_true>[, <value_if_false>])
```

A function argument is optional when documentation shows it enclosed within square brackets. If **logical_test** evaluates to FALSE and **value_if_false** isn't provided, the function will return BLANK.

Many Excel summarization functions are available, including SUM, COUNT, AVERAGE, MIN, MAX, and many others.

The only difference is that in DAX, you pass in a column reference, whereas in Excel, you pass in a range of cells.

Many Excel mathematic, text, date and time, information, and logical functions are available as well. For example, a small sample of Excel functions that are available in DAX include ABS, ROUND, SQRT, LEN, LEFT, RIGHT, UPPER, DATE, YEAR, MONTH, NOW, ISNUMBER, TRUE, FALSE, AND, OR, NOT, and IFERROR.

Functions that don't originate from Excel

Two useful DAX functions that aren't specific to modeling and that don't originate from Excel are DISTINCTCOUNT and DIVIDE.

DISTINCTCOUNT function

You can use the DISTINCTCOUNT DAX function to count the number of distinct values in a column. This function is especially powerful in an analytics solution. Consider that the count of customers is different from the count of *distinct* customers.

The latter doesn't count repeat customers, so the difference is "How many customers" compared with "How many *different* customers."

DIVIDE function

You can use the DIVIDE DAX function to achieve division. You must pass in numerator and denominator expressions. Optionally, you can pass in a value that represents an *alternate result*. The DIVIDE function's syntax is:

```
DIVIDE(<numerator>, <denominator>[, <alternate_result>])
```

The DIVIDE function automatically handles division by zero cases. If an alternate result isn't passed in, and the denominator is zero or BLANK, the function returns BLANK.

When an alternate result is passed in, it's returned instead of BLANK.

This function is convenient because it saves your expression from having to first test the denominator value. The function is also better optimized for testing the denominator value than the IF function.

The performance gain is significant because checking for division by zero is expensive. What's more, using the DIVIDE function results in a more concise and elegant expression.

We recommend that you use the DIVIDE function whenever the denominator is an expression that could return zero or BLANK. In the case that the denominator is a constant value, we recommend that you use the divide operator (/). In this case, the division is guaranteed to succeed, and your expression will perform better because it will avoid unnecessary testing.

Use DAX Operators

Your DAX formulas can use operators to create expressions that perform arithmetic calculations, compare values, work with strings, or test conditions.

Many DAX operators and precedence order are the same as those found in Excel.

Arithmetic operators

The following table lists the arithmetic operators.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponentiation

Remember, when you are dividing two expressions, and when the denominator could return zero or BLANK, it's more efficient and safer to use the DIVIDE DAX function.

Comparison operators

The following table lists the comparison operators, which are used to compare two values. The result is either TRUE or FALSE.

Operator	Description
=	Equal to
==	Strict equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

All comparison operators, except **strict equal to (==)**, treat BLANK as equal to the number zero, an empty string (""), the date December 30, 1899, or FALSE.

It means that the expression [Revenue] = 0 will be TRUE when the value of [Revenue] is either zero or BLANK. In contrast, [Revenue] == 0 is TRUE only when the value of [Revenue] is zero.

Text concatenation operator

Use the ampersand (&) character to connect, or concatenate, two text values to produce one continuous text value. For example, consider the following calculated column definition:

```
Model Color = 'Product'[Model] & "-" & 'Product'[Color]
```

Logical operators

Use logical operators to combine expressions that produce a single result. The following table lists all logical operators.

Operator	Description
&&	Creates an AND condition between two expressions where each has a Boolean result. If both expressions return TRUE, the combination of the expressions also returns TRUE; otherwise the combination returns FALSE.
(double pipe)	Creates an OR condition between two logical expressions. If either expression returns TRUE, the result is TRUE; only when both expressions are FALSE is the result FALSE.
IN	Creates a logical OR condition between each row that is being compared to a table. Note: The table constructor syntax uses braces.
NOT	Inverts the state of a Boolean expression (FALSE to TRUE, and vice versa).

An example that uses the IN logical operator is the **ANZ Revenue** measure definition, which uses the CALCULATE DAX function to enforce a specific filter of two countries/regions: Australia and New Zealand.

You'll be introduced to the powerful CALCULATE function when you learn how to modify the filter context.

```
ANZ Revenue =
CALCULATE(
    [Revenue],
    Customer[Country-Region] IN {
        "Australia",
        "New Zealand"
    }
)
```

Operator precedence

When your DAX formula includes multiple operators, DAX uses rules to determine the evaluation order, which is known as an *operator precedence*. Operations are ordered according to the following table.

Operator	Description
^	Exponentiation
-	Sign (as in -1)
* and /	Multiplication and division
NOT	NOT
+ and -	Addition and subtraction
&	Concatenation of two strings of text
=, <, >, <=, >=, <>	Comparison

When the operators have equal precedence value, they're ordered from left to right.

In general, operator precedence is the same as what's found in Excel. If you need to override the evaluation order, then group operations within parentheses.

For example, consider the following calculated column definition:

Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * 1 - [Unit Price Discount Pct]

This sample calculated column definition produces an incorrect result because multiplication happens before the subtraction. The following correct calculated column definition uses parentheses to ensure that the subtractions happen before the multiplications.

Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * (1 - [Unit Price Discount Pct])

Remembering operator precedence rules can be challenging, especially for DAX beginners. Consequently, we recommend that you test your formulas thoroughly. When the formulas don't produce the correct result due to an incorrect evaluation order, you can experiment by adding parentheses to adjust the evaluation order. You can also add parentheses to improve the readability of your formulas.

Implicit conversion

When writing a DAX formula that uses operators to combine different data types, you don't need to explicitly convert types. Usually, DAX automatically identifies the data types of referenced model objects and performs implicit conversions where necessary to complete the specified operation.

However, some limitations might exist on the values that can be successfully converted. If a value or a column has a data type that's incompatible with the current operation, DAX returns an error. For example, the attempt to multiply a date value will create an error because it isn't logical.

BLANK is handled differently, depending on the operator that is used. It's handled similar to how Excel treats BLANK, but differently to how databases (SQL) treat NULL. BLANK is treated as zero when acted on by arithmetic operators and as an empty string when concatenated to a string.

Remembering how BLANK is handled can be challenging, especially for DAX beginners. Consequently, we recommend that you test your formulas thoroughly. When BLANKs create unexpected results, consider using the **IF** and **ISBLANK** DAX functions to test for BLANK, and then respond in an appropriate way.

Use DAX Variables

You can declare DAX variables in your formula expressions.

When you declare at least one variable, a RETURN clause is used to define the expression, which then refers to the variables.

We recommend that you use variables because they offer several benefits:

- Improving the readability and maintenance of your formulas.
- Improving performance because variables are evaluated once and only when or if they're needed.
- Allowing (at design time) straightforward testing of a complex formula by returning the variable of interest.

The following example shows a formula that declares a variable. The **Revenue YoY %** measure definition is rewritten to declare a variable that's assigned the value of the prior year's revenue.

```
Revenue YoY % =  
VAR RevenuePriorYear =  
    CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    )  
RETURN  
    DIVIDE(  
        [Revenue] - RevenuePriorYear,  
        RevenuePriorYear  
    )
```

Notice that the RETURN clause refers to the variable twice. This improved measure definition formula will run in at least half the time because it doesn't need to evaluate the prior year's revenue twice.

Add Measures to Power BI Models

Introduction

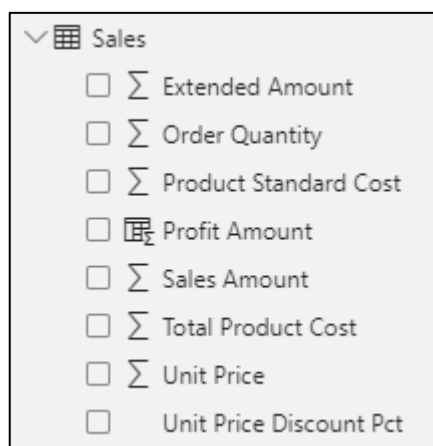
Measures in Microsoft Power BI models are either *implicit* or *explicit*. Implicit measures are automatic behaviors that allow visuals to summarize model column data.

Explicit measures, also known simply as *measures*, are calculations that you can add to your model. This module focuses on how you can use implicit measures.

In the **Fields** pane, a column that's shown with the sigma symbol (Σ) indicates two facts:

- It's a numeric column.
- It will summarize column values when it is used in a visual (when added to a field well that supports summarization).

In the following image, notice that the **Sales** table includes only fields that can be summarized, including the **Profit Amount** calculated column.



As a data modeler, you can control if and how the column summarizes by setting the **Summarization** property to **Don't summarize** or to a specific aggregation function.

When you set the **Summarization** property to **Don't summarize**, the sigma symbol will no longer show next to the column in the **Fields** pane.

Implicit measures allow the report author to start with a default summarization technique and lets them modify it to suit their visual requirements.

Numeric columns support the greatest range of aggregation functions:

- Sum
- Average
- Minimum
- Maximum
- Count (Distinct)
- Count
- Standard deviation
- Variance
- Median

Summarize non-numeric columns

Non-numeric columns can be summarized. However, the sigma symbol does not show next to non-numeric columns in the **Fields** pane because they don't summarize by default.

Text columns allow the following aggregations:

- First (alphabetically)
- Last (alphabetically)
- Count (Distinct)
- Count

Date columns allow the following aggregations:

- Earliest
- Latest
- Count (Distinct)
- Count

Boolean columns allow the following aggregations:

- Count (Distinct)
- Count

Benefits of implicit measures

Several benefits are associated with implicit measures.

Implicit measures are simple concepts to learn and use, and they provide flexibility in the way that report authors visualize model data.

Additionally, they mean less work for you as a data modeler because you don't have to create explicit calculations.

Limitations of implicit measures

Implicit measures do have limitations.

Despite setting an appropriate summarization method, report authors could choose to aggregate a column in unsuitable ways.

For example, in the matrix visual, you could modify the aggregate function of **Unit Price** to **Sum**.

Fiscal Year	Sales Amount	Sum of Unit Price
FY2018	\$23,860,891.17	\$13,905,519.77
2017 Jul	\$1,423,357.32	\$1,164,795.52
2017 Aug	\$2,057,902.45	\$1,115,258.56
2017 Sep	\$2,523,947.55	\$1,291,296.17
2017 Oct	\$561,681.48	\$561,681.48
2017 Nov	\$4,764,920.16	\$2,159,760.38
2017 Dec	\$596,746.56	\$596,746.56

The report visual obeys your setup, but it has now produced a **Sum of Unit Price** column, which presents misleading data.

The most significant limitation of implicit measures is that they only work for simple scenarios, meaning that they can only summarize column values that use a specific aggregation function. Therefore, in situations when you need to calculate the ratio of each month's sales amount over the yearly sales amount, you'll need to produce an explicit measure by writing a Data Analysis Expressions (DAX) formula to achieve that more sophisticated requirement.

Implicit measures don't work when the model is queried by using Multidimensional Expressions (MDX). This language expects explicit measures and can't summarize column data. It's used when a Power BI semantic model is queried by using Analyze in Excel or when a Power BI paginated report uses a query that is generated by the MDX graphical query designer.

Create Simple Measures

You can write a DAX formula to add a measure to any table in your model. A measure formula must return a scalar or single value.

In tabular modeling, no such concept as a calculated measure exists. The word *calculated* is used to describe calculated tables and calculated columns. It distinguishes them from tables and columns that originate from Power Query, which doesn't have the concept of an explicit measure.

Measures don't store values in the model. Instead, they're used at query time to return summarizations of model data. Additionally, measures can't reference a table or column directly; they must pass the table or column into a function to produce a summarization.

A *simple* measure is one that aggregates the values of a single column; it does what implicit measures do automatically.

The COUNTDAX function counts the number of non-BLANK values in a column, while the DISTINCTCOUNT DAX function counts the number of distinct values in a column.

Create Compound Measures

When a measure references one or more measures, it's known as a *compound measure*.

For this example, you will modify the **Profit** measure by using the following measure definition. Format the measure with two decimal places.

Profit =
[Revenue] - [Cost]

Next, add the **Profit** measure to the matrix visual.

Now that your model provides a way to summarize profit, you can delete the **Profit Amount** calculated column.

By removing this calculated column, you've optimized the semantic model. Removing this column results in a decreased semantic model size and shorter data refresh times.

The **ProfitAmount** calculated column wasn't required because the **Profit** measure can directly produce the required result.

Create Quick Measures

Microsoft Power BI Desktop includes a feature named Quick Measures.

This feature helps you to quickly perform common, powerful calculations by generating the DAX expression for you.

Many categories of calculations and ways to modify each calculation are available to fit your needs. Moreover, you are able to see the DAX that's generated by the quick measure and use it to jumpstart or expand your DAX knowledge.

Compare Calculated Columns with Measures

DAX beginners often experience a degree of confusion about calculated columns and measures. The following section reviews the similarities and differences between both.

Regarding similarities between calculated columns and measures, both are:

- Calculations that you can add to your semantic model.
- Defined by using a DAX formula.
- Referenced in DAX formulas by enclosing their names within square brackets.

The areas where calculated columns and measures differ include:

- **Purpose** - Calculated columns extend a table with a new column, while measures define how to summarize model data.
- **Evaluation** - Calculated columns are evaluated by using *row context* at data refresh time, while measures are evaluated by using *filter context* at query time. Filter context is introduced in a later module; it's an important topic to understand and master so that you can achieve complex summarizations.
- **Storage** - Calculated columns (in Import storage mode tables) store a value for each row in the table, but a measure never stores values in the model.
- **Visual use** - Calculated columns (like any column) can be used to filter, group, or summarize (as an implicit measure), whereas measures are designed to summarize.

Summary

Implicit measures are automatic behaviors that are supported by visuals, while explicit measures use DAX formulas that summarize model data.

Explicit measures are important because they allow you to create complex DAX formulas to achieve the precise calculations that your report visuals need.

Add Calculated Tables and Columns to Power BI Desktop Models

Introduction

You can write a Data Analysis Expressions (DAX) formula to add a *calculated table* to your model. The formula can duplicate or transform existing model data to produce a new table.

A calculated table can't connect to external data; you must use Power Query to accomplish that task.

A calculated table formula must return a table object. The simplest formula can duplicate an existing model table.

Calculated tables have a cost: They increase the model storage size and they can prolong the data refresh time. The reason is because calculated tables recalculate when they have formula dependencies to refreshed tables.

A calculated table only duplicates data; it doesn't duplicate any model properties or objects like column visibility or hierarchies. You'll need to set them up for the new table, if required.

It's possible to rename columns of a calculated table.

Calculated tables are useful to work in scenarios when multiple relationships between two tables exist, as previously described. They can also be used to add a date table to your model. Date tables are required to apply special time filters known as *time intelligence*.

Create a date table

In the next example, a second calculated table will be created, this time by using the CALENDARAUTO DAX function.

Create the **Due Date** calculated table by using the following definition.

Due Date = CALENDARAUTO(6)

The CALENDARAUTO DAX function takes a single optional argument, which is the last month number of the year, and returns a single-column table. If you don't pass in a month number, it's assumed to be 12 (for December). For example, at Adventure Works, their financial year ends on June 30 of each year, so the value 6 (for June) is passed in.

The function scans all date and date/time columns in your model to determine the earliest and latest stored date values. It then produces a complete set of dates that span all dates in your model, ensuring that full years of dates are loaded.

For example, if the earliest date that is stored in your model is October 15, 2021, then the first date that is returned by the CALENDARAUTO function would be July 1, 2021. If the latest date that is stored in the model is June 15, 2022, then the last date that is returned by the CALENDARAUTO function would be June 30, 2022.

Effectively, the CALENDARAUTO function guarantees that the following requirements to *mark a date table* are met:

- The table must include a column of data type Date.
- The column must contain complete years.
- The column must not have missing dates.

You can also create a date table by using the **CALENDAR** DAX function and passing in two date values, which represent the date range. The function generates one row for each date within the range. You can pass in static date values or pass in expressions that retrieve the earliest/latest dates from specific columns in your model.

When the table rows and distinct values are the same, it means that the column contains unique values. That factor is important for two reasons: It satisfies the requirements to mark a date table, and it allows this column to be used in a model relationship as the one-side.

The **Due Date** calculated table will recalculate each time a table that contains a date column refreshes. In other words, when a row is loaded into the **Sales** table with an order date of July 1, 2022, the **Due Date** table will automatically extend to include dates through to the end of the next year: June 30, 2023.

The **Due Date** table requires additional columns to support the known filtering and grouping requirements, specifically by year, quarter, and month.

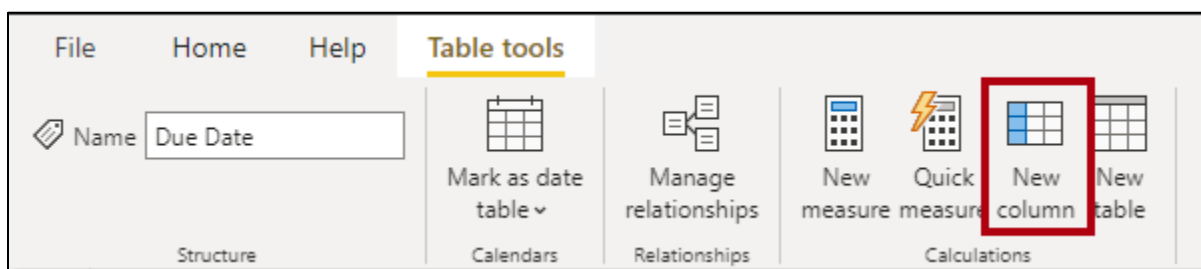
Create Calculated Columns

You can write a DAX formula to add a *calculated column* to any table in your model. A calculated column formula must return a scalar or single value.

Calculated columns in import models have a cost: They increase the model storage size and they can prolong the data refresh time. The reason is because calculated columns recalculate when they have formula dependencies to refreshed tables.

In data view, in the **Fields** pane, ensure that the **Due Date** table is selected. Before you create a calculated column, first rename the **Date** column to **Due Date**.

Now, you can add a calculated column to the **Due Date** table. To create a calculated column, in the **Table tools** contextual ribbon, from inside the **Calculations** group, select **New column**.



In the formula bar, enter the following calculated column definition and then press **Enter**.

```
Due Fiscal Year =  
"FY"  
& YEAR('Due Date'[Due Date])  
+ IF(  
    MONTH('Due Date'[Due Date]) > 6,  
    1  
)
```

The calculated column definition adds the **Due Fiscal Year** column to the **Due Date** table. The following steps describe how Microsoft Power BI evaluates the calculated column formula:

1. The addition operator (+) is evaluated before the text concatenation operator (&).
2. The YEAR DAX function returns the whole number value of the due date year.
3. The IF DAX function returns the value when the due date month number is 7-12 (July to December); otherwise, it returns BLANK. (For example, because the Adventure Works financial year is July-June, the last six months of the calendar year will use the next calendar year as their financial year.)
4. The year value is added to the value that is returned by the IF function, which is the value one or BLANK. If the value is BLANK, it's implicitly converted to zero (0) to allow the addition to produce the fiscal year value.
5. The literal text value "FY" concatenated with the fiscal year value, which is implicitly converted to text.

Learn About Row Context

Now that you've created calculated columns, you can learn how their formulas are evaluated. The formula for a calculated column is evaluated for each table row. Furthermore, it's evaluated within row context, which means *the current row*. Consider the **Due Fiscal Year** calculated column definition:

```
Due Fiscal Year =  
"FY"  
& YEAR('Due Date'[Due Date])  
+ IF(  
    MONTH('Due Date'[Due Date]) <= 6,  
    1  
)
```

When the formula is evaluated for each row, the 'Due Date'[Due Date] column reference returns the column value for *that row*. You might be familiar with this concept from working with formulas in Excel tables.

However, row context doesn't extend beyond the table. If your formula needs to reference columns in other tables, you have two options:

- If the tables are related, directly or indirectly, you can use the RELATED or RELATEDTABLE DAX function. The RELATED function retrieves the value at the one-side of the relationship, while the RELATEDTABLE retrieves values on the many-side. The RELATEDTABLE function returns a table object.
- When the tables aren't related, you can use the LOOKUPVALUE DAX function.

Generally, try to use the RELATED function whenever possible. It will usually perform better than the LOOKUPVALUE function due to the ways that relationship and column data is stored and indexed.

Now, as an example, we can add the following calculated column definition to the **Sales** table:

```
Discount Amount =  
(  
    Sales[Order Quantity]  
    * RELATED('Product'[List Price])  
) - Sales[Sales Amount]
```

The calculated column definition adds the **DiscountAmount** column to the **Sales** table. Power BI evaluates the calculated column formula for each row of the **Sales** table.

The values for the **Order Quantity** and **Sales Amount** columns are retrieved within row context.

However, because the **List Price** column belongs to the **Product** table, the RELATED function is required to retrieve the list price value *for the sale product*.

Row context is used when calculated column formulas are evaluated. It's also used when a class of functions, known as *iterator functions*, are used.

Iterator functions provide you with flexibility to create sophisticated summarizations. Iterator functions are described in a later module.

Choose a Technique to Add a Column

There are three techniques that you can use to add columns to a model table:

- **Add columns to a view or table (as a persisted column), and then source them in Power Query.**
 - This option only makes sense when your data source is a relational database and if you have the skills and permissions to do so.
 - However, it's a good option because it supports ease of maintenance and allows reuse of the column logic in other models or reports.
- **Add custom columns (using M) to Power Query queries.**
- **Add calculated columns (using DAX) to model tables.**

Regardless of which technique you use, it results in the same outcome. Report users can't determine the origin of a column.

Typically, they're not concerned with how the column was created but rather that it delivers the right data.

When multiple ways are available to add a column, you can consider using the approach that best fits your skills and that's supported by the language (M or DAX).

However, the preference is to add custom columns in Power Query, whenever possible, because they load to the model in a more compact and optimal way.

When you need to add a column to a calculated table, make sure that you create a calculated column.

Otherwise, we recommend that you only use a calculated column when the calculated column formula:

- **Depends on summarized model data.**
- **Needs to use specialized modeling functions that are only available in DAX, such as the RELATED and RELATEDTABLE functions.**
 - Specialized functions can also include the DAX parent and child hierarchies, which are designed to naturalize a recursive relationship into columns.
 - For example, in an employee table where each row stores a reference to the row of the manager (who is also an employee).

Modify DAX Filter Context in Power BI Desktop Models

Introduction

Filter context describes the filters that are applied during the evaluation of a measure or measure expression. Filters can be applied directly to columns, like a filter on the **Fiscal Year** column in the **Date** table for the value **FY2020**.

Additionally, filters can be applied indirectly, which happens when model relationships propagate filters to other tables. For example, the **Sales** table receives a filter through its relationship with the **Date** table, filtering the **Sales** table rows to those with an **OrderDateKey** column value in **FY2020**.

Calculated tables and calculated columns aren't evaluated within filter context. Calculated columns are evaluated in row context, though the formula can transition the row context to filter context, if it needs to summarize model data.

Not all filters are applied at report design time. Filters can be added when a report user interacts with the report.

They can modify filter settings in the **Filters** pane, and they can cross-filter or cross-highlight visuals by selecting visual elements like columns, bars, or pie chart segments.

These interactions apply additional filters to report page visuals (unless interactions have been disabled).

It's important to understand how filter context works. It guides you in defining the correct formula for your calculations.

As you write more complex formulas, you'll identify times when you need to add, modify, or remove filters to achieve the desired result.

The key to writing complex measures is mastering these concepts:

- Understanding how filter context works.
- Understanding when and how to modify or remove filters to achieve a required result.
- Composing a formula to accurately and efficiently modify filter context.

Modify Filter Context

You can use the CALCULATE DAX function to modify filter context in your formulas. The syntax for the CALCULATE function is as follows:

```
CALCULATE(<expression>, [[<filter1>], <filter2>]...)
```

The function requires passing in an expression that returns a scalar value and as many filters as you need. The expression can be a measure (which is a named expression) or any expression that can be evaluated in filter context.

Filters can be Boolean expressions or table expressions. It's also possible to pass in filter modification functions that provide additional control when you're modifying filter context.

When you have multiple filters, they're evaluated by using the AND logical operator, which means that all conditions must be TRUE at the same time.

The **CALCULATETABLE** DAX function performs exactly the same functionality as the CALCULATE function, except that it modifies the filter context that's applied to an expression that returns a table object.

Apply Boolean expression filters

A Boolean expression filter is an expression that evaluates to TRUE or FALSE. Boolean filters must abide by the following rules:

- They can reference only a single column.
- They cannot reference measures.
- They cannot use functions that scan or return a table that includes aggregation functions like SUM.

Apply table expression filters

A table expression filter applies a table object as a filter. It could be a reference to a model table; however, it's likely a DAX function that returns a table object.

Commonly, you'll use the FILTER DAX function to apply complex filter conditions, including those that can't be defined by a Boolean filter expression.

The FILTER function is classed as an iterator function, and so you would pass in a table, or table expression, and an expression to evaluate for each row of that table.

The FILTER function returns a table object with exactly the same structure as one that the table passed in.

Its rows are a subset of those rows that were passed in, meaning the rows where the expression evaluated as TRUE.

The following example shows a table filter expression that uses the FILTER function:

```
Revenue High Margin Products =  
CALCULATE(  
    [Revenue],  
    FILTER(  
        'Product',  
        'Product'[List Price] > 'Product'[Standard Cost] * 2  
    )  
)
```

In this example, the FILTER function filters all rows of the Product table that are in filter context.

Each row for a product where its list price exceeds double its standard cost is displayed as a row of the filtered table.

Therefore, the **Revenue** measure is evaluated for all products that are returned by the FILTER function.

All filter expressions that are passed in to the CALCULATE function are table filter expressions.

A Boolean filter expression is a shorthand notation to improve the writing and reading experience. Internally, Microsoft Power BI translates Boolean filter expressions to table filter expressions,

Filter behaviour

Two possible standard outcomes occur when you add filter expressions to the CALCULATE function:

- If the columns (or tables) aren't in filter context, then new filters will be added to the filter context to evaluate the CALCULATE expression.
- If the columns (or tables) are already in filter context, the existing filters will be overwritten by the new filters to evaluate the CALCULATE expression.

Use Filter Modifier Functions

When using the CALCULATE function, you can pass in filter modification functions, which allow you to accomplish more than adding filters alone.

Remove filters

Use the REMOVEFILTERS DAX function as a CALCULATE filter expression to remove filters from filter context. It can remove filters from one or more columns or from all columns of a single table.

The REMOVEFILTERS function is relatively new. In previous versions of DAX, you removed filters by using the **ALL** DAX function or variants including the **ALLEXCEPT** and the **ALLNOBLANKROW** DAX functions.

These functions behave as both filter modifiers and as functions that return table objects of distinct values.

These functions are mentioned now because you're likely to find documentation and formula examples that remove filters by using them.

Preserve filters

You can use the KEEPFILTERS DAX function as a filter expression in the CALCULATE function to preserve filters.

Use inactive relationships

An inactive model relationship can only propagate filters when the USERELATIONSHIP DAX function is passed as a filter expression to the CALCULATE function.

When you use this function to engage an inactive relationship, the active relationship will automatically become inactive.

Modify relationship behavior

You can modify the model relationship behavior when an expression is evaluated by passing the [CROSSFILTER](#) DAX function as a filter expression to the CALCULATE function.

It's an advanced capability.

The CROSSFILTER function can modify filter directions (from both to single or from single to both) and even disable a relationship.

Examine Filter Context

The VALUES DAX function lets your formulas determine what values are in filter context.

The VALUES function syntax is as follows:

VALUES(<TableNameOrColumnName>)

The function requires passing in a table reference or a column reference. When you pass in a table reference, it returns a table object with the same columns that contain rows for what's in filter context.

When you pass in a column reference, it returns a single-column table of unique values that are in filter context.

The function always returns a table object and it's possible for a table to contain multiple rows. Therefore, to test whether a specific value is in filter context, your formula must first test that the VALUES function returns a single row.

Two functions can help you accomplish this task: the HASONEVALUE and the SELECTEDVALUE DAX functions.

- The HASONEVALUE function returns TRUE when a given column reference has been filtered down to a single value.
- The SELECTEDVALUE function simplifies the task of determining what a single value could be.

When the function is passed a column reference, it'll return a single value, or when more than one value is in filter context, it'll return BLANK (or an alternate value that you pass to the function).

Three other functions that you can use to test filter state are:

- ISFILTERED - Returns TRUE when a passed-in column reference is *directly* filtered.
- ISCROSSFILTERED - Returns TRUE when a passed-in column reference is *indirectly* filtered.
 - A column is cross-filtered when a filter that is applied to another column in the same table, or in a related table, affects the reference column by filtering it.
- ISINSCOPE - Returns TRUE when a passed-in column reference is the level in a hierarchy of levels.

Perform Context Transition

What happens when a measure or measure expression is evaluated within row context? This scenario can happen in a calculated column formula or when an expression in an iterator function is evaluated.

In the following example, you will add a calculated column to the **Customer** table to classify customers into a loyalty class. The scenario is simple: When the revenue that is produced by the customer is less than \$2500, the customer is classified as **Low**; otherwise they're classified as **High**.

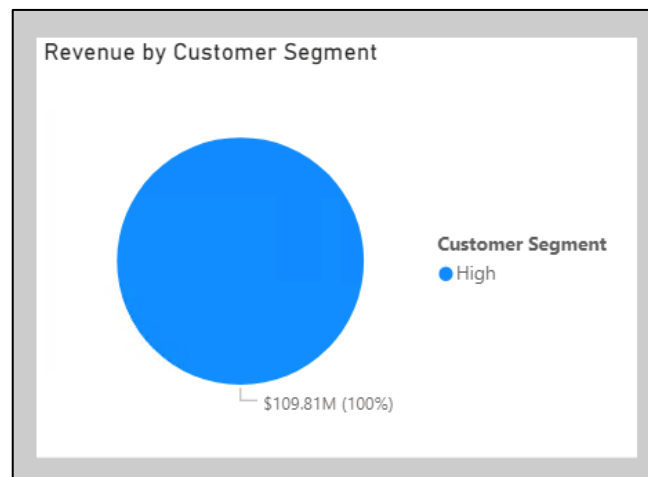
Customer Segment =

VAR CustomerRevenue = SUM(Sales[Sales Amount])

RETURN

IF(CustomerRevenue < 2500, "Low", "High")

On **Page 4** of the report, add the **Customer Segment** column as the legend of the pie chart.



Notice that only one **Customer Segment** value exists. The reason is because the calculated column formula produces an incorrect result:

Each customer is assigned the value of **High** because the expression `SUM(Sales[Sales Amount])` isn't evaluated in a filter context. Consequently, each customer is assessed on the sum of every **Sales Amount** column value in the **Sales** table.

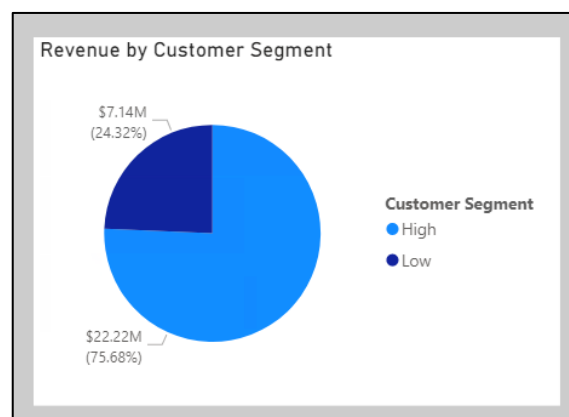
To force the evaluation of the `SUM(Sales[Sales Amount])` expression *for each customer*, a context transition must take place that applies the row context column values to filter context.

You can accomplish this transition by using the `CALCULATE` function without passing in filter expressions.

Modify the calculated column definition so that it produces the correct result.

```
Customer Segment =  
VAR CustomerRevenue = CALCULATE(SUM(Sales[Sales Amount]))  
RETURN  
    IF(CustomerRevenue < 2500, "Low", "High")
```

In the pie chart visual add the new calculated column to the Legend well, verify that two pie segments now display.



In this case, the `CALCULATE` function applies row context values as filters, known as *context transition*. To be accurate, the process doesn't quite work that way when a unique column is on

the table. When a unique column is on the table, you only need to apply a filter on that column to make the transition happen. In this case, Power BI applies a filter on the **CustomerKey** column for the value in row context.

If you reference measures in an expression that's evaluated in row context, context transition is automatic. Thus, you don't need to pass measure references to the CALCULATE function.

Modify the calculated column definition, which references the **Revenue** measure, and notice that it continues to produce the correct result.

```
Customer Segment =
VAR CustomerRevenue = [Revenue]
RETURN
    IF(CustomerRevenue < 2500, "Low", "High")
```

Now, you can complete the **Sales Commission** measure formula. To produce a total, you need to use an iterator function to iterate over all regions in filter context. The iterator function expression must use the CALCULATE function to transition the row context to the filter context. Notice that it no longer needs to test whether a single **Country** column value in the **Sales Territory** table is in filter context because it's known to be filtering by a single country/region (because it's iterating over the regions in filter context and a region belongs to only one country/region).

Modifying the **Sales Commission** measure definition to use the SUMX iterator function:

```
Sales Commission =
SUMX(
    VALUES('Sales Territory'[Region]),
    CALCULATE(
        [Revenue]
        * IF(
            VALUES('Sales Territory'[Country]) = "United States",
            0.15,
            0.1
        )
    )
)
```

The table visual now displays a sales commission total for all regions.

Region	Revenue	Sales Commission
Australia	\$10,655,335.96	\$1,065,533.60
Canada	\$16,355,770.46	\$1,635,577.05
Central	\$7,909,009.01	\$1,186,351.35
France	\$7,251,555.65	\$725,155.56
Germany	\$4,878,300.38	\$487,830.04
Northeast	\$6,939,374.48	\$1,040,906.17
Northwest	\$16,084,942.55	\$2,412,741.38
Southeast	\$7,879,655.07	\$1,181,948.26
Southwest	\$24,184,609.60	\$3,627,691.44
United Kingdom	\$7,670,721.04	\$767,072.10
Total	\$109,809,274.20	\$14,130,806.96

Use DAX Time Intelligence Functions in Power BI Desktop Models

Introduction

Time intelligence relates to calculations over time. Specifically, it relates to calculations over dates, months, quarters, or years, and possibly time. Rarely would you need to calculate over time in the sense of hours, minutes, or seconds.

In Data Analysis Expressions (DAX) calculations, time intelligence means *modifying the filter context for date filters*.

For example, at the Adventure Works company, their financial year begins on July 1 and ends on June 30 of the following year. They produce a table visual that displays monthly revenue and year-to-date (YTD) revenue.

Year	Revenue	Revenue YTD
FY2018	\$23,860,891.17	\$23,860,891.17
2017 Jul	\$1,423,357.32	\$1,423,357.32
2017 Aug	\$2,057,902.45	\$3,481,259.78
2017 Sep	\$2,523,947.55	\$6,005,207.32
2017 Oct	\$561,681.48	\$6,566,888.80
2017 Nov	\$4,764,920.16	\$11,331,808.96
2017 Dec	\$596,746.56	\$11,928,555.52
2018 Jan	\$1,327,674.63	\$13,256,230.15
2018 Feb	\$3,936,463.31	\$17,192,693.45
2018 Mar	\$700,873.18	\$17,893,566.64
2018 Apr	\$1,519,275.24	\$19,412,841.88
2018 May	\$2,960,378.09	\$22,373,219.97
2018 Jun	\$1,487,671.19	\$23,860,891.17
FY2019	\$34,070,108.50	\$34,070,108.50
2018 Jul	\$2,939,691.00	\$2,939,691.00
2018 Aug	\$3,964,801.20	\$6,904,492.20
2018 Sep	\$3,287,605.93	\$10,192,098.13

The filter context for **2017 August** contains each of the 31 dates of August, which are stored in the **Date** table. However, the calculated year-to-date revenue for **2017 August** applies a different filter context. It's the first date of the year through to the last date in filter context. In this example, that's July 1, 2017 through to August 31, 2017.

Time intelligence calculations modify date filter contexts. They can help you answer these time-related questions:

- What's the accumulation of revenue for the year, quarter, or month?
- What revenue was produced for the same period last year?
- What growth in revenue has been achieved over the same period last year?
- How many new customers made their first order in each month?
- What's the inventory stock on-hand value for the company's products?

Use DAX Time Intelligence Functions

DAX includes several time intelligence functions to simplify the task of modifying date filter context. You could write many of these intelligence formulas by using a `CALCULATE` function that modifies date filters, but that would create more work.

Many DAX time intelligence functions are concerned with standard date periods, specifically years, quarters, and months. If you have irregular time periods (for example, financial months that begin mid-way through the calendar month), or you need to work with weeks or time periods (hours, minutes, and so on), the DAX time intelligence functions won't be helpful. Instead, you'll need to use the `CALCULATE` function and pass in hand-crafted date or time filters.

Date table requirement

To work with time intelligence DAX functions, you need to meet the prerequisite model requirement of having at least one *date table* in your model. A date table is a table that meets the following requirements:

- It must have a column of data type Date (or date/time), known as the *date column*.
- The date column must contain unique values.
- The date column must not contain BLANKs.
- The date column must not have any missing dates.
- The date column must span full years. A year isn't necessarily a calendar year (January-December).
- The date table must be indicated as a date table.

Summarizations over time

One group of DAX time intelligence functions is concerned with summarizations over time:

- `DATESYTD` - Returns a single-column table that contains dates for the year-to-date (YTD) in the current filter context. This group also includes the `DATESMTD` and `DATESQTD` DAX functions for month-to-date (MTD) and quarter-to-date (QTD). You can pass these functions as filters into the `CALCULATE` DAX function.
- `TOTALYTD` - Evaluates an expression for YTD in the current filter context. The equivalent QTD and MTD DAX functions of `TOTALQTD` and `TOTALMTD` are also included.
- `DATESBETWEEN` - Returns a table that contains a column of dates that begins with a given start date and continues until a given end date.
- `DATESINPERIOD` - Returns a table that contains a column of dates that begins with a given start date and continues for the specified number of intervals.

While the `TOTALYTD` function is simple to use, you are limited to passing in one filter expression. If you need to apply multiple filter expressions, use the `CALCULATE` function and then pass the `DATESYTD` function in as one of the filter expressions.

For the `TOTALYTD` function, the syntax is as follows:

```
TOTALYTD(<expression>, <dates>, [, <filter>][, <year_end_date>])
```

The function requires an expression and, as is common to all time intelligence functions, a reference to the date column of a marked date table. Optionally, a single filter expression or the year-end date can be passed in (required only when the year doesn't finish on December 31).

Comparisons over time

Another group of DAX time intelligence functions is concerned with shifting time periods:

- **DATEADD** - Returns a table that contains a column of dates, shifted either forward or backward in time by the specified number of intervals from the dates in the current filter context.
- **PARALLELPERIOD** - Returns a table that contains a column of dates that represents a period that is parallel to the dates in the specified dates column, in the current filter context, with the dates shifted a number of intervals either forward in time or back in time.
- **SAMEPERIODLASTYEAR** - Returns a table that contains a column of dates that are shifted one year back in time from the dates in the specified dates column, in the current filter context.
- Many helper DAX functions for navigating backward or forward for specific time periods, all of which returns a table of dates. These helper functions include **NEXTDAY**, **NEXTMONTH**, **NEXTQUARTER**, **NEXTYEAR**, and **PREVIOUSDAY**, **PREVIOUSMONTH**, **PREVIOUSQUARTER**, and **PREVIOUSYEAR**.

The **Revenue YoY %** measure demonstrates a good use of DAX variables. The measure improves the readability of the formula and allows you to unit test part of the measure logic (by returning the **RevenuePriorYear** variable value).

Additionally, the measure is an optimal formula because it doesn't need to retrieve the prior year's revenue value twice. Having stored it once in a variable, the **RETURN** clause uses the variable value twice.

Additional Time Intelligence Calculations

Other DAX time intelligence functions exist that are concerned with returning a single date. You'll learn about these functions by applying them in two different scenarios.

The **FIRSTDATE** and the **LASTDATE** DAX functions return the first and last date in the current filter context for the specified column of dates.

Calculate new occurrences

Another use of time intelligence functions is to count new occurrences. The following example shows how you can calculate the number of new customers for a time period. A new customer is counted in the time period in which they made their first purchase.

Your first task is to add the following measure to the **Sales** table that counts the number of distinct customers *life-to-date* (LTD).

```
Customers LTD =  
VAR CustomersLTD =  
    CALCULATE(  
        DISTINCTCOUNT(Sales[CustomerKey]),  
        DATESBETWEEN(  
            'Date'[Date],  
            BLANK(),  
            MAX('Date'[Date])  
        ),  
        'Sales Order'[Channel] = "Internet"  
    )  
RETURN  
    CustomersLTD
```

Add the **Customers LTD** measure to the matrix visual. Notice that it produces a result of distinct customers LTD until the end of each month.

Year	Revenue	Revenue YTD	Revenue YoY %	Customers LTD
 FY2018	\$23,860,891.17	\$23,860,891.17		2,459
2017 Jul	\$1,423,357.32	\$1,423,357.32		289
2017 Aug	\$2,057,902.45	\$3,481,259.78		448
2017 Sep	\$2,523,947.55	\$6,005,207.32		609
2017 Oct	\$561,681.48	\$6,566,888.80		783
2017 Nov	\$4,764,920.16	\$11,331,808.96		1,013
2017 Dec	\$596,746.56	\$11,928,555.52		1,201
2018 Jan	\$1,327,674.63	\$13,256,230.15		1,394
2018 Feb	\$3,936,463.31	\$17,192,693.45		1,571
2018 Mar	\$700,873.18	\$17,893,566.64		1,790
2018 Apr	\$1,519,275.24	\$19,412,841.88		1,992
2018 May	\$2,960,378.09	\$22,373,219.97		2,214
2018 Jun	\$1,487,671.19	\$23,860,891.17		2,459

The DATESBETWEEN function returns a table that contains a column of dates that begins with a given start date and continues until a given end date. When the start date is BLANK, it will use the first date in the date column. (Conversely, when the end date is BLANK, it will use the last date in the date column.)

In this case, the end date is determined by the MAX function, which returns the last date in filter context. Therefore, if the month of August 2017 is in filter context, then the MAX function will return August 31, 2017 and the DATESBETWEEN function will return all dates through to August 31, 2017.

Snapshot calculations

Occasionally, fact data is stored as snapshots in time. Common examples include inventory stock levels or account balances. A snapshot of values is loaded into the table on a periodic basis.

When summarizing snapshot values (like inventory stock levels), you can summarize values across any dimension except date. Adding stock level counts across product categories produces a meaningful summary, but adding stock level counts across dates does not. Adding yesterday's stock level to today's stock level isn't a useful operation to perform (unless you want to average that result).

When you are summarizing snapshot tables, measure formulas can rely on DAX time intelligence functions to enforce a single date filter.

Optimise a Model for Performance in Power BI

Introduction

Performance optimization, also known as performance tuning, involves making changes to the current state of the semantic model so that it runs more efficiently. Essentially, when your semantic model is optimized, it performs better.

You might find that your report runs well in test and development environments, but when deployed to production for broader consumption, performance issues arise. From a report user's perspective, poor performance is characterized by report pages that take longer to load and visuals taking more time to update.

This poor performance results in a negative user experience.

As a data analyst, you will spend approximately 90 percent of your time working with your data, and nine times out of ten, poor performance is a direct result of a bad semantic model, bad Data Analysis Expressions (DAX), or the mix of the two.

The process of designing a semantic model for performance can be tedious, and it is often underestimated. However, if you address performance issues during development, you will have a robust Power BI semantic model that will return better reporting performance and a more positive user experience.

Ultimately, you will also be able to maintain optimized performance. As your organization grows, the size of its data grows, and its semantic model becomes more complex. By optimizing your semantic model early, you can mitigate the negative impact that this growth might have on the performance of your semantic model.

A smaller sized semantic model uses less resources (memory) and achieves faster data refresh, calculations, and rendering of visuals in reports. Therefore, the performance optimization process involves minimizing the size of the semantic model and making the most efficient use of the data in the model, which includes:

- Ensuring that the correct data types are used.
- Deleting unnecessary columns and rows.
- Avoiding repeated values.
- Replacing numeric columns with measures.
- Reducing cardinalities.
- Analysing model metadata.
- Summarizing data where possible.

Review Performance of Measures, Relationships, and Visuals

If your semantic model has multiple tables, complex relationships, intricate calculations, multiple visuals, or redundant data, a potential exists for poor report performance. The poor performance of a report leads to a negative user experience.

To optimize performance, you must first identify where the problem is coming from; in other words, find out which elements of your report and semantic model are causing the performance issues. Afterward, you can take action to resolve those issues and, therefore, improve performance.

Identify report performance bottlenecks

To achieve optimal performance in your reports, you need to create an efficient semantic model that has fast running queries and measures. When you have a good foundation, you can improve the model further by analyzing the query plans and dependencies and then making changes to further optimize performance.

You should review the measures and queries in your semantic model to ensure that you are using the most efficient way to get the results that you want. Your starting point should be to identify bottlenecks that exist in the code. When you identify the slowest query in the semantic model, you can focus on the biggest bottleneck first and establish a priority list to work through the other issues.

Analyze performance

You can use **Performance analyzer** in Power BI Desktop to help you find out how each of your report elements is performing when users interact with them.

For example, you can determine how long it takes for a particular visual to refresh when it is initiated by a user interaction.

Performance analyzer will help you identify the elements that are contributing to your performance issues, which can be useful during troubleshooting.

Before you run **Performance analyzer**, to ensure you get the most accurate results in your analysis (test), make sure that you start with a clear visual cache and a clear data engine cache.

- **Visual cache** - When you load a visual, you can't clear this visual cache without closing Power BI Desktop and opening it again.
- To avoid any caching in play, you need to start your analysis with a clean visual cache.

To ensure that you have a clear visual cache, add a blank page to your Power BI Desktop (.pbix) file and then, with that page selected, save and close the file.

Reopen the Power BI Desktop (.pbix) file that you want to analyze. It will open on the blank page.

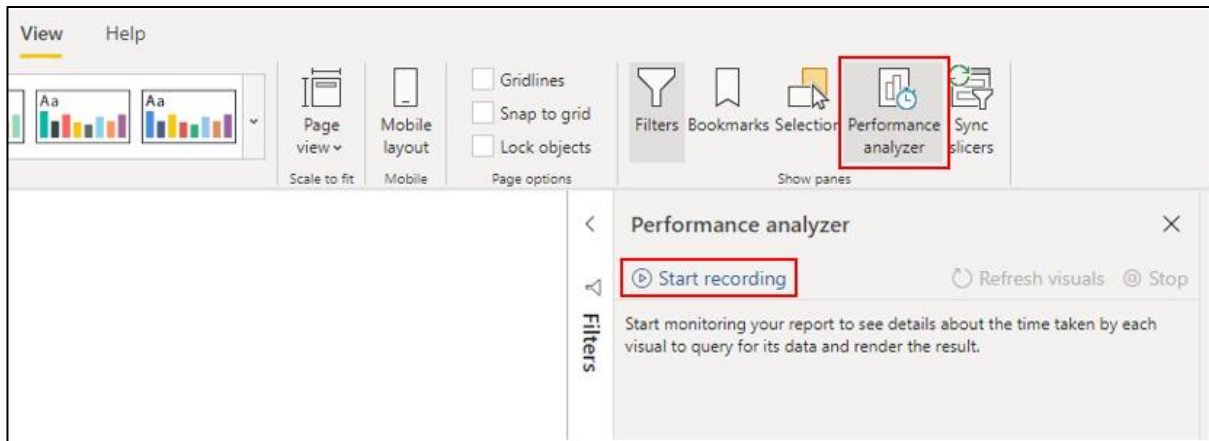
- **Data engine cache** - When a query is run, the results are cached, so the results of your analysis will be misleading. You need to clear the data cache before rerunning the visual.

To clear the data cache, you can either restart Power BI Desktop or connect DAX Studio to the semantic model and then call Clear Cache.

When you have cleared the caches and opened the Power BI Desktop file on the blank page, go to the **View** tab and select the **Performance analyzer** option.

To begin the analysis process, select **Start recording**, select the page of the report that you want to analyze, and interact with the elements of the report that you want to measure.

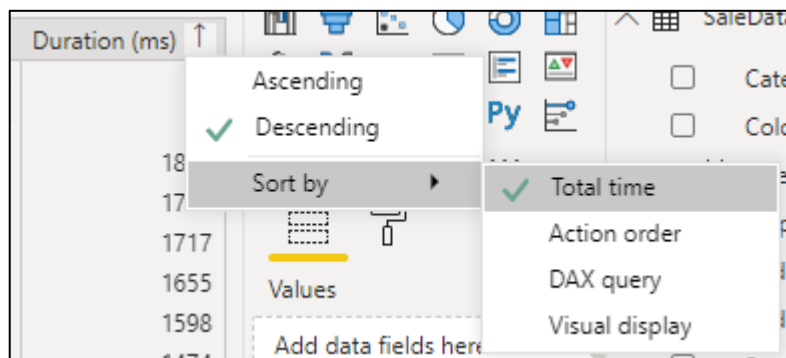
You will see the results of your interactions display in the **Performance analyzer** pane as you work. When you are finished, select the **Stop** button.



Review results

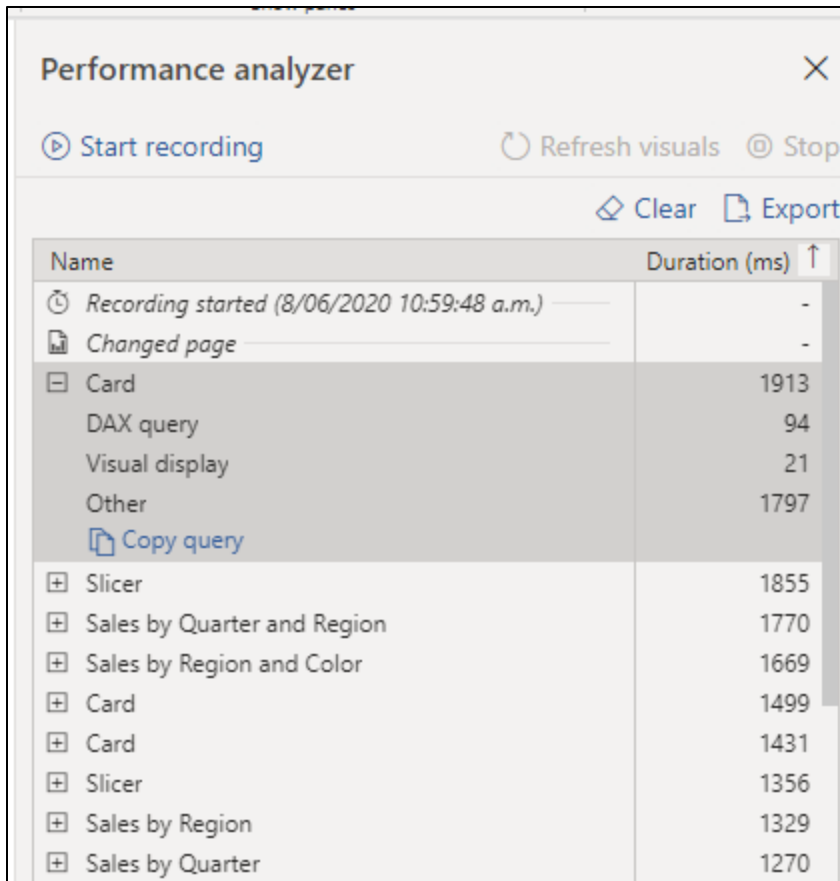
You can review the results of your performance test in the **Performance analyzer** pane.

To review the tasks in order of duration, longest to shortest, right-click the **Sort** icon next to the **Duration (ms)** column header, and then select **Total time** in **Descending** order.



The log information for each visual shows how much time it took (duration) to complete the following categories of tasks:

- **DAX query** - The time it took for the visual to send the query, along with the time it took Analysis Services to return the results.
- **Visual display** - The time it took for the visual to render on the screen, including the time required to retrieve web images or geocoding.
- **Other** - The time it took the visual to prepare queries, wait for other visuals to complete, or perform other background processing tasks.
 - If this category displays a long duration, the only real way to reduce this duration is to optimize DAX queries for other visuals, or reduce the number of visuals in the report.



Name	Duration (ms)
Recording started (8/06/2020 10:59:48 a.m.)	-
Changed page	-
Card	1913
DAX query	94
Visual display	21
Other	1797
Copy query	
Slicer	1855
Sales by Quarter and Region	1770
Sales by Region and Color	1669
Card	1499
Card	1431
Slicer	1356
Sales by Region	1329
Sales by Quarter	1270

The results of the analysis test help you to understand the behaviour of your semantic model and identify the elements that you need to optimize.

You can compare the duration of each element in the report and identify the elements that have a long duration.

You should focus on those elements and investigate why it takes them so long to load on the report page.

To analyze your queries in more detail, you can use DAX Studio, which is a free, open-source tool that is provided by another service.

Resolve issues and optimize performance

The results of your analysis will identify areas for improvement and opportunities for performance optimization. You might find that you need to carry out improvements to the visuals, the DAX query, or other elements in your semantic model. The following information provides guidance on what to look for and the changes that you can make.

Visuals

If you identify visuals as the bottleneck leading to poor performance, you should find a way to improve performance with minimal impact to user experience.

Consider the number of visuals on the report page; fewer visuals means better performance. Ask yourself if a visual is really necessary and if it adds value to the end user. If the answer is no, you should remove that visual.

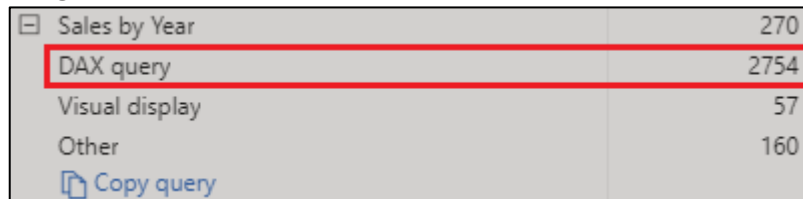
Rather than using multiple visuals on the page, consider other ways to provide additional details, such as drill-through pages and report page tooltips.

Examine the number of fields in each visual. The more visuals you have on the report, the higher chance for performance issues. In addition, the more visuals, the more the report can appear crowded and lose clarity.

The upper limit for visuals is 100 fields (measures or columns), so a visual with more than 100 fields will be slow to load. Ask yourself if you really need all of this data in a visual. You might find that you can reduce the number of fields that you currently use.

DAX query

When you examine the results in the **Performance analyzer** pane, you can see how long it took the Power BI Desktop engine to evaluate each query (in milliseconds). A good starting point is any DAX query that is taking longer than 120 milliseconds. In this example, you identify one particular query that has a large duration time.



Sales by Year	270
DAX query	2754
Visual display	57
Other	160

Performance analyzer highlights potential issues but does not tell you what needs to be done to improve them. You might want to conduct further investigation into why this measure takes so long to process. You can use DAX Studio to investigate your queries in more detail.

Semantic model

If the duration of measures and visuals are displaying low values (in other words they have a short duration time), they are not the reason for the performance issues. Instead, if the DAX query is displaying a high duration value, it is likely that a measure is written poorly or an issue has occurred with the semantic model. The issue might be caused by the relationships, columns, or metadata in your model, or it could be the status of the **Auto date/time** option, as explained in the following section.

Relationships

You should review the relationships between your tables to ensure that you have established the correct relationships. Check that relationship cardinality properties are correctly configured. For example, a one-side column that contains unique values might be incorrectly configured as a many-side column. You will learn more about how cardinality affects performance later in this module.

Columns

It is best practice to not import columns of data that you do not need. To avoid deleting columns in Power Query Editor, you should try to deal with them at the source when loading data into Power BI Desktop. However, if it is impossible to remove redundant columns from the source query or the data has already been imported in its raw state, you can always use Power Query Editor to examine each column.

Ask yourself if you really need each column and try to identify the benefit that each one adds to your semantic model. If you find that a column adds no value, you should remove it from your semantic model. For example, suppose that you have an ID column with thousands of unique rows. You know that you won't use this particular column in a relationship, so it will not be used in a report. Therefore, you should consider this column as unnecessary and admit that it is wasting space in your semantic model.

When you remove an unnecessary column, you will reduce the size of the semantic model which, in turn, results in a smaller file size and faster refresh time. Also, because the semantic model contains only relevant data, the overall report performance will be improved.

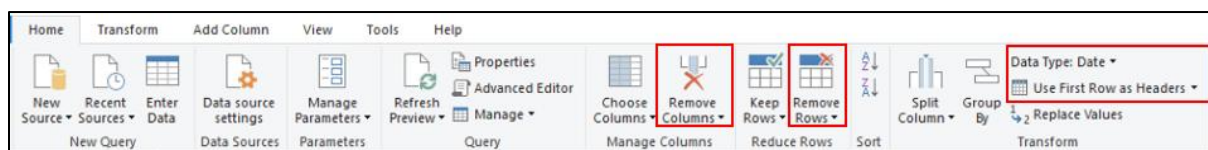
Metadata

Metadata is information about other data. Power BI metadata contains information on your semantic model, such as the name, data type and format of each of the columns, the schema of

the database, the report design, when the file was last modified, the data refresh rates, and much more.

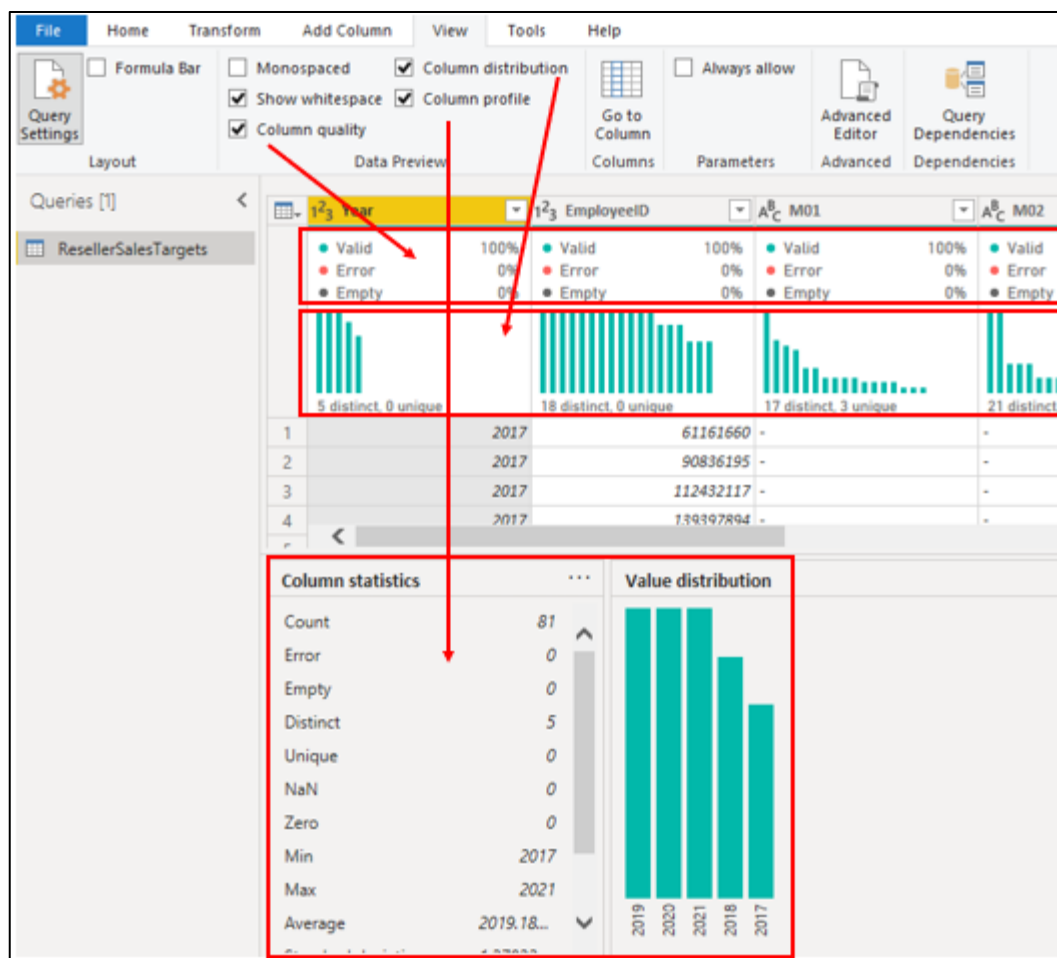
When you load data into Power BI Desktop, it is good practice to analyze the corresponding metadata so you can identify any inconsistencies with your semantic model and normalize the data before you start to build reports. Running analysis on your metadata will improve semantic model performance because, while analyzing your metadata, you will identify unnecessary columns, errors within your data, incorrect data types, the volume of data being loaded (large semantic models, including transactional or historic data, will take longer to load), and much more.

You can use Power Query Editor in Power BI Desktop to examine the columns, rows, and values of the raw data. You can then use the available tools, such as those highlighted in the following screenshot, to make the necessary changes.



The Power Query options include:

- **Unnecessary columns** - Evaluates the need for each column. If one or more columns will not be used in the report and are therefore unnecessary, you should remove them by using the **Remove Columns** option on the **Home** tab.
- **Unnecessary rows** - Checks the first few rows in the semantic model to see if they are empty or if they contain data that you do not need in your reports; if so, it removes those rows by using the **Remove Rows** option on the **Home** tab.
- **Data type** - Evaluates the column data types to ensure that each one is correct. If you identify a data type that is incorrect, change it by selecting the column, selecting **Data Type** on the **Transform** tab, and then selecting the correct data type from the list.
- **Query names** - Examines the query (table) names in the **Queries** pane. Just like you did for column header names, you should change uncommon or unhelpful query names to names that are more obvious or names that the user is more familiar with.
 - You can rename a query by right-clicking that query, selecting **Rename**, editing the name as required, and then pressing **Enter**.
- **Column details** - Power Query Editor has the following three data preview options that you can use to analyze the metadata that is associated with your columns. You can find these options on the **View** tab, as illustrated in the following screenshot.
 - **Column quality** - Determines what percentage of items in the column are valid, have errors, or are empty.
 - If the Valid percentage is not 100, you should investigate the reason, correct the errors, and populate empty values.
 - **Column distribution** - Displays frequency and distribution of the values in each of the columns.
 - **Column profile** - Shows column statistics chart and a column distribution chart.



If you are reviewing a large semantic model with more than 1,000 rows, and you want to analyze that whole semantic model, you need to change the default option at the bottom of the window.

Select **Column profiling based on top 1000 rows** > **Column profiling based on entire data set**.



Other metadata that you should consider is the information about the semantic model as a whole, such as the file size and data refresh rates. You can find this metadata in the associated Power BI Desktop (.pbix) file. The data that you load into Power BI Desktop is compressed and stored to the disk by the VertiPaq storage engine. The size of your semantic model has a direct impact on its performance; a smaller sized semantic model uses less resources (memory) and achieves faster data refresh, calculations, and rendering of visuals in reports.

Auto date/time feature

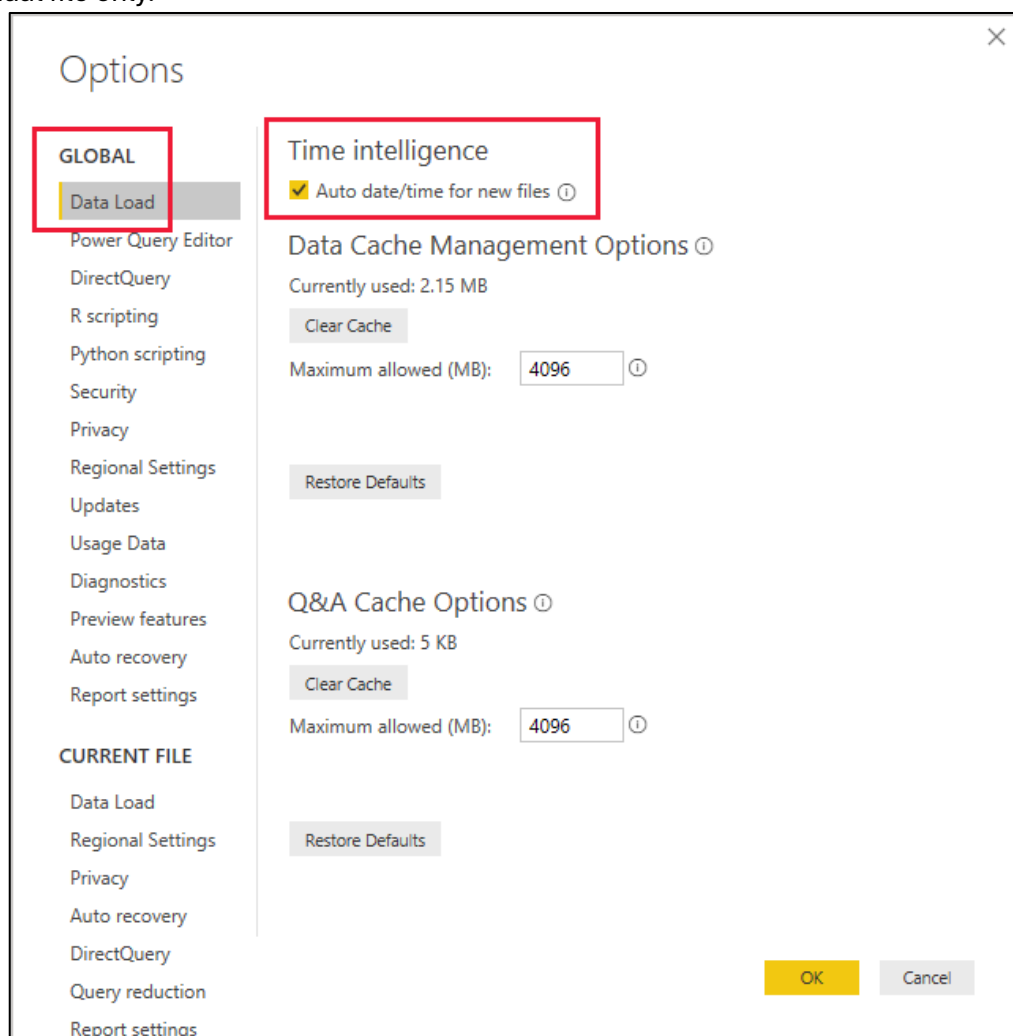
Another item to consider when optimizing performance is the **Auto date/time** option in Power BI Desktop. By default, this feature is enabled globally, which means that Power BI Desktop automatically creates a hidden calculated table for each date column, provided that certain

conditions are met. The new, hidden tables are in addition to the tables that you already have in your semantic model.

The **Auto date/time** option allows you to work with time intelligence when filtering, grouping, and drilling down through calendar time periods. We recommend that you keep the **Auto date/time** option enabled only when you work with calendar time periods and when you have simplistic model requirements in relation to time.

If your data source already defines a date dimension table, that table should be used to consistently define time within your organization, and you should disable the global **Auto date/time** option. Disabling this option can lower the size of your semantic model and reduce the refresh time.

You can enable/disable this **Auto date/time** option globally so that it applies to all of your Power BI Desktop files, or you can enable/disable the option for the current file so that it applies to an individual file only.



To enable/disable this **Auto date/time** option, go to **File > Options and settings > Options**, and then select either the **Global** or **Current File** page. On either page, select **Data Load** and then, in the **Time Intelligence** section, select or clear the check box as required.

Use Variables to Improve Performance and Troubleshooting

You can use variables in your DAX formulas to help you write less complex and more efficient calculations. Variables are underused by developers who are starting out in Power BI Desktop, but they are effective and you should use them by default when you are creating measures. Some expressions involve the use of many nested functions and the reuse of expression logic.

These expressions take a longer time to process and are difficult to read and, therefore, troubleshoot. If you use variables, you can save query processing time. This change is a step in the right direction toward optimizing the performance of a semantic model.

The use of variables in your semantic model provides the following advantages:

- **Improved performance** - Variables can make measures more efficient because they remove the need for Power BI to evaluate the same expression multiple times. You can achieve the same results in a query in about half the original processing time.
- **Improved readability** - Variables have short, self-describing names and are used in place of an ambiguous, multi-worded expression. You might find it easier to read and understand the formulas when variables are used.
- **Simplified debugging** - You can use variables to debug a formula and test expressions, which can be helpful during troubleshooting.
- **Reduced complexity** - Variables do not require the use of EARLIER or EARLIEST DAX functions, which are difficult to understand. These functions were required before variables were introduced, and were written in complex expressions that introduced new filter contexts. Now that you can use variables instead of those functions, you can write fewer complex formulas.

Use variables to improve performance

To illustrate how you can use a variable to make a measure more efficient, the following table displays a measure definition in two different ways. Notice that the formula repeats the expression that calculates "same period last year" but in two different ways: the first instance uses the normal DAX calculation method and the second one uses variables in the calculation. The second row of the table shows the improved measure definition.

This definition uses the VAR keyword to introduce a variable named **SalesPriorYear**, and it uses an expression to assign the "same period last year" result to that new variable. It then uses the variable twice in the DIVIDE expression.

Without variable

Sales YoY Growth =

```
DIVIDE (
    ( [Sales] - CALCULATE ( [Sales], PARALLELPERIOD ( 'Date'[Date], -12, MONTH ) ) ),
    CALCULATE ( [Sales], PARALLELPERIOD ( 'Date'[Date], -12, MONTH ) )
)
```

With variable

Sales YoY Growth =

```
VAR SalesPriorYear =
    CALCULATE ( [Sales], PARALLELPERIOD ( 'Date'[Date], -12, MONTH ) )
VAR SalesVariance =
    DIVIDE ( ( [Sales] - SalesPriorYear ), SalesPriorYear )
RETURN
    SalesVariance
```

In the first measure definition in the table, the formula is inefficient because it requires Power BI to evaluate the same expression twice. The second definition is more efficient because, due to the variable, Power BI only needs to evaluate the **PARALLELPERIOD** expression once.

If your semantic model has multiple queries with multiple measures, the use of variables could cut the overall query processing time in half and improve the overall performance of the semantic model. Furthermore, this solution is a simple one; imagine the savings as the formulas get more complicated, for instance, when you are dealing with percentages and running totals.

Use variables to improve readability

In addition to improved performance, you might notice how the use of variables makes the code simpler to read.

When using variables, it is best practice to use descriptive names for the variables. In the previous example, the variable is called **SalesPriorYear**, which clearly states what the variable is calculating. Consider the outcome of using a variable that was called **X**, **temp** or **variable1**; the purpose of the variable would not be clear at all.

Using clear, concise, meaningful names will help make it easier for you to understand what you are trying to calculate, and it will be much simpler for other developers to maintain the report in the future.

Use variables to troubleshoot multiple steps

You can use variables to help you debug a formula and identify what the issue is. Variables help simplify the task of troubleshooting your DAX calculation by evaluating each variable separately and by recalling them after the **RETURN** expression.

In the following example, you test an expression that is assigned to a variable. In order to debug you temporarily rewrite the **RETURN** expression to write to the variable. The measure definition returns only the **SalesPriorYear** variable because that is what comes after the **RETURN** expression.

```
Sales YoY Growth % =  
VAR SalesPriorYear = CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))  
VAR SalesPriorYear% = DIVIDE((([Sales] - SalesPriorYear), SalesPriorYear)  
RETURN SalesPriorYear%
```

The **RETURN** expression will display the **SalesPriorYear%** value only. This technique allows you to revert the expression when you have completed the debugging. It also makes calculations simpler to understand due to reduced complexity of the DAX code.

Reduce Cardinality

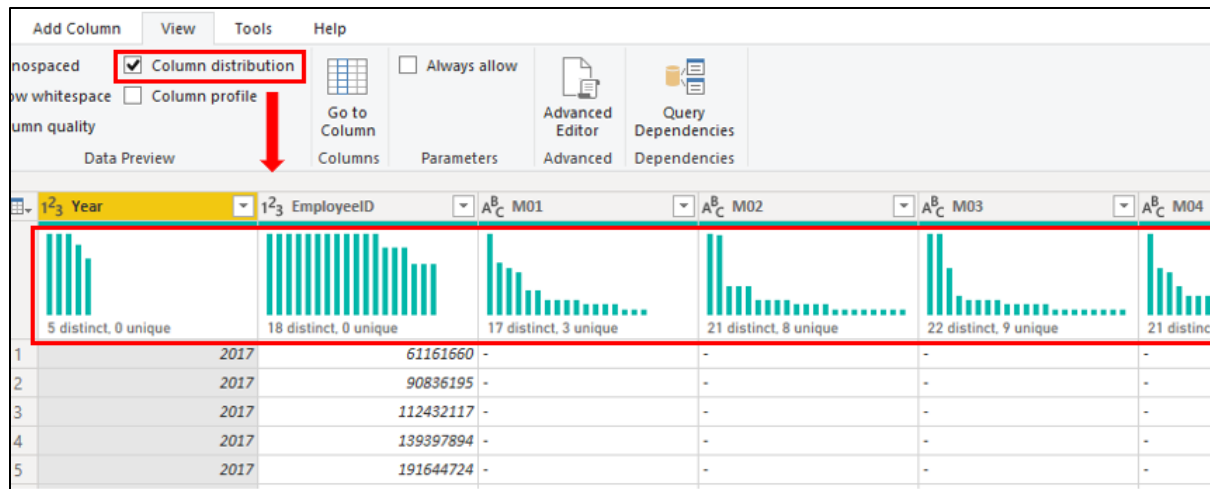
Cardinality is a term that is used to describe the uniqueness of the values in a column. Cardinality is also used in the context of the relationships between two tables, where it describes the direction of the relationship.

Identify cardinality levels in columns

Previously, when you used Power Query Editor to analyze the metadata, the **Column distribution** option on the **View** tab displayed statistics on how many distinct and unique items were in each column in the data.

- **Distinct values count** - The total number of different values found in a given column.

- **Unique values count** - The total number of values that only appear once in a given column.



A column that has a lot of repeated values in its range (unique count is low) will have a low level of cardinality. Conversely, a column that has a lot of unique values in its range (unique count is high) will have a high level of cardinality.

Lower cardinality leads to more optimized performance, so you might need to reduce the number of high cardinality columns in your semantic model.

Reduce relationship cardinality

When you import multiple tables, it is possible that you'll do some analysis by using data from all those tables. Relationships between those tables are necessary to accurately calculate results and display the correct information in your reports.

Power BI Desktop helps make creating those relationships easier. In fact, in most cases, you won't have to do anything, the autodetect feature does it for you. However, you might occasionally have to create relationships or need to make changes to a relationship. Regardless, it's important to understand relationships in Power BI Desktop and how to create and edit them.

When you create or edit a relationship, you can configure additional options. By default, Power BI Desktop automatically configures additional options based on its best guess, which can be different for each relationship based on the data in the columns.

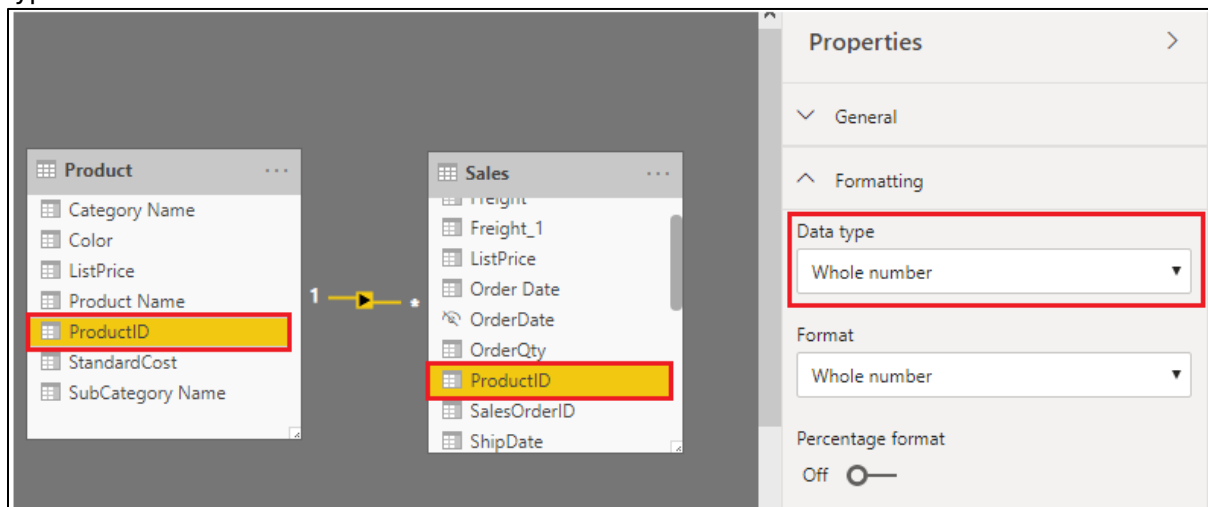
The relationships can have different cardinality. Cardinality is the direction of the relationship, and each model relationship must be defined with a cardinality type. The cardinality options in Power BI are:

- **Many-to-one (*:1)** - This relationship is the most common, default type. It means that the column in one table can have more than one instance of a value, and the other related table, often known as the lookup table, has only one instance of a value.
- **One-to-one (1:1)** - In this relationship type, the column in one table has only one instance of a particular value, and the other related table has only one instance of a particular value.
- **One-to-many (1:*)** - In this relationship type, the column in one table has only one instance of a particular value, and the other related table can have more than one instance of a value.

- **Many-to-many (:)** - With composite models, you can establish a many-to-many relationship between tables, which removes requirements for unique values in tables. It also removes previous workarounds, such as introducing new tables only to establish relationships.

During development, you are creating and editing relationships in your model, so when you are building new relationships in your model, regardless of what cardinality you have chosen, always ensure that both of the columns that you are using to participate in a relationship are sharing the same data type. Your model will never work if you try to build a relationship between two columns, where one column has a text data type and another column has an integer data type.

In the following example, the **ProductID** field has the data type **Whole number** in the Product and Sales tables. The columns with data type **Integer** perform better than columns with data type **Text**.



Improve performance by reducing cardinality levels

Power BI Desktop offers different techniques that you can use to help reduce the data that is loaded into semantic models, such as summarization. Reducing the data that is loaded into your model will improve the relationship cardinality of the report. For this reason, it is important that you strive to minimize the data that will be loaded into your models. This case is especially true for large models, or models that you anticipate will grow to become large over time.

Perhaps the most effective technique to reduce a model size is to use a summary table from the data source. Where a detail table might contain every transaction, a summary table would contain one record per day, per week, or per month. It might be an average of all of the transactions per day, for instance.

For example, a source sales fact table stores one row for each order line. Significant data reduction could be achieved by summarizing all sales metrics if you group by date, customer, and product, and individual transaction detail is not needed.

Consider then that an even more significant data reduction could be achieved by grouping by date at month level. It could achieve a possible 99 percent reduction in model size; but, reporting at day level or an individual order level is no longer possible.

Deciding to summarize fact-type data will always involve a tradeoff with the detail of your data. A disadvantage is that you might lose the ability to drill into data because the detail no longer exists. This tradeoff could be mitigated by using a mixed model design.

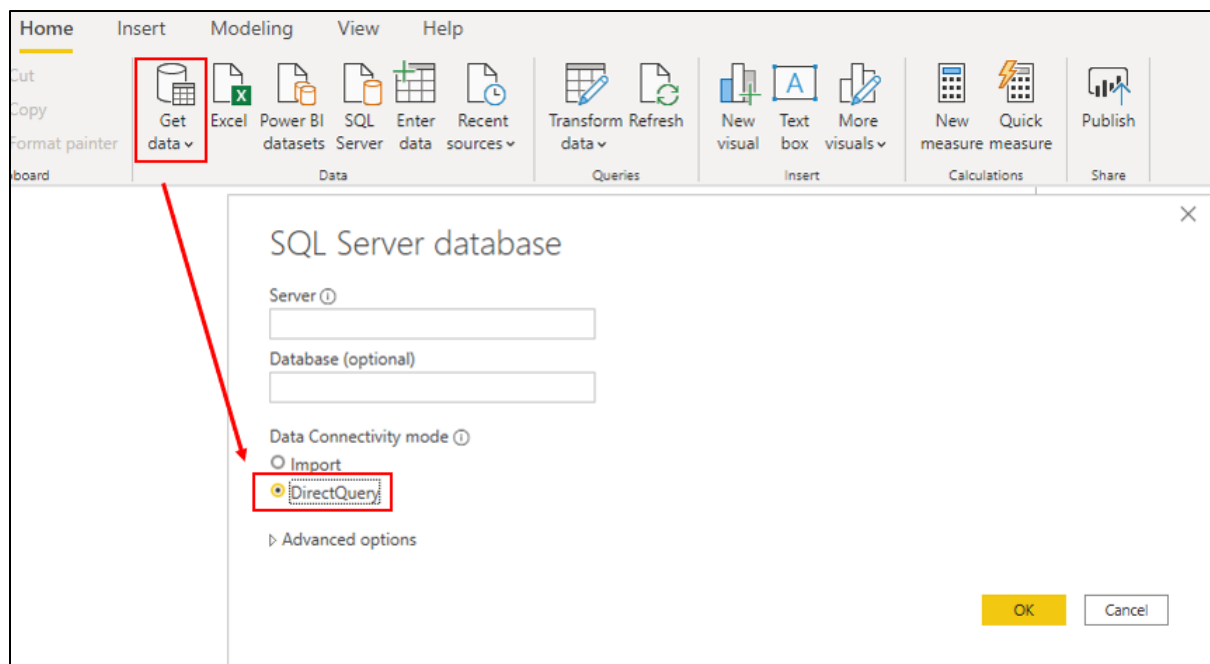
In Power BI Desktop, a Mixed mode design produces a composite model. Essentially, it allows you to determine a storage mode for each table. Therefore, each table can have its **Storage Mode** property set as **Import** or **DirectQuery**.

An effective technique to reduce the model size is to set the **Storage Mode** property for larger fact-type tables to **DirectQuery**. This design approach can work well in conjunction with techniques that are used to summarize your data. For example, the summarized sales data could be used to achieve high performance "summary" reporting.

A drill-through page could be created to display granular sales for specific (and narrow) filter context, displaying all in-context sales orders. The drill-through page would include visuals based on a DirectQuery table to retrieve the sales order data (sales order details).

Optimize DirectQuery Models with Table Level Storage

DirectQuery is one way to get data into Power BI Desktop. The DirectQuery method involves connecting directly to data in its source repository from within Power BI Desktop. It is an alternative to importing data into Power BI Desktop.



When you use the DirectQuery method, the overall user experience depends heavily on the performance of the underlying data source. Slow query response times will lead to a negative user experience and, in the worst-case scenarios, queries might time out. Also, the number of users who are opening the reports at any one time will impact the load that is placed on the data source.

For example, if your report has 20 visuals in it and 10 people are using the report, 200 queries or more will exist on the data source because each visual will issue one or more queries.

Unfortunately, the performance of your Power BI model will not only be impacted by the performance of the underlying data source, but also by other uncontrollable factors, such as:

- Network latency; faster networks return data quicker.
- The performance of the data source's server and how many other workloads are on that server. For example, consider the implications of a server refresh taking place while hundreds of people are using the same server for different reasons.

Therefore, using DirectQuery poses a risk to the quality of your model's performance. To optimize performance in this situation, you need to have control over, or access to, the source database.

Implications of using DirectQuery

It is best practice to import data into Power BI Desktop, but your organization might need to use the DirectQuery data connectivity mode because of one of the following reasons (benefits of DirectQuery):

- It is suitable in cases where data changes frequently and near real-time reporting is required.
- It can handle large data without the need to pre-aggregate.
- It applies data sovereignty restrictions to comply with legal requirements.
- It can be used with a multidimensional data source that contains measures such as SAP Business Warehouse (BW).

If your organization needs to use DirectQuery, you should clearly understand its behavior within Power BI Desktop and be aware of its limitations. You will then be in a good position to take action to optimize the DirectQuery model as much as possible.

Behaviour of DirectQuery connections

When you use DirectQuery to connect to data in Power BI Desktop, that connection behaves in the following way:

- When you initially use the **Get Data** feature in Power BI Desktop, you will select the source. If you connect to a relational source, you can select a set of tables and each one will define a query that logically returns a set of data. If you select a multidimensional source, such as SAP BW, you can only select the source.
- When you load the data, no data is imported into the Power BI Desktop, only the schema is loaded. When you build a visual within Power BI Desktop, queries are sent to the underlying source to retrieve the necessary data. The time it takes to refresh the visual depends on the performance of the underlying data source.
- If changes are made to the underlying data, they won't be immediately reflected in the existing visuals in Power BI due to caching. You need to carry out a refresh to see those changes. The necessary queries are present for each visual, and the visuals are updated accordingly.
- When you publish the report to the Power BI service, it will result in a semantic model in Power BI service, the same as for import. However, no data is included with that semantic model.
- When you open an existing report in Power BI service, or build a new one, the underlying source is again queried to retrieve the necessary data. Depending on the location of the original source, you might have to configure an on-premises data gateway.
- You can pin visuals, or entire report pages, as dashboard tiles. The tiles are automatically refreshed on a schedule, for example, every hour. You can control the frequency of this refresh to meet your requirements. When you open a dashboard, the tiles reflect the data at the time of the last refresh and might not include the latest changes that are made to

the underlying data source. You can always refresh an open dashboard to ensure that it's up-to-date.

Limitations of DirectQuery connections

The use of DirectQuery can have negative implications. The limitations vary, depending on the specific data source that is being used. You should take the following points into consideration:

- **Performance** - As previously discussed, your overall user experience depends heavily on the performance of the underlying data source.
- **Security** - If you use multiple data sources in a DirectQuery model, it is important to understand how data moves between the underlying data sources and the associated security implications. You should also identify if security rules are applicable to the data in your underlying source because, in Power BI, every user can see that data.
- **Data transformation** - Compared to imported data, data that is sourced from DirectQuery has limitations when it comes to applying data transformation techniques within Power Query Editor. For example, if you connect to an OLAP source, such as SAP BW, you can't make any transformations at all; the entire external model is taken from the data source. If you want to make any transformations to the data, you will need to do this in the underlying data source.
- **Modeling** - Some of the modeling capabilities that you have with imported data aren't available, or are limited, when you use DirectQuery.
- **Reporting** -- Almost all the reporting capabilities that you have with imported data are also supported for DirectQuery models, provided that the underlying source offers a suitable level of performance. However, when the report is published in Power BI service, the Quick Insights and Q&A features are not supported. Also, the use of the Explore feature in Excel will likely result in poorer performance.

Optimize performance

Continuing with the Tailwind Traders scenario, during your review of the semantic model, you discover that the query used DirectQuery to connect Power BI Desktop to the source data. This use of DirectQuery is the reason why users are experiencing poor report performance. It's taking too long to load the pages in the report, and tables are not refreshing quickly enough when certain selections are made. You need to take action to optimize the performance of the DirectQuery model.

You can examine the queries that are being sent to the underlying source and try to identify the reason for the poor query performance. You can then make changes in Power BI Desktop and the underlying data source to optimize overall performance.

Optimize data in Power BI Desktop

When you have optimized the data source as much as possible, you can take further action within Power BI Desktop by using **Performance analyzer**, where you can isolate queries to validate query plans.

You can analyze the duration of the queries that are being sent to the underlying source to identify the queries that are taking a long time to load. In other words, you can identify where the bottlenecks exist.

You don't need to use a special approach when optimizing a DirectQuery model; you can apply the same optimization techniques that you used on the imported data to tune the data from the DirectQuery source. For example, you can reduce the number of visuals on the report page or

reduce the number of fields that are used in a visual. You can also remove unnecessary columns and rows.

Optimize the underlying data source (connected database)

Your first stop is the data source. You need to tune the source database as much as possible because anything you do to improve the performance of that source database will in turn improve Power BI DirectQuery.

The actions that you take in the database will do the best.

Consider the use of the following standard database practices that apply to most situations:

- Avoid the use of complex calculated columns because the calculation expression will be embedded into the source queries.
 - It is more efficient to push the expression back to the source because it avoids the push down. You could also consider adding surrogate key columns to dimension-type tables.
- Review the indexes and verify that the current indexing is correct. If you need to create new indexes, ensure that they are appropriate.

Customize the Query reduction options

Power BI Desktop gives you the option to send fewer queries and to disable certain interactions that will result in a poor experience if the resulting queries take a long time to run.

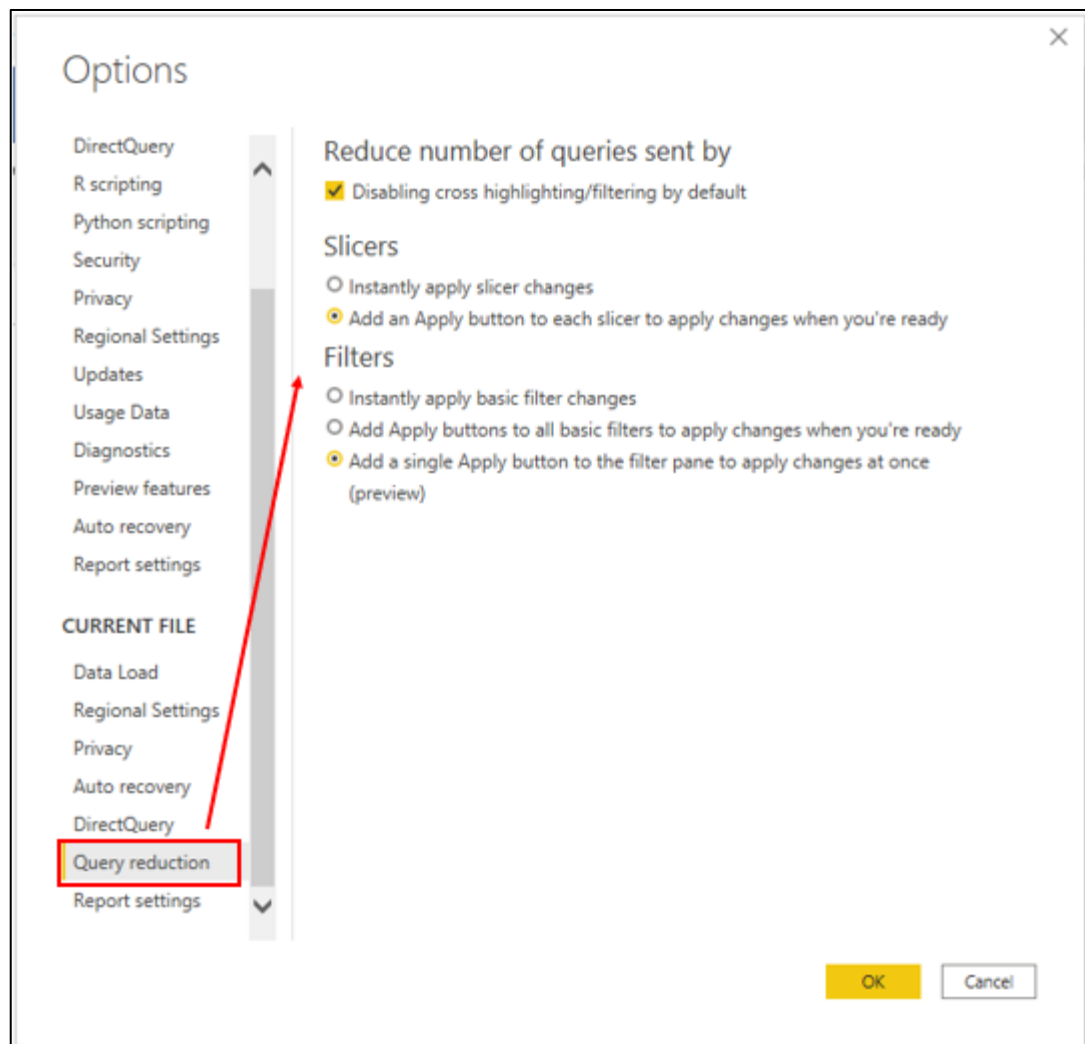
Applying these options prevents queries from continuously hitting the data source, which should improve performance.

In this example, you edit the default settings to apply the available data reduction options to your model.

You access the settings by selecting **File > Options and settings > Options**, scrolling down the page, and then selecting the **Query reduction** option.

The following query reduction options are available:

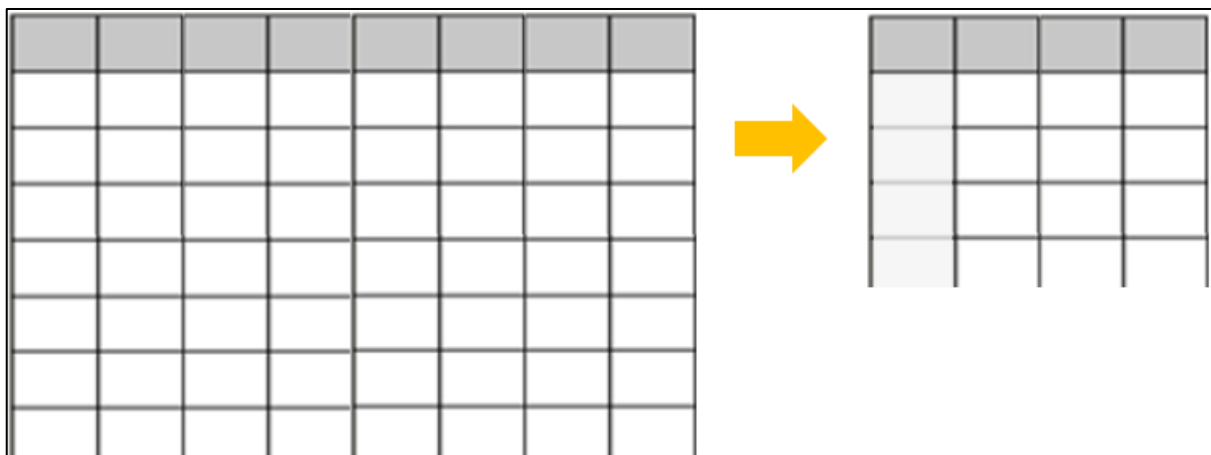
- **Reduce number of queries sent by** - By default, every visual interacts with every other visual. Selecting this check box disables that default interaction.
 - You can then optionally choose which visuals interact with each other by using the **Edit interactions** feature.
- **Slicers** - By default, the **Instantly apply slicer changes** option is selected. To force the report users to manually apply slicer changes, select the **Add an apply button to each slicer to apply changes when you're ready** option.
- **Filters** - By default, the **Instantly apply basic filter changes** option is selected.
 - To force the report users to manually apply filter changes, select one of the alternative options:
 - **Add an apply button to all basic filters to apply changes when you're ready**
 - **Add a single apply button to the filter pane to apply changes at once (preview)**



Create and Manage Aggregations

When aggregating data, you summarize that data and present it in at a higher grain (level). For example, you can summarize all sales data and group it by date, customer, product, and so on.

The aggregation process reduces the table sizes in the semantic model, allowing you to focus on important data and helping to improve the query performance.



Your organization might decide to use aggregations in their semantic models for the following reasons:

- If you are dealing with a large amount of data (big data), aggregations will provide better query performance and help you analyze and reveal the insights of this large data. Aggregated data is cached and, therefore, uses a fraction of the resources that are required for detailed data.
- If you are experiencing a slow refresh, aggregations will help you speed up the refresh process. The smaller cache size reduces the refresh time, so data gets to users faster. Instead of refreshing what could be millions of rows, you would refresh a smaller amount of data instead.
- If you have a large semantic model, aggregations can help you reduce and maintain the size of your model.
- If you anticipate your semantic model growing in size in the future, you can use aggregations as a proactive step toward future proofing your semantic model by lessening the potential for performance and refresh issues and overall query problems.

Create aggregations

Before you start creating aggregations, you should decide on the grain (level) on which you want to create them. In this example, you want to aggregate the sales data at the day level.

When you decide on the grain, the next step is to decide on how you want to create the aggregations. You can create aggregations in different ways and each method will yield the same results, for example:

- If you have access to the database, you could create a table with the aggregation and then import that table into Power BI Desktop.
- If you have access to the database, you could create a view for the aggregation and then import that view into Power BI Desktop.
- In Power BI Desktop, you can use Power Query Editor to create the aggregations step-by-step.

Automatic aggregations are both self-training and self-optimizing. They enable model owners of any skill level to improve query performance, providing faster report visualizations for large models.

With automatic aggregations:

- Report visualizations are faster - An optimal percentage of report queries are returned by an automatically maintained in-memory aggregations cache instead of backend data source systems. Outlier queries that aren't returned by the in-memory cache are passed directly to the data source using DirectQuery.
- Balanced architecture - When compared to pure DirectQuery mode, most query results are returned by the Power BI query engine and in-memory aggregations cache. Query processing load on data source systems at peak reporting times can be significantly reduced, which means increased scalability in the data source backend.
- Easy setup - Model owners can enable automatic aggregations training and schedule one or more refreshes for the model. With the first training and refresh, automatic aggregations begin creating an aggregations framework and optimal aggregations. The system automatically tunes itself over time.
- Fine-tuning - With a simple and intuitive user interface in the model settings, you can estimate the performance gains for a different percentage of queries returned from the in-memory aggregations cache and make adjustments for even greater gains. A single slide bar control helps you easily fine-tune your environment.

Build Power BI Visuals and Reports

Scope Report Design Requirements

Introduction

As with any project, a good starting point when designing reports is to define clear goals. In your reporting project, those goals should strive to help you determine your reporting requirements for:

- Audiences
- Report type
- User interface requirements
- User experience requirements

Identify the Audience

Identifying the audience is one of the most important steps in the report design process. It enables the report author to create a final result that can be efficiently used and will meet the needs of the report consumer.

The three broad report consumer audiences are:

- Executive
- Analyst
- Information worker

An **executive** is a person who is charged with making plans and decisions that often involve a medium or long-term focus. Executives are responsible for making the business run smoothly. For example, the C-level executives at the Contoso Skateboard Store would be an executive audience.

An **analyst** is a person who provides guidance to the organization. Analysts can be responsible for a range of tasks, often with goals of determining the effectiveness of business strategies, developing or improving processes, or implementing change. A business analyst (or data analyst) in the sales division of the Contoso Skateboard Store is an example of an analyst audience.

An **information worker** is someone who uses data to help make decisions or take actions. Often, these decisions and actions are operational in that they are done on a daily basis. The inventory manager at the Contoso Skateboard Store, who needs up-to-date information about stock levels, is an example of an information worker audience.

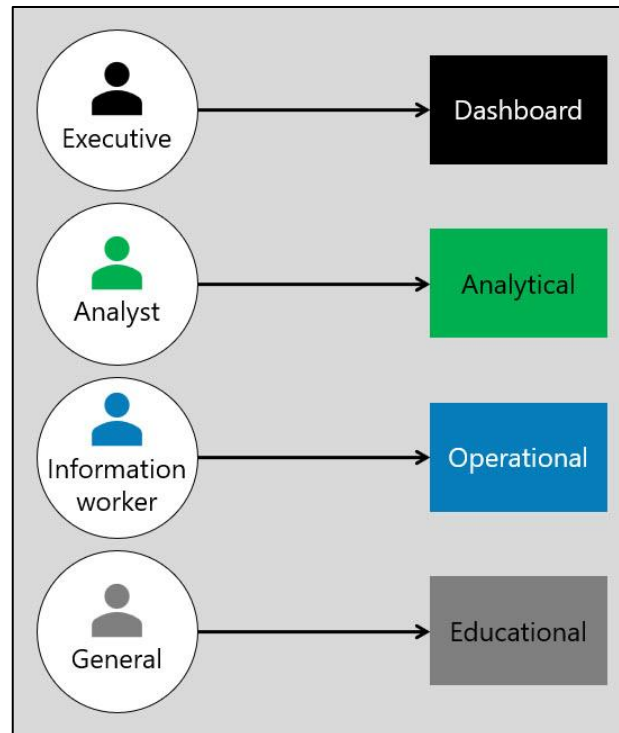
Determine Report Types

Generally, report design can be classified by report type. Often, a direct mapping between the report audience and the report type occurs.

Audience needs can be met by one, or possibly a combination, of four report types:

- Dashboard
- Analytical
- Operational
- Educational

Commonly, executives work with dashboards, analysts work with analytical reports, and information workers work with operational reports.



Each report type has a different approach to the user interface (UI) and user experience (UX) requirements.

Dashboard

The primary goal of a dashboard is to interpret the story as quickly as possible. User interactions are limited by insights that are highly curated toward the audience. Report visuals are focused, self-explanatory, and clearly labelled. A dashboard directly communicates the meaning behind the data to minimize misinterpretation or confusion.

A good example of a dashboard is an *executive dashboard*, which often presents high-level metrics that are displayed on a single page. Dashboards help answer questions such as "How are we doing?" or "Are we there yet?"

Analytical reports

An analytical report is the most common type of report that can serve various report consumer use cases while providing a structured space for analysis.

The primary goal of an analytical report is to help report consumers discover answers to a broad array of questions by interacting with the report and its visuals. Analytical reports often have many slicers to filter report data, and they often contain complex visuals that expose in-depth detail of the data.

Report pages are often expressly designed for interactivity with a focus on UX features.

Multiple pathways are often provided for the report consumer to follow, which allows them to explore a topic of interest, share their findings, or return to where they started.

Report consumers can remove layers and add context and detail by incorporating interactive features. Common interactive features include drill down, drill through, and tooltips.

A good example of an analytical report is one that extends beyond the "How are we doing?" type of question to answer the "Why did that happen?" or "What might happen next?" type of questions.

Operational reports

Operational reports are designed to give the report consumer the ability to monitor current or real-time data, make decisions, and act on those decisions. Operational reports can include buttons that allow the report consumer to navigate within the report and also beyond the report to perform actions in external systems. Frequently, operational reports serve as a hub for action that is used by report consumers as part of their daily activity and workload.

This type of report should minimize the number of analytical features to ensure that focus remains on the operation that it's designed to serve. A streamlined user experience is the primary aim for this report type because excessive clicking or illogical flow can lead to high dissatisfaction.

A good example of an operational report is one that allows monitoring of a manufacturing production line. When an unexpected event arises, such as equipment malfunction, a button could allow workers to start a maintenance request.

Educational reports

Educational reports assume that the report consumer is unfamiliar with the data or context. So the reports must provide clear narrative detail and guidance to help with understanding. This type of report is often used in journalism and by governments to disseminate information to large audiences that have varying levels of understanding of the subject.

A good example of an educational report is one that describes the rollout of COVID-19 vaccination progress and that can be filtered by the home geographic region of the report consumer.

Define User Interface Requirements

UI requirements relate to how reports are consumed and to the appearance and behaviour of reports. Aspects to consider include form factor, input method, style and theme, and accessibility.

Form factor

In the context of report design, form factor describes the size of the hardware that is used to open reports, and to page orientation (portrait or landscape).

Generally, today's computers are accompanied by large-sized monitors and, occasionally, multiple monitors. Large monitor screen sizes are ideal for viewing reports in web browsers, especially when they include several visuals or complex visuals by using landscape orientation.

Mobile devices, which are typically used when the report consumer is away from their desk, have a smaller form factor.

Phones and tablets display content in portrait orientation by default. Smaller form factors demand a different design approach compared to reports that are designed for larger form factors.

Accordingly, when you design for a smaller form factor, strive for fewer and less complex visuals. Also, visuals should be large to help ease viewing and interactivity.

Mobile devices encompass augmented or mixed reality technologies, allowing head-mounted devices to display reports that are superimposed over the reality of the surroundings.

Occasionally, the design of a report that is intended for a large form factor can be refactored to create a mobile view. The mobile view might contain a subset of visuals from the full-sized report.

Input method

When defining UI requirements, also consider input methods that are supported by devices or applications.

While a computer has a keyboard and pointing device (mouse), mobile devices rely on common gestures, such as tap, double-tap, drag, pinch, spread, or press. Report consumers who are using mobile devices can also use on-screen keyboards, voice control, or barcode and QR code readers.

Augmented or mixed reality devices rely heavily on hand gestures or body movement. Knowing how your report consumers are likely interacting with reports should influence your report designs.

Consider the number of visuals that display together on the page, the complexity of visuals, spacing between visuals, and the use of interactive design elements such as tooltips, slicers, buttons, or filters.

When a report is embedded in an application, input can be received programmatically from the application.

For example, when the application user opens a page to view history for a specific customer, a filter automatically passes to an embedded report to filter and display sales history for that customer.

Style and theme

UI requirements should also consider style and theme. Strive to design reports with a consistent and distinctive appearance that is determined by a deliberate theme.

The report theme should express your organizational branding or aim to complement it. At a minimum, the theme should include the following elements:

- A brand mark or logo symbol.
- A palette of colors that align to, or complement, organizational branding. The colors should also be sufficiently different so that, when applied to visuals, they provide appropriate contrast when appearing side by side.
- Text settings, including font selection, sizes, and color.

To manage style and theme changes effectively and efficiently, design reports that use images and themes that are stored in a central repository.

This approach improves change management: Changes that are applied to the repository can automatically cascade through to reports.

Accessibility

UI requirements should also factor in accessibility. Reports need to communicate to the broadest audience possible.

So you should consider how report consumers with no-to-low vision or other physical disability can fully experience the reports.

While you might not be aware of report users with accessibility issues today, it's a good idea to design for accessibility from the outset.

Accessibility requirements can influence your report designs in terms of form factor, input method, and style and theme.

To support people with low or no vision, consider using:

- clear and large-sized fonts,
- well-spaced and large visuals,
- sufficiently contrasting colors,
- and intuitive report navigation that can be understood by keyboard and screen readers.

Some physical disabilities can inhibit or prevent report consumers from fully interacting with reports in the way that you intend.

When possible, ensure that you offer alternative paths for consumers to follow to achieve the interactive design outcomes. For example, adding alt text to visuals for screen reader support, and setting tab order for keyboard navigation.

Define User Experience Requirements

UX requirements relate to how reports deliver the expected report consumer needs.

To assess user experience requirements, you should consider the audience, report type, and the UI requirements.

Other UX requirements can include:

- Support for interactions, such as:
 - Drill up, drill down, or drill through to details.
 - Navigation within the report or to other reports.
 - Filters or slicers that can be applied to report visuals, specific pages, or all pages.
 - Data export as specific data formats, such as Microsoft Excel or a comma-separated value (CSV) file.
- Support for ad hoc questions to retrieve a response in the form of a data visualization.
- Configuring of data alerts to notify people when specific data values change or exceed predefined thresholds.
- Links to open webpages.
- Actions to open applications, write back data entry values, or trigger workflows.
- What-if analysis that allows the report consumer to modify "what-if" values to understand the consequences of different scenarios.
 - For example, what-if analysis could allow consumers to predict sales revenue based on different consumer demand estimates.
- Page layouts that can extend over multiple pages and are suitable for printing as multi-page documents.
- Printing the report to a physical printer or as a PDF document.

- Subscribing to the report so that it can be automatically delivered as a document on a scheduled basis.
- Adding commentary, feedback, or engaging in a conversation about the report.

Design Power BI Reports

Introduction

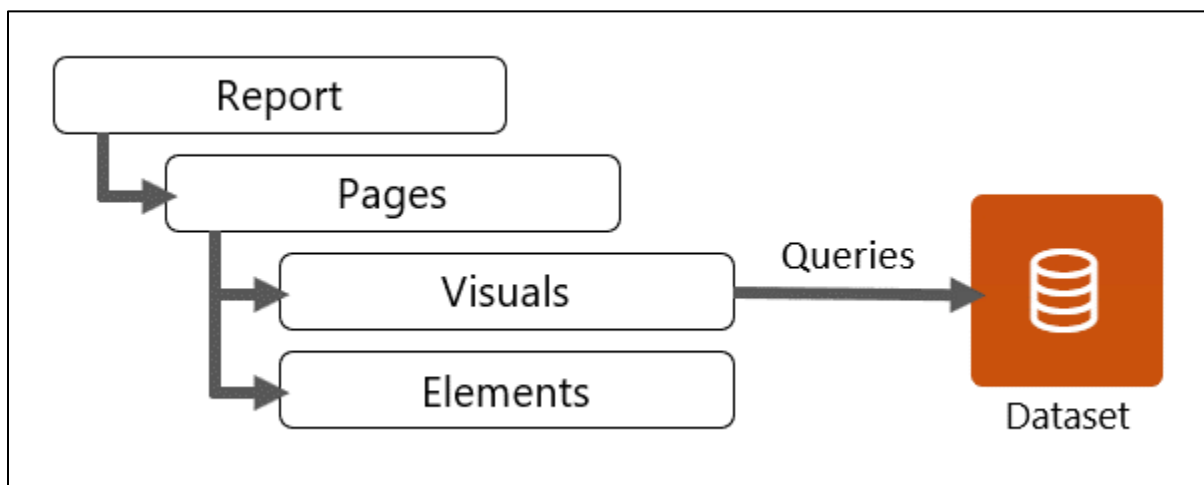
You can design Microsoft Power BI reports in Power BI Desktop or Power BI service (web portal). Power BI mobile apps don't support report design; they support only the report consumer experience.

Power BI Desktop is only supported on Windows operating systems. As a design tool, it also supports the development of reports and semantic models. In this module, the focus is only on report design.

Report structure

Structurally, a Power BI report connects to a single semantic model (semantic model), and it has at least one report page. However, it's common that reports have multiple pages. On each page, *report objects* are laid out. Report objects include:

- **Visuals** - Visualizations of semantic model data.
- **Elements** - Provide visual interest but don't use semantic model data. Elements include text boxes, buttons, shapes, and images.



Report pages

Similar to Microsoft Excel worksheets, you can add, rename, resequence, hide, duplicate, or delete Power BI report pages.

Duplicating pages can help expedite report development, especially when you're copying a completed and polished report page.

However, take care that you don't over complicate the report design if you can accomplish a duplicated page by filtering a single page. For example, instead of creating one page for each customer, you could use a slicer on a single page to filter by customer.

Report consumers navigate to a visible page by selecting a page tab (in Power BI Desktop) or by selecting the page in the **Pages** pane (in Power BI service).

The decision to create more pages depends on your report requirements. Strive for a report design that expresses the data in a logical flow on the page and between pages. A well-designed report often provides a high-level summary on the first page with supporting detail on the following pages.

Occasionally, it might make sense to have a single report with many pages. Other times, it could make sense to separate pages into different reports. This scenario is especially true when you design those pages for a different audience or when you need to secure, share, or distribute them differently.

The reason is because reports are a security and publication unit, and report pages belong to reports and can't be secured or published independently of the report.

You can hide pages when they're not yet ready for use; they're a *work in progress*. However, you might commonly hide them because you want to control how they're accessed.

You can provide report page navigation with buttons or by drilling through from a visual. It's also possible to design a page as a tooltip, which is revealed when a report consumer hovers over a visual.

You can configure various page-level settings, which can be set in the **Format** options. Commonly applied settings include page information, page size, and page background.

Design the Analytical Report Layout

When report consumers look at a report, an automatic and unconscious process occurs when they comprehend what they see. Therefore, you should adhere to basic report design principles to support this process to help effectively communicate meaning from data.

Report design is a blend of science and art, and many possible report designs are available that help achieve the audience and interface requirements.

What matters most is that the report design effectively communicates the data to meet requirements.

An analytical report can consist of one or more pages, and pages comprise report objects.

Report objects can be data visuals, which represent query results, or decorations like images, background shapes, or text.

You can begin to design a report layout by determining the number, sequence, and purpose of the pages.

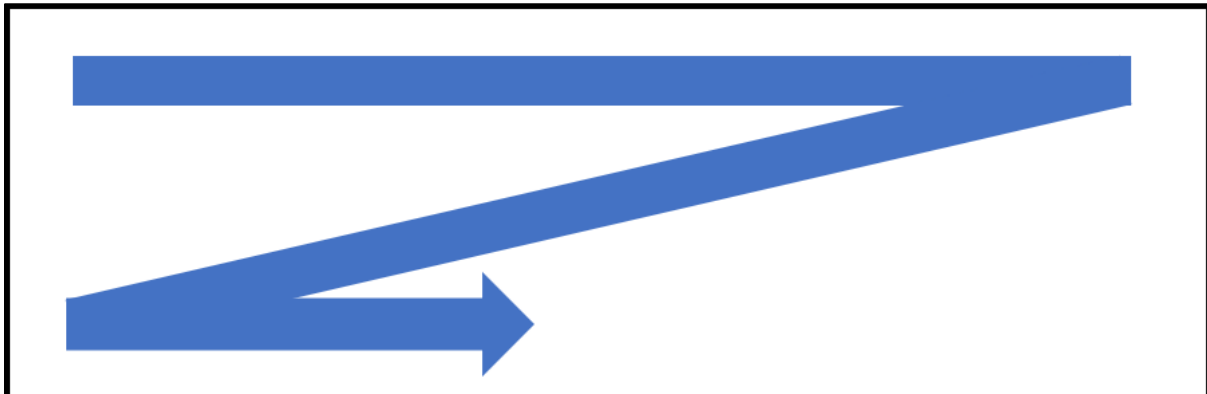
Make sure that you avoid combining subjects or opposing objectives on the same page. Then, design each page layout with specific report objects that are relevant to the requirements.

Additionally, good report design layouts should consider design principles of placement, balance, contrast, proximity, and repetition.

Always keep in mind that the *less is more* adage applies; simplicity and clarity lead to good design.

Placement

Good placement of report objects contributes to an ordered report design. Generally, you should place the most important information in the upper-left corner of the page and arrange the report elements from left to right and top to bottom.



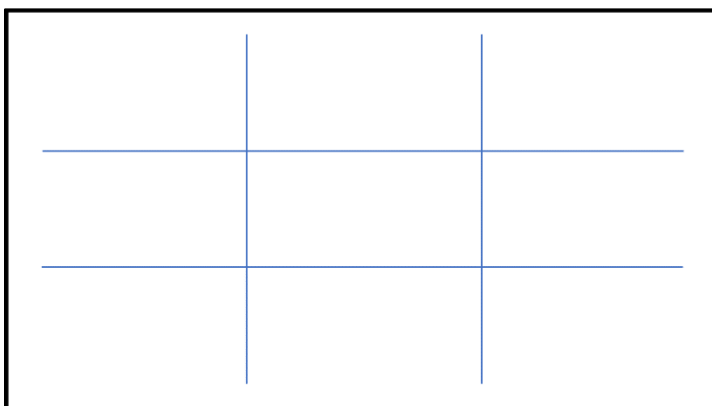
This placement applies to audiences who predominantly read left to right (LTR). When your audience reads right to left (RTL), as is the case with some written languages such as Arabic and Hebrew, place the most important information in the *upper-right* corner and arrange the report elements from *right to left*.

Arrange report objects so that the vertical and horizontal edges align because it looks ordered and is pleasing to the eye.

Position-related report objects in logical groups. An ordered report layout creates a connection between visual elements and avoids clutter that can result from a seemingly random placement of report objects.

Additionally, aligning report objects in a visually pleasing layout can convey more energy and interest than simply centring or randomly placing report objects. Consider applying the *rule of thirds*, which is a visual arts rule that can be applied to report object placements in an analytical report.

The rule proposes that a page layout should be divided into an invisible grid of nine equal parts.



The grid is formed by two equally spaced horizontal lines and two equally spaced vertical lines. Then, report objects can be placed within the cells of the grid.

Balance

Another important consideration when you are laying out report objects is *balance*. Balance is concerned with stability and

structure in design. In the context of a report layout, balance refers to the weight that is distributed across the report page by the placement of objects of the same or different sizes.

Balance can be *symmetrical* or *asymmetrical*. Symmetrical balance is achieved by distributing the weight evenly on both halves of the page. Asymmetrical balance is achieved through contrast.

Consider using the *golden ratio* as a guide to produce asymmetrical balance. The ratio is based on the Fibonacci Sequence, where two quantities are in the golden ratio if their ratio is the same as the ratio of their sum to the larger of the two quantities.

For centuries, the golden ratio has influenced art and architecture to produce works that are harmonious and balanced.

If applied to report design, the golden ratio will align a page to have one large visual to draw initial attention, which is then supported by smaller visuals that provide context.

Proximity

In a report layout, proximity is concerned with the nearness of report objects. When a report page consists of multiple groups of related objects, you should use space to visually separate them.

In the following report design, notice the top-left section labelled key metrics. Related visuals are placed near one another. They are also purposefully and consistently aligned forming a clear section.

Contrast

Contrast can be used to combine two opposing objects. The use of contrasting colors, fonts, font properties, or lines can emphasize important objects of the report design.

Use this principle to direct report consumers to where they should look or which data visual they should interact with first.

Repetition

Repetition in a report design creates association and consistency. Good use of repetition can help strengthen a report layout by tying related report objects together.

Many key metrics can be presented in single-value cards. This repetitious design allows report consumers to quickly understand and interpret the metrics.

Design Visually Appealing Reports

Having the correct data and selecting the correct visuals is important. Equally as important is ensuring that the report is visually appealing. A well-designed report should guide the report consumer to quickly find and understand the answers to their questions.

While striving to produce an appealing report, bear in mind that the report should be user-friendly. Moreover, it might be possible to add more visuals to a well-designed report page without it appearing cluttered.

Space

Space is essential for an effective report design because it helps reduce clutter and increase readability. Spacing applies to the report page margins and the spacing between report objects.

Margins

Margins include the border area, or edge, around each page. Having a consistently spaced border area frames the report objects.

Because there isn't a report page property to set margins, it's up to you to lay out objects in way that results in a consistent border area. Margin sizes should be equal on the left and right, with possible variation on the top and bottom.

Space across the top or bottom can show branding, titles, slicers, or other information that needs to be separated from the visuals.

Object spacing

Ensure that you provide sufficient space surrounding, or within, report objects.

When visual headers are enabled, be sure to test that they don't overlap with nearby objects because overlapping objects can make interacting with visual header icons difficult. Appropriate spacing between visuals will help you avoid this problem.

Consider using different space depth to visually separate sections of related objects. However, keep in mind that too much space can result in an unbalanced report layout and could draw the report consumer's attention away from what matters. Moderation is key: Always strive to produce an evenly spaced and balanced report.

Size

Size can relate to the page size and visual size.

Page size

You can set the page size to predefined or custom dimensions.

Additionally, you can set custom dimensions that are larger than the available screen size so that the report consumer will need to interact with scrollbars to view the entire page.

However, a large page size that is filled with visuals might take time to render, and visuals might not render in a top-to-bottom order.

Visual size

Generally, the more important the visual, the larger its size. Report consumers will quickly focus on larger visuals. When similar visuals are on the page, such as a series of card visuals, they should be equally sized.

Many visuals are responsive to size, so the visual will look appealing in either a small or large size. Consider that a line chart visual might appear like a sparkline when it's sized as small. In this case, only a few axis and data labels might appear. When the line chart visual is sized larger, more detail will be revealed, including many more axis and data labels.

While report consumers can use focus mode to enlarge a single visual, the visual should still clearly communicate its data when viewed at actual size on the report page. Focus mode can help consumers better interpret the data or more easily interact with the visual, such as expanding into levels of a matrix or decomposition tree visual.

Alignment

When multiple visuals are on the report page, ensure that they're properly aligned, meaning that the edges of visuals should be in alignment and the spacings between visuals are consistent. Alignment also relates to format options. For example, the alignment of titles and legends *within visuals* should be consistent.

Consider laying out the report page with different sections and aligning visuals appropriately within the sections. Sections can be *implied* or *explicit*.

Implied sections

Define implied sections by aligning groups of visuals in close proximity.

Explicit sections

You can define explicit sections by using coloured shapes and overlaying aligned visuals on those shapes.

Color

Use color sparingly and meaningfully because overusing it can be distracting. Stick to a few softer colors as a base, possibly aligned with corporate colors. Softer colors ensure that the data is the focus in your report. Reserve the use of bolder colors to highlight exceptions.

Ensure that colors are sufficiently contrasting. Color contrast is especially important to create accessible reports for report consumers who have low vision.

Consistency

Strive for consistency when you are laying out and configuring report objects. Consistency should apply to everything in your report design, including spacing, margins, size, alignment, and especially to object format options.

Format options include the selection of font, font size, font weight, colors, and many other styling options. Format options also include the enabling of visual features like axis labels and data labels.

The quickest way to enforce consistency is to use a *report theme*. A report theme applies format settings to your entire report, ensuring consistent application of colors, fonts, pages, and visual format options, including the **Filters** pane styles.

Consider using one of the many built-in themes from the theme gallery. You can use one as a starting point and then customize it to better meet your requirements. Alternatively, you can create a new theme, which can be considerable work initially, but will provide you with granular control.

Be aware that the theme will be overridden when you explicitly configure a format option. For example, you can explicitly set a color by entering a HEX value instead of selecting a color from the palette.

Try to limit overriding the report theme to an exception basis because if you switch themes, overridden properties won't update.

Be sure to export the report theme, which is a JSON file, and then apply it to other reports to ensure consistency across all reports.

Report Objects

Report objects are laid out on each report page and include:

- **Visuals** - Visualizations of semantic model data.
- **Elements** - Provide visual interest but don't use semantic model data.

All report objects have some properties in common, which can be set in various sections of the **Format** options. Commonly applied settings include general properties (location, size, and alt text), title, background, border, and shadow.

Visuals

Visuals are visualizations of semantic model data. Power BI includes over 30 *core visuals*, which are built in and available to all reports. You can access the core visuals in the first section of the **Visualizations** pane.

You can also extend the core visuals with *custom visuals*.

Custom visuals are sourced from Microsoft AppSource (an app store for business applications such as Microsoft Office 365, Microsoft Dynamics 365, Power BI, or separate Microsoft Azureweb apps) or uploaded as a .pbviz file.

You can use the following general methodology to add and configure a visual:

1. Select the visual type in the **Visualizations** pane and then position and size it on the page.
2. Map semantic model fields to populate the visual. Each visual has one or more *wells*. The number and type of wells differ between visuals. Some visuals, like the slicer, have a single well, but others have many more.
3. Optionally, apply visual-level filters.
4. Modify field mappings, possibly renaming fields, modifying summarization behaviour (or restricting summarization), or enabling the **Show items with no data** option.
5. Optionally, modify the sort field and the sort direction (in ascending or descending order).
6. Optionally, apply format options to produce the desired result and style.
7. Optionally (and when supported), use the **Analytics** options to overlay supporting data, like minimum or maximum lines, or Artificial Intelligence (AI) results like anomaly detection and forecasts.

Elements

Elements provide visual interest but don't use semantic model data

The four types of elements are:

- **Text box** - Add rich text, such as the title, to the report page. You can selectively apply font type and font size and then set the color, styling (bold, italics, or underline), and paragraph alignment (right, centre, or left). You can insert hyperlinks, too.
- **Buttons** - Add buttons so that report consumers can interact with the report as they would an app. Buttons can perform different actions, such as return to the previous page, navigate to a specific page, drill through to a page, select a bookmark, open Q&A, or open a web URL in the default web browser.
- **Shapes** - Add a shape to the report as decoration or to behave like a button to perform an action. Almost any conceivable shape is possible, from basic shapes to arrows and even hearts. Shapes can include text and can be formatted and styled in many different ways.
- **Image** - Upload an image, such as your company logo, to the reports page. Supported image types include BMP, JPEG, GIF, TIFF, and PNG. Similar to the **Shapes** element, images can behave like buttons to perform an action.

The text box deserves a special mention because it's capable of embedding *dynamic values* that are sourced from the report semantic model into paragraphs of text. When the page is filtered, dynamic values are filtered. Technically, the text box isn't a visual. However, in this instance, it behaves like one. It's also available as the *smart narrative* visual, which automatically summarizes data by using text descriptions and insights.

At design time, you can start with a text box, or you can add the smart narrative visual from the **Visualizations** pane. However you start, the end result will be the same. Then, you can add a dynamic value by using Q&A to ask a question. Additionally, you can format the values.

Create Visual Calculations in Power BI Desktop

Introduction

Visual calculations in Power BI are Data Analytics Expressions (DAX) calculations defined on a visual. Unlike measures, they aren't stored in the model, but on the visual you're working with.

Visual calculations make it easier to create calculations that were previously hard to create, leading to simpler DAX, easier maintenance, and better performance.

For example, you're asked to add the following calculations to your report:

- A comparison of sales over years.
- A running sum of cost by category.
- A moving average of profit over time.

Visual calculations make these complex calculations easier by providing a user-friendly interface and real-time validation when applying the calculation on the visual.

Understand Visual Calculations

Visual calculations are a simple way to add calculations to your Power BI report. They're DAX calculations defined on a visual and are defined visually. Compared to measures, the DAX required is easier to write, understand, and maintain.

Visual calculations are defined on a visual, such as a table, bar chart, or a matrix. They provide easy to use functions for calculations that are commonly used in organizations.

While creating a visual calculation, you don't have to worry about the complexity of the model. You just work with what is on the visual, making it easier to understand and check your work.

A visual calculation can refer to any data in the visual including columns, measures, or other visual calculations.

This ability removes the complexity of the semantic model and simplifies the process of adding calculations to your report. You can use visual calculations to complete common business calculations, such as running sums or moving averages.

Benefits of using a visual calculation

DAX in visual calculations works differently from other calculation options. The formula isn't stored in the model, but on the visual itself.

This means they can only refer to what's on the visual. If you need to use something from the model, you must add it to the visual first.

This approach simplifies things by avoiding the complexity of filter context and the model.

Visual calculations combine the straightforward context of calculated columns with the flexibility of on-demand calculations from measures. They work on aggregated data instead of detailed data, which often improves performance. When you can choose between a new measure or a visual calculation, the visual calculation usually performs better.

Since visual calculations are part of the visual, they can refer to the visual structure, offering more flexibility.

Create Visual Calculations

Creating a visual calculation is straightforward - select a visual and choose **New calculation**. The visual calculation window consists of three major sections, as shown from top to bottom in the following image:

- The **visual preview** which shows the visual you're working with.
- A **formula bar** where you can add visual calculations.
- The **visual matrix** which shows the data in the visual, and displays the results of visual calculations as you add them. Any styling or theming you apply to your visual isn't applied to the visual matrix.



To add a visual calculation, type the expression in the formula bar.

For example, the following code calculates the profit by subtracting Total Product Cost from Sales Amount.

Profit = [Sales Amount] – [Total Product Cost]

By default, most visual calculations on a visual are evaluated row-by-row, like a calculated column.

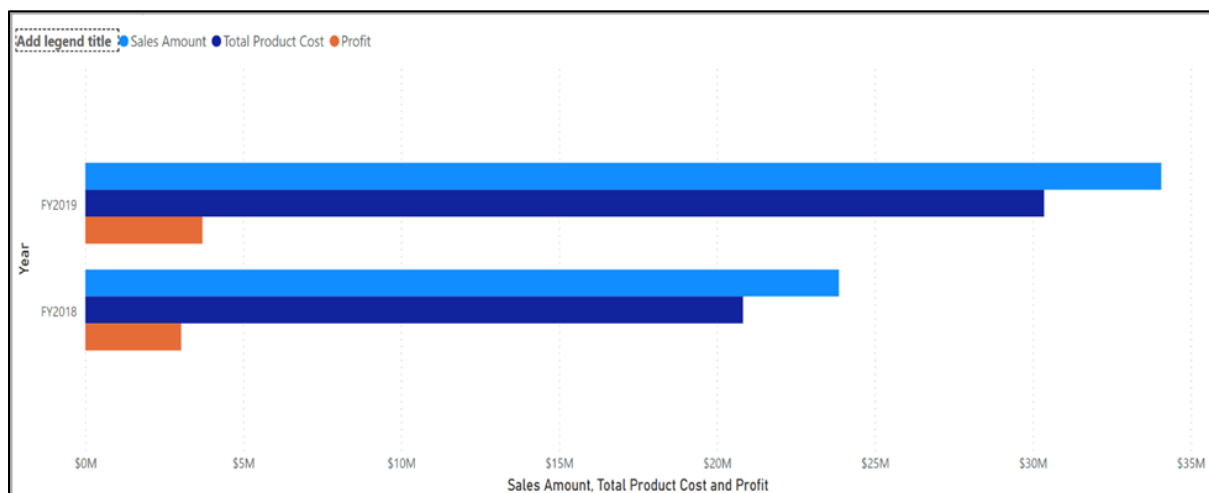
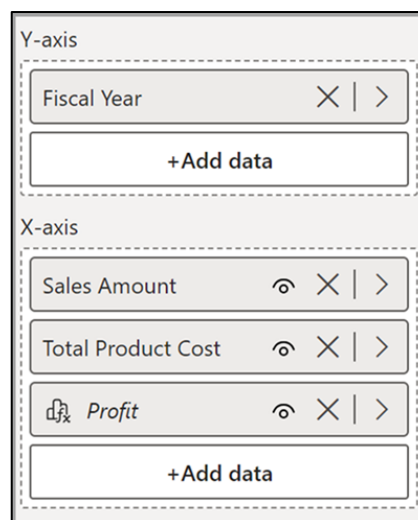
In the previous example, for each row of the visual matrix the current Sales Amount and Total Product Cost are subtracted, and the result is returned in the Profit column.

Although possible, there's no need to add an aggregation function like SUM as you would in a measure.

In fact, it's better not to add such aggregates when they're not necessary, so you can more easily distinguish between measures and visual calculation expressions.

Hide fields from a visual

As you add visual calculations, they're shown in the list of fields on the visual and on the visual itself:



In visual calculations edit mode, you can hide fields from the visual just like you can hide columns and tables in the modeling view. For example, if you wanted to only show the *Profit* visual calculation, you can hide *Sales Amount* and *Total Product Cost* from view.



Hiding fields doesn't remove them from the visual or from the visual matrix, so your visual calculations can still refer to them and continue to work.

A hidden field is still shown on the visual matrix but is not shown on the resulting visual.

It's a recommended practice to only include hidden fields if they're necessary for your visual calculations to work.

Work with templates

Visual calculations include templates to make it easier to write common calculations.

You can find templates by selecting the template button and choosing a template, such as:

- **Running sum** calculates the sum of values, adding the current value to the preceding values.
- **Moving average** calculates an average of a set of values in a given window by dividing the sum of the values by the size of the window.
- **Percent of parent** calculates the percentage of a value relative to its parent.
- **Average of children** calculates the average value of the set of child values.
- **Versus previous** compares a value to a preceding value.
- **Versus next** compares a value to a subsequent value.

Each template has a corresponding function which is added to the formula bar when you choose a template.

You can also add your own expressions without relying on templates.

Available DAX functions

You can use many of the existing DAX functions in visual calculations.

Since visual calculations work within the confines of the visual matrix, functions that rely on model relationships such as USERELATIONSHIP, RELATED, or RELATEDTABLE aren't available.

Visual calculations also introduce a set of functions specific to visual calculations. Many of these functions are easier to use shortcuts to DAX window functions:

Function	Description	Example	Shortcut to
<u>COLLAPSE</u>	Calculation is evaluated at a higher level of the axis.	Percent of parent = DIVIDE([Sales Amount], COLLAPSE([Sales Amount], ROWS))	N/A
<u>COLLAPSEALL</u>	Calculation is evaluated at the total level of the axis.	Percent of grand total = DIVIDE([Sales Amount], COLLAPSEALL([Sales Amount], ROWS))	N/A
<u>EXPAND</u>	Calculation is evaluated at a lower level of the axis.	Average of children = EXPAND(AVERAGE([Sales Amount]), ROWS)	N/A
<u>EXPANDALL</u>	Calculation is evaluated at the leaf level of the axis.	Average of leaf level = EXPANDALL(AVERAGE([Sales Amount]), ROWS)	N/A
<u>FIRST</u>	Refers to the first row of an axis.	ProfitVSFirst = [Profit] – FIRST([Profit])	<u>INDEX(1)</u>
<u>ISATLEVEL</u>	Reports whether a specified column is present at the current level.	IsFiscalYearAtLevel = ISATLEVEL([Fiscal Year])	N/A
<u>LAST</u>	Refers to the last row of an axis.	ProfitVSLast = [Profit] – LAST([Profit])	<u>INDEX(-1)</u>
<u>MOVINGAVERAGE</u>	Adds a moving average on an axis.	MovingAverageSales = MOVINGAVERAGE([Sales Amount], 2)	<u>WINDOW</u>
<u>NEXT</u>	Refers to a next row of an axis.	ProfitVSNext = [Profit] – NEXT([Profit])	<u>OFFSET(1)</u>
<u>PREVIOUS</u>	Refers to a previous row of an axis.	ProfitVSPrevious = [Profit] – PREVIOUS([Profit])	<u>OFFSET(-1)</u>
<u>RANGE</u>	Refers to a slice of rows of an axis.	AverageSales = AVERAGEX(RANGE(1), [Sales Amount])	<u>WINDOW</u>
<u>RUNNINGSUM</u>	Adds a running sum on an axis.	RunningSumSales = RUNNINGSUM([Sales Amount])	<u>WINDOW</u>

Use Parameters in Visual Calculations

Visual calculations have optional parameters to help you create complex calculations with minimal code.

Use the Axis parameter

Many functions have an optional **Axis** parameter, which can only be used in visual calculations. Axis influences how the visual calculation traverses the visual matrix.

The Axis parameter is set to the first axis in the visual by default.

For many visuals the first axis is ROWS, which means that the visual calculation is evaluated row-by-row in the visual matrix, from top to bottom.

The following parameter values control how the data is calculated:

Value	Description	Icon
ROWS	Vertically across rows from top to bottom.	
COLUMNS	Horizontally across columns from left to right.	
ROWS COLUMNS	Vertically across rows from top to bottom, continuing column by column from left to right.	
COLUMNS ROWS	Horizontally across columns from left to right, continuing row by row from top to bottom.	

Use the Reset parameter

Many functions have an optional **Reset** parameter that is available in visual calculations only. Reset influences if and when the function resets its value to 0 or switches to a different scope while traversing the visual matrix.

The Reset parameter is set to None by default, which means the visual calculation is never restarted. The following list describes the only valid values for the Reset parameter:

- **NONE** is the default value and doesn't reset the calculation.
- **HIGHESTPARENT** resets the calculation when the value of the highest parent on the axis changes.
- **LOWESTPARENT** resets the calculations when the value of the lowest parent on the axis changes.
- **A numerical value** which refers to the fields on the axis, with the highest field being one.

To better understand this concept, let's consider an axis that has three fields on multiple levels: Year, Quarter, and Month.

As seen in the following example, **HIGHESTPARENT** is **Year** and **LOWESTPARENT** is **Quarter**, which affect how the running sum is calculated.

- **RUNNINGSUM([Sales Amount], HIGHESTPARENT)** starts from 0 **for every year**.
- **RUNNINGSUM([Sales Amount], LOWESTPARENT)** starts from 0 **for every Quarter**.

Lastly, a visual calculation that is defined as **RUNNINGSUM([Sales Amount])** doesn't reset, and continues adding the Sales Amount value for each month to the previous values without restarting.

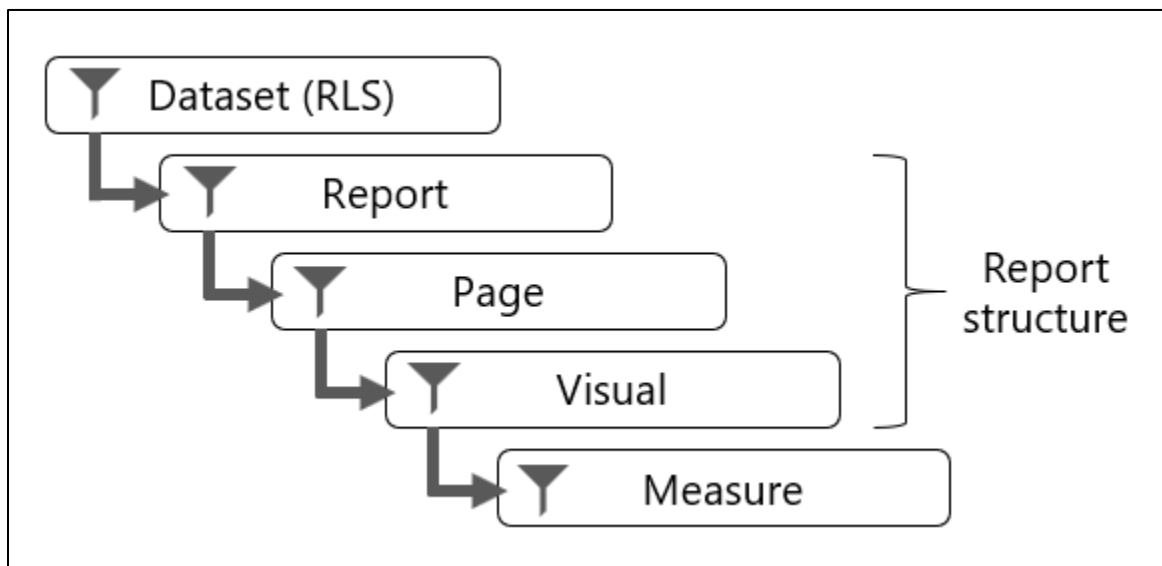
Configure Power BI Report Filters

Introduction to Designing Reports for Filtering

Filtering can occur at five different levels of a Microsoft Power BI report:

- Semantic model (RLS)
- Report
- Page
- Visual
- Measure

Report, page, and visual level filters apply to the structure of the report.



Semantic model (RLS)

Every Power BI report queries a single semantic model, which is a Power BI artifact that represents a semantic model. The semantic model can enforce row-level security (RLS) to restrict access to a subset of data, and different users will see different data. A report can't determine whether the semantic model will enforce RLS, and it can't override RLS.

When you filter RLS-filtered tables, take care to avoid situations where some report consumers see no data (BLANK) in the report. For example, consider a model that enforces RLS to restrict data visibility to specific countries/regions. If at design time, you have permission to see Australian data and you filter the report by Australia, a report consumer that has permission to see only United States data won't see data at all.

Report structure

The structure of a Power BI report is hierarchical. The top level is the report, the second level comprises its pages, while the third level comprises report objects, which consist of visuals and elements. In the **Filters** pane, you can add a filter to any of these levels.

Measure

A measure is a model object that is designed to summarize data. Measure formulas, which are written in Data Analysis Expressions (DAX), can modify filter context by using the `CALCULATE` or `CALCULATETABLE` functions. These functions are powerful and provide you with the flexibility to add, remove, or modify filters. A set of DAX functions, known as time intelligence functions, also

modify filter context. These functions can override any filters that are applied to the report structure.

At report design time in Microsoft Power BI Desktop, you can create measures (except when the model is a live connection to SQL Server Analysis Services multidimensional model). These measures belong to the report, and so they're called *report-level measures*.

A good example of a measure that overrides report filters is a three-month moving average calculation. To compute the result for March, the filter context for month must expand to encompass January, February, and March. The CALCULATE function or a time intelligence function can modify the filter context to produce that result.

Apply Filters to the Report Structure

At report design time, use the **Filters** pane to apply filters to the report structure.

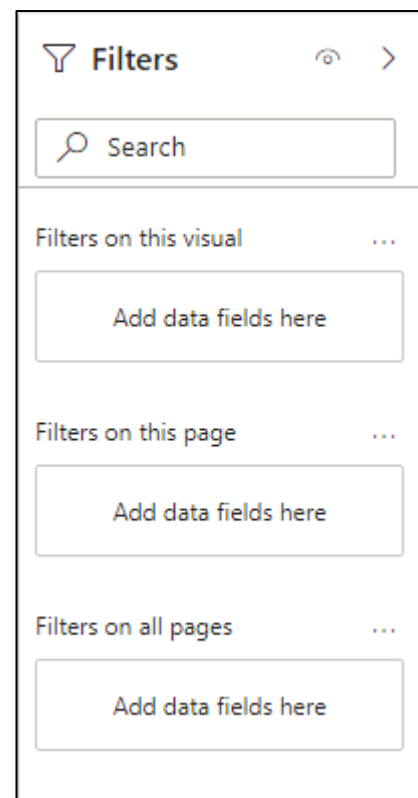
The **Filters** pane has three sections:

- Filters on all pages
- Filters on this page
- Filters on this visual

The **Filters on all pages** section defines *report-level filters*, which apply to all report pages and visuals. Report-level filters are global filters.

The **Filters on this page** section defines *page-level filters*, which apply to all visuals on a specific page, adding to any existing report-level filters. For example, if a report-level filter restricts data to the country of United States, and a page-level filter restricts data to the state of Washington, then both filters are in effect: Country is United States and State-Province is Washington.

The **Filters on this visual** section defines *visual-level filters*. This section is only visible when one visual is in focus because visual-level filters apply only to a single visual; they add to any existing report-level and page-level filters. Extending the previous example, if a visual-level filter restricts data to the city of Seattle, then three filters are in effect: Country is United States, State-Province is Washington, and City is Seattle.



Unlike report and page-level filters, a visual-level filter can filter by using a measure. When a measure filters a visual, it's used to eliminate groups. For example, consider a column chart visual that groups by store. A measure filter could eliminate groups (stores) where the total store sales are less than a certain amount.

Filters apply to a single field and use one of following filter types:

- Basic
- Advanced
- Top N
- Relative date and Relative time

The **Basic filter** type allows you to select items from a list of distinct values that are found in the field. You can also restrict the filter to a single selection instead of multi-selection.

The **Advanced** filter type allows you to create more complex conditions by using data type-specific operators:

- **Text field operators** - Test for conditions such as "contains," "starts with," "is blank," "is empty," and others.
- **Numeric field operators** - Test for conditions such as "is less than," "is less than or equal to," and others.
- **Date field operators** - Test for conditions such as "is after," "is on or after," and others.

You can combine multiple tests by using a logical AND/OR operator.

The **Top N** filter type applies to text and date fields that are available only in visual-level filters. This filter type helps you filter by the top (or bottom) number of items, like the top five products by revenue. To configure the filter, you must pass in a field that's summarized, like sales revenue.

The **Relative date** and **Relative time** filter types apply to date fields only, making it possible to filter by *relative date or time*. Relative filters allow the report consumer to filter by past, present, or future time periods based on the current date and time.

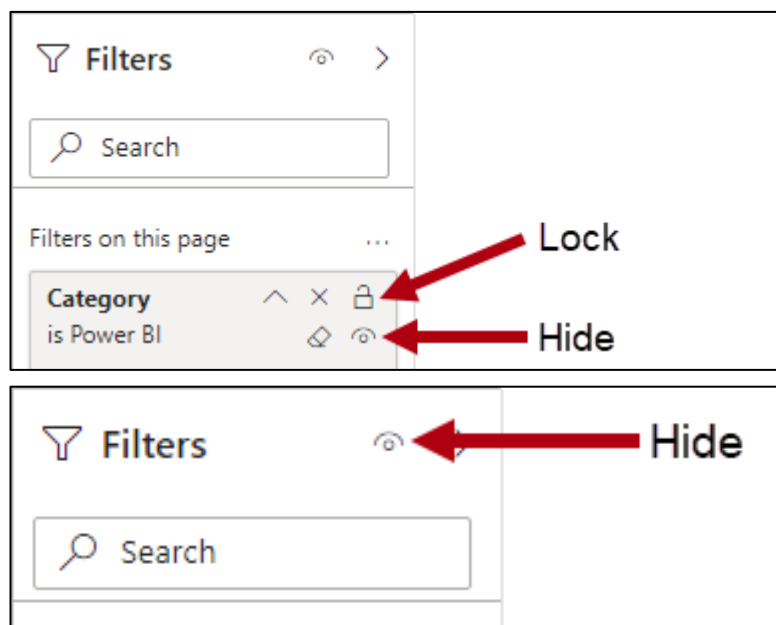
If an overwhelming number of filters are available, use the search box to quickly narrow down the number of filters.

You can also sort the filters alphabetically within each section in ascending or descending order. Also, to determine the model field for each filter, hover the cursor over a filter card to reveal the full reference to the model in terms of its table and the field name.

You can lock filters to ensure that report consumers can't remove or modify them. It's a good idea to lock filters that are critical to the design of the report, page, or visual. Additionally, you can hide filters.

A hidden filter isn't visible to report consumers. Consider hiding a filter when the report consumer doesn't need to know about it, such as when filters are cleaning up the data, perhaps by removing BLANKs.

You can hide the entire **Filters** pane to ensure that report consumers can't open it.



Apply Filters with Slicers

The slicer is a core visual with one purpose: *filter other visuals*. It's one of the most common visuals that you add to a report page because it presents an intuitive way for report consumers to

filter data. As a report author, you have considerable control over how the visual is laid out and formatted and how it functions.

By default, slicers filter all other visuals on the page. However, you can edit visual interactions to restrict filtering between two visuals. We cover visual interactions later in this module. Sync slicers can also filter visuals on other pages.

It might be tempting to think that slicers apply page-level filters because that's the default result. However, it's important to understand that a slicer is a visual that propagates filters to other visuals on the same page or (when synced) across other pages.

You can configure a slicer by using one or more fields from the same table or a hierarchy. When configured to use multiple fields or a hierarchy, the slicer presents an expandable tree structure of items.

The slicer layout is responsive to the data type of the field. Field data types are either text, numeric, or date. By default, a text field will produce a list slicer, a numeric field will produce a numeric range "between" filter, and a date field will produce a date range "between" filter, allowing value selection with calendar controls.

At design time, you can modify the slicer layout so that lists become dropdown lists. Dropdown lists use much less space on the report page.

Numeric and date ranges offer additional layouts, allowing you to select a single value that acts as the lower or upper boundary of the filter. The reason why numeric and date slicers have additional layouts is because these data types represent continuous values.

Therefore, the slicer layouts allow filtering by ranges of continuous values.

To change the slicer style, select Format your visual > Slicer settings > Visual > Options > Style.

A slicer that is based on a date field offers additional layouts to filter by *relative date or time*. Relative filters allow the report consumer to filter by past, present, or future time periods based on the current date and time. For example, a relative date slicer could filter by the current date (today).

Possible layouts for the field data type are:

- **Text field** - List (default) or dropdown
- **Numeric field** - List, dropdown, between (default), less than or equal to, or greater than or equal to
- **Date field** - List, dropdown, between (default), before, after, relative date, or relative time

Dropdown slicer layouts are common; they help minimize the size of the slicer to provide more room on the page for other visuals. An additional benefit when using dropdown slicers that might not be immediately apparent is that they only query the semantic model when expanded open. Therefore, they can also help expedite report page rendering.

List and dropdown slicers support format options to control the selection of items. When you enable the **Single select** option, the slicer allows only a single item selection. That approach makes sense for a slicer like scenario, where the options are Actual, Budget, or Forecast. In this case, it only makes sense to filter by one scenario at a time.

Other configuration options are available for you to modify slicer behaviour and its look. Review the product documentation for more details.

Design Reports with Advanced Filtering

Beyond filters and slicers, report authors can employ other filtering techniques, such as:

- Visual interactions
- Drillthrough
- Report tooltip
- Bookmarks
- Report options
- Query reduction options

Visual interactions

By default, when report consumers interact with visuals, filters are propagated to other visuals on the report page. This way, visuals behave like slicers. For example, a report consumer can select a column of a column chart visual to filter other visuals on the page. To remove the cross filters, they can either select the column again or select a different visual.

While cross filtering, you can introduce additional cross filters, even from other visuals, by pressing the **Ctrl** key.

Cross filtering (and cross highlighting) works between any two visuals. However, at design time, you can edit visual interactions between any pair of visuals and in either direction. Additionally, you can disable cross filtering or modify the interaction to use cross highlighting instead.

Drillthrough

Add drillthrough pages to allow report consumers to drill from visuals. By default, the drillthrough action propagates all filters that apply to the visual to the drillthrough page.

Report tooltip

Add report tooltips that will appear when report consumers hover the cursor over visuals. By default, the report tooltip receives all filters that apply to the visual.

Bookmarks

Bookmarks capture a specific view of a report, including filters, slicers, the page selection, and the state of visuals. Report authors and report consumers can create them.

Bookmarks that are created by a report consumer are known as *personal bookmarks*.

Because bookmarks can capture filter state, when invoked, they will apply that filter state. You can invoke bookmarks directly from the **Bookmarks** pane, or you can invoke them indirectly by selecting a button, image, or shape.

You can create a bookmark to capture the default state of slicers. A button on the report page could have the text **Reset slicers**, and when invoked, it will use the bookmark.

Report options

Report authors can configure report or visual settings to control filter options and behavior. With report options, you can:

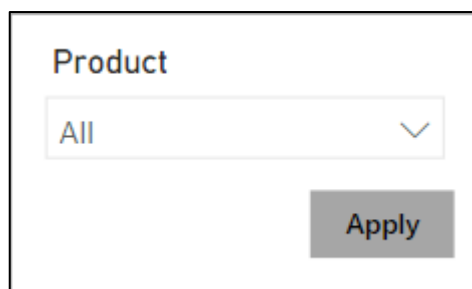
- Disable persistent filters. (Persistent filters are covered in Unit 5.)

- Hide visual headers for all visuals or for a specific visual. When the visual header is hidden, report consumers can't hover the cursor over the Filter icon to determine the filters that are applied to the visual. (The **Filter** icon is covered in Unit 5.)
- Hide the **Filter** icon for a specific visual.
- Restrict report consumers from changing filter types (for example, basic to advanced) in the **Filters** pane.
- Remove the search box in the **Filters** pane.

The default settings provide the most flexibility for report consumers. Only restrict options when you have a compelling reason to do so.

Query reduction options

Report authors can configure report settings to reduce the number of queries that are sent to the semantic model. Fewer queries will result in better responsiveness as report consumers update filters or slicers or cross filter report pages.



Consider an example where a report consumer wants to select multiple items in a single slicer or wants to update two slicers.

By default, each action results in the re-rendering of visuals, so unnecessary queries are sent to the semantic model.

This process slows down report rendering and places an unnecessary burden on the semantic model and any DirectQuery data sources.

Consider enabling the query reduction options when the semantic model uses DirectQuery table storage or when imported data volumes are large and calculations are complex and slow.

Report settings can:

- Disable cross highlighting/filtering by default. All visuals, except for slicers, don't propagate filters to other visuals unless they are explicitly added as a visual interaction.
- Add an **Apply** button to slicers. The slicer selection will only filter the report page when the report consumer selects the **Apply** button.
- Add an **Apply** button to all basic filters in the **Filters** pane. A single filter will apply when the report consumer selects the **Apply** button in the filter card.
- Add an **Apply** button to the **Filters** pane. All filters will apply simultaneously when the report consumer selects the **Apply** button. This option is helpful when report consumers want to update many filters at the same time.

Consumption-Time Filtering

In reading view, report consumers can use many different filter techniques when viewing a Power BI report, such as:

- Using slicers.

- Using filters.
- Applying interactive filtering actions.
- Determining applied filters.
- Working with persistent filters.

As a report author, use these filter techniques to design reports that take advantage of these capabilities, and at times, disable them (when you have good reason to do so).

Slicers

Slicers provide an intuitive experience for report consumers. They are potentially the most-used interactive technique in Power BI reports. Report consumers can modify or clear slicer filters. They can also enable the search feature, allowing them to quickly locate specific items to filter by.

Filters

If the **Filters** pane isn't hidden, a report consumer can open the pane to inspect the filters. When filters aren't locked, report consumers can modify them. Specifically, they can:

- Select the eraser icon to clear the filter.
- Apply a new filter selection.
- Change the filter type, such as from basic to advanced (unless disabled for the report).
- Use the search box to search for values to filter by (unless disabled for the report).

Equally as important is understanding what report consumers can't do, such as:

- Add new filters.
- Remove filters.
- Change the filter type to **Top N**, or **relative date or time** (for date/time fields).
- Enforce or disable single selection.

You can add or remove filters by adding query string parameters in the URL. For embedded reports, the embedding app can use the client APIs to programmatically add or remove filters and change filter types to **Top N** or **relative date or time**.

Interactive filtering actions

Many report consumer interactions apply filters. These interactions can:

- Slice to propagate filters to other visuals.
- Include/exclude filters.
- Cross filter.
- Drill through to a report page.
- Apply bookmarks.

Include/exclude groups

Report consumers can include or exclude one or more groups in a visual. *Include* keeps the selection and removes all other groups. *Exclude* removes one or more groups from the visual. These filters appear in the **Filters** pane as filter cards titled **Included** or **Excluded**. The report consumer can use the **Filters** pane to view, modify, or remove them.

Cross filter

Report consumers can temporarily cross filter the visuals on the report page by selecting one or more elements in a visual.

By default, cross filtering (or cross highlighting) works between any two visuals. However, the report author can edit visual interactions between any pair of visuals and in either direction. Then, they can disable cross filtering or modify the interaction to use cross highlighting instead.

Report page drillthrough

When the report author designs a report for drillthrough, report consumers can drill from a visual to another report page. By default, the drillthrough action propagates all filters that apply to the visual to the drillthrough page.

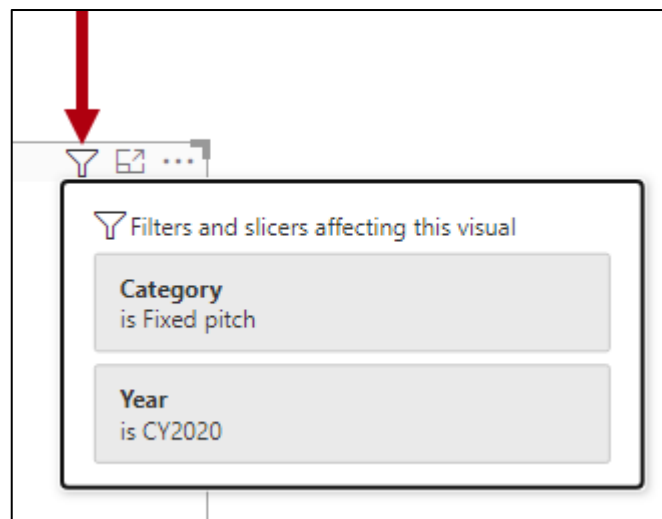
For example, a report consumer modifies slicer values, uses an **Include** filter to filter by a specific month, and cross filters by a specific product. These filters will propagate to the drillthrough page.

Bookmarks

Report consumers can apply bookmarks that apply filters.

Determine applied filters

Occasionally, report consumers want to know (or verify) what filters, if any, apply to a specific visual. They can complete that action by hovering the cursor over the Filter icon in the visual header. A pop-up window will appear, describing the filters that affect the visual. The pop-up window shows the accumulation of filters that are applied by slicers or filters, including any cross filters.



Hidden filters aren't visible in the pop-up window.

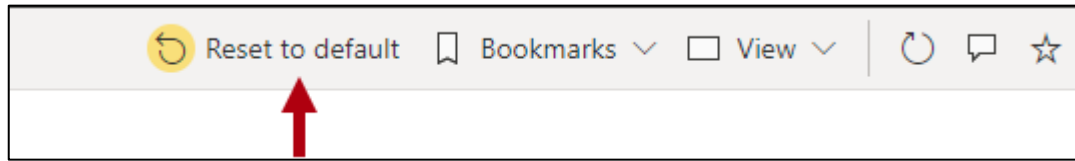
If the **Filter** icon isn't available for a visual, it could be because:

- The report settings disable visual headers for the entire report.
- The visual settings disable the visual header for the visual.
- The visual settings disable the visual header **Filter** icon for the visual.

Persistent filters

Persistent filters are a feature that saves report consumer's slicer and filter settings. It automatically applies the settings when the report consumer reopens the report. That way, Power

BI remembers previously applied filters. You can revert to default filters (that are saved in the report by the report author) by selecting **Reset to default**.



If the **Reset to default** button is not available, it could be because persistent filters is disabled for the report. This button will also reset the state of visuals, including drillthrough location and sorting.

Select Report Filter Techniques

Filters and slicers are two different techniques that can achieve the same outcome. Typically, the decision to use a filter or slicer is a trade-off between performance and design.

Consider that the **Filters** pane will perform better (because Power BI doesn't need to render slicer visuals), and it doesn't take up space on the report page. However, you should also consider that slicers have the formatting flexibility to seamlessly integrate into the report page, creating the most refined experience possible for report consumption.

Many ways are available for you to format a slicer, and the most important consideration is to ensure that it fits into the report design with an intuitive placement and user experience that meets requirements.

Advantages and disadvantages that are associated with the **Filters** pane are as follows:

- Advantages:
 - The **Filters** pane is consistently located on the right side of the report.
 - It allows you to configure advanced filter types, like **Top N**, or allows you to use more complex expressions, like "contains," "does not contain," "is blank," and others.
 - The search box and sort functions (to order filters within the sections) are powerful tools to use when many filters exist.
 - Filters result in faster report rendering because no visual rendering is required.
 - You can lock and/or hide filters.
 - When the **Apply** button is enabled, you can submit all filter updates at once, resulting in fewer queries.
 - You can filter by using measures (only at visual level).
- Disadvantages:
 - The **Filters** pane offers less design flexibility, though you can style it by using specific fonts and colors.
 - Careful consideration should be made to hide certain filters to avoid confusing report consumers.
 - It can be easy to lose track of applied visual-level filters.

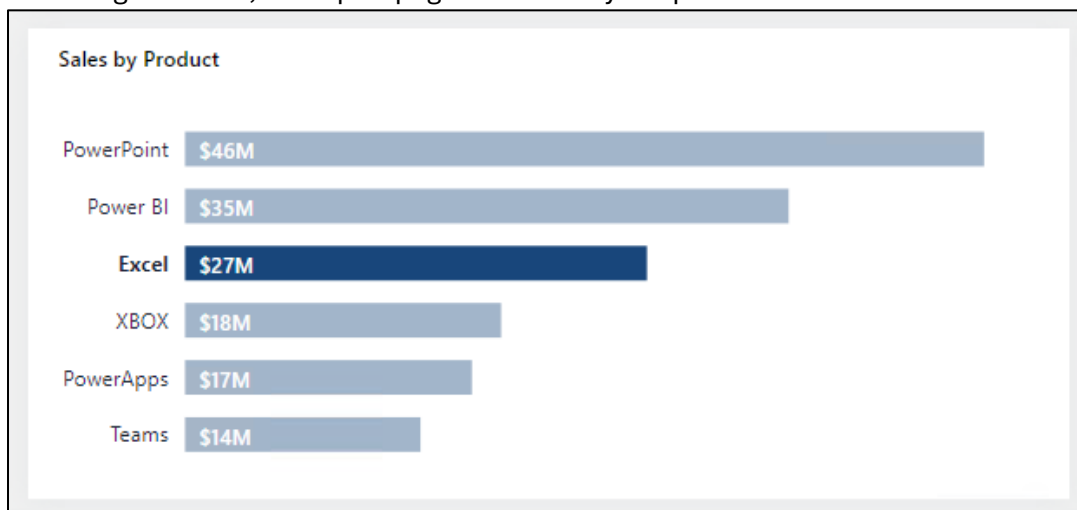
Advantages and disadvantages that are associated with using slicers are as follows:

- Advantages:
 - You can place slicers anywhere on the report page to produce an intuitive layout.
 - Slicers are highly configurable to achieve the required functionality and style.
 - You can configure hierarchical slicers (based on a hierarchy or by using multiple fields that are sourced from the same table).

- They can display their selection (filter context) directly on the report page.
 - They can display images when you are using a field that is categorized as an image URL.
 - They can filter only certain visuals by editing visual interactions.
 - You can filter slicers as you would any visual. For example, you can apply a filter to a slicer to remove the BLANK item.
 - You can sort slicer items.
 - Synced slicers can filter other pages in the report.
- Disadvantages:
 - Slicers can have an impact on report render performance due to the visual render time.
 - Slicer filtering options are less complex than the advanced filter options that are possible in the **Filters** pane. For example, you can't configure a **Top N** slicer.

Bear in mind that you can enable filtering by using visuals. For example, a sales by product bar chart can behave like a slicer when report consumers select a single or multiple bars. However, a downside to this approach is that some report consumers might not be aware that visuals can cross filter other visuals.

In the following bar chart, the report page is filtered by the product Excel.



Tips to help you produce successful report designs include:

- Use either filters or slicers. Avoid using both filter techniques because it can create confusion.
- In the **Filters** pane, consider locking or hiding visual-level filters to avoid confusing report consumers. (Often, report consumer shouldn't modify or see visual-level filters.)
- Create a bookmark to reset all slicers to default values. Then, add a button to the page to invoke the bookmark. For example, the button could be captioned as **Reset slicers**.
- When a requirement is in place to lay out many slicers, consider creating a page that is dedicated to showing all slicers. For example, the page could be named **Slicers**. Sync the slicers to other pages and then set the slicers as hidden on those pages. This design technique will require that report consumers should always go to the **Slicers** page to modify slicer settings. To help them, you can add a page navigation button at a consistent location on each page so that they can easily return to the **Slicers** page.
- Consider using other visuals in place of slicers. Be sure to teach report consumers how to cross filter by using these visuals.

Enhance Power BI Report Designs for the User Experience

Design Reports to Show Details

An analytical report layout can integrate a guided analytical experience. That way, the report design is aligned to the automatic and unconscious processes that take place when the report consumer looks at the report.

The guided analytical experience allows navigation between three levels:

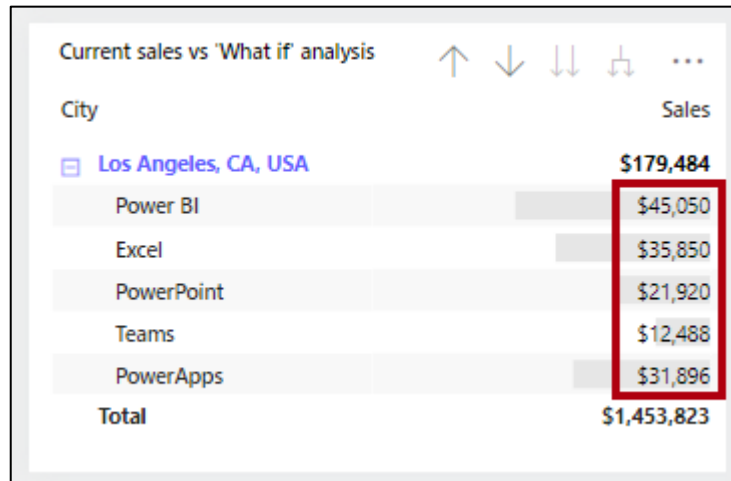
- High-level metrics
- Supporting visuals
- Details, when required

You can show details by using four different techniques:

- Use drillable visuals
- Add tooltips
- Add drillthrough
- Embed paginated reports

Use drillable visuals

The simplest way to show details is to use drillable visuals. The matrix visual is an excellent choice because it allows drilling on rows and/or columns. Therefore, by assigning hierarchies or multiple fields, report consumers can drill to the level of detail that they want. For example, the matrix rows could show years, and the report consumer could drill down to quarter, month, and date levels.



Ensure that the visual size is sufficient for consumers to view drill-down details, or you can teach your report consumers how to use focus mode to enlarge the visual. Also, you can add buttons to provide a quick way for report consumers to drill to specific levels.

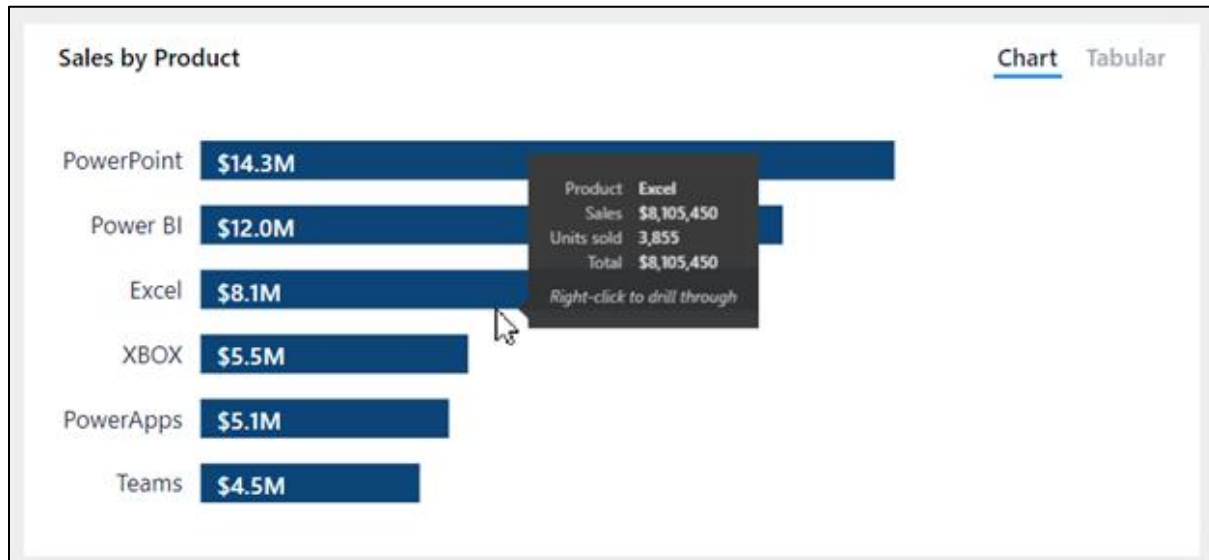
Add tooltips

Adding tooltips is a simple way to reveal details or provide supporting metrics. In Power BI service, tooltips appear when the report consumer hovers the cursor over an element of a report object. An element could be a column of a column chart or a bubble or region in a map. In a mobile app, tooltips appear when consumers are using the tap-and-hold gesture.

Power BI reports support two types of tooltips: visual and page.

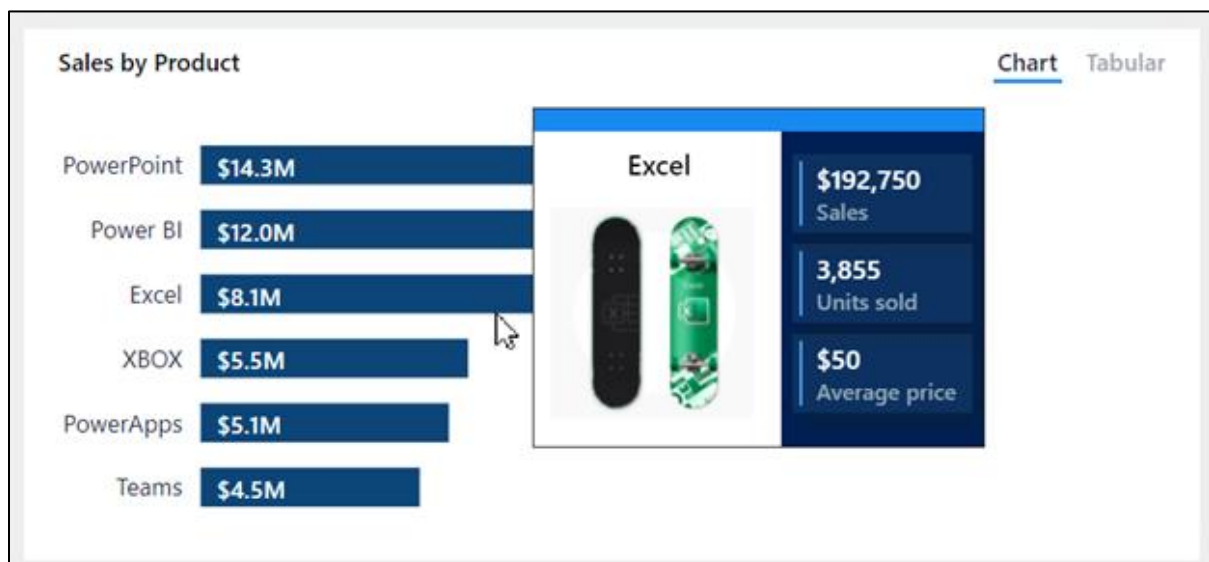
Visual tooltips

Many Power BI visuals include a **Tooltips** well. Fields that are added to this well is summarized in a tooltip. For example, a column chart visual that shows sales revenue by month could include order quantity as a tooltip. While the height of the column reveals the monthly revenue amount, when a report consumer hovers over the column, a tooltip describes the order quantity for that month.



Page tooltips

Page tooltips allow your report consumers to gain deeper insights quickly and efficiently from visuals. As its name suggests, a page tooltip is a report page that receives the filter context of the source element. In this way, they are closely related to drillthrough pages, with the difference being that the page tooltip overlays a small page of visuals.



You can associate page tooltips with different report objects, such as:

- **Visuals** - On a visual-by-visual basis, you can configure which visuals will reveal your page tooltip. For each visual, you can have the visual reveal no tooltip, default to the visual tooltips (which is configured in the visual **Fields** pane), or use a specific page tooltip.

- **Visual headers** - You can configure specific visuals to display a page tooltip. Your report users can reveal the page tooltip when they hover the cursor over the visual header icon. Make sure that you educate your users about this icon.

Page tooltips don't support interactivity. If you want your report consumers to interact with the visuals, create a drillthrough page instead.

Add drillthrough

Drillthrough will navigate the report consumer to a different page, possibly in a different report, and it can show details. The main benefit of drillthrough is that you don't need to clutter a report page with details. Instead, the details are available when required.

A common use case for drillthrough is to allow report consumers to achieve the following flow:

1. View a report page.
2. Identify a visual element to analyze in depth.
3. Right-click the visual element to drill through.
4. Perform complimentary analysis.
5. Return to the source report page.

To enable drillthrough, you can create a report page that provides details. This design approach allows consumers to view supporting transactions and only when needed

Report consumers might not be aware that drillthrough is supported by a visual. To promote visibility, you can add a button that performs drillthrough.

Embed paginated reports

Use the **Paginated report** visual to embed a Power BI paginated report in your report.

Paginated reports can connect directly to source databases, providing you with the ability to present transaction-level detail that's not stored in the Power BI semantic model.

You can map fields from your Power BI semantic model to report parameter values for your paginated report visual, allowing report consumers to cross filter the paginated report. This way, it can behave like a drillthrough page.

In the visual format options, you can enable the toolbar so that report consumers can use the export command.

This command can export the paginated report in many different formats, including Microsoft Excel, PDF, Accessible PDF, CSV, PowerPoint, Microsoft Word, MHTML, and XML.

Also, unlike the 150,000-row limitation that is imposed by Power BI reports, paginated reports can export up to 1,000,000 rows to Excel while retaining all formatting of the paginated report.

Design Reports to Highlight Values

Highlighting values in reports helps you to quickly communicate important and relevant facts about data. Power BI supports several techniques to highlight values:

- Conditional formatting
- Overlaid analytics
- Anomaly detection
- Specialized visuals

Conditional formatting

You can use conditional formatting to format table and matrix visuals. With this approach, you can apply rules to highlight specific cells by using:

- Background color
- Font color
- Data bars
- Icons

The following image shows an example of a matrix visual with icons. The icons highlight the **On hand** values.

Consider that report consumers might be blind or have low vision. Take care not to highlight by using only color. Instead, consider using icons that can use shape and color to communicate status.

City	Units sold	On hand
 Los Angeles, CA, USA	394	850 
 Oakland, California, USA	226	459 
 Sacramento, CA, USA	527	2,780 
 San Diego, CA, USA	412	1,550 
 San Francisco, CA, USA	733	1,697 
 San Jose, California, USA	324	1,056 
Total	2705	8,536

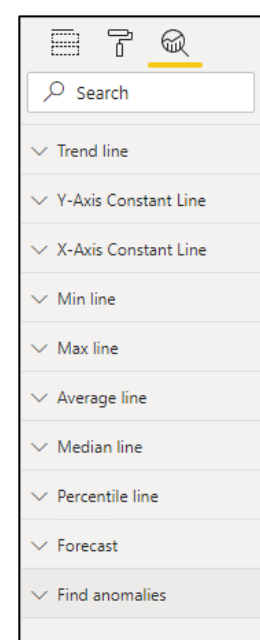
Overlaid analytics

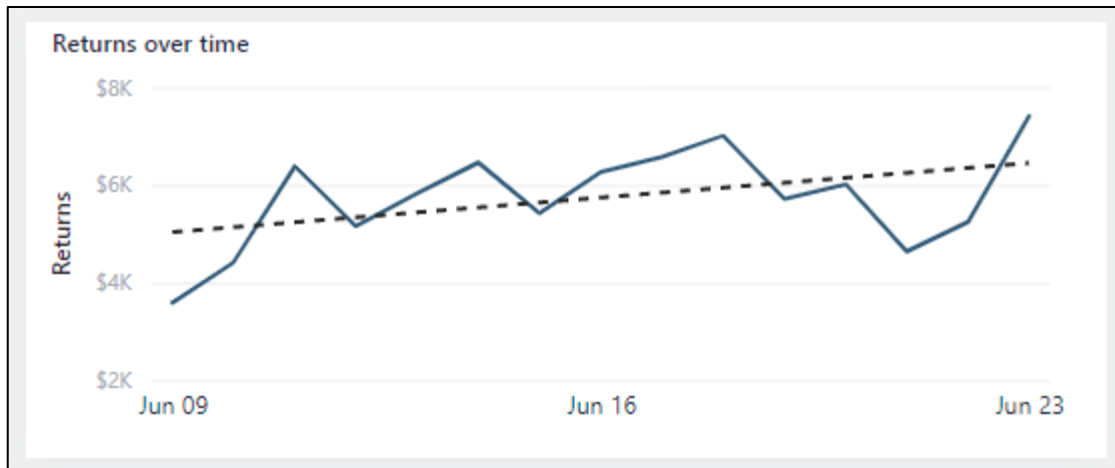
You can also overlay analytics to highlight values. Certain visuals, like the line chart visual, allow you to add analytic options.

Options include trend lines, constant lines, minimum or maximum lines, and many others.

The following image is an example of a trend line.

The upward gradient of the trend line clearly indicates that returns are increasing over time.





Anomaly detection

When a time series (date field) is on the X-axis, two additional analytic options are available.

These options use AI to produce a forecast or find anomalies. The anomaly detection feature can help you highlight exceptional values.

The following example shows a line chart visual. Power BI detected an anomaly on a particular date and marked it with a shape. When the report consumer hovers the cursor over the shape, a tooltip appears that describes expected values.



Select the anomaly shape to open the **Anomalies** pane, which lists a series of visuals that provide further possible explanations.

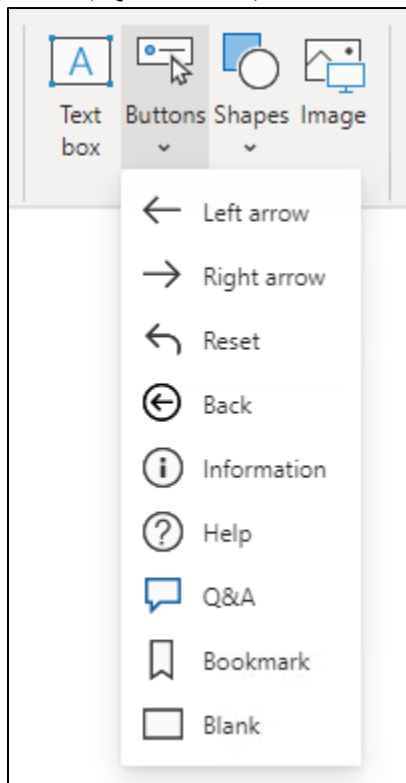
Specialized visuals

Some specialized visuals can help to highlight values in your reports; specifically, the **Key influencers** visual and the **Decomposition Tree** visual.

Design Reports That Behave Like Apps

The button element can take your Power BI reports to the next level by making them behave like an app. Buttons support six different actions that take effect when the button is selected.

When you are inserting a button, many preconfigured buttons are available for you to choose from. These preconfigured buttons include icons and preconfigured actions, such as the Back button, Q&A button, and Bookmark button.



You can assign any type of action to images and shapes so that these elements can behave like buttons.

The **Back** action navigates to the previous page. It's useful in drillthrough scenarios, allowing the report consumer to quickly return to where they drilled from.

The **Bookmark** action activates a selected bookmark. Bookmarks can turn a Power BI report into a guided analytical experience, maximize available page real estate, and provide user-friendly interactions. Bookmarks are covered in the next unit.

The **Drillthrough** action is assigned a target drillthrough destination page. The button remains disabled until drillthrough becomes a valid action, which is the case when you are interacting with a visual that can navigate to the drillthrough page.

When the **Drillthrough** action is selected, Power BI navigates to the drillthrough page, propagating appropriate filters.

The **Page navigation** action directs the report consumer to a specific report page. The page can be a specific page that is assigned at design time or a measure that returns the page name. Using a measure allows Power BI to dynamically determine the page based on the filter context.

A measure is a model object that summarizes data. When assigned to the **Page navigation** action, measure formulas, which are written in Data Analysis Expressions (DAX), must return a text value that represents the page name.

The **Q&A** action opens a pop-up window that allows the report consumer to explore data by using intuitive, natural language capabilities and receive answers as data visualizations. Unlike the **Q&A** visual, a button that is assigned the **Q&A** action doesn't occupy significant space on the report page.

The **Web URL** action opens the URL by using the default web browser. Like the **Page navigation** action, this action can be a specific URL or one that is returned by a measure. The measure can produce a URL that appends filter context to the query string. For example, if the report consumer filters the page by a single customer, the measure can return a URL to an external system that includes the customer key in the query string.

Consider using the **Web URL** action to create a bridge to resources that are commonly referenced in tandem. **Web URL** actions can be highly effective in prompting direct action in external systems.

Work with Bookmarks

Bookmarks are a powerful way to turn a Power BI report into a guided analytical experience. Bookmarks can also help maximize available page space and provide user-friendly interactions. The possibilities are nearly endless, spanning from simple outcomes, such as resetting filters, to more complex behaviors that swap visuals or navigate visual depth.

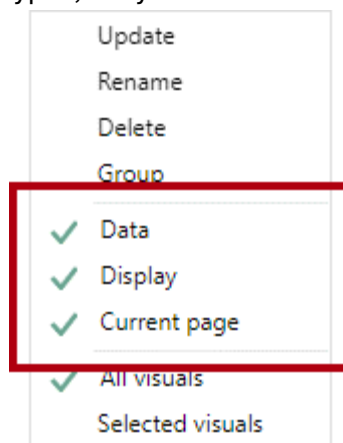
Commonly, you will add buttons to a report design and configure their actions to apply a bookmark. Shapes and images can also apply bookmarks.

To understand what you can achieve with bookmarks, you need to know how to configure them. You can configure the state that they capture and the scope of visuals that they effect.

To ensure that all actions produce the intended result, be as prescriptive as possible when deciding on the enabled states and scope.

Bookmark state

Bookmarks capture different state, relating to data, display, and the current page. By default, a new bookmark captures all state types, but you can decide to disable any of them.



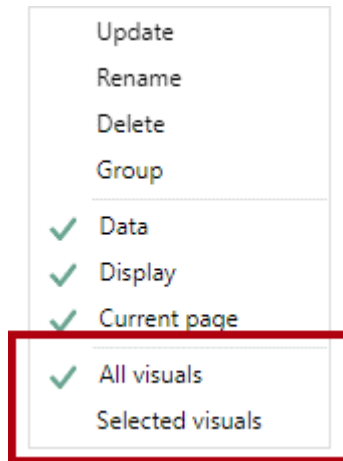
The **Data** state captures anything that impacts the queries that Power BI sends to the semantic model. For example, if a slicer is included in the scope of the bookmark, the **Data** state retains the applied slicer items when the bookmark was created (or updated). It will also capture sort order and the drill depth of a visual because the query is impacted.

The **Display** state is related to the visibility of a report object. Objects consist of visuals and also elements like textboxes, buttons, shapes, and images. By using the **Selection** pane, you can hide or unhide objects and groups of objects. Additionally, you can swap visuals on a report page by creating bookmarks that capture hidden and unhidden objects.

The **Current page** state determines whether the bookmark will direct the report consumer to the bookmarked page or apply the current page. Disabling the **Current page** state is rare, but you should consider some creative use cases. For example, on a page tooltip, a bookmark can change the visuals without navigating from the page that the report consumer has selected.

Bookmark scope

Bookmarks can apply to all page visuals or specific visuals that you select.



The **All visuals** scope is turned on by default, meaning that the bookmark applies to all report objects, even if hidden.

The **Selected visuals** scope will target only those visuals that are selected when the bookmark was updated.

Press the **Ctrl** key to select multiple visuals. The simplest way to select multiple visuals is to use the **Selection** pane.

Also, when you are creating a selected visuals bookmark that shows/hides several objects, create a group of those objects and configure visibility for the group. That way, you can add or remove objects from the group without the need to update the bookmark.

Setting **Bookmark** scope is best described with an example. For a demonstration on the use of the **Selected visuals** scope, watch the following video.

Bookmark examples

By combining different bookmark states and scopes, you can create intuitive experiences.

Make sure that you include bookmark requirements in all design decisions. For optimal experiences, you should create bookmarks at the end of report development after you have added all report elements. This approach will help you avoid the complex and time-consuming updating of bookmarks when you're adding new elements.

The ensuing sections highlight common use cases for bookmarks.

Reset slicers

You can provide a simple way for report consumers to quickly reset slicers to a default state by following these steps:

1. Configure the bookmark to capture the **Data** state.
2. Configure the bookmark to use the **Selected visuals** scope, targeting the slicers that you want to reset. The **Selected visuals** scope is key because you don't want to impact other slicers or visuals. For example, if a drillable visual is on the page, the bookmark shouldn't overwrite the drill state when it resets the slicers.
3. Set the slicers to the default values.
4. Update the bookmark.
5. Assign the bookmark to a button action.

Swap visuals

Swapping a visual means replacing it with another visual. For example, your report can allow the report consumer to select the type of visual, perhaps a chart or a table. Alternatively, you might allow the report user to determine the unit system, whether it's metric or imperial.

Swapping a visual requires two bookmarks, and you can do so by following these steps:

1. Configure each bookmark to capture the **Display** state but not the **Data** state. These settings will preserve any sorting or additional filtering.
2. Configure each bookmark to use the **Selected visuals** scope, targeting the initially visible visual and the hidden visual.
3. Update the first bookmark, with one visual as visible and the other as hidden.
4. Update the second bookmark by using the inverse visibility state.
5. Assign the bookmarks to button actions.

No performance is impacted by having hidden visuals on a page. Hidden visuals don't run queries.

Swapping visuals will provide flexible consumption options to the report consumer while retaining optimal performance and making the most out of the report page space.

Drill down multiple visuals and direct depth navigation

When visuals have many levels of drill depth, buttons and bookmarks can provide report consumers with a quick and simple way to arrive at the required depth.

You can create a bookmark for each required drill depth with the following steps:

1. Configure each bookmark to capture the **Data** state, which preserves the drill state of visuals.
2. Configure each bookmark to use the **Selected visuals** scope, targeting the visuals to update.
3. In the visuals, drill down to the required level, and then update the corresponding bookmark.
4. Assign the bookmarks to button actions.

In the **Bookmark** scope, consider including a shape or alternate button that appears as highlighted. That way, report consumers will be aware of the selected drill level.

Pop-up overlays

Use buttons and bookmarks to overlay an image, shape, or text box. A good example of a pop-up overlay is one that provides built-in assistance.

In the following example, when the report consumer selects the **Help** button in the lower-left corner of the page, a full-page image becomes visible.

The image includes many coach marks, which point to the targeted objects on the report page.

Also, the image uses transparency so that the underlying report design remains partially visible.

When the report consumer selects anywhere on the page, the image becomes hidden (due to an action on the image that uses a bookmark).

Two bookmarks are required for creating a pop-up overlay:

1. Configure the first bookmark to capture the **Display** state.
2. Ensure that the overlay object is visible, and then update the bookmark.
3. Assign the bookmark to a **Help** button action.
4. Configure the second bookmark to capture the **Display** state.
5. Ensure that the overlay object is hidden, and then update the bookmark.
6. Assign the bookmark to the overlay object.

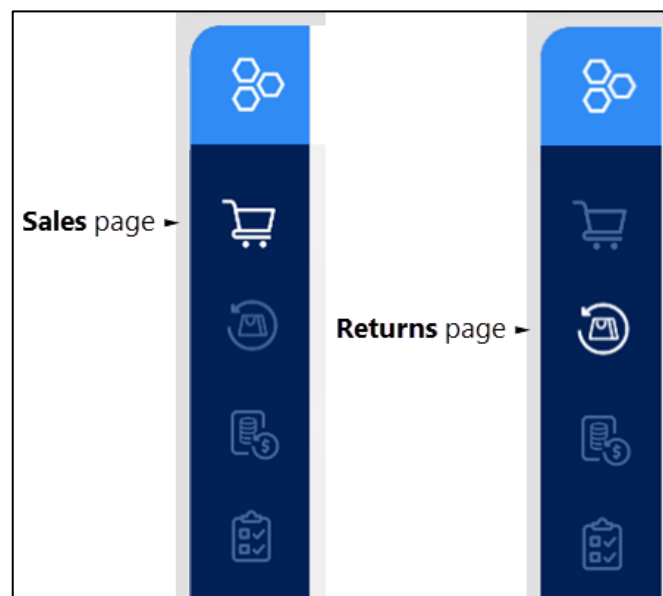
Design Reports with Navigation

Many different button actions are available to help you configure report navigation experiences. While buttons enable navigation, the report and button layout is critical to ensure that report consumers can efficiently navigate within the report.

To ensure seamless integration into your report design, these UX components should be considered early in the design process. Retrofitting buttons and actions to a report design can lead to an unintuitive and disjointed experience for report consumers.

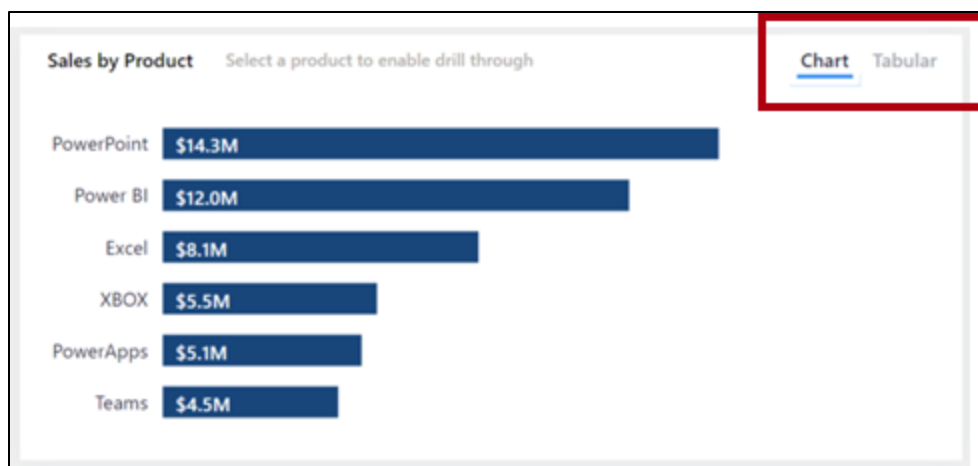
Buttons should be placed in relevant and consistent locations. A button is only valuable if it's easy to find. To promote ease of discovery, ensure that buttons are placed in a relevant and consistent location on the report page.

In the following example, buttons that are located along the left side of the report page allow report consumers to navigate between pages. Most important, they're placed in the same location and in the same order on all report pages. The button that represents the current page is formatted in a way to highlight the icon.



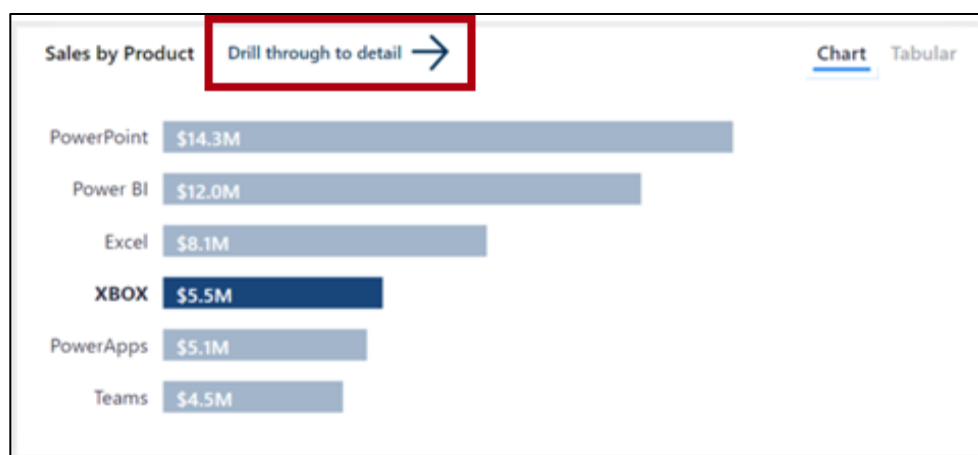
When a button only impacts one section of a report, you can place it within that section so that it's clear to report consumers that the action applies only to that section.

In the following image, two buttons are located in the upper-right corner of a section that allows the report consumer to switch between visual types.



Similarly, if a button action drills through to a page, place it as close as possible to the visual that activates it.

In the following image, the **Drill through to detail** button is located above the visual that activates it.



On the drillthrough page, ensure that a back button will provide a simple way for the report consumer to return to where they drilled from.

An established convention is to use a left-pointing arrow icon for the button.

Power BI automatically adds a back button, which is located in the upper-left corner of the page. The button icon is a left-pointing arrow.

ONTOSO
CATEGORY: STORE

SALES DETAIL

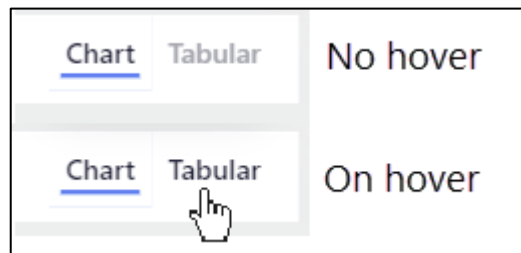
Select a date range: 6/9/2021 to 6/23/2021

Type	Category	Product	Store	City	Date	Price	Sales
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/09/2021	\$1,200	\$12,279
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/10/2021	\$1,200	\$12,538
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/11/2021	\$1,200	\$11,070
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/12/2021	\$1,200	\$9,142
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/13/2021	\$1,200	\$9,200
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/14/2021	\$1,200	\$10,988
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/15/2021	\$1,200	\$8,898
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/16/2021	\$1,200	\$7,660
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/17/2021	\$1,200	\$7,884
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/18/2021	\$1,200	\$3,470
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/19/2021	\$400	\$2,684
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/20/2021	\$400	\$3,330
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/21/2021	\$400	\$2,381
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/22/2021	\$400	\$2,300
In store	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/23/2021	\$400	\$1,876
Online	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/09/2021	\$2,400	\$22,217
Online	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/10/2021	\$2,400	\$24,798
Online	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/11/2021	\$2,400	\$19,198
Online	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/12/2021	\$2,400	\$16,766
Online	XBOX	XBOX	Contoso	Los Angeles, CA, USA	06/13/2021	\$2,400	\$15,512
Total							\$840,000 \$5,501,560

Buttons need to be clearly interactive. If a button doesn't appear to be interactive, the report consumer might not notice it. To help in their discovery, you can apply two formatting options: on-hover formatting and descriptive tooltips.

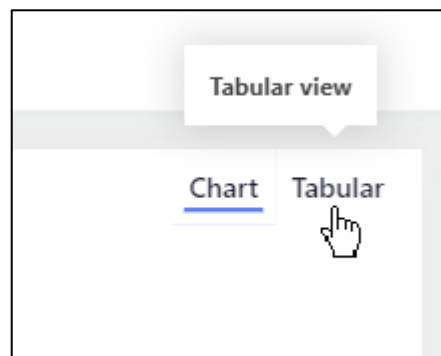
On-hover formatting

You can configure buttons to change their state when a hover over is available. When the report consumer hovers the cursor over a button, the cursor icon changes to a pointing finger, which indicates that the button is selectable. In this example, the **Tabular** button text is light gray by default, but when they hover the cursor over, it darkens to black.



Descriptive tooltips

In addition to on-hover formatting, you can configure tooltips to concisely communicate what the button does.



Enable and enter the tooltip text in the **Action** section for the button.

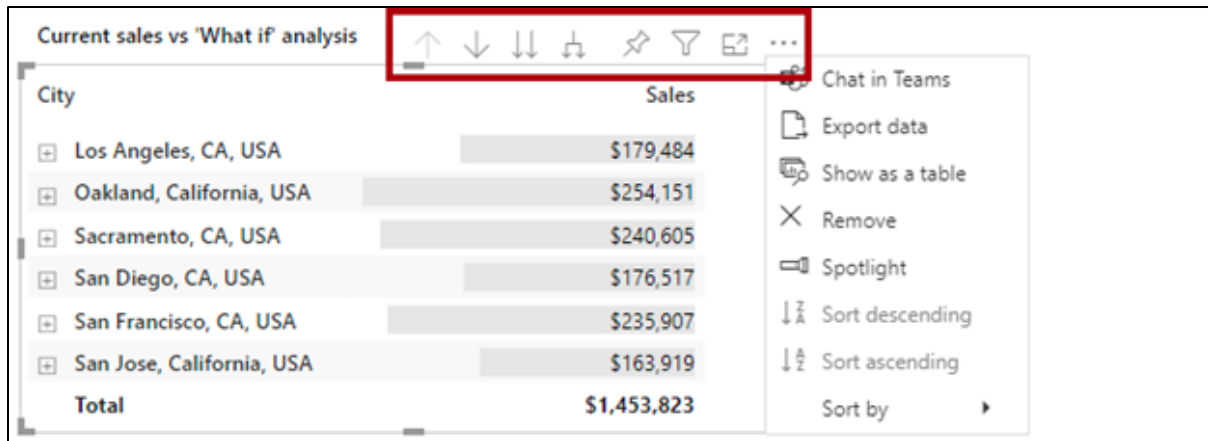
Work with Visual Headers

Visual headers make many interactions available to report authors and consumers. All report objects, including visuals and elements (like buttons), have a visual header.

The visual header appears when you hover the cursor over the report object, and they can launch actions like focus mode, drill up, or drill down.

You can also access the **More options** menu by selecting the ellipsis (...) button. This menu includes sorting options, export, spotlight, and many others.

By default, visual headers are enabled for all report objects.



Visual headers might appear in the upper or lower part of the object. Power BI attempts to show visual headers in the upper right of the object if sufficient space is available between the upper part of the object and the edge of the page. If no space is available in the upper part, Power BI will place the visual headers in the lower-right corner of the visual. If no space is available in either location, it will place the headers inside the visual in the upper right.

Always leave sufficient space for the visual headers to appear in the upper-right section of objects. This approach ensures a consistent experience for report consumers. Also, take care not to overlap objects because superimposed visual headers make it difficult for consumers to see or select icons.

A report setting is available for you as the report author to disable all visual headers. When visual headers are disabled, they will still be visible when editing the report, but they will be hidden for report consumers (in reading view).

However, situations might occur where enabling this option is a good idea because it limits most interactions. It might make sense to disable visual interactions when you are delivering a Power BI report to an audience that's unfamiliar with Power BI or when you don't expect them to interact with the report.

Visual headers can be turned off for a single object. A good report design turns off visual headers for objects that don't need them. Therefore, consider turning off visual headers for slicers and buttons and any other object that won't be used to launch actions. Fewer visual headers will produce a cleaner and less distracting end result.

For those objects that have visual headers, consider disabling functionality that's not appropriate or useful. You can disable all visual header icons, including the **More options (...)** menu.

At design time, the icons will still appear to help you create the report. However, when the report opens in reading view, the icons won't appear.

By enabling the visual header tooltip icon, you can enter a text value or reference a page tooltip.

Designing Reports with Built-In Assistance

Training report consumers on how to use and navigate your reports can be difficult because every report that you create will be different. Features and functionality that are present in one report might not be present in another.

Creating intuitive and simple-to-navigate reports is important. When approaching a report design project, you should always aim to provide "no training required" experiences. Strive to add built-in assistance whenever possible.

Several design techniques that can provide built-in assistance include:

- Information page
- Visual header tooltip icon
- Button with overlay

Every report consumer will approach a report with a different expectation of how to interact with it.

Therefore, to avoid handling support requests, take extra initial effort to ensure that the functionality is as discoverable as possible.

Information page

Adding an **information page** is the most simplistic technique. Dedicate an entire report page that includes instructions and definitions.

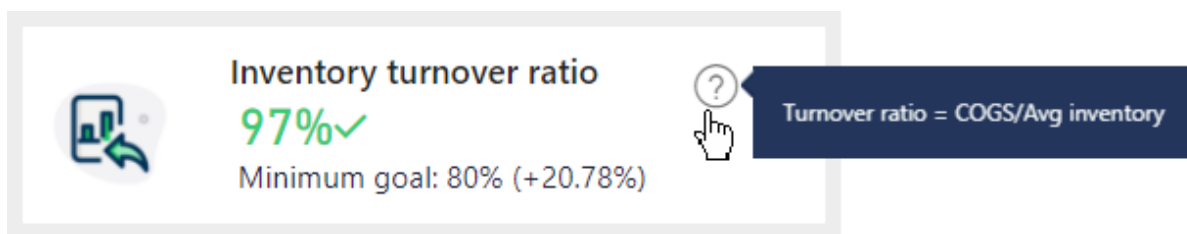
Consider adding a back button to the page. Then, add a button in a consistent location on each page that navigates to the information page.

Configure these buttons to use the **Information** or **Help** icon.

Visual header tooltip icon

Within the visual header options, you can enable the visual header tooltip. It adds the **Help (?)** icon to the visual header.

Then, you can assign a page tooltip or text. This technique is beneficial for providing visual-specific guidance, such as more detail that describes definitions, calculation logic, or the source of data.



Button with overlay

You can use buttons and bookmarks to overlay an image, shape, or text box that provides built-in assistance.



Tune Report Performance

When you have finished creating your report, the performance of that report depends on how quickly data can load onto the report page. You should test your report in the Power BI Report Server to see how it works from a user's perspective.

If you experience issues, or if the report users have reported issues, you need to investigate the cause of those issues and take measures to tune the report for more optimized performance.

Analyze performance

To investigate the cause of issues, your first step is to use the **Performance analyzer** tool within Power BI Desktop. **Performance analyzer** allows you to discover how each of your report elements, such as visuals and DAX formulas, are performing.

Performance analyzer provides you with logs that measure (in time duration) how each of your report elements performs when users interact with them.

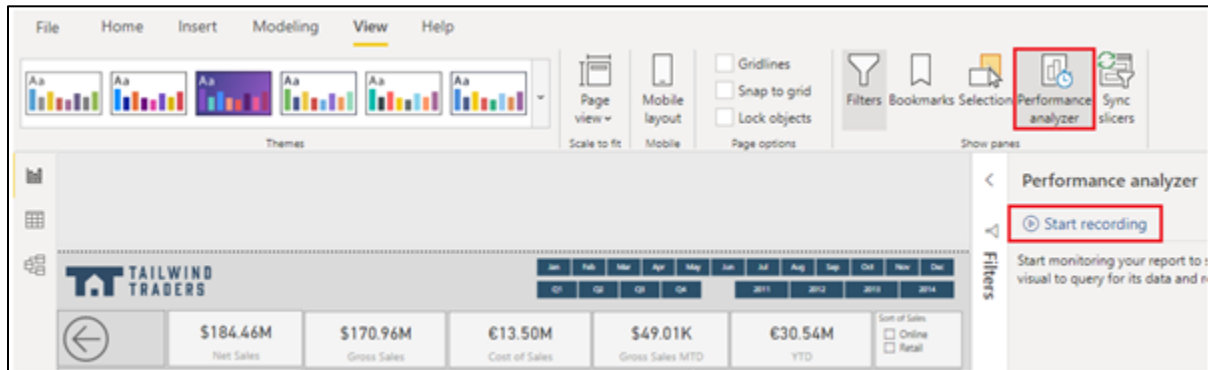
By examining the durations in the logs, you can identify which elements of the report are the most (or least) resource intensive. You can find where bottlenecks exist, which is a good starting point for making changes.

Before you run **Performance analyzer**, ensure that you clear the visual cache and data engine cache; otherwise, the results will not be accurate. Also, you should set up the report so that it opens on a blank page.

When you have cleared the caches and opened the report on the blank page, to run the **Performance analyzer**, go to the **View** tab, select **Performance analyzer**, and then select **Start recording**.

Interact with your report as you would expect a user to, and then stop the recording. The results of your interactions will display in the **Performance analyzer** pane as you work. When you are finished, select the **Stop** button. Then, you can analyze the results in the **Performance**

analyzer pane. Performance results of each item in the report will display, in milliseconds, under the **Duration** column. The following image shows that all items on the report take less than two seconds to load, which is acceptable. You can expand an item in the list to view more detailed information and identify the exact cause of the issue, such as the DAX query, the visual display, or something else (other).



Performance analyzer	
Start recording	Refresh visuals Stop
Clear Export	
Name	Duration (ms)
Recording started (16/06/2020 00:13:52)	-
Changed page	-
Net Sales	1435
MTD card	1491
DAX query	46
Visual display	31
Other	1414
Copy query	
YTD card	1983
Cost of Sales	1550
Cross Sales	1655
Gross Sales by Year by Month	1312
Net Sales by Country	1831
Top 10 Products	1355
Top 10 Customer	1587
Product Category	2023
Sort of Sales	1034
Slicer	1726
Slicer	1190
Slicer	1896
Image	305
Button	115

If you want to examine the DAX query, select **Copy query** and then paste it into DAX Studio for further analysis.

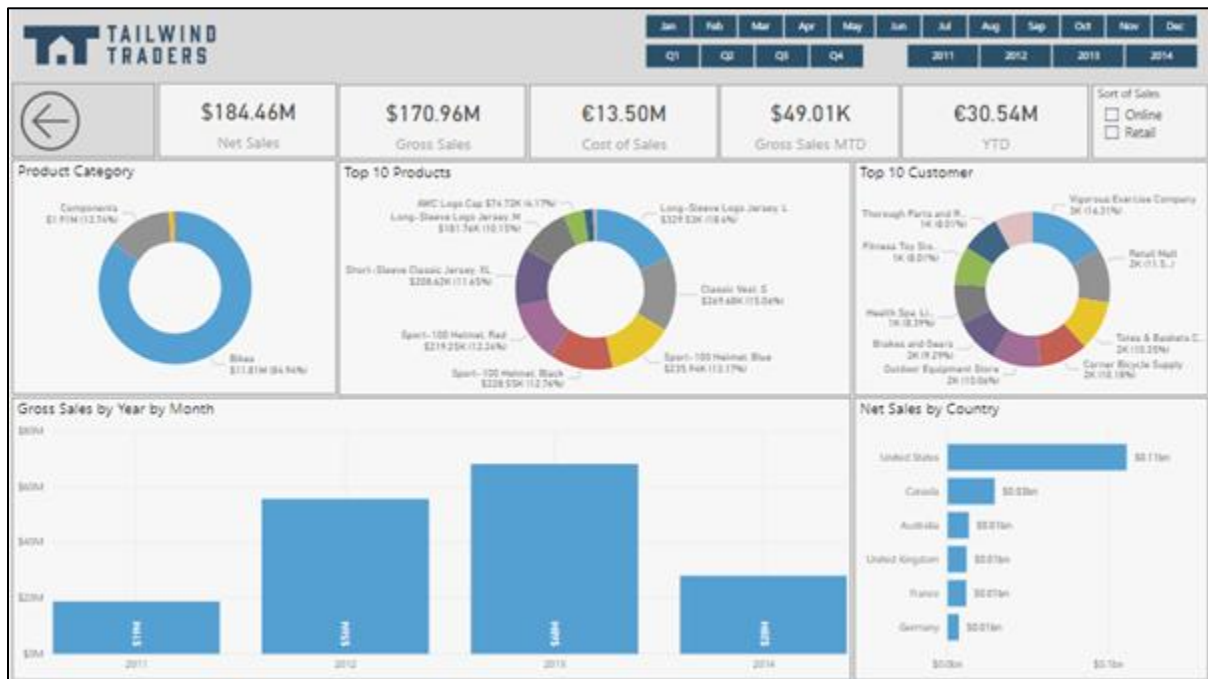
DAX Studio is a free, open-source tool that is provided by another source that you can download and install on your computer.

Tune performance

The results of your analysis will identify areas for improvement and highlight items that you need to optimize.

A common reason for poor performance is too many visuals on the same page.

The following image shows an example of a busy page that contains several visuals.



If you identify visuals as the bottleneck that has caused poor performance, take the following measures to tune the report:

- Reduce the number of visuals on the report page because fewer visuals means better performance. If a visual is not necessary and doesn't add value to the user, you should remove it. Rather than using multiple visuals on the page, consider other ways to provide additional details, such as drillthrough pages and report page tooltips.
- Reduce the number of fields in each visual. The upper limit for visuals is 100 fields, so a visual with more than 100 fields will be slow to load (and will look cluttered and confusing). Identify fields that are not valuable to the visual and then remove them.

If you find that visuals are not causing the performance issues, you should assess the DAX query results that are displayed in the **Performance analyzer** pane and investigate those results further. For example, you might need to look elsewhere in your semantic model, such as the relationships and columns.

In circumstances where you have made necessary changes to tune report performance and have established that the report is performing well, but some users are still experiencing poor performance, other factors might be affecting performance.

These factors include the bandwidth, server, firewall, and other external, uncontrollable factors. You might need to speak to the IT team in your organization to see if they can explain why users are experiencing poor performance when using your reports.

Perform Analytics in Power BI

Introduction

Analytics encompasses emerging industry practices, such as data mining, big data analytics, machine learning, AI, and predictive analytics. It is a term that is used to describe the technical aspects of analytics that have predictive capabilities and can be used to solve business problems.

Analytics can transform raw data into an extensive collection of information that categorizes data to identify and analyze behavioural data and patterns. Organizations can use this information to analyze the current state of their operations and to predict future behaviour and trends by asking "what-if" questions. Additionally, analytics can help with fraud detection, image recognition, sentiment analysis, overall general employee productivity, and it also often replaces cumbersome manual processes.

Traditionally, data analysis was a complex task that was carried out by engineers. Today, data analysis is more accessible to, and understood by, many people within organizations, across all teams. Power BI is an exceptional tool for quickly pulling actionable insights from data. It allows you to build visuals and metrics for your data in reports and dashboards so that you and your users can analyze data insights at a high level and drill down into those insights for more detailed information.

The inherent functionality of Power BI will help you accomplish this task. You can develop quick insights and share them in reports and dashboards with different teams within the organization. The advanced analytics capabilities of Power BI will enable you to identify categories and trends, see how data changes over time, and much more. From this information, you can make predictive semantic models and, therefore, help your organization make more robust business decisions, plans, and forecasts.

Explore Statistical Summary

Data is often intertwined with statistics because statistics are one way in which you can explore your data. Statistics show you the distribution of your data and help you to identify key takeaways and trends and determine whether outliers exist.

The statistical summary is the information that provides a quick and simple description of your data. Power BI has many functions that help you to conduct a statistical analysis, such as Data Analysis Expressions (DAX) functions, visuals such as histograms and bell curves, advanced analytics visuals, and statistical programming languages such as Python and R.

Exploring the statistical summary gives the user a high-level view of the available data, where they can see clusters, patterns on behavioural data, data averages, and more. They can gain insights about their data that will help drive business decisions.

Power BI Desktop has a number of DAX functions that you can use to get quick statistics based on your data. You can access these quick functions by right-clicking the **Values** field in the **Visualizations** pane.

However, to avoid performance issues, it's better to create the statistical measures yourself by using DAX functions to calculate average, sum, min, max, and so on.

Histogram

Histograms and bell curves are the most common way to display statistics about your semantic models. In Power BI terms, you can represent a histogram with one of the bar or column chart visuals and represent a bell curve with an area chart visual.

A typical bar or column chart visual in Power BI relates two data points: a measure and a dimension. A histogram differs slightly from a standard bar chart in that it only visualizes a single data point.

Top N analysis

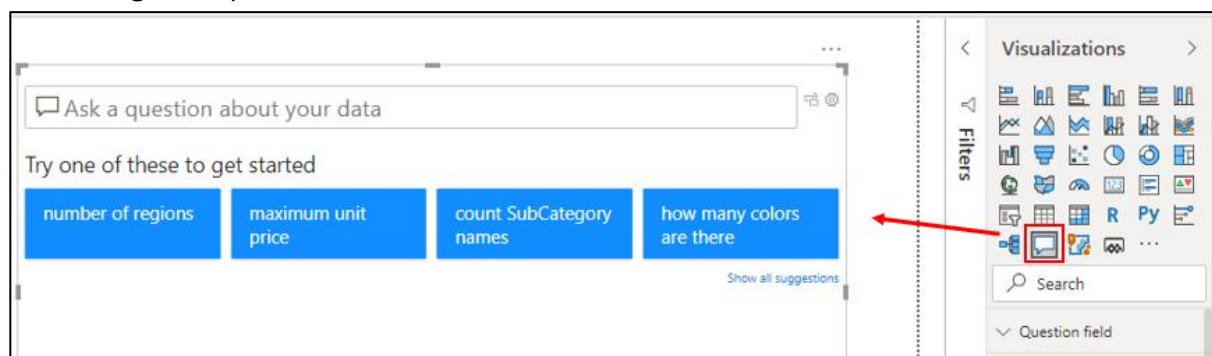
The TOPN DAX function returns the top N rows of a specified table. The Top N analysis is a great way to present data that might be important, such as the top 10 selling products, top 10 performers in an organization, or top 10 customers. Alternatively, you can look at it from the other perspective and present the bottom 10 items in a list, in other words, the worst performers. Depending on the requirements, you might want to use one or both of these options.

In this example, the Supply Chain team wants to know what the top 10 selling products are. You accomplish this task in one of three ways: by using a Q&A visual, using a Top N filter, or writing a DAX formula.

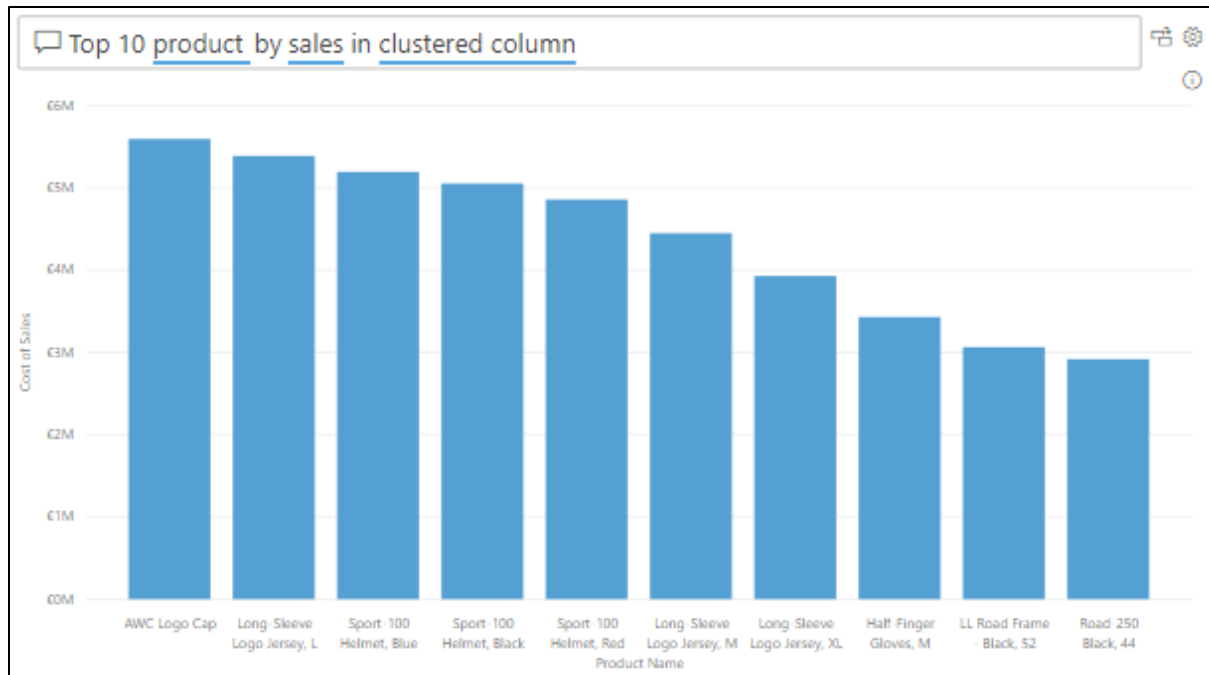
Use the Q&A visual to find the top N

You've created a report for the Supply Chain team, and now the team members have questions about various other views or insights that they are interested in. Power BI has a built-in Q&A visual that allows users to ask their own questions and get answers so you don't have to address each individual question. The Q&A visual is an effective tool because it allows users to quickly get answers about the data independently, which saves time for everyone involved. The Q&A visual is unique in that it does not require knowledge of Power BI to use the visual; users can ask their question and they, too, can create insightful visuals.

Add the **Q&A** visualization to your report, and then reposition the visual and customize its formatting, as required.

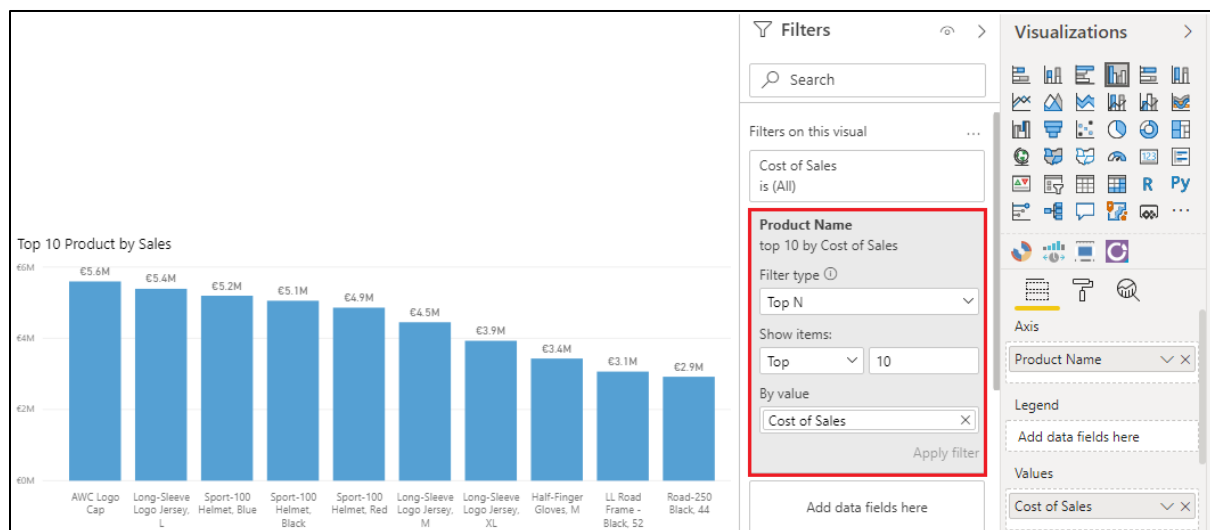


Now, you can use the visual to get answers. In this case, you want to know what the top 10 selling products are, so you enter a question such as, "What are my top 10 products by sales?" Power BI will automatically display those results for you.



Use a Top N filter type

Top N is a filtering option that is available on the **Filters** pane. Select the field that you want to analyze on your report page (in this example, it's the **Product Name** field). In the **Filters** pane, expand the **Filter type** list and select **Top N**. In the **Show items** settings, select **Top** and **10**. Then, select **Cost of Sales** as the value that you want to filter the field by. The visual updates accordingly.



Use a TOPN DAX function

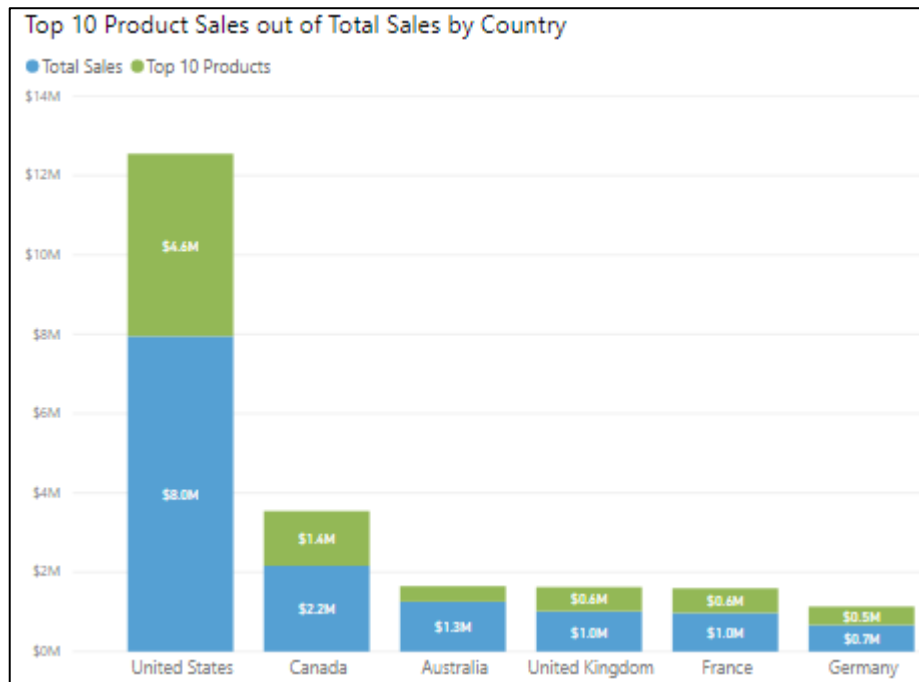
You can also calculate your top 10 products in DAX by using the TOPN function. This option could be useful if you want to present the top 10 in a different context, such as how much of the top 10 best-selling products contributed toward the overall total sales.

Start by creating a new measure called **Top 10 Products**. Then, use the TOPN function, along with the SUMX function, to calculate your top 10 products by total sales, as follows:

Top 10 Products =

SUMX (TOPN (10, Product, Product[Total Sales]), [Total Sales])

The following image shows the top 10 products versus total sales for comparison.



You can adjust the DAX formula to present the same result in percentages.

Identify Outliers with Power BI Visuals

An outlier is a type of anomaly in your data, something that you didn't expect or that surprised you, based on historical averages or results.

You will want to identify outliers to isolate data points that significantly differ from other data points, and then take action to investigate the reasons for the differences.

The results of this analysis can make a significant impact on business decision making.

Consider the scenario where you are analyzing data for a shipping warehouse. You notice that the number of orders increased above average for a specific product category. You first want to identify the product category. Then, you want to ask several questions about the outlier:

- Did above average shipments happen that day?
- Did this anomaly occur in a specific warehouse?
- Did a single event cause the increase in orders for that specific category?
- Did this event occur on other days in the last month, quarter, year, or prior year?

Power BI allows you to identify outliers in your data, but you need to first determine the logic behind what constitutes an outlier.

You can use trigger points, such as calculations, around what you would consider the outlier to be.

The process of identifying outliers involves segmenting your data into two groups: one group is the outlier data and the other group is not. You could use calculated columns to identify outliers, but the results would be static until you refresh the data. A better way to identify outliers is to use a visualization or DAX formula because these methods will ensure that your results are dynamic.

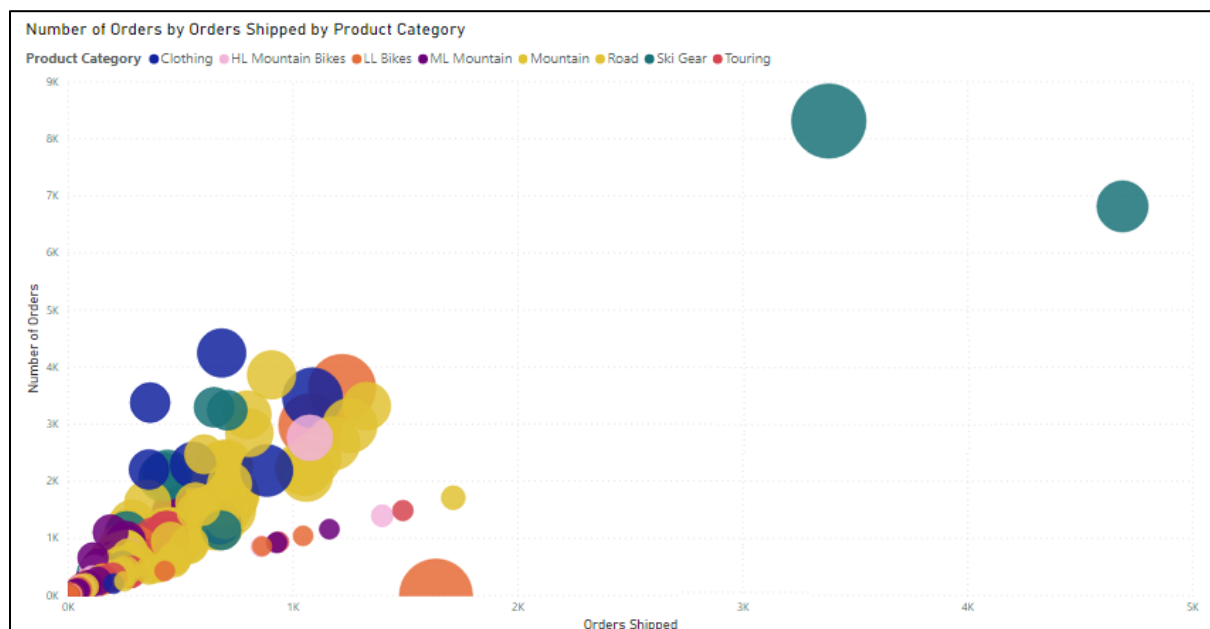
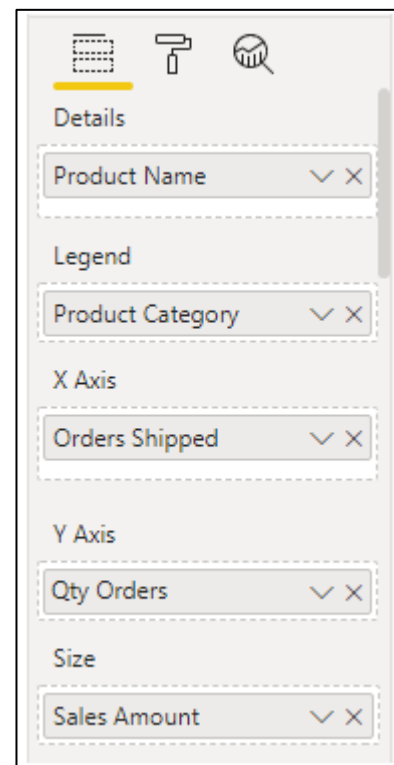
When you have identified the outliers in your data, you can then use slicers or filters to highlight those outliers. Additionally, you can add a legend to your visuals so that the outliers can be identified among the other data. You can then drill in to the outlier data for more detailed analysis.

Use a visual to identify outliers

The best visual to use for identifying outliers is the scatter chart, which shows the relationship between two numerical values. Scatter charts display patterns in large sets of data and are, therefore, ideal for displaying outliers.

When you add a scatter chart to your Power BI report, you put your fields of interest in the **X Axis** and **Y Axis** sections, respectively. In this case, the **Orders Shipped** field is on the x-axis, and the **Qty Orders** field is on the y-axis.

The visual will update to display the data according to the selected fields, and you'll be able to clearly identify the outliers in that data; they are the isolated items that are away from the bulk of the data.



Now that you can identify the outliers in your data, you can investigate the reasons for their existence and take corrective action.

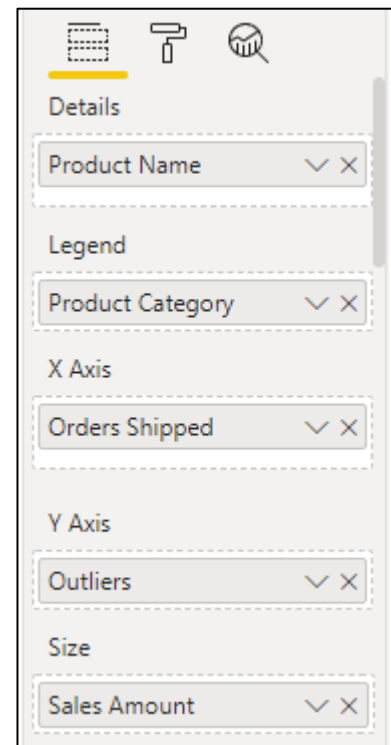
Use DAX to identify outliers

You can use DAX to create a measure that will identify the outliers in your data, such as in the following formula:

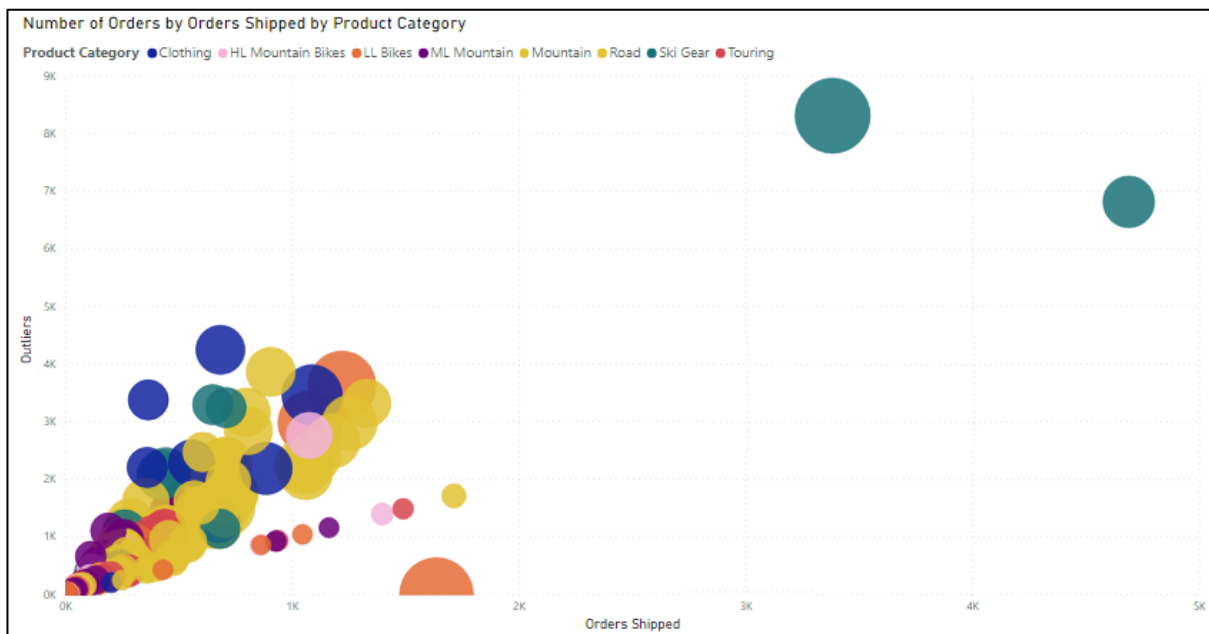
```
Outliers =
CALCULATE (
    [Order Qty],
    FILTER (
        VALUES ( Product[Product Name] ),
        COUNTROWS ( FILTER ( Sales, [Order Qty] >= [Min Qty] ) ) >
0
    )
)
```

Order Qty is a measure in the Sales table, and Min Qty refers to the lowest order quantity in the Sales table.

When you have created a new outlier measure, you can group your products into categories by using the grouping feature, as you previously did when creating a histogram. You then need to add a scatter chart visual, as you did in the previous section, because this is the best visualization option for displaying outliers. When you've added the scatter chart, populate it with the fields that are associated with your DAX formula and outlier measure.



In the scatter chart, you'll be able to identify the outliers in your data. You can then investigate the reasons for their existence and take corrective action.



Group and Bin Data for Analysis

When you create visuals, Power BI Desktop aggregates your data into groups, based on the values that it finds in the underlying data. You can refine how those default groups are presented. You can also create new groups by grouping two or more data points in a visual or putting values into equal-sized groups (binning).

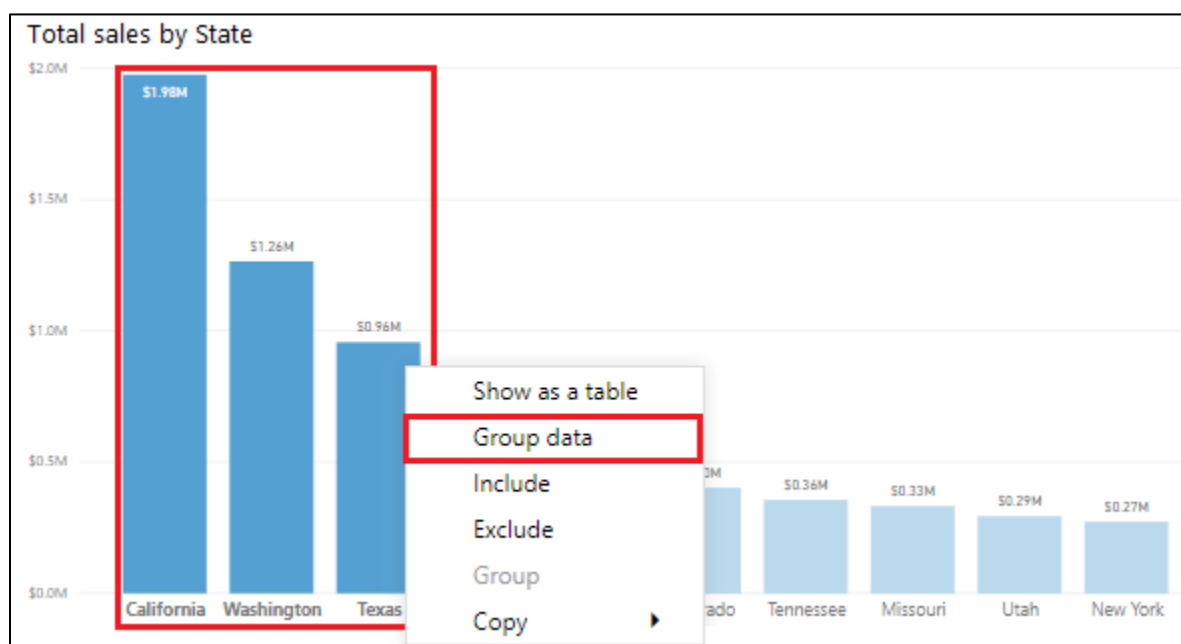
Grouping is used for categories of data. Binning is similar to grouping, but it is used for grouping continuous fields, such as numbers and dates.

You can use the grouping and binning features to ensure that the visuals in your reports display your data according to your preference. Using these features will help you to clearly view, analyze, and explore the data and trends in your visuals. Additionally, you'll be able to identify clusters, patterns of behaviour, data averages, and more. The results of this analysis will provide your users with more specific insights on their data, which can help drive business decisions.

Create a group

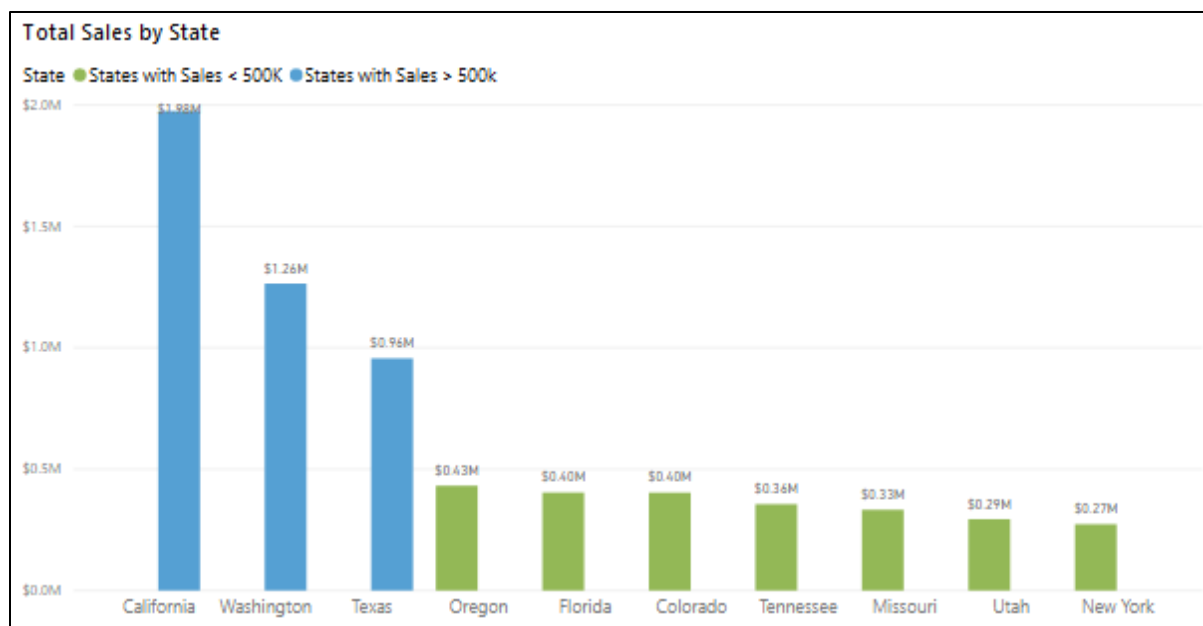
The following image shows a bar chart in which Power BI has automatically segmented the data in the way that it found most useful: Total Sales by State. However, you want to group some of the bars (states) together so that you can view them as one category, which will help the Sales team identify the cities with the highest sales.

To create the group, use **Ctrl + click** to select the data points on the visual that you want to group. In this case, it's states with sales greater than 500,000 dollars. Right-click one of those selected data points and then select the **Group data** option.



When the group is created, notice that the visual updates to take into account the new group.

The following image shows that the other states, which are the states with lower sales (less than 500,000 dollars) have been grouped together and are highlighted in a different shade.



The new group field displays in the **Legend** bucket for the visual and is listed in the **Fields** pane. When you create a group, you can change the way that the data is displayed in the visual. For example, you might want to switch the values in each axis. You can also use the group in any of the other visuals in your report. To do so, drag the group field from the **Fields** pane and then drop it into the visual in which you want to use it.

Edit a group

Continuing with the previous example, you now want to edit the categories that make up your group. Right-click the group field in either the **Legend** bucket or the **Fields** pane, and then select **Edit Groups**.

In the **Groups** window that displays are a list of the groups and the different items within those groups. The following image shows the **States with Sales > 500k** group and its members, along with the **Other** group (**States with Sales < 500k**) that contains all other values that have not been put into the first group. If you refresh your data, and new items appear in the ungrouped values list, they'll all go into the **Other** group.

You can now make changes to the group. You can rename any group by double-clicking the group title in the **Groups and members** section and entering a new name. You can add ungrouped values into an existing group, remove values from an existing group, and create a new group.

Create bin groups

The process of binning allows you to group your numerical and time field data into "bins" of equal size. This approach allows you to visualize and identify trends in your data in more meaningful ways. Binning allows you to right-size the data that Power BI Desktop displays.

In this example, you want to create bins (groups) for the **Order Qty** field. Start in the **Fields** pane by right-clicking the **Order Qty** field that you want to create the bins for, and then select **New Group**. On the **Groups** window that displays, set the **Bin size** to the size that you want, adjust other settings as required, and then select **OK**.

The screenshot shows the 'Groups' dialog box with the following settings:

Field	Value
Name	Order Qty (Bins)
Field	OrderQty
Group type	Bin
Bin Type	Size of bins
Min value	1
Max value	44
Bin size	5

Below the input fields, there is a text box with the text: "Binning splits numeric or date/time data into equally sized groups. The default bin size is calculated based on your data."

At the bottom of the dialog, there is a "Reset to default" button and "OK" and "Cancel" buttons.

When you have set up the bin group, you'll see a new field in the **Fields** pane with **(bins)** appended to its name. You can then drag that field onto the canvas to use the bin size in a visual.

Apply Clustering Techniques

Clustering allows you to identify a segment (cluster) of data that is similar to each other but dissimilar to the rest of the data. The process of clustering is different to that of grouping, which you accomplished previously.

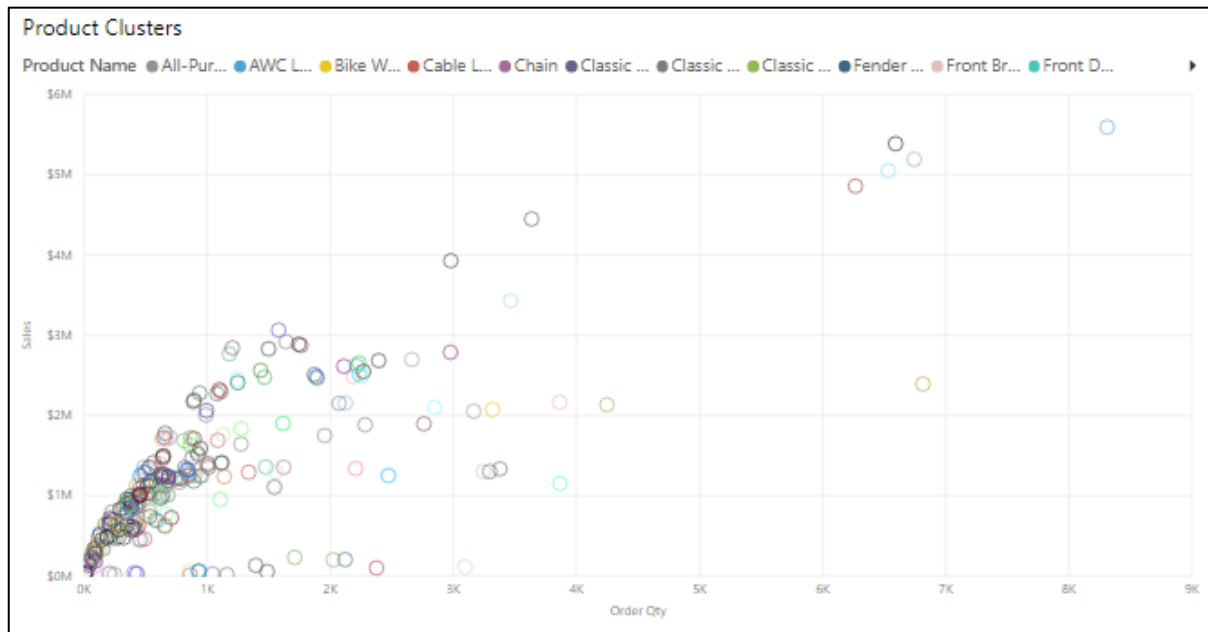
The Power BI clustering feature allows you to quickly find groups of similar data points in a subset of your data.

It analyses your semantic model to identify similarities and dissimilarities in the attribute values, and then it separates the data that has similarities into a subset of the data. These subsets of data are referred to as clusters.

For example, you might want to look for patterns in your sales data, such as the behaviour of customers overall. You can segment the customers into clusters according to their similarities, such as age or location.

Start by adding the scatter chart visualization to your report and then add the required fields to the visual. In this example, you will add the **Order Qty** field to the x-axis, the **Sales** field to the y-axis and **Unit Price** to the **Values** field.

The following image shows considerable data in the scatter chart, so it is difficult to discern any natural groups.



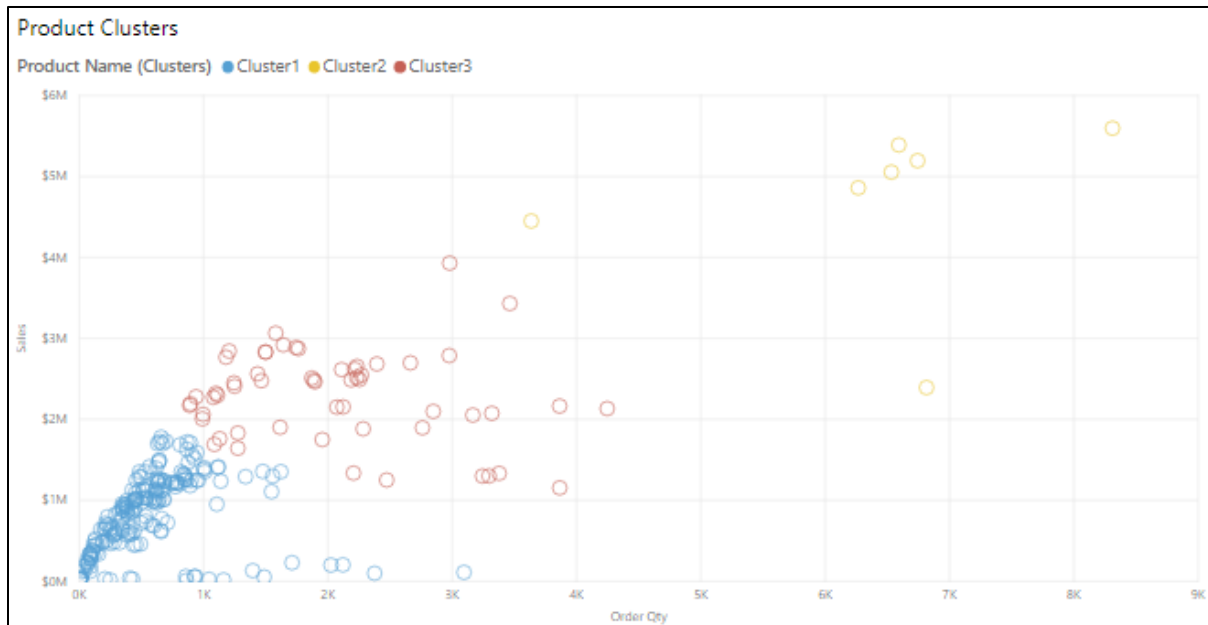
To apply clustering to your scatter chart, select **More options (...)** in the upper-right corner of the visual and then select **Automatically find clusters**.

On the **Clusters** window that displays, you can edit the default name, field, and description, if required. However, for this example, you want to change the number of clusters. The following image shows that the **Number of clusters** box is blank by default, which means that Power BI automatically finds the number of clusters that it determines as making the most sense with your data.

Enter the number of clusters that you want (3) into the box and then select **OK**. Power BI will run the clustering algorithm and create a new categorical field with the different cluster groups in it. Now, when you look at the visual, you can clearly see the clusters that are in your data and proceed to perform analysis on them.

The 'Clusters' configuration window shows the following settings:

- Name:** Product Name (Clusters)
- Field:** Product Name
- Description:** Clusters for Product Name
- Cluster list:**
 - Cluster1 (203 items)
 - Cluster2 (7 items)
 - Cluster3 (56 items)
 - (Blank) (29 items)
- Number of clusters:** 3
- Message:** Changing the number of clusters will rerun clustering and discard cluster names.
- Buttons:** OK, Cancel



The new cluster field is added to your scatter chart's **Legend** field well bucket, which you can now use as a source of cross-highlighting like any other **Legend** field. You can also find the new cluster field in your field list and use it in new visuals, just like any other field.

Clusters

Name: Field:

Description:

Cluster1 (40 items)

Cluster2 (1 items)

Cluster3 (10 items)

Cluster4 (7 items)

Cluster5 (81 items)

Cluster6 (28 items)

Cluster7 (67 items)

Cluster8 (4 items)

Cluster9 (3 items)

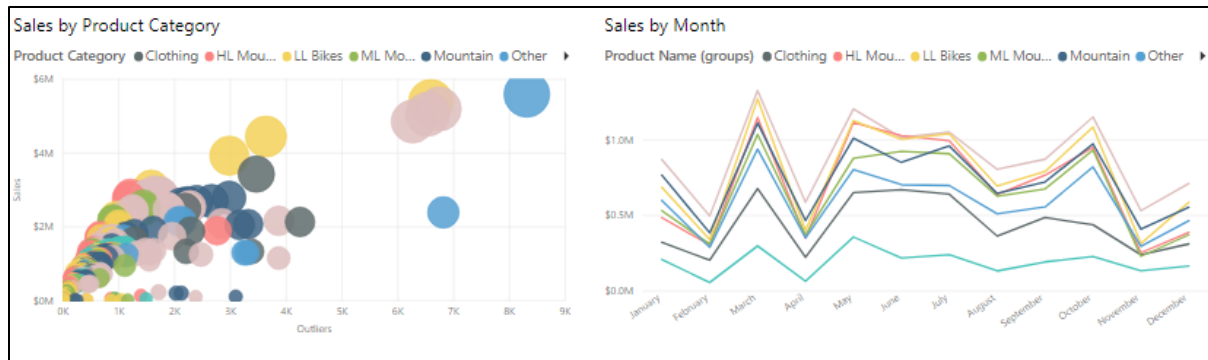
Cluster10 (25 items)

Number of clusters

Changing the number of clusters will rerun clustering and discard cluster names.

If you want to edit the cluster, right-click the cluster field and select **Edit clusters**.

In the preceding example, when you applied clustering to the scatter chart, you could only use two measures. If you want to find clusters by using more than two measures, you can use a table visual instead, add all the fields you want to use, and then run the clustering algorithm by using the same process.



Conduct Time Series Analysis

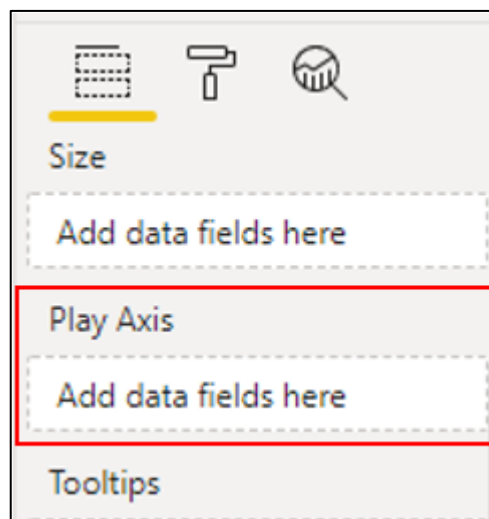
Time series analysis involves analyzing a series of data in time order to identify meaningful information and trends and make predictions. The result of time series analysis is the best data that you can use for forecasting activities.

Time series analysis often involves the use of visuals such as Gantt charts, project planning, and stock movement semantic models. In Power BI, you can use visuals to view how your data is progressing over time, which in turn allows you to make observations, such as if any major events disrupted your data.

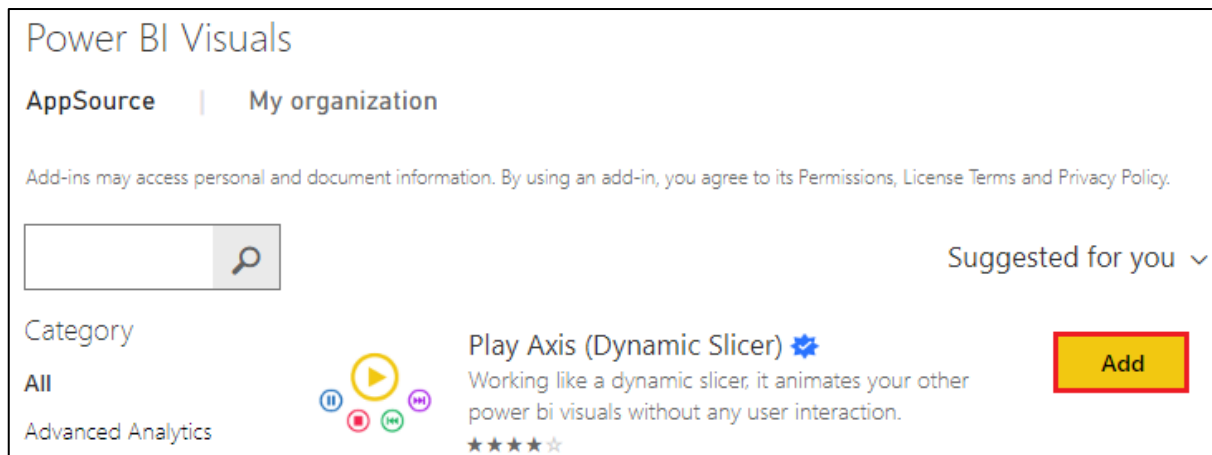
To conduct a time series analysis in Power BI, you need to use a visualization type that is suitable for displaying trends and changes over time, such as a line chart, area chart, or scatter chart. You can also import a time series custom visual into Power BI Desktop from Microsoft AppSource. The following example uses a standard scatter chart.

In addition to the range of time series custom visuals, Microsoft AppSource has an animation custom visual called **Play Axis** that works like a dynamic slicer and is a compelling way to display time trends and patterns in your data without user interaction. This visual is similar to the one that Hans Rosling used in his original presentation and is used alongside the scatter chart in the following example.

Some organizations prefer not to use custom visuals for security or other reasons. Before you import custom visuals, check with your organization to see if they are allowed or not. If they are not allowed, you can instead use the **Play Axis** visual that is available for scatter chart visualizations within Power BI Desktop because it has similar functionality.



Next, you will import the animation custom visual to use with the visuals. In the **Visualizations** pane, select the **Get more visuals** icon and then select **Get more visuals**. On the **Power BI Visuals** window that displays, search for **play axis**, and then select the **Add** button for the **Play Axis (Dynamic Slicer)** visual.

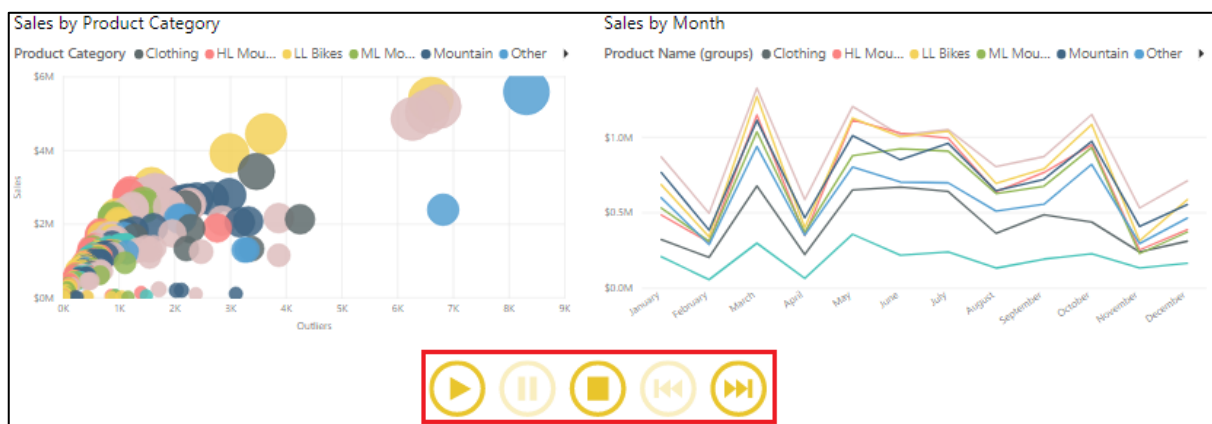


A message will display, stating that the visual was successfully imported. When you return to Power BI Desktop, you'll see the new **Play Axis** icon in the **Visualizations** pane.

Select the page, and then select the **Play Axis** icon to add that visual to the page.

With the new visual selected, select the field (**Quarter**) that you want to use as the slicer in the **Play Axis** animation.

Animation controls become available on the visual.

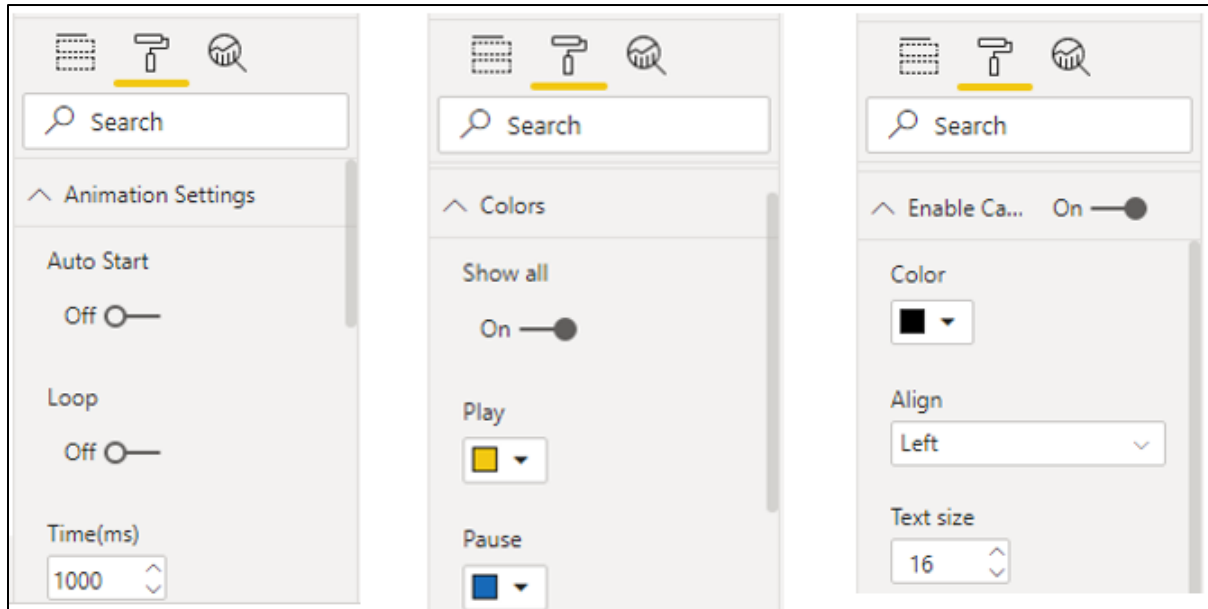


You can now resize and reposition the new visual and customize its formatting so that it's consistent with the other visuals on the page.

Specific formatting options that you might want to use include:

- In the **Animation Settings** section, you can control the play functionality of the **Play Axis** visual, such as making the animation automatically start, continue looping, and then change the speed at which the animation occurs.

- In the **Colors** section, you can change the appearance of the **Play Axis** visual by adjusting its overall color, or selecting the **Show all** option, and then changing the color of each control button.
- The **Enable Caption On** section allows you to turn on/off the text that is displayed next to the visual or adjust the formatting of it.



When you have set up the **Play Axis** visual to meet your requirements, you are ready to use it with your other visuals. Select the **Play** button and then watch how the data in each visual on the page evolves over the time. You can use the control buttons to pause the animation, restart it, and so on.

Use the Analyze Feature

The **Analyze** feature provides you with additional analysis that is generated by Power BI for a selected data point. You might want to use this feature to determine if Power BI has found something that you haven't seen before, or if you want Power BI to give you a different insight into your data. This feature is particularly useful for analyzing why your data distribution looks the way that it does.

This feature does not work if you have non-numeric filters applied to your visual and/or if you have measure filters applied.

Create What-If Parameters

You can use *what-if* parameters to run scenarios and scenario-type analysis on your data. What-if parameters are powerful additions to your Power BI semantic models and reports because they enable you to look at historical data to analyze potential outcomes if a different scenario had occurred.

Additionally, what-if parameters can help you look forward, to predict or forecast what could happen in the future.

You can use what-if parameters in multiple situations, such as to determine the effect of increased sales to deeper discounts, or to let sales consultants see their compensation if they meet certain sales goals or percentages.

After you have created a what-if parameter, the parameter and the measure will become part of your model; therefore, they will be available throughout the report and can be used on other report pages. Additionally, because the parameter and measure are part of the model, you can delete the slicer from the report page. If you want it back, you can drag the what-if parameter from the **Fields** list onto the canvas and then change the visual type to a slicer.

Use a what-if parameter

If you want to use the what-if parameter after you've created it, you will need to create a new measure whose value adjusts with the slider. You can create complex and unique measures that let your report users visualize the variable of your what-if parameter.

Use Specialised Visuals

Power BI includes several specialized visuals that provide a considerable interactive experience for report consumers. Often, these specialized visuals are called AI visuals because Power BI uses machine learning to discover and display insights from data. These visuals provide a simple way to deliver a rich and interactive experience to your report consumers.

The three main AI visuals are:

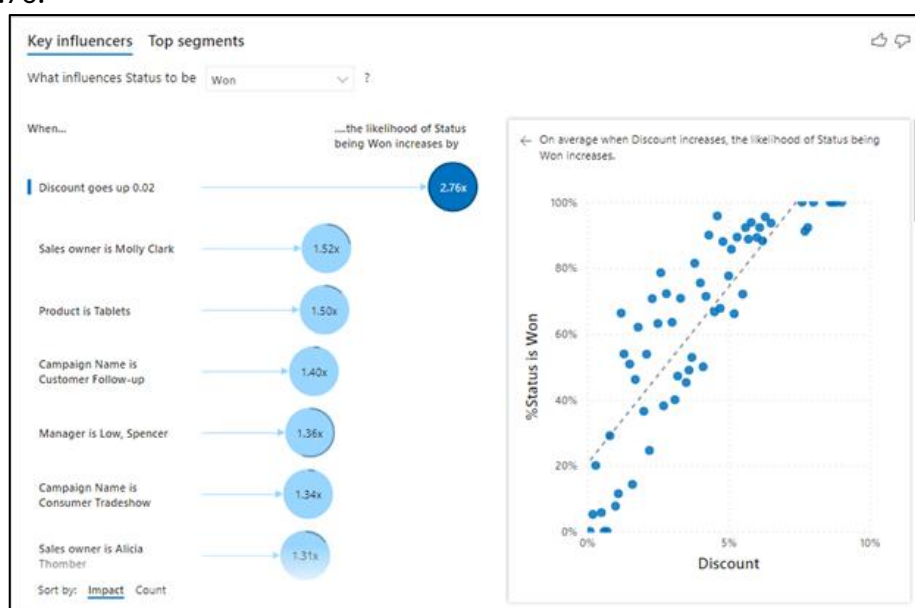
- Key influencers
- Decomposition tree
- Q&A

When adding an AI visual to your report, make sure that you size it to become as large as possible. That way, report consumers can fully interact with and explore the data in the visual.

Key influencers

The **Key influencers** visual helps report consumers understand the factors that drive a particular metric, like sales revenue. By using AI, Power BI will analyze the data, rank the factors that matter, and then present them as key influencers.

In the following example, the visual helps report consumers understand the important dimensions that contribute to won sales opportunities. According to the visual, when the discount rate increases by two percent, the likelihood of a won sales opportunity increases by a factor of 2.76.



Essentially, the **Key influencers** visual is many visuals inside one frame. When you select a key influencer, an adjacent visual will show a representation of the influencer as a comparison against the remainder of the data.

Additionally, the **Key influencers** visual includes the **Top segments** view, which shows the highest-ranking segments that contribute to a particular metric.

Decomposition tree

The **Decomposition Tree** visual helps report consumers visualize data across multiple dimensions. It automatically aggregates data and enables consumers to drill down into dimensions in any order.

As a result, it's a valuable tool for ad hoc exploration and conducting root cause analysis.

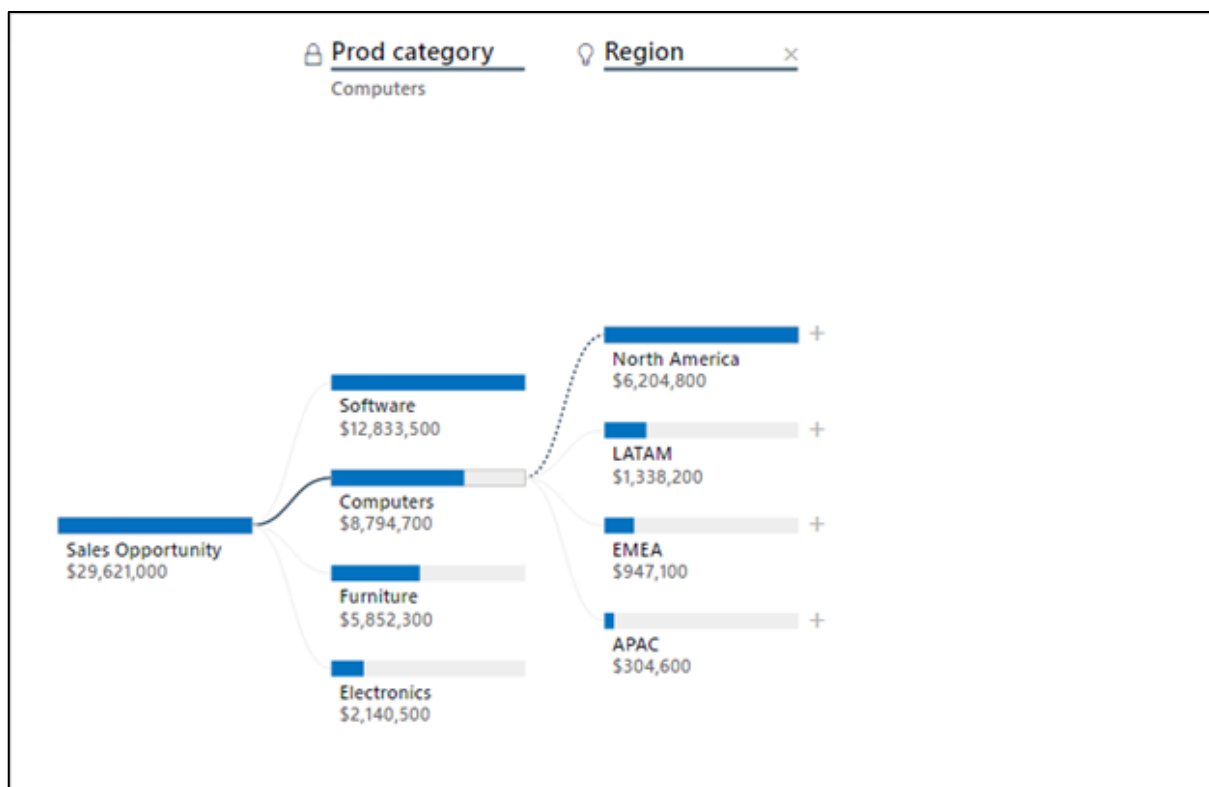
As an AI visual, a decomposition tree provides a guided exploration experience that helps by finding the next dimension for consumers to drill down into.

In the following example, a **Decomposition Tree** visual helps report consumers to understand the breakdown of sales opportunity revenue.

For a demonstration of the **Decomposition Tree** visual, watch the following video.

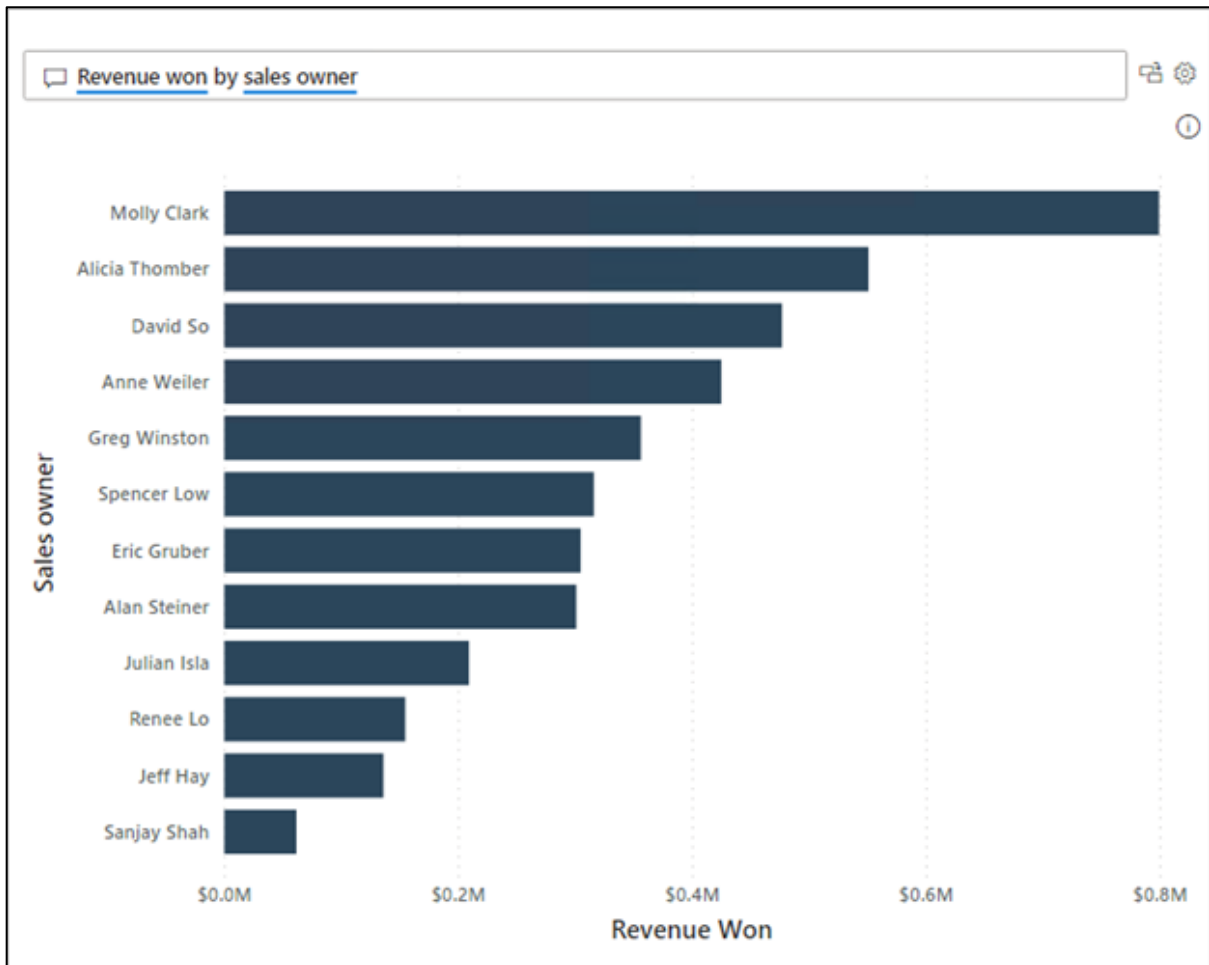
The lock icon next to the **Prod Category** split means that the report author added the split and set it so that it can't be removed.

Additionally, the light bulb icon next to the **Region** split means that the report consumer used an AI split to discover the best split.



Q&A

The **Q&A** visual allows report consumers to ask questions of their data and receive responses as data visualizations. Similar to how web search engines parse and understand complex search requests, report consumers can write natural language questions (in English).



To optimize the Q&A experience, ensure that the data field names are user-friendly. You can also enhance the semantic model with synonyms and terms. Additionally, you can hide fields, such as fields that are used in model relationships, to restrict their use in Q&A. You can also add suggested questions that become prompts in the **Q&A** visual.

Manage Workspaces and Semantic Models in Power BI

Create and Manage Workspaces in Power BI

Introduction

You've likely had the chance to load and transform data from numerous sources, build visuals, create DAX equations, and even publish a report or two to Microsoft Power BI. The next step on your data analysis journey is to share these reports with your wider audiences and organizations. You can accomplish this task in a workspace, which is a feature of Power BI. A workspace is a centralized repository in which you can collaborate with colleagues and teams to create collections of reports and dashboards.

Workspaces offer the following benefits:

- Focused collaboration efforts. You can use workspaces to house reports and dashboards for use by multiple teams.
- Ability to share and present reports and dashboards in a single environment.
- Assurance that the highest level of security is maintained by controlling who can access semantic models, reports, and dashboards.

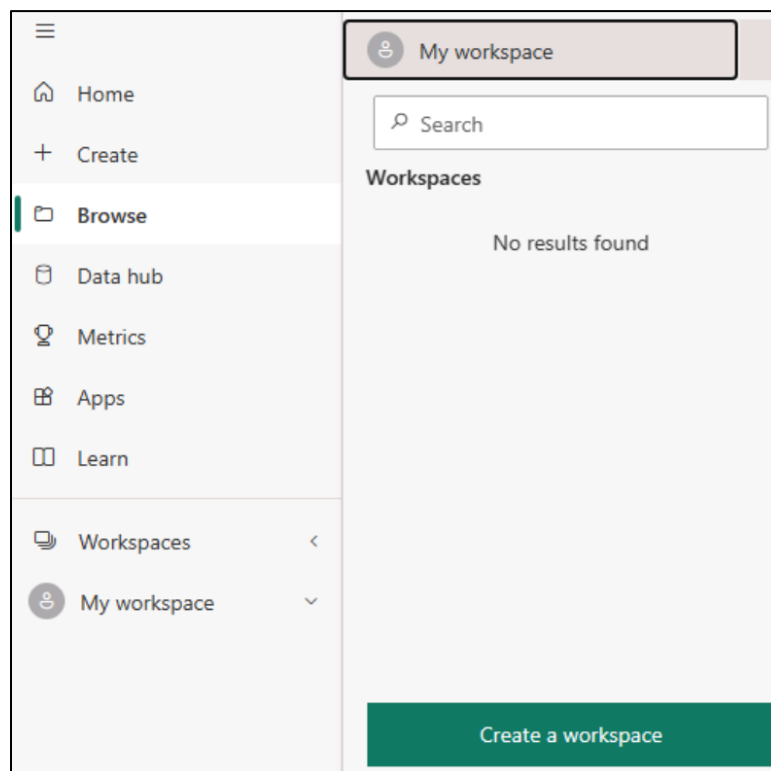
Distribute a Report or Dashboard

Consider a scenario where you have created a few reports for the Sales team at Tailwind Traders. The issue that you have encountered is determining how to make these reports viewable and shareable. By creating a workspace in Power BI, you can house your reports in one location, make them shareable, collaborate with other teams, and update reports.

Create a workspace

Your first task is to create a workspace by following these steps:

1. Go to Power BI service.
2. Select the **Workspaces** menu from left navigation blade.
3. Select the **Create a workspace** button at the bottom of the resulting panel.



4. In the **Create a workspace** window, enter information in the **Workspace name** and **Description** fields and then upload a **Workspace image**.

Create a workspace

Workspace image

 Upload Delete

Workspace name *

Sales

Available

Description

Describe this workspace

[Learn more about workspace settings](#)

Advanced ^

Contact list

☒ Workspace admins

☐ Specific users and groups

Enter users and groups

Workspace OneDrive

(Optional)

Save Cancel

5. In the **Advanced** drop-down menu, you can create a **Contact list** of users who will receive notifications if issues with the workspace occur.

By default, these users are the workspace admins, but you can also add specific users. You can also add this workspace to a specific OneDrive and then choose whether this workspace will be a part of a dedicated capacity or not.

Dedicated capacities are Power BI Premium features that ensure that your workspace will have its own computational resources as opposed to sharing resources with other users.

6. After you have filled out pertinent fields on the **Create a workspace** window, select **Save**.

You have now created a workspace.

Assign workspace roles

Now that you've successfully created a workspace, the Sales team wants to collaborate with other teams to build additional dashboards and reports. As the workspace owner, you want to ensure that appropriate access is given to members of the Products team because their team includes stakeholders and developers. Workspace roles allow you to designate who can do what within a workspace.

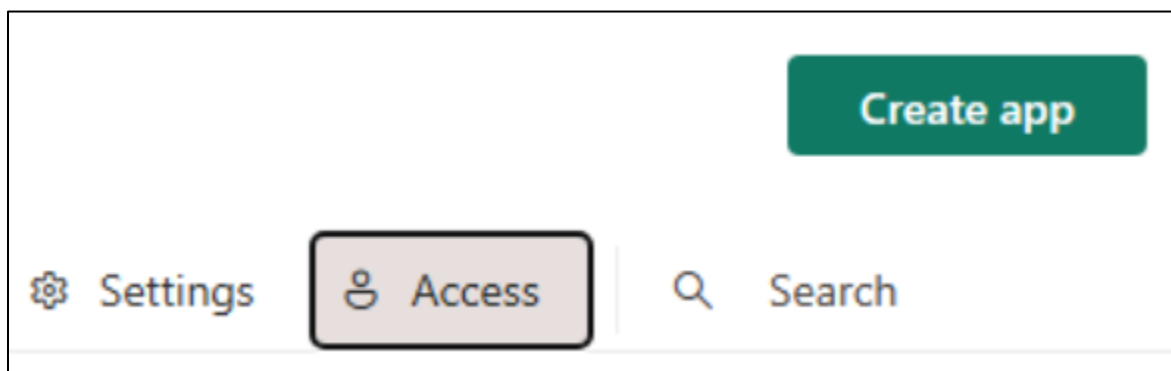
There are four roles for workspaces, and it's advised that you grant the minimum access necessary to collaborators. For consumers, skip workspace role assignment, and provide access through the app instead in the next section.

The four roles are listed below, in order of most permissive to least, along with select permissions. For full permissions, review the Roles in workspaces documentation.

- **Admin**
 - Update and delete the workspace
 - Add or remove people, including other admins
- **Member**
 - Add members or others with lower permissions
 - Publish, unpublish, and change permissions for an app
- **Contributor**
 - Create, edit, and delete content, such as reports, in the workspace
 - Publish reports to the workspace
- **Viewer**
 - View and interact with an item
 - Read data that's stored in workspace dataflows

If the workspace is backed by a Premium capacity, a non-Pro user can view content within the workspace under the **Viewer** role.

To assign these roles to users, go to the workspace that you've created and, in the upper-left corner of the ribbon, select **Access**.



In the resulting **Access** window, you can add email addresses of individual users, mail-enabled security groups, distribution lists, Microsoft 365 groups, and regular security groups, and then assign them to their specific roles.

You can also change the user's assigned role at the bottom of the page and delete the user from the workspace by selecting the ellipsis (...) next to their name.

Access

Sales

Add admins, members, or contributors. [Learn more](#)

Enter email addresses

Member

Add

Search

NAME	PERMISSION
MOD Administrator ⓘ	Admin

Create and configure an app

After creating an app workspace and assigning your collaborator-specific roles, you want to add content to your app workspace. Content can be in the form of reports, dashboards, semantic models, dataflows, and so on.

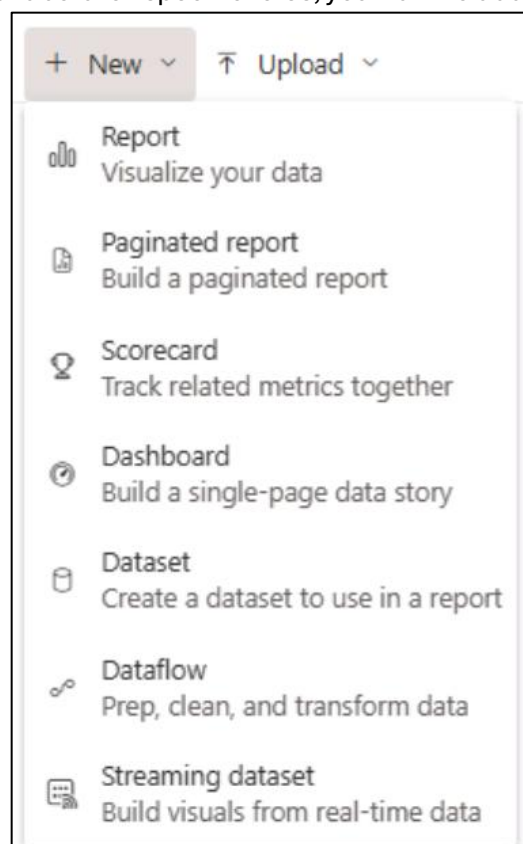
An app is a published, read-only window into your data for mass distribution and viewing. When ready to share apps with your users, you can publish the app. This process requires a Power BI Pro license.

Consuming and viewing an app also requires a Pro license, or the workspace must be hosted in a Premium capacity.

You can now publish from Power BI Desktop to your new workspace, and upload saved files or create new items from within the workspace.

When you are ready to publish your app with its collection of reports, dashboards, and semantic models, return to the workspace and select **Create app** in the upper-right corner of the ribbon.

The **Build your app** experience starts on the Setup page, where you add a name and description for the app. You can also customize the theme color and add a logo if desired.



① Setup*

② Content*

③ Audience*

Build your app

App name *

Sales

Description *

Enter a summary

Describe your app.

App logo



Upload

Delete

App theme color



Contact Information

☒ Show app publisher

☐ Show items contacts from the workspace

☐ Show specific individuals or groups

Enter a name or email address

Global App Settings

☐ Install this app automatically.

☐ Hide app navigation pane.

☐ Allow users to make a copy of the reports in this app.

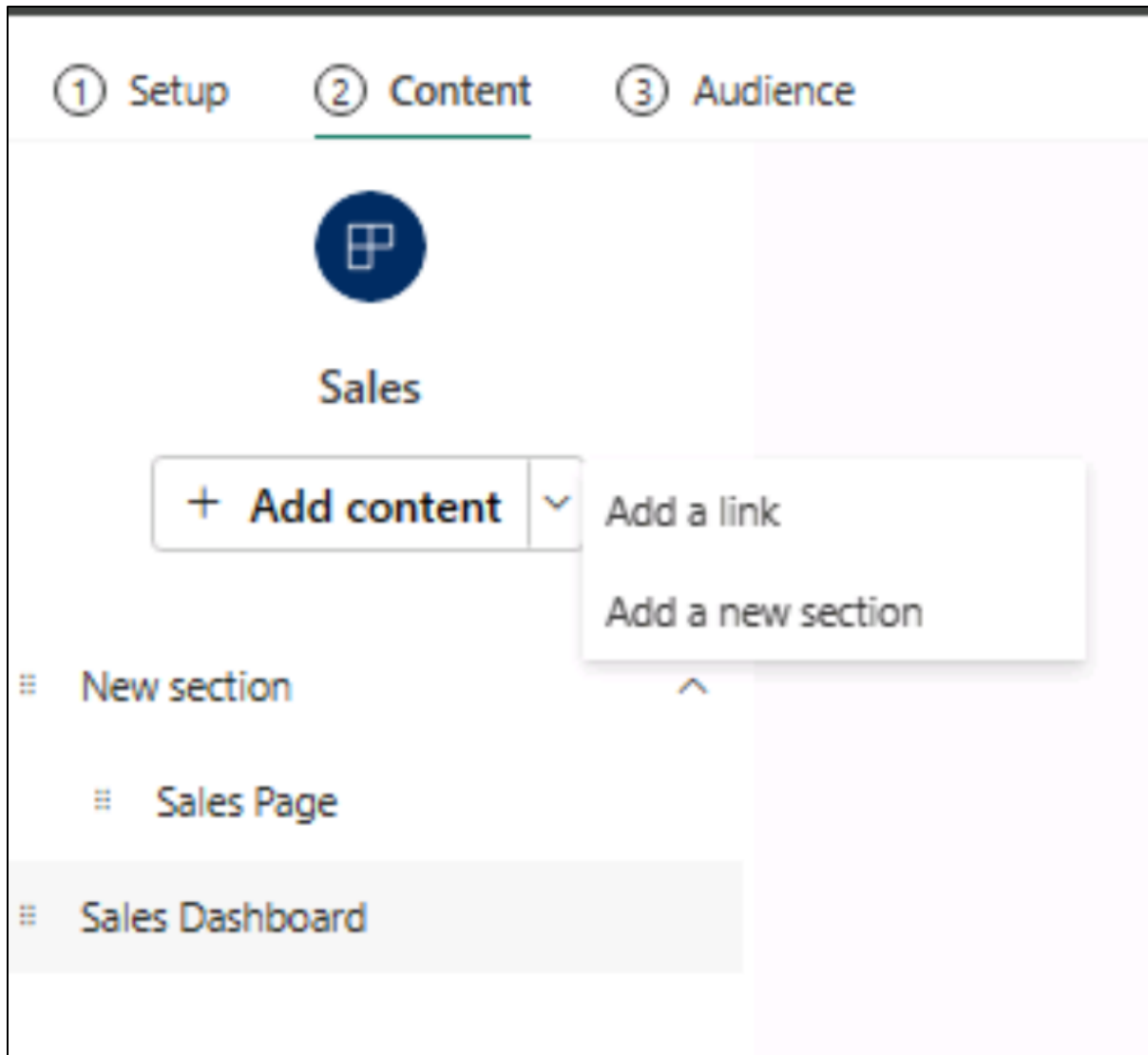
Support site

Share where your users can find help

Use the **Contact Information** and **Support Site** fields to help users contact the appropriate person(s) and how to find help for the app.

Under the **Content** tab, you can choose which content to include and change the viewing order.

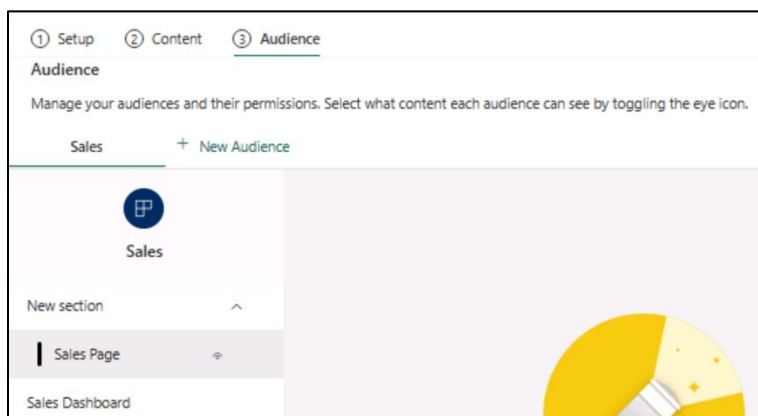
You can add content in the workspace, new sections for grouping, and external links.



In the **Audience** tab, you're now able to choose one or more audiences with different viewing options.

First, you'll select which reports you want visible to the default audience created. Select what content each audience sees by toggling the eye icon on the right.

In the following screenshot, the audience is called "Sales" by default, but you can right-click and rename it.



After selecting the viewable content, you can **Manage Audience Access**. You can **Grant access to the Entire organization or Specific users or groups**. For Specific users or groups, you can enter any mail-enabled account accessible within your Power BI tenant.

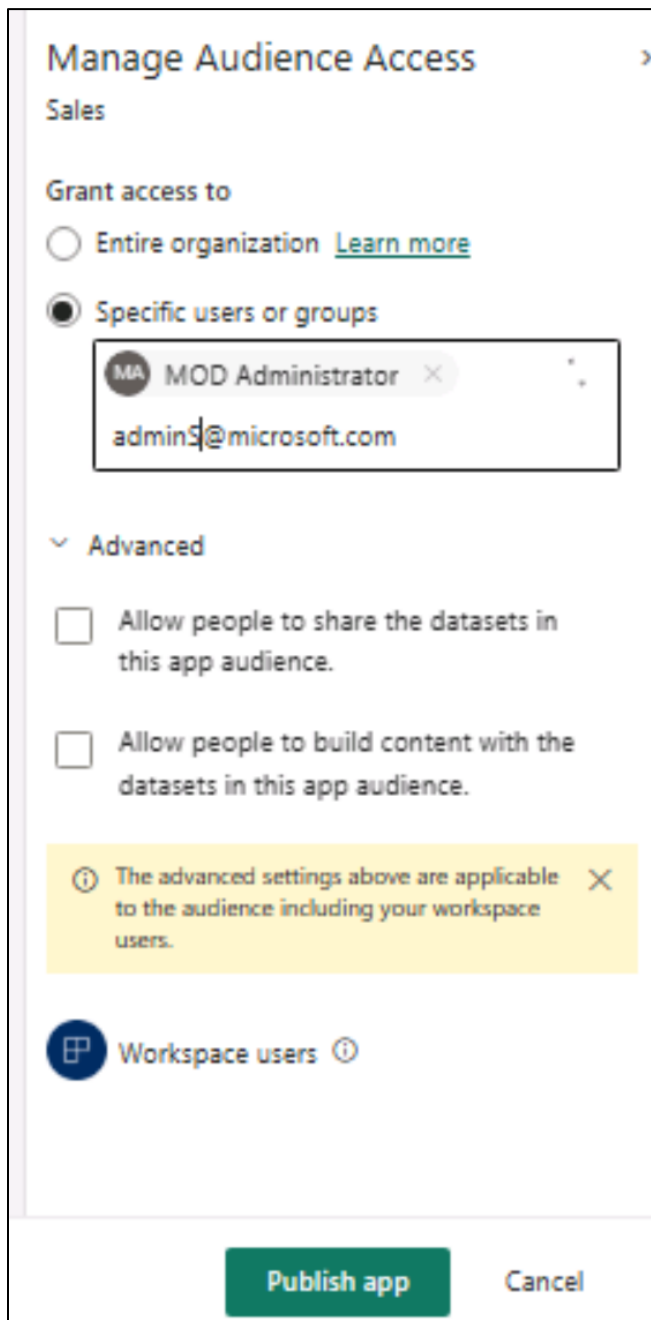
In the **Advanced** section, you can choose to grant additional permissions, individually or neither:

- Allow people to share the semantic models in the app audience
- Allow people to build content with the semantic models in the app audience

Lastly, notice that **Workspace users** are already included in the audience by default. This goes back to the roles we covered earlier.

Entire organization may not be accessible due to settings configured by your Power BI Administrator. Additionally, not all email addresses may be available, such as external accounts.

When you're ready, select **Publish app**. Congratulations, you just published an app! You'll receive a notification with the link to distribute to consumers, and an option to go to the app.



When publishing, there's a notification that it may take 5-10 minutes or longer to reflect changes, depending on your tenant and size of report.

It's not possible to guarantee changes will be reflected in 10 minutes or less.

Update workspaces

After publishing your app, you realize that you want to make updates within your workspace.

Don't worry, it's as easy as publishing your app.

From the workspace, the **Create app** button will now say **Update app**. Select Update app, then move to the appropriate section and make your changes.

When ready to save your changes, select the **Update app** button in the bottom where **Publish App** was before.

Monitor Usage and Performance

Knowing about the usage and performance of your workspace is crucial because it:

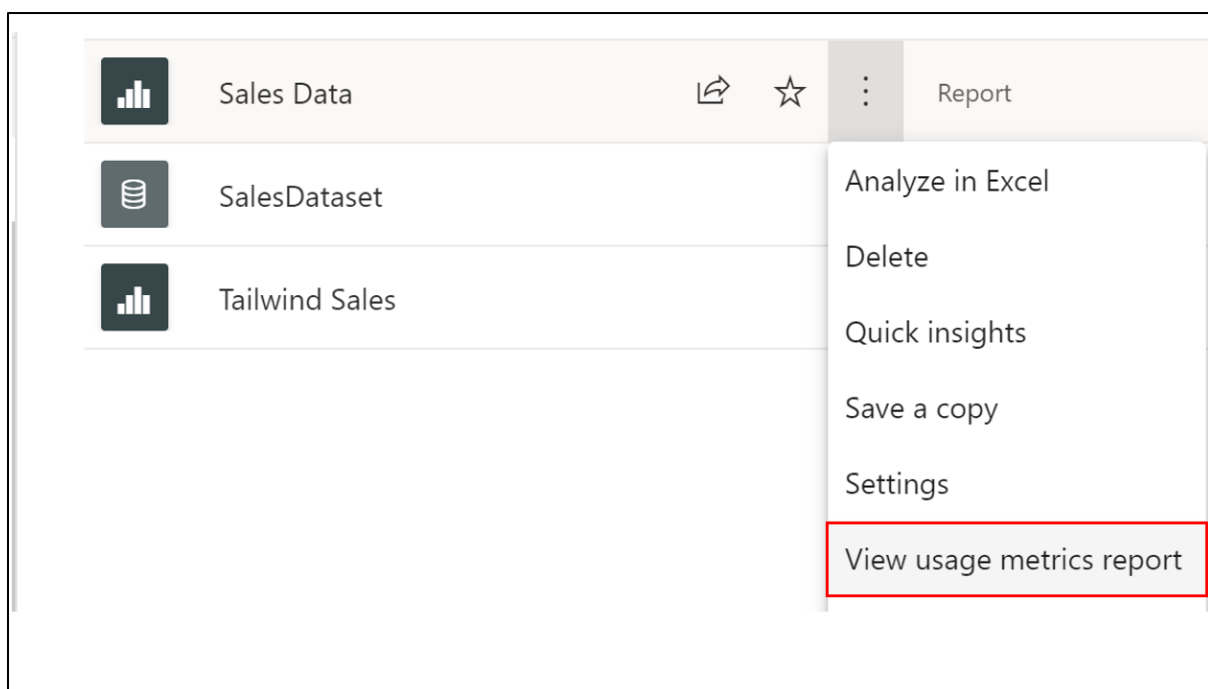
- Focuses your efforts for improvement. If you know the areas that experience the worst performance, you can concentrate your efforts for improvement in those areas.
- Quantifies the impact of your reports. Usage metrics help you determine your reports' success.

These performance and usage metrics are available features that you can use in a workspace. With these metrics, you can view who's using your reports, what actions are being done on the reports, and what performance issues exist.

Configure and view usage metric reports

Usage metric reports are available for Power BI Pro users and can only be accessed by users with the role types of **Admin**, **Member**, or **Contributor**.

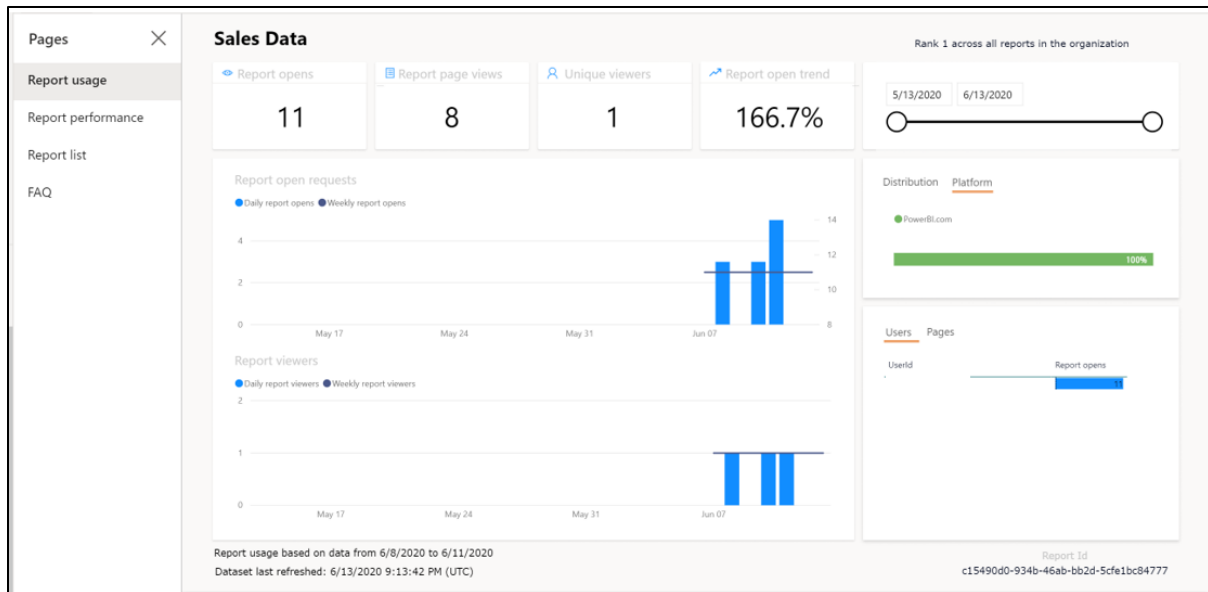
To view usage metric reports, go to the pertinent workspace. Find the report or dashboard that you want to see usage metrics for. For example, if you want to see the usage metrics report for **Sales Data**, select the ellipsis (...), and then select **View usage metrics report** from the drop-down menu.



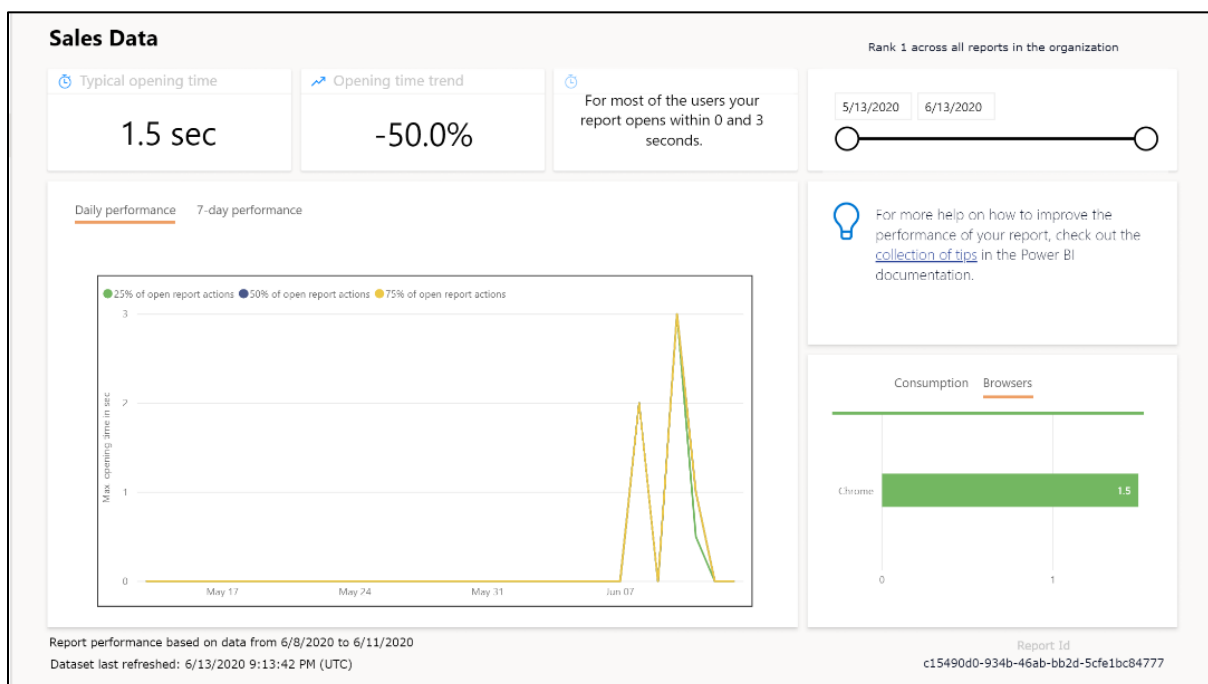
When the usage metrics report is ready for viewing, you will receive a prompt that will direct you to a dashboard. In the **Report usage** tab, you can view such details as:

- **Viewers per day**, **Unique viewers per day** (which doesn't include users who returned to the same reports multiple times), and **Shares per day** charts
- **Total Views**, **Total Viewers**, and **Total Shares** KPI cards
- **Total views and shares ranking** (compares how your report is doing in comparison to other reports in the app)
- **Views by Users** (details about each specific user that viewed the dashboard)

You can also filter by the distribution method of the report (for example, through sharing or from the workspace directly) and platform type (for example, mobile or web).



You can also view performance metrics on the **Report performance** tab, as shown in the following screenshot.



On the **Report performance** tab, you can view metrics such as:

- **Typical opening time** - How long it takes, at the fiftieth percentile, to open the report.
- **Opening time trend** - How the typical opening time changes over time. This metric can tell you how the report is performing as the number of users starts to grow.
- **Daily/7-Day Performance** charts - Highlight the performance for 10, 50, and 90 percent of the open-report actions every day and over a seven-day period.
- Filters for date, so you can see how the performance changes according to the day.

Recommend a Development Lifecycle Strategy

The development process is iterative; it typically requires building an initial solution, testing the solution in a different environment, returning to make necessary revisions, and eventually releasing a final product.

This process is known as a development life cycle. This process can take place in several different ways and in different environments.

Deployment pipeline (Premium)

The deployment pipeline feature in Power BI manages content in dashboards, reports, and semantic models between different environments in the development life cycle.

With this feature, you can develop and test Power BI content in one centralized location and streamline the process before deploying the final content to your users. This Power BI Premium feature requires you to be a Capacity admin.

The advantages of using the deployment pipeline are:

- **Increased productivity** - Through this feature, you can reuse previous deployment pipelines, ensuring that efforts aren't duplicated.
- **Faster delivery of content** - Report development becomes more streamlined, meaning that it takes less time to get to production.
- **Lower human intervention required** - Having the ability to reuse deployment pipelines means a decreased chance of error associated with moving content from one environment to another.

Development environments

Typically, development and collaboration occur in different stages. Reports and dashboards are built in and iterated on a series of controlled stages, or environments, where several tasks occur:

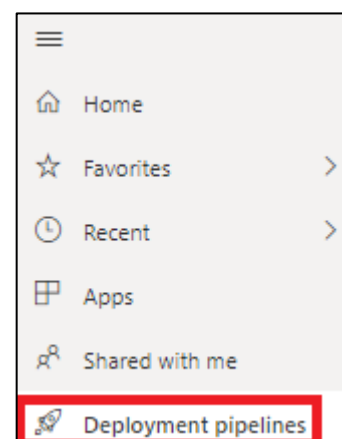
- **Development** - The location in which dashboard developers or semantic modelers can build new content with other developers. This stage is first in the deployment pipeline.
- **Test** - Where a small group of users and user acceptance testers can see and review new reports, provide feedback, and test the reports with larger semantic models for bugs and data inconsistencies before it goes into production.
- **Production** - Where an expansive user audience can use tested reports that are reliable and accurate. This stage is the final one of the deployment pipeline.

You can choose which one of these development environments that you want to include in your deployment pipeline, according to your business needs. For example, you can choose to only include the Test and Production environments, if necessary.

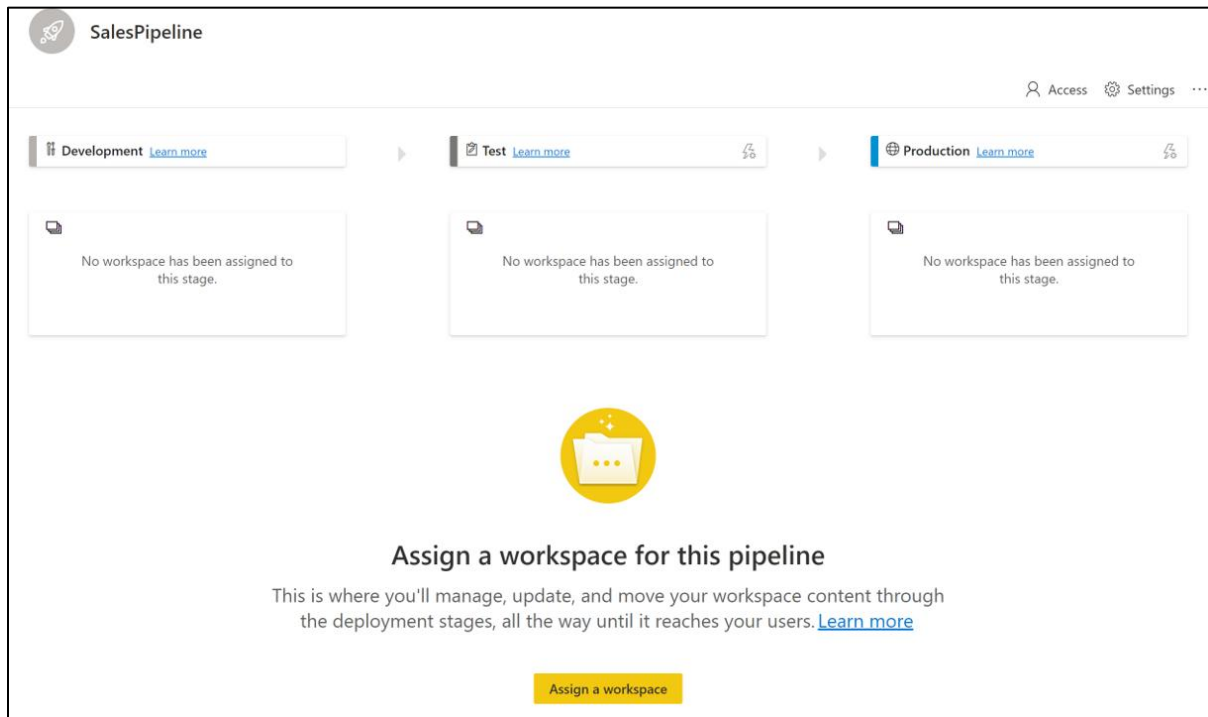
Configuration of deployment pipelines

In the scenario with Tailwind Traders, you want to create a deployment pipeline. To configure a deployment pipeline, go to Power BI service, and then follow these steps:

1. On the ribbon on the left side of the page, select **Deployment pipelines**, as shown in the following screenshot.
2. On the resulting page, select **Create a pipeline**.
3. Create a deployment pipeline called **SalesPipeline**. Enter the **Pipeline name** as **SalesPipeline** and enter a description, if necessary.

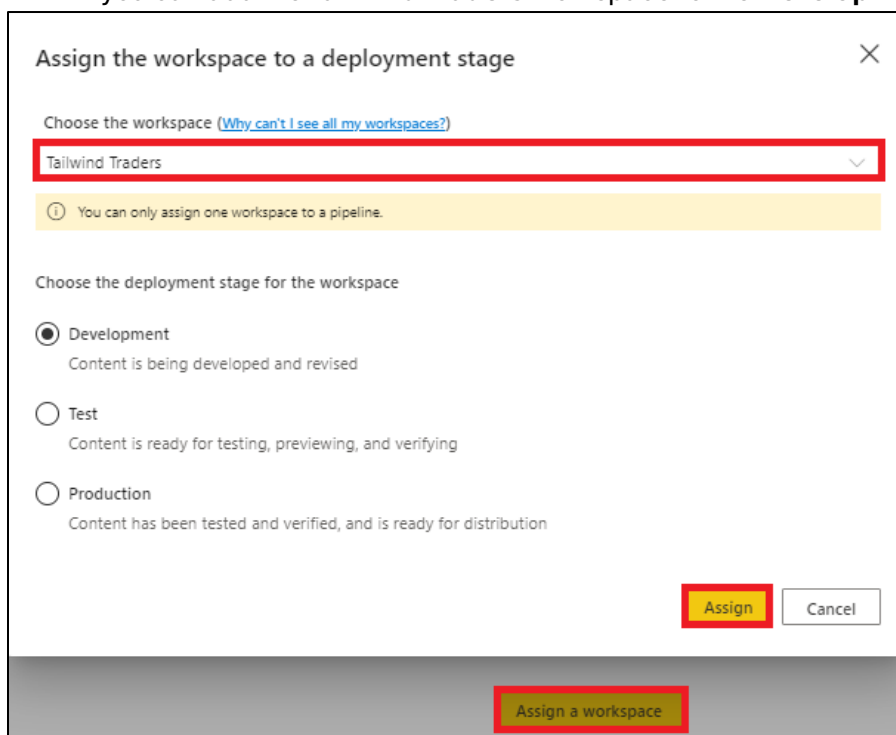


4. Select **Create**, which will take you to the following screen.



This view shows you the steps of the development life cycle: **Development**, **Test**, and **Production**.

5. To create your pipeline, assign workspaces to each of these stages to facilitate where your reports and dashboards will be housed during each stage.
6. Select **Assign a workspace** to begin.
7. You will be directed to the **Assign the workspace to a deployment stage** window, where you can add the **Tailwind Traders** workspace to the **Development** environment.



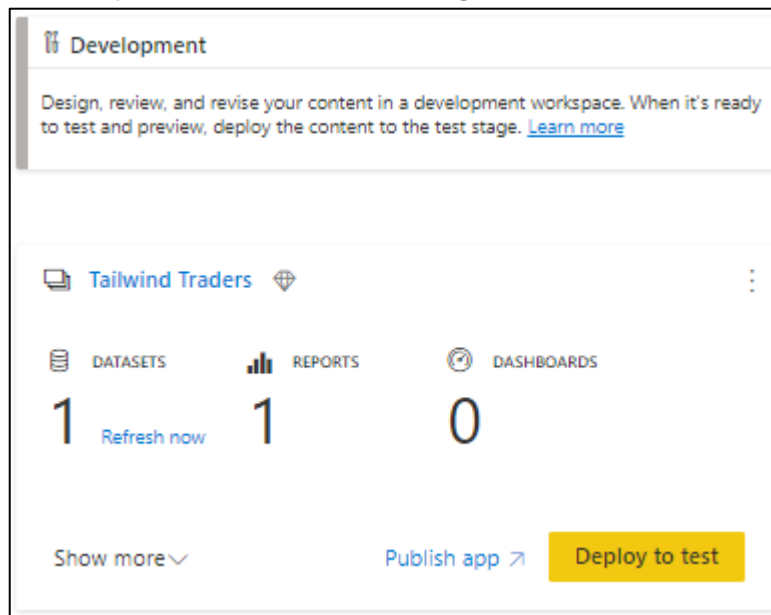
Only workspaces that are assigned to a Premium capacity will appear.

Additionally, you can only assign a single workspace to each pipeline.

Power BI will auto generate the two other workspaces that are used in the pipeline.

8. If you already have **Development**, **Test**, and **Production** workspaces, choose one that you want to work with and then select **Assign**.

If this step is successful, you should see the resulting view.



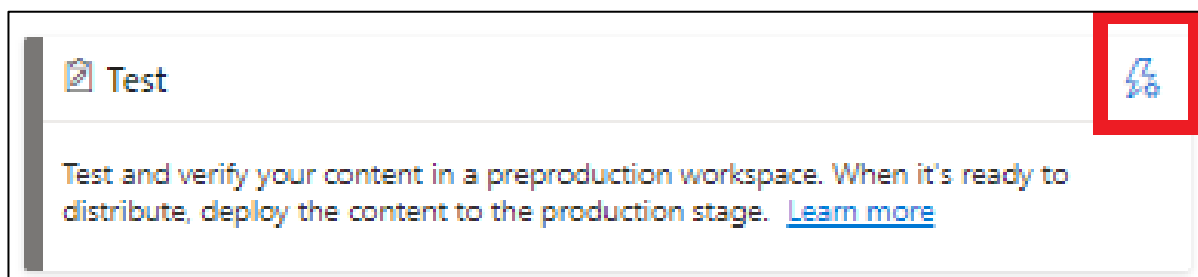
The preceding image shows how many semantic models, reports, and dashboards that you have in the current **Development** environment. At every stage, you have the option to publish the associated workspace as an app by selecting **Publish app**.

9. To view all objects that constitute the workspace, select **Show more**.

Testing stage

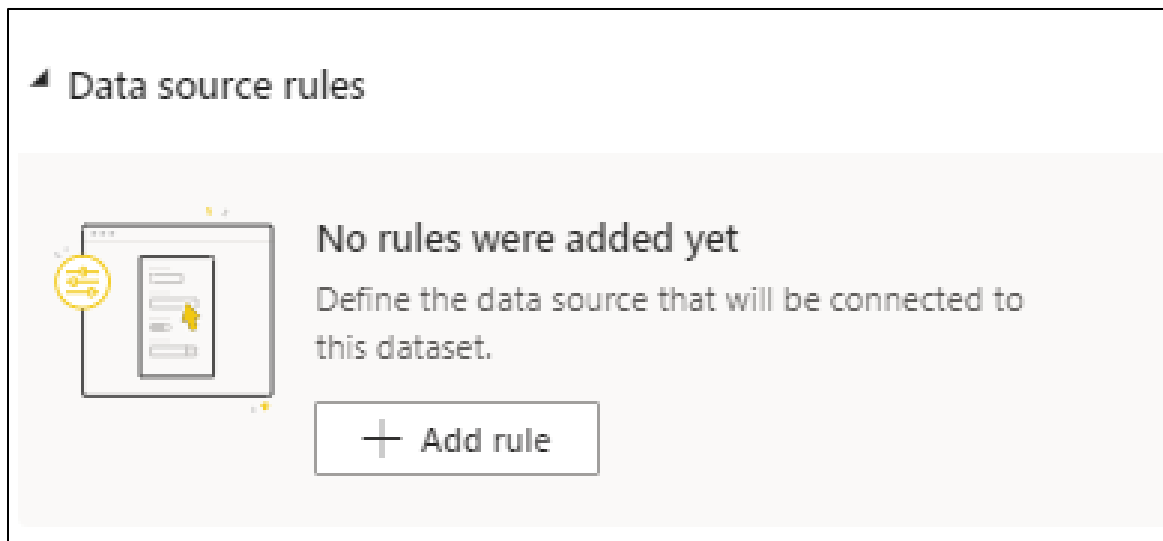
After you have collaborated with the teams and built a testing-ready report, you are ready to proceed to the testing phase. Select **Deploy to test**, which will create a new workspace. This workspace, by default, has the same name as the initial workspace but includes the **[Test]** suffix. You can change the name by entering the workspace's settings within the deployment pipeline interface.

Testing should emulate conditions that objects will experience after they've been deployed for users. Therefore, Power BI allows you to change the source of data that is used during testing. To accomplish this task, you will first need to enter the environment's deployment settings by selecting the lightning icon, as shown in the following screenshot.

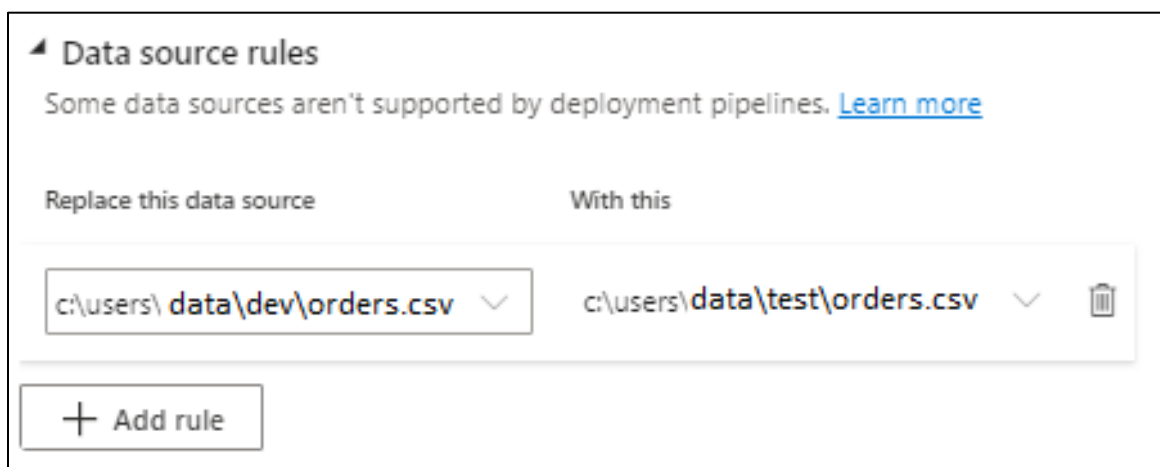


In the resulting **Settings** window, select the correct semantic model. In this example, you want the **OrdersFigures** semantic model to be used for testing but with a different data source. To

accomplish this task, create parameters in Power Query Parameters (which will be discussed in a later module) or add a new rule, which is the process that is used for this example. Under the **Data source rules** drop-down menu, select **+ Add rule**.



On the **Data source rules** section, you can change the data source (which was used in development) to a new source, which is used for testing the reports (**orders.csv** in the following example). When you are finished, select **Save** at the bottom of the card.

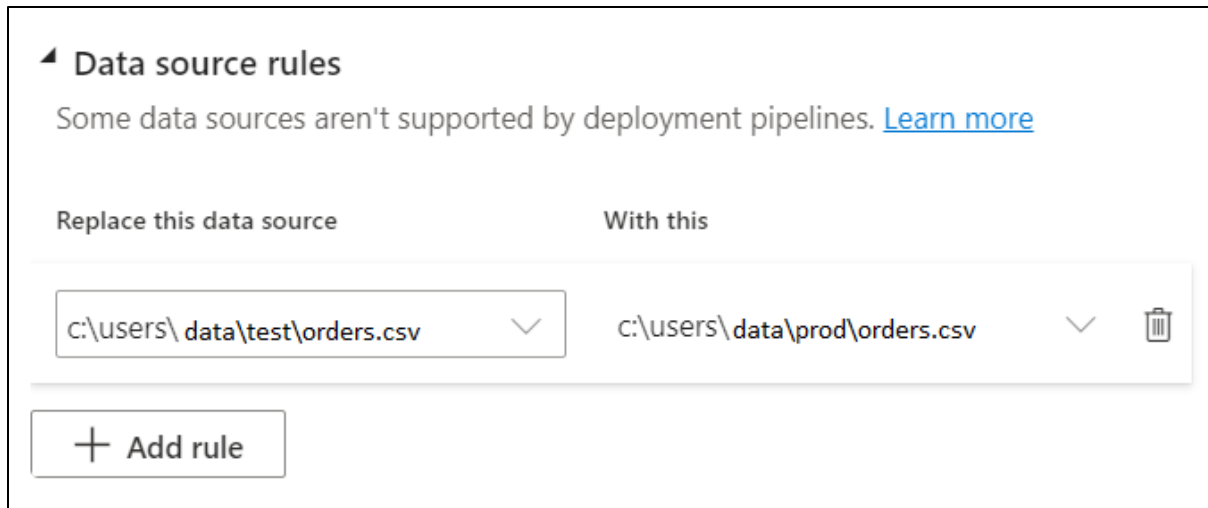


Production stage

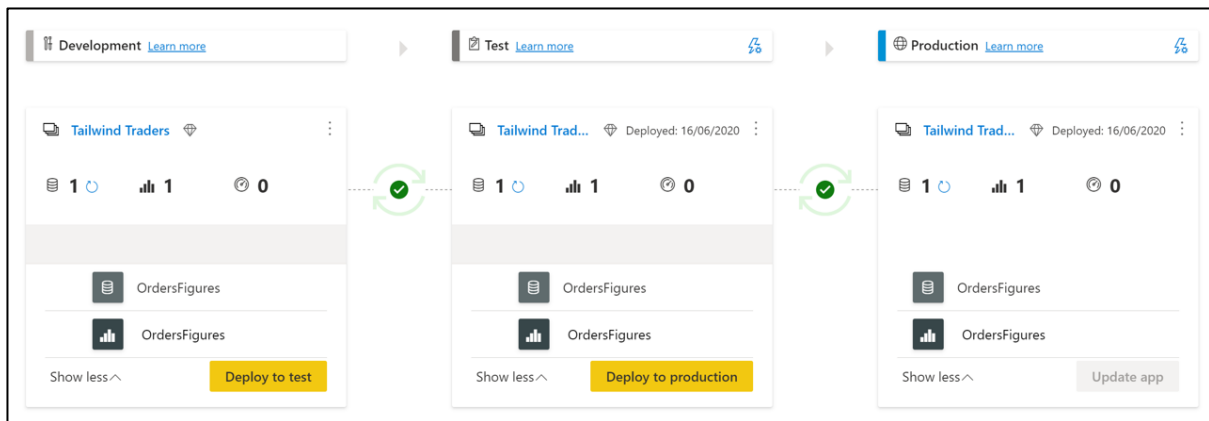
Now, you are close to completing the pipeline, transitioning from development to testing, and finally to production.

At this stage, you need to create a data source rule for the **OrdersFigures** semantic model in the workspace to ensure that you are using production data.

In this instance, you will be changing your source from the test to the production folder version of the orders.csv file, as shown in the following screenshot.



After performing a semantic model refresh, your production workspace will be ready. You can package the workspace as an app, which is available for users. Currently, your deployment pipeline will appear as shown in the following figure.



You have successfully created a deployment pipeline from the development to the testing phase.

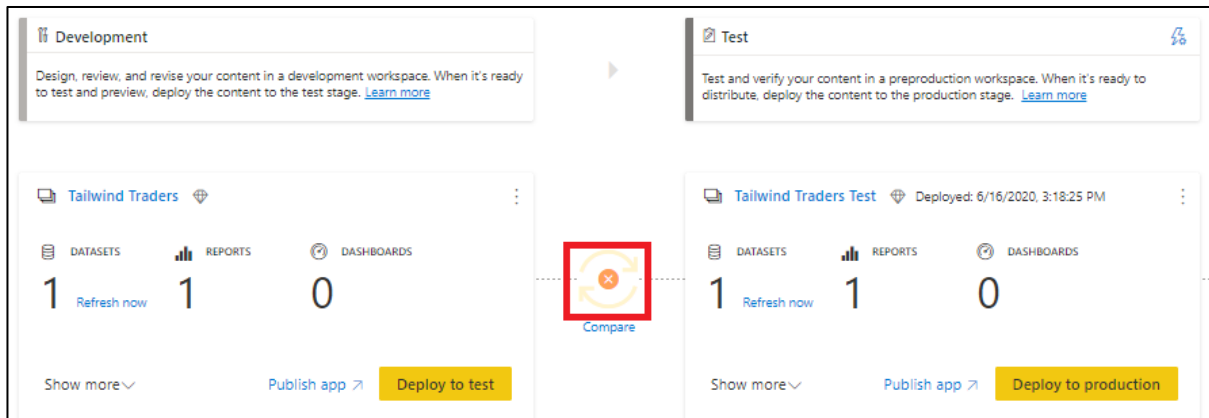
The following section describes additional operations that you can conduct in the development pipeline.

Additional operations in the development pipeline

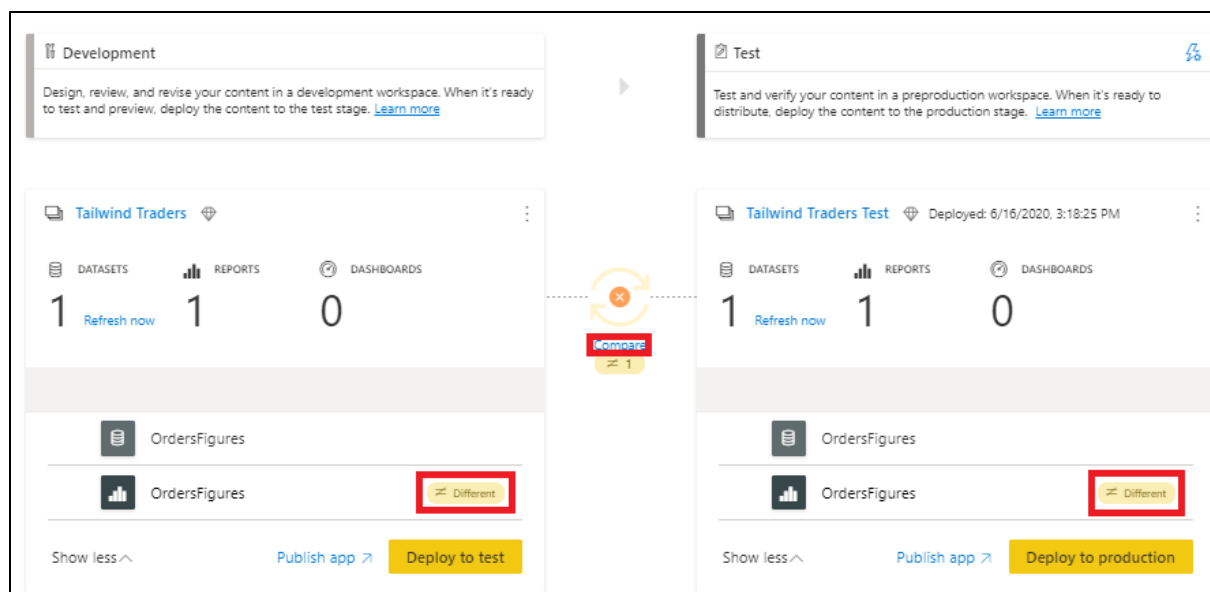
You have created a deployment pipeline and have begun collaborating with other report developers.

You receive notification that one of the other developers has modified a report.

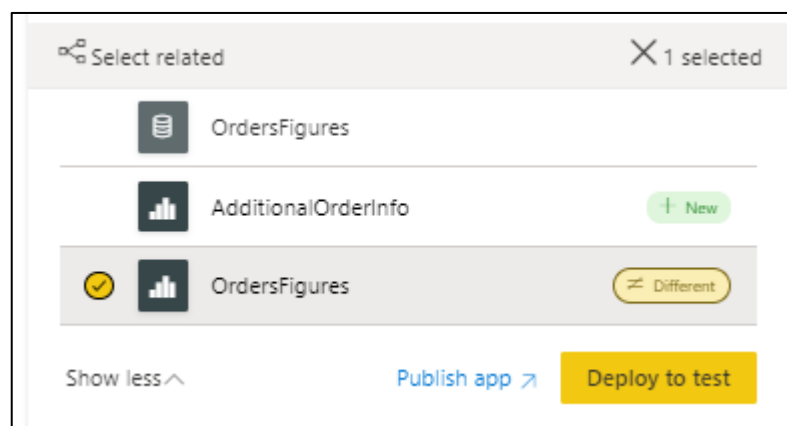
To see the changes to this report, select the **Compare** button, as shown in the following screenshot.



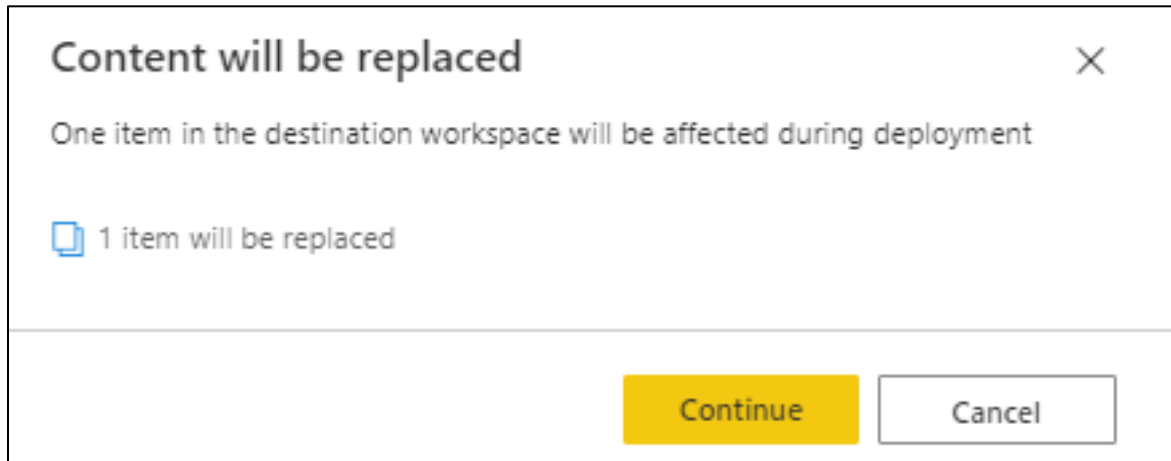
Selecting **Compare** reveals that the **OrdersFigures** report differs between the Development and Test environments.



The difference is typically registered as added or removed objects. If you decide that the changes shouldn't be deployed to the next phase, you can choose to ignore the changes. For instance, the other developer has added a report called **AdditionalOrderInfo** in the Development environment, but you don't want to deploy these changes. By selecting a specific report and then selecting **Deploy to test**, you can effectively choose which reports that you want to move from environment to environment, as shown in the following figure.



As the following message indicates, only one change will be carried over.



Exercise caution with this tool. Reports are dependent on their semantic models. If a semantic model has changed, but you don't deploy it with an associated report, the report will not behave correctly.

We recommend that you use deployment pipelines in Power BI service. This tool ensures that the development life cycle is streamlined and that you can create one centralized location to collaborate, keep track of, and deploy your reports.

Troubleshoot Data by Viewing Its Lineage

The **Lineage** view feature in Power BI allows you to quickly refresh semantic models and see the relationships between the artifacts in a workspace and their external dependencies.

Consider this module's continuing scenario with Tailwind Traders as an example. Thus far, you've developed several reports and have published them to the Tailwind workspace. However, because you are also collaborating with the Products team, it has become increasingly difficult to track which reports need to be refreshed and which semantic models are in which report.

Consequently, you want the ability to determine which semantic models need to be refreshed because you've been receiving reports of stale data. The path of data from its source to the destination can often be a considerable challenge, more so if you have multiple semantic models.

The **Lineage** view feature can help you accomplish this task efficiently and almost effortlessly.

Data lineage

Data lineage refers to the path that data takes from the data source to the destination.

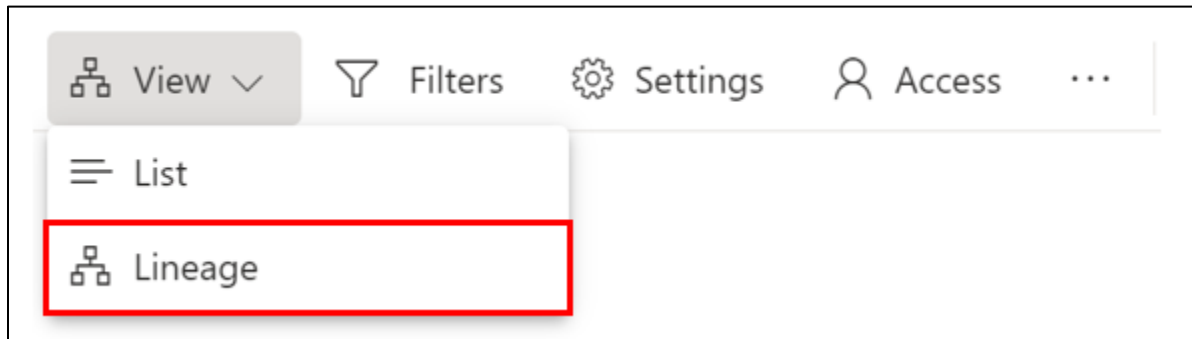
The **Lineage** view feature in Power BI is crucial because it:

- Simplifies the troubleshooting process because you can see the path that the data takes from source to destination and determine pain points and bottlenecks.
- Allows you to manage your workspaces and observe the impact of a single change in one semantic model to reports and dashboards.
- Saves time by simplifying your task of identifying reports and dashboards that haven't been refreshed.

Use the Lineage view

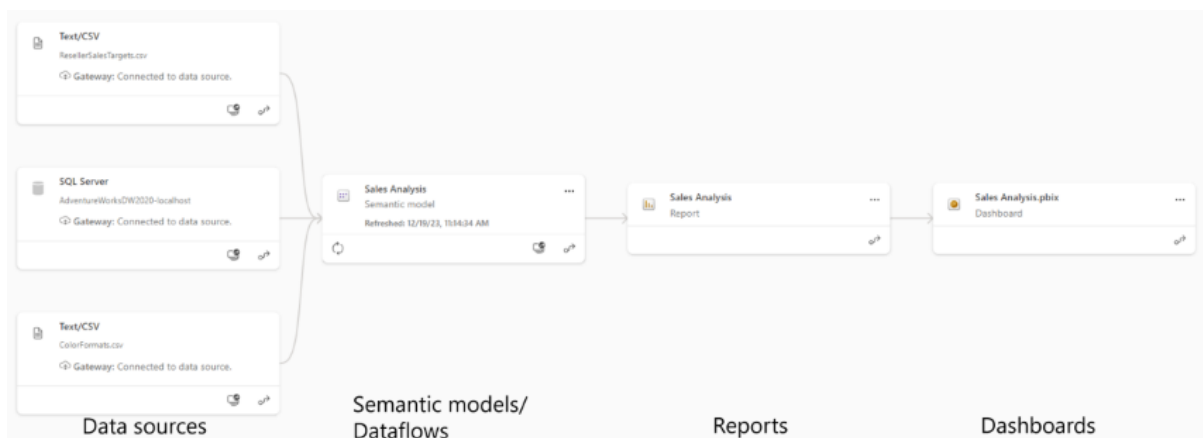
The **Lineage** view is only accessible to **Admin**, **Contributor**, and **Member** roles. Additionally, it requires a Power BI Pro license and is only available for app workspaces.

To access the **Lineage** view, go to the workspace, and then select **Lineage** from the **View** drop-



down menu on the top ribbon.

When the view canvas opens, you can begin to explore this view. The following example shown an excerpt of the data lineage for the **Tailwind Sales** workspace.



This view shows all the artifacts in your workspace. Artifacts include data sources, semantic models and dataflows, reports, and dashboards.

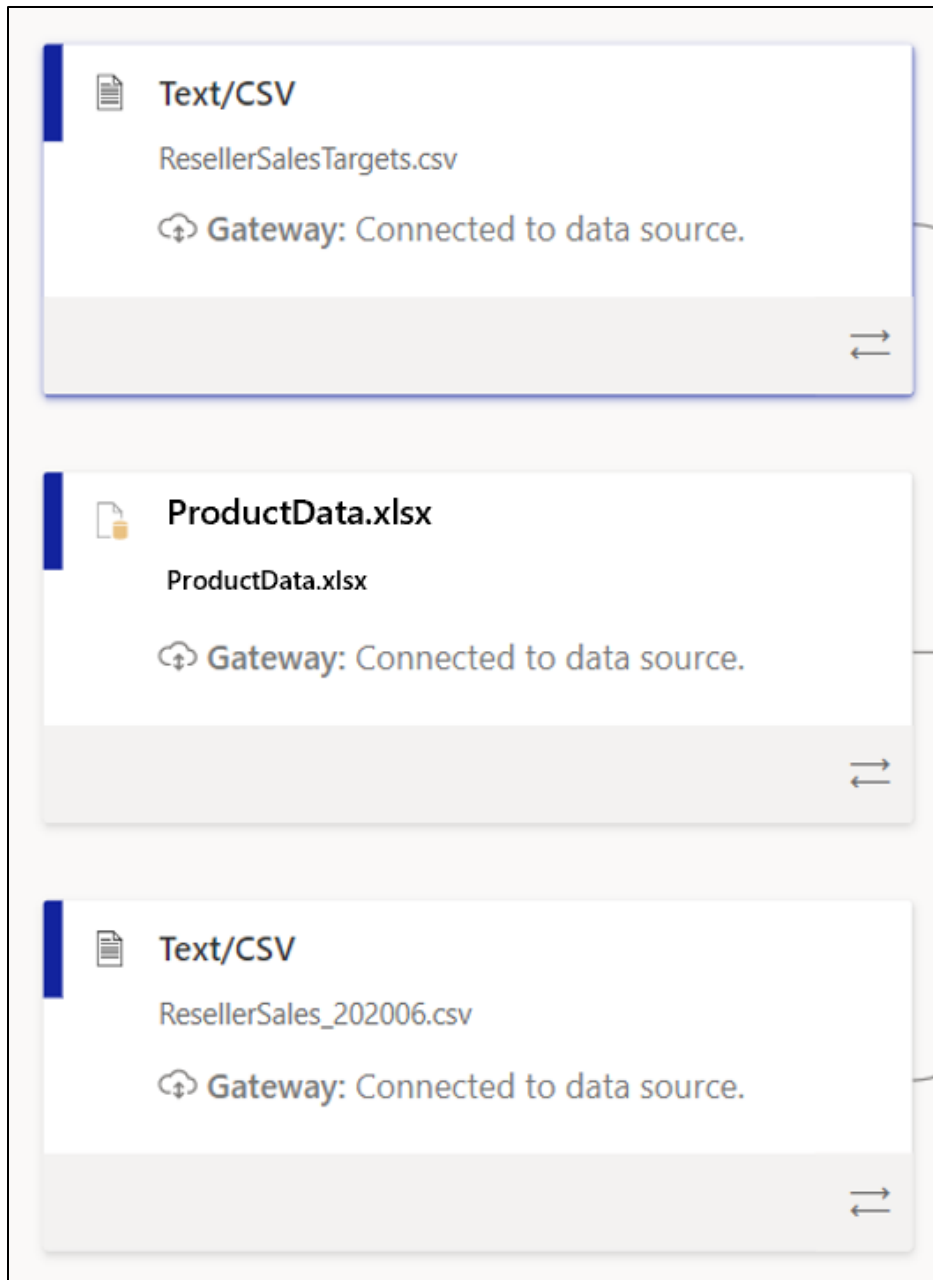
Each card represents an artifact, and the arrows in between these cards represent the flow of data or the relationship between different artifacts.

By following the arrows from left to right, you can observe the flow of data from the source to the destination, which will often be a dashboard.

Typically, the flow would be **data sources** > **semantic models/dataflows** > **reports** > **dashboards**.

Data sources

Each of the following cards is a data source that is used in your workspace.

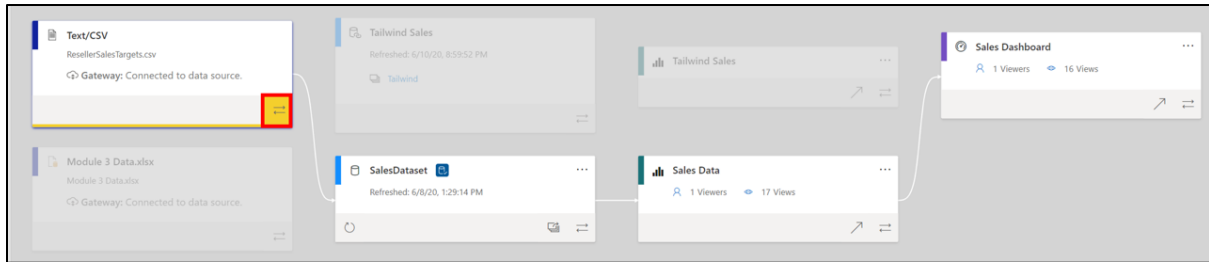


The card tells you the type of data source (for example, **Text/CSV**) and the **Gateway**, which tells you the source of your data.

If you are connected to the data through an on-premises data gateway, this card will tell you more information about the gateway.

Additionally, if you double-click the card, you will get more details about the data source, such as the file path and the connection status.

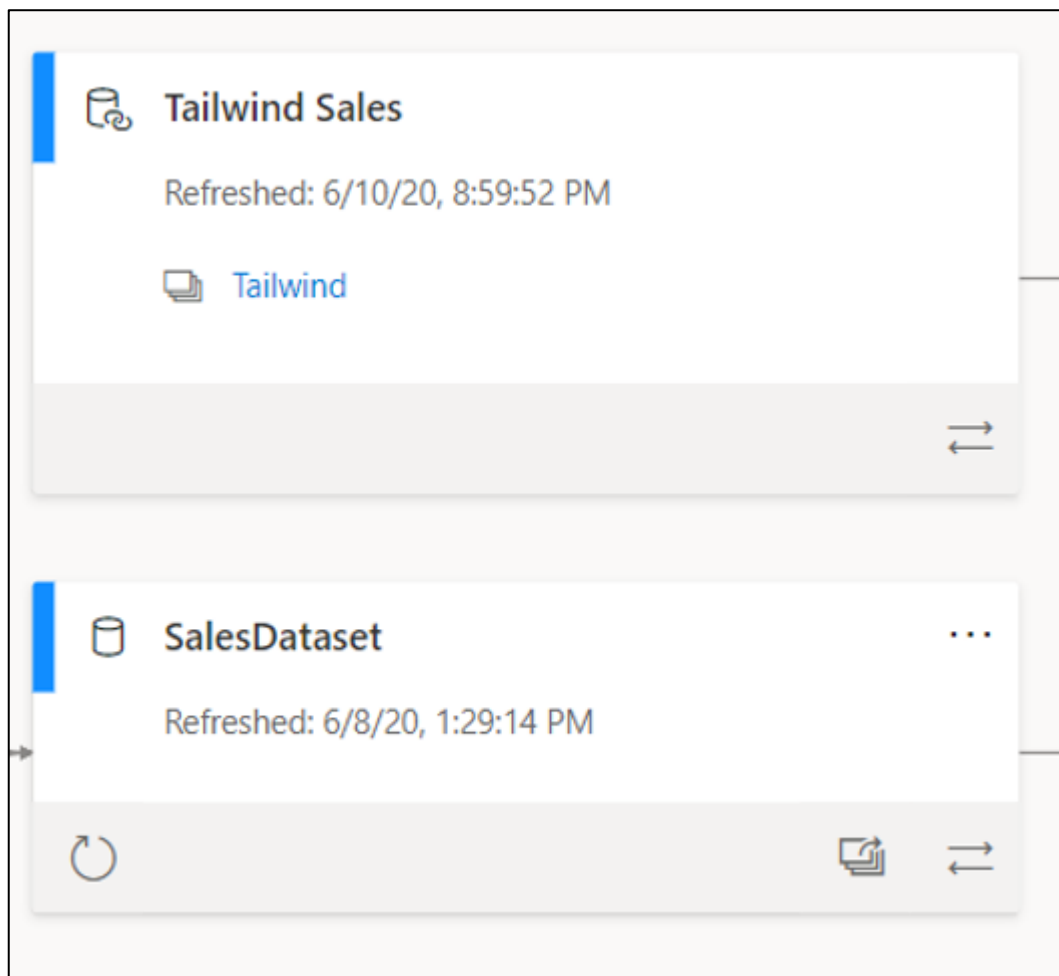
Selecting the lower-right icon on the card will highlight the path from the data source to the destination, as shown in the following screenshot, which clarifies the exact path that the data takes.



Next are the semantic models/dataflows, which are marked in blue.

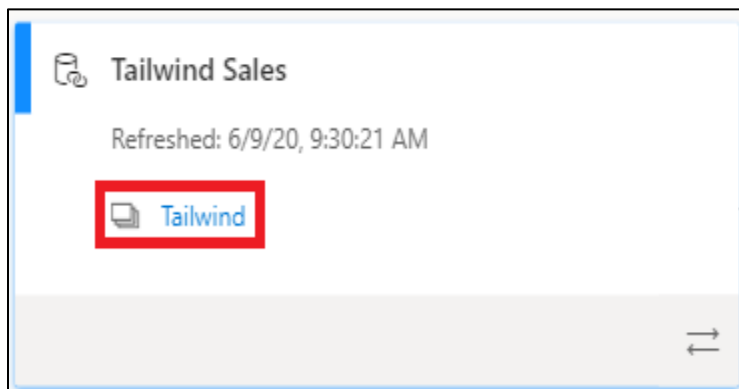
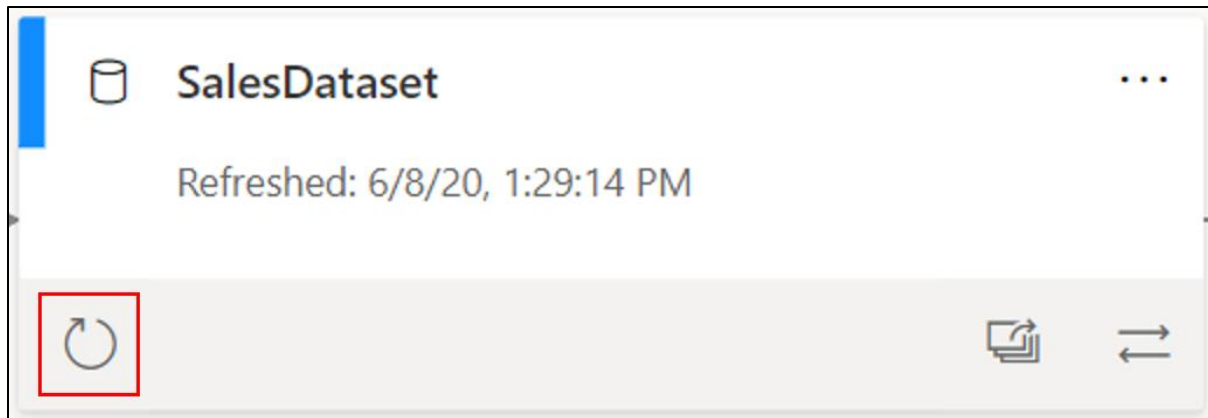
Semantic models/dataflows

Often, semantic models and dataflows can connect to external data sources, such as SQL Server, or to external semantic models in other workspaces. The following examples show semantic model and dataflow cards on the **Lineage** view.



The **Lineage** view uses arrows to connect objects, such as semantic models, with their data sources.

On these cards, you can see when the semantic model was last refreshed, and you can refresh the semantic model by selecting the arrow icon on the lower-left corner of the card, as shown in the following screenshot.



This component is a powerful troubleshooting feature that helps ensure that your semantic model refreshes are quick and uncomplicated.

Returning to the initial quandary with Tailwind Traders, you wanted to determine if the company had stale semantic models and then quickly refresh the data.

By using the **Lineage** view feature, you can go through the different semantic models in one view and then use the **Refresh data** button to refresh semantic models that you determine as stale.

Additionally, if a semantic model or dataflow belongs to a different workspace (in this case, the **Tailwind** workspace), it will be indicated on the card, as shown in the following screenshot.

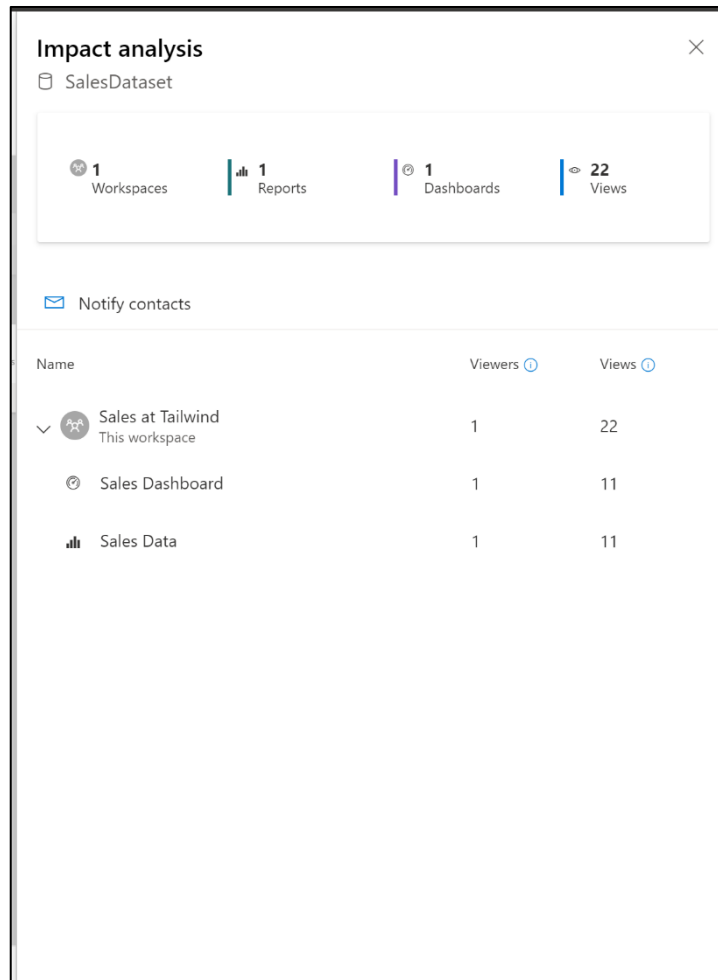
By double-clicking on any card, you can view the metadata, such as the sensitivity, by whom it was configured, the last refresh date, and the names and count of tables within this semantic model, as shown in the following figure.

You can also view the impact of this semantic model across workspaces

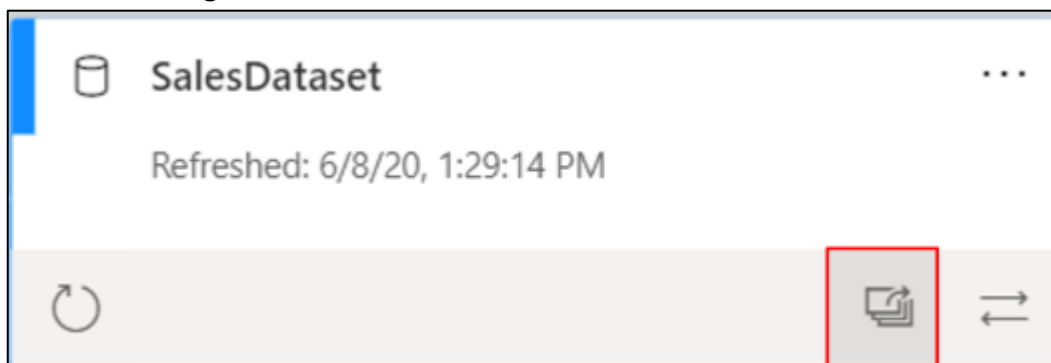
By selecting the overlapping window icon on the lower-right corner of a semantic model card, you can determine the impact analysis of the semantic model.

Next refresh --	
Total Tables 17	
Budget	▼
Calendar	▼
Country	▼
CountryName	▼
Customers	▼
Emp	▼
Employee	▼
Order	▼
Product	▼
Product ID	▼
Query2	▼

On the **Impact**



analysis window, you can see how many workspaces, reports, and dashboards that this semantic model is a part of and how many views that this semantic model has gathered, as shown in the following screenshot.



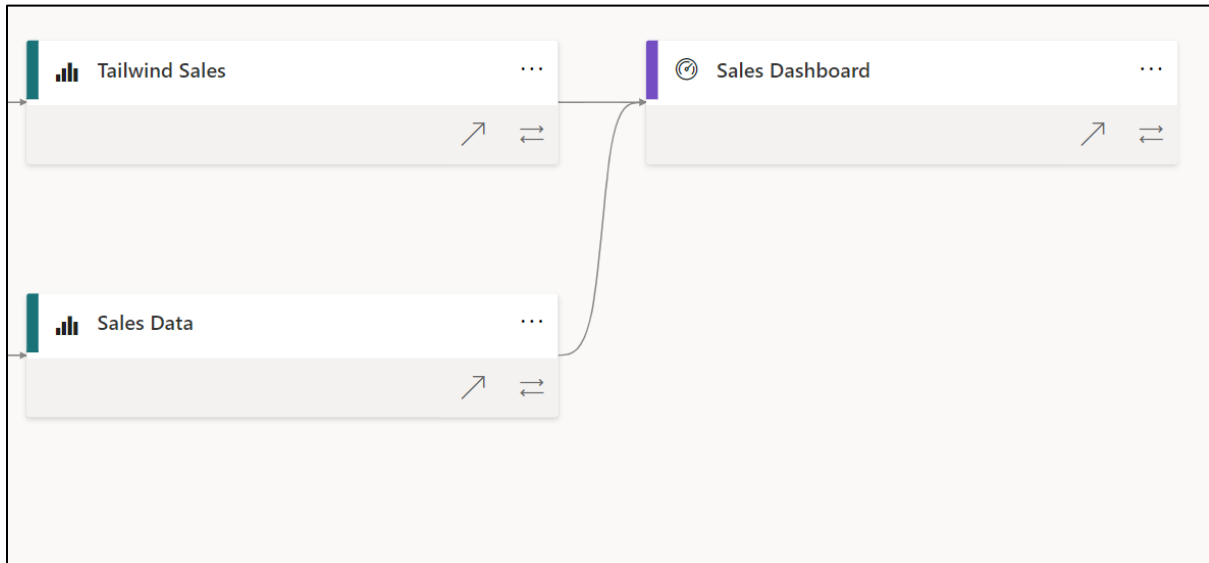
The bottom of the **Impact Analysis** window includes more detail about which specific reports and dashboards that this semantic model is part of.

Additionally, you can select **Notify contacts**, which allows you to notify semantic model owners (or any other user) of changes in the semantic model.

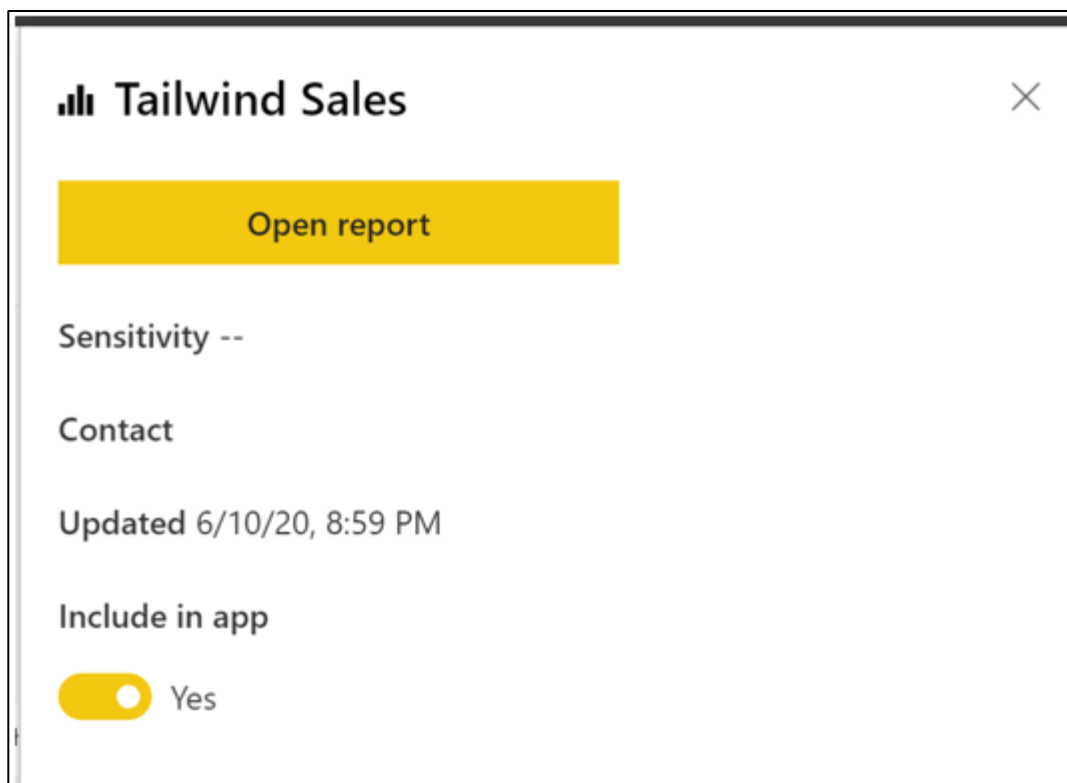
Impact analysis is useful because it allows you to pinpoint semantic models that aren't being used or looked at.

Reports and dashboards

The reports and dashboards cards have similar functionality as the data source and semantic model cards.

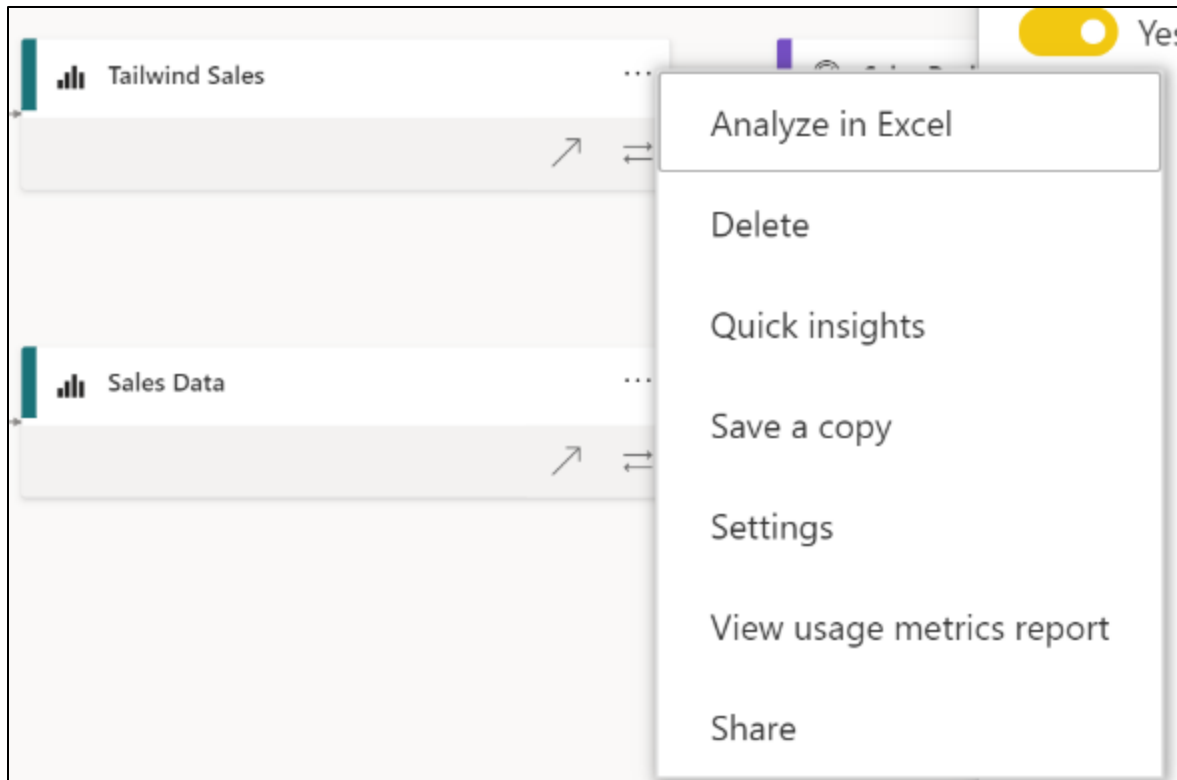


Selecting a card will bring up a window in which you can view the metadata about the report or dashboard. In this window, you can also go directly to the report or dashboard. You can also enable or disable whether you want to include this report or dashboard within the app.



This card also contains useful options under the ellipsis (...), as shown in the following figure.

From this menu, you can select to analyze the report in Microsoft Excel, delete a report, create Quick Insights, save a copy to a workspace, and more.



Configure Data Protection

As enterprises grow, so does their data. Often, strict requirements and regulations must be applied to ensure that this sensitive data is secure. Power BI provides a few different ways to help you accomplish this task:

- Use Microsoft sensitivity labels to label dashboards, reports, semantic models, and dataflows by using the same taxonomy that is used to classify and protect files in Microsoft 365.
- Add more protection measures such as encryption and watermarks when you are exporting the data.
- Use Microsoft Cloud App Security to monitor and investigate activities in Power BI.

Sensitivity labels

Sensitivity labels specify which data can be exported. These labels are configured externally to Power BI, and Power BI allows you to quickly use them in your reports and dashboards.

These labels allow you to define and protect content, even outside of Power BI. semantic models, dataflows, reports, and dashboards can use this mechanism, and all users in your corporation can use this feature unless exceptions have been defined.

Manage Semantic Models in Power BI

Introduction

When your semantic models are published to your organization's workspace in Microsoft Power BI service, everyone who needs access to those semantic models can find them in a central location, which provides opportunities for collaboration between teams.

It also reduces the duplication of effort because one semantic model can be used by multiple users for different business reasons.

For instance, one semantic model can be used to create multiple Power BI reports. Because preparing and cleaning data can be time-consuming, sharing semantic models can be a productivity boost for report authors.

The sharing of semantic models needs to be actively managed for optimal organizational performance. For example, you can automate the refresh process so that it becomes more efficient and users always have access to the latest data.

You can also promote certain semantic models over others so that users can clearly identify the best semantic models to use.

Management of semantic models also involves the implementation of parameters within those semantic models to help with decision making and solving business problems.

For example, you can use parameters to change the server or database name of your semantic model or a file path for a data source.

You can also use parameters to configure incremental refreshes of your data and to run "what-if" scenarios and conduct scenario-type analysis on the data.

Another key area of semantic model management is setting up and maintaining a gateway so that you and other users can access your on-premises data source from the cloud.

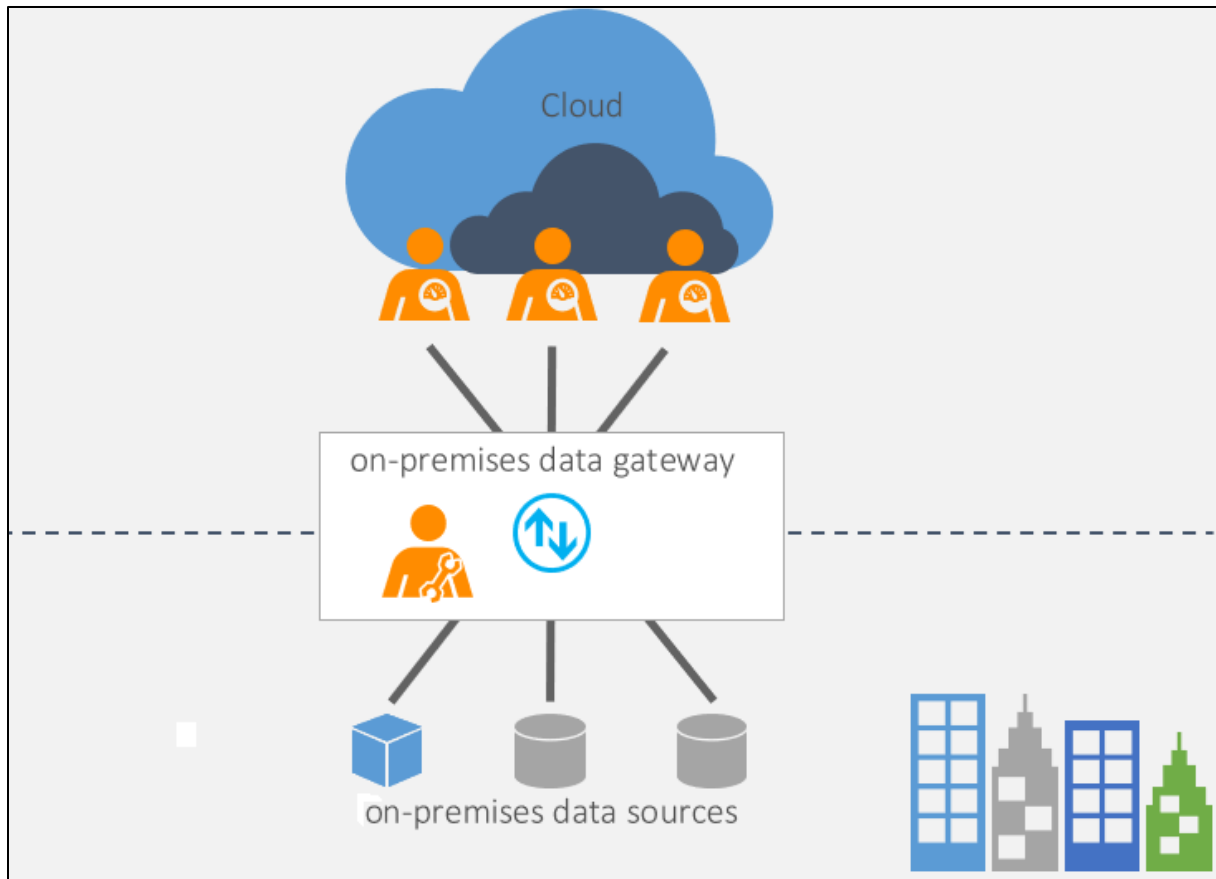
You also need to prepare for potential issues that might arise regarding this gateway, which could interrupt user access to the semantic models.

The effect of a service connectivity issue could be detrimental to user productivity; if they can't access the data, they can't do their jobs, and the organization's decision-making capability will be at a standstill. Being prepared to deal with such issues in a timely manner is critical.

Use Power BI Gateway to Connect to On-Premises Data Sources

Gateway software acts like a bridge; it allows organizations to retain databases and other data sources on their on-premises networks and access that on-premises data in cloud services, such as Power BI and Microsoft Azure Analysis Services.

A gateway facilitates quick, behind-the-scenes communication that flows from a user in the cloud to your on-premises data source and then back again to the cloud.



Two types of on-premises gateways are:

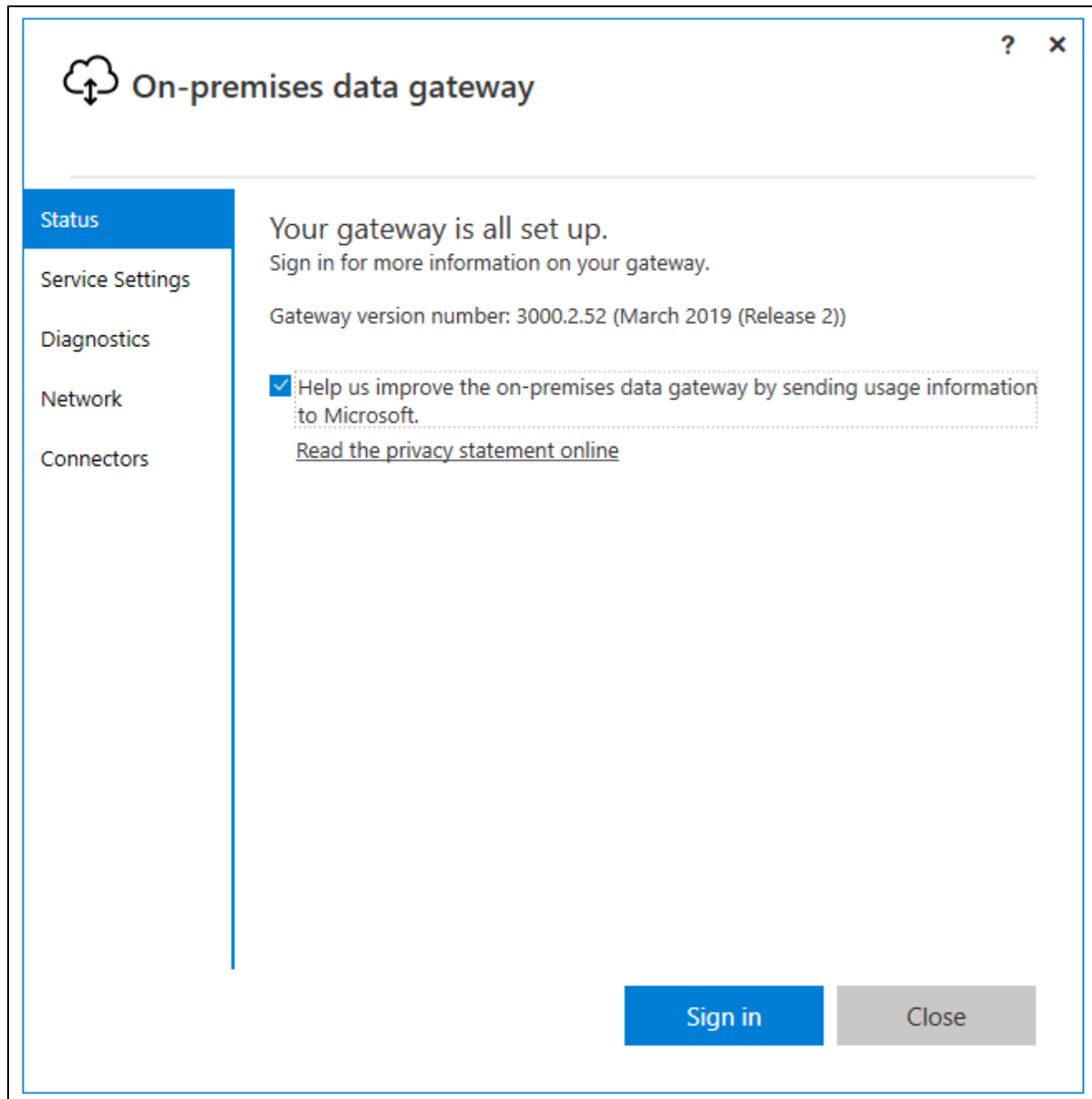
- **Organization mode** - Allows multiple users to connect to multiple on-premises data sources and is suitable for complex scenarios.
- **Personal mode** - Allows one user to connect to data sources. This type of gateway can be used only with Power BI and it can't be shared with other users, so it is suitable in situations where you're the only one in your organization who creates reports.
 - You will install the gateway on your local computer, which needs to stay online for the gateway to work.

Use an on-premises gateway

Before you can connect to your on-premises data source, you need to install the on-premises data gateway, and then configure it to suit your organizational needs.

Usually, this task is completed by an admin in your organization.

When the on-premises gateway is installed and configured, you can start the gateway and then sign in by using your Microsoft 365 organization account.



When you are working in the cloud and interacting with an element that is connected to an on-premises data source, the following actions occur:

- The cloud service creates a query and the encrypted credentials for the on-premises data source.
 - The query and credentials are sent to the gateway queue for processing.
- The gateway cloud service analyzes the query and pushes the request to Microsoft Azure Service Bus.
- Service Bus sends the pending requests to the gateway.
- The gateway gets the query, decrypts the credentials, and then connects to one or more data sources with those credentials.
- The gateway sends the query to the data source to be run.
- The results are sent from the data source back to the gateway and then to the cloud service. The service then uses the results.

Configure a Semantic Model Scheduled Refresh

The **Scheduled refresh** feature in Power BI allows you to define the frequency and time slots to refresh a particular dataset. Scheduling the refresh of your data will save you time because you don't have to manually refresh the data. It also ensures that users can access the most up-to-date data.

In this example, you are creating a report, but then realize that the version of the sales data that you're using isn't the most up to date. You check the refresh status and notice that it was last refreshed 10 days ago, and no refresh is scheduled to take place.

All

Content

Datasets + dataflows

Name	Type	Owner	Refreshed	Next refresh
 TailwindTraders2	Dataset	TailwindTraders	06/19/20, 01:58:14	N/A

Considering how important it is to have accurate sales data; you need to find a solution. Usually, the data is updated weekly, but you don't want to return to the report every week to manually refresh the semantic model, and you know that you occasionally forget to do so.

Therefore, you decide to use the **Scheduled refresh** functionality in Power BI to solve this problem.

Set up a refresh schedule

Before you can set up a refresh schedule, you need to have created a gateway connection.

To set up a refresh schedule for your semantic model, follow these steps:

1. Go to the **semantic models + dataflows** page.
2. Hover over the semantic model for which you want to set up the schedule and then select the **Schedule refresh** icon.



3. On the **Settings** page, turn on the **Scheduled refresh** feature.
4. Select the **Refresh frequency** and ensure that the correct time zone is selected.
5. Add the time(s) that you want the refresh to occur. You can configure up to eight daily time slots, if your semantic model is on shared capacity, or 48 time slots on Power BI Premium.
6. When you have finished configuring the scheduled refresh, select **Apply**.

While you can set a time for the refresh, be aware that the refresh might not take place at that exact time. Power BI starts scheduled refreshes on a best effort basis. The goal is to initiate the

refresh within 15 minutes of the scheduled time slot, but a delay of up to one hour can occur if the service can't allocate the required resources sooner.

In this example, you want the system to refresh the sales data on a daily basis, at 6:00 AM, 10:00 AM, and 3:00 PM, as illustrated in the following image.

Scheduled refresh

Keep your data up to date

Configure a data refresh schedule to import data from the data source into the dataset.

☒ On

Refresh frequency

Daily

Time zone

(UTC-08:00) Pacific Time (US and Cani

Time

[Add another time](#)

Send refresh failure notifications to

☒ Semantic model owner

☐ These contacts:

Enter email addresses

Apply **Discard**

When you have configured a refresh schedule, the semantic model settings page informs you of the next refresh time, as shown in the following image.

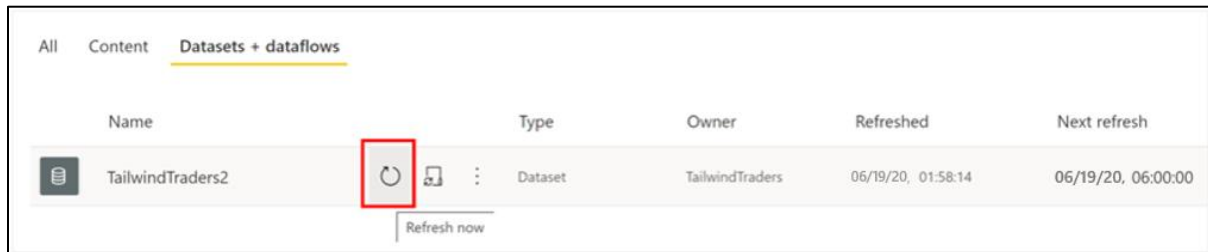
Name	Type	Owner	Refreshed	Next refresh
 TailwindTraders2	Dataset	TailwindTraders	06/19/20, 01:58:14	06/19/20, 06:00:00

Perform an on-demand refresh

In addition to the scheduled refreshes, you can refresh a dataset at any time by performing an on-demand refresh. This type of refresh doesn't affect the next scheduled refresh time.

For example, you might want to refresh now because you need to view the most recent data and can't wait for the next refresh time, or you might want to test your gateway and data source configuration.

To perform an on-demand refresh, on the **semantic models + dataflows** page, hover over the semantic model that you want to refresh and then select the **Refresh now** icon.



Check the refresh status and history

You can check the refresh status and history at any time. This feature is useful if you want to find out when the last refresh occurred and when the next one is scheduled. It is also good practice to check the status of your semantic models occasionally to check if refresh errors have occurred.

Power BI deactivates your refresh schedule after four consecutive failures or when the service detects an unrecoverable error that requires a configuration update, such as invalid or expired credentials. It is not possible to change the consecutive failures threshold.

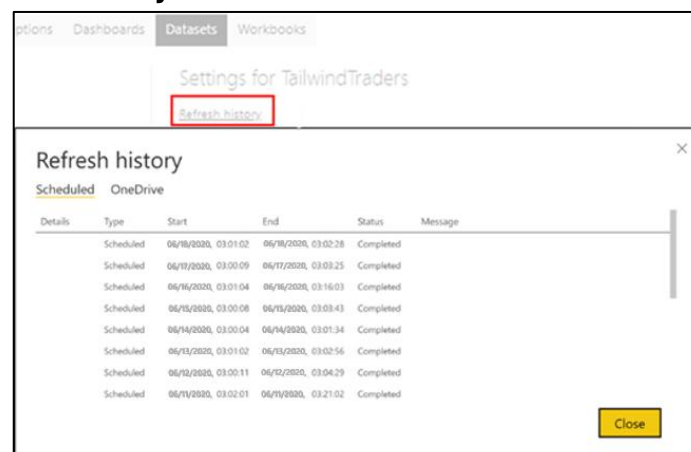
A quick way to check the refresh status is to view the list of semantic models in a workspace.

Name	Type	Owner	Refreshed	Next refresh
TailwindTraders2	Dataset	TailwindTraders	06/19/20, 01:58:14	06/19/20, 06:00:00

If a semantic model displays a small warning icon, you'll know that the semantic model is currently experiencing an issue. Select the warning icon to get more information.

Name	Type	Owner	Refreshed	Next refresh
TailwindTraders2	Dataset	TailwindTraders	06/19/20, 02:14:13	06/19/20, 06:00:00

You should also check the refresh history occasionally to review the success or failure status of past synchronization cycles. To view the refresh history, open the semantic model's settings page and then select **Refresh history**.



Configure Incremental Refresh Settings

The **Incremental refresh** feature in Power BI is a popular feature because it allows you to refresh large semantic models quickly and as often as you need, without having to reload historical data each time.

Incremental refresh should only be used on data sources and queries that support query folding. If query folding isn't supported, incremental refresh could lead to a bad user experience because, while it will still issue the queries for the relevant partitions, it will pull all data, potentially multiple times.

Traditionally, complex code was required for running incremental refreshes, but you can now define a refresh policy within Power BI Desktop. The refresh policy is applied when you publish to Power BI service, which then does the work of managing partitions for optimized data loads, resulting in the following benefits:

- **Quicker refreshes** - Only data that needs to be changed gets refreshed. For example, if you have five years' worth of data, and you only need to refresh the last 10 days because that is the only data that has changed, the incremental refresh will refresh only those 10 days of data. Undoubtedly, the time it takes to refresh 10 days of data is much shorter than five years of data.
- **More reliable refreshes** - You no longer need to keep your long-running data connections open to schedule a refresh.
- **Reduced resource consumption** - Because you only need to refresh the smaller the amount of data, the overall consumption of memory and other resources is reduced.

In this example, the Sales team has come to you with a dilemma. The data in their report is already out-of-date. It isn't feasible for you to manually refresh the data by adding a new file because the refreshes need to happen regularly to match the frequency of the sales transactions that are occurring.

Also, the manual refresh task is becoming more difficult because the semantic models have millions of rows. Consequently, you need to implement a better data refresh solution.

You can define an incremental refresh policy to solve this business problem. This process involves the following steps:

1. Define the filter parameters.
2. Use the parameters to apply a filter.
3. Define the incremental refresh policy.
4. Publish changes to Power BI service.

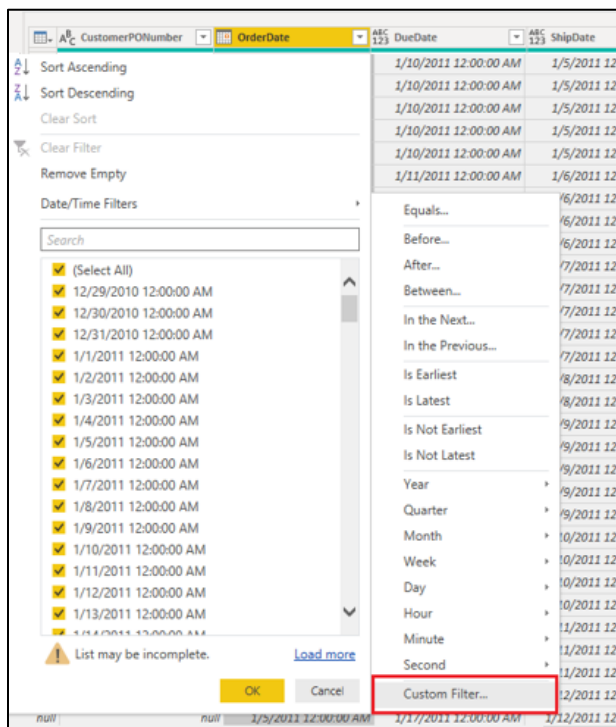
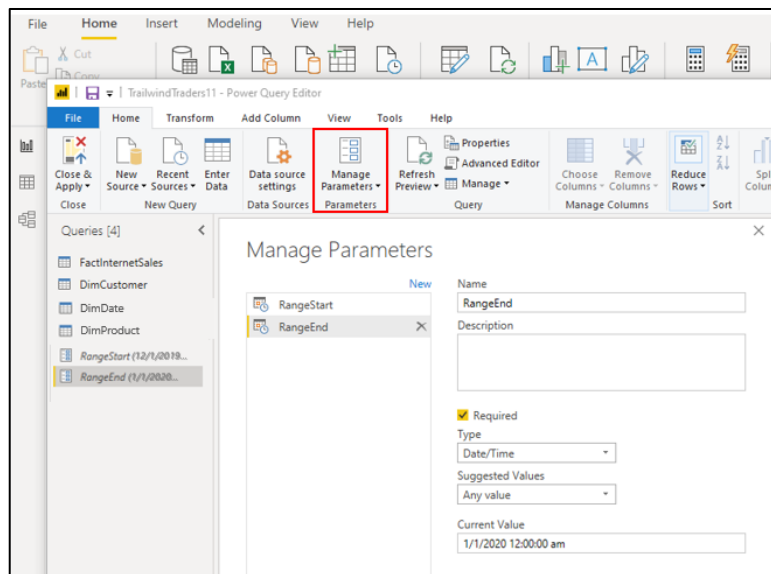
Define filter parameters

Whether you are using incremental refresh or not, large semantic models are commonly filtered when they are imported into Power BI Desktop because the PBIX file is limited by the memory resources that are available on the desktop computer. For incremental refresh, the semantic models are filtered by two date/time parameters: **RangeStart** and **RangeEnd**. These parameters have a dual purpose.

In Power BI Desktop, they are the filtering window because they restrict the used data to the range that is listed in the start and end dates. After they have been published to the service, the parameters are taken over to be the sliding window to determine what data to pull in.

To define the parameters for the incremental refresh, follow these steps:

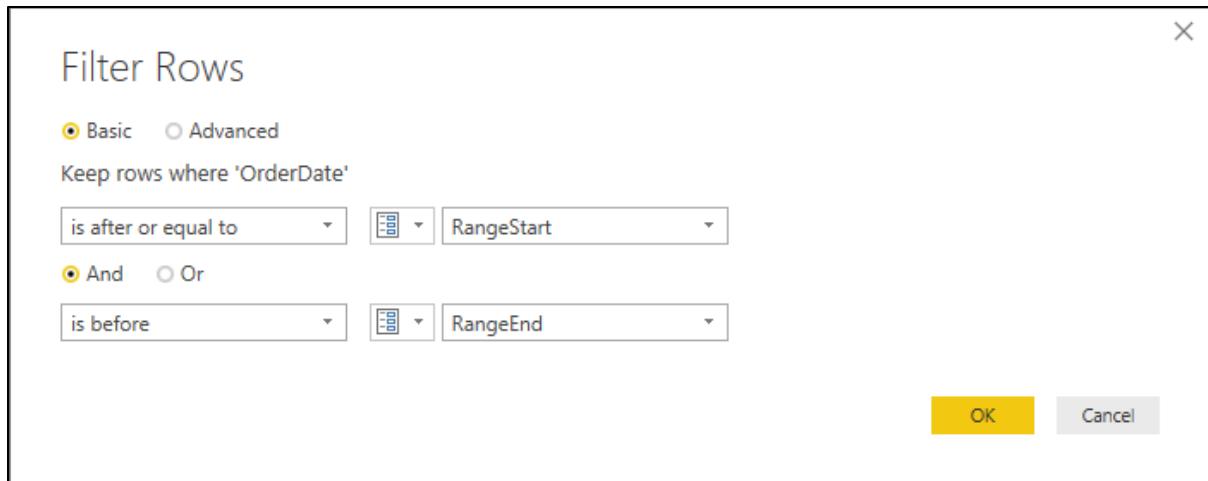
1. Open your semantic model in Power Query Editor.
2. On the **Home** tab, select **Manage Parameters**.
3. On the **Parameters** window that displays, add two new parameters, **RangeStart** and **RangeEnd**, ensuring that for both parameters, the **Type** is set to **Date/Time** and the **Suggested Value** is set to **Any value**.
4. Regarding the **Current Value**, for the **RangeStart** parameter, enter the date on which you want to begin the import, and for the **RangeEnd** parameter, enter the date on which you want the import to end.



Apply the filter

When you have defined the new parameters, you can apply the filter by following these steps:

1. Go to the applicable **Date** column and then right-click that column and select **Custom Filter**.
2. In the **Filter Rows** window that displays, to avoid the double counting of rows, make sure that you keep rows where **OrderDate** is after or equal to the **RangeStart** parameter and before the **RangeEnd** parameter.



3. Select **Close and Apply** from the Power Query Editor.

You should now see a subset of the semantic model in Power BI Desktop.

Define the incremental refresh policy

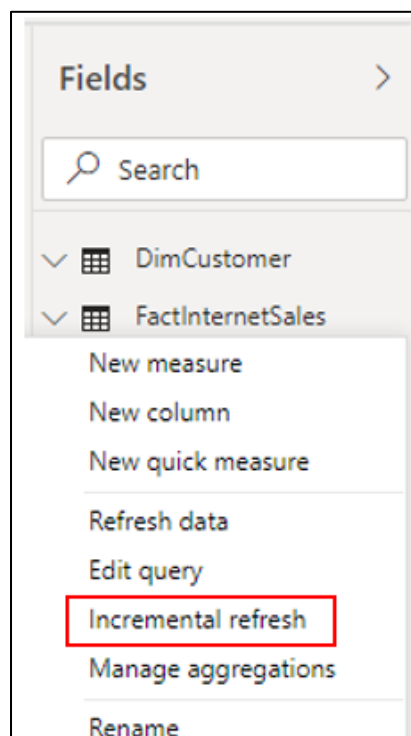
When you have filtered the data, you can define the incremental refresh policy for the data table, which sets up the refresh process.

Right-click the applicable table and then select **Incremental refresh**.

On the **Incremental refresh** window that displays, turn on the **Incremental refresh** option.

Then, configure the refresh as required.

In this example, you will define a refresh policy to store data for five full calendar years, plus data for the current year up to the current date, and incrementally refresh 10 days of data.



Incremental refresh and real-time data

Refresh large tables faster with incremental refresh. Plus, get the latest data in real time with DirectQuery (Premium only). [Learn more](#)

ⓘ These settings will apply when you publish the dataset to the Power BI service. Once you do that, you won't be able to download it back to Power BI Desktop. [Learn more](#)

1. Select table

Orders

2. Set import and refresh ranges

☒ Incrementally refresh this table

Archive data starting

Years

 before refresh date

Data imported from 1/1/2017 to 11/17/2022 (inclusive)

Incrementally refresh data starting

Days

 before refresh date

Data will be incrementally refreshed from 11/18/2022 to 11/20/2022 (inclusive)

3. Choose optional settings

☐ Get the latest data in real time with DirectQuery (Premium only) [Learn more](#)

Selected table cannot be folded for DirectQuery.

☐ Only refresh complete days [Learn more](#)

☐ Detect data changes [Learn more](#)

4. Review and apply

Archived

Incremental Refresh

The first refresh operation in Power BI service will load the historical data for the last five years. The subsequent refresh operations are incremental, and they will refresh the data that was changed in the last 10 days up to the current date. The incremental refreshes will also remove calendar years that are older than five years prior to the current date.

Publish to Power BI service

When you have defined the incremental refresh policy in Power BI Desktop, to apply that refresh policy, you need to publish the report to Power BI service.

Manage and Promote Semantic Models

Business intelligence involves collaboration, and sharing semantic models across workspaces is a powerful way to collaborate within your organization. However, if your organization has many different semantic models that can be accessed by many users, you might want to take measures to manage those semantic models. For instance, you might want to direct your users to the most up-to-date and highest-quality semantic models in your workspaces, or you might want to restrict the reuse of semantic models across your workspaces.

To ensure that your organization has consistent data for making decisions and a healthy data culture, it's important to create and share optimized semantic models and then endorse those semantic models as the *one source of truth*. Report creators can then reuse those endorsed semantic models to build accurate, standardized reports.

Power BI provides two ways to endorse your semantic models:

- **Promotion** - Promote your semantic models when they're ready for broad usage.
- **Certification** - Request certification for a promoted semantic model. Certification can be a highly selective process, so only the truly reliable and authoritative semantic models are used across the organization.

In this example, you and the other teams are using a workspace in Power BI service to organize all your reports and dashboards. However, you begin to receive emails from confused users who expected to see a sales report and are now looking at a product report instead. You need to make some changes to direct your users to the semantic models that they should be accessing, and you can accomplish this task with the endorsing capability in Power BI.

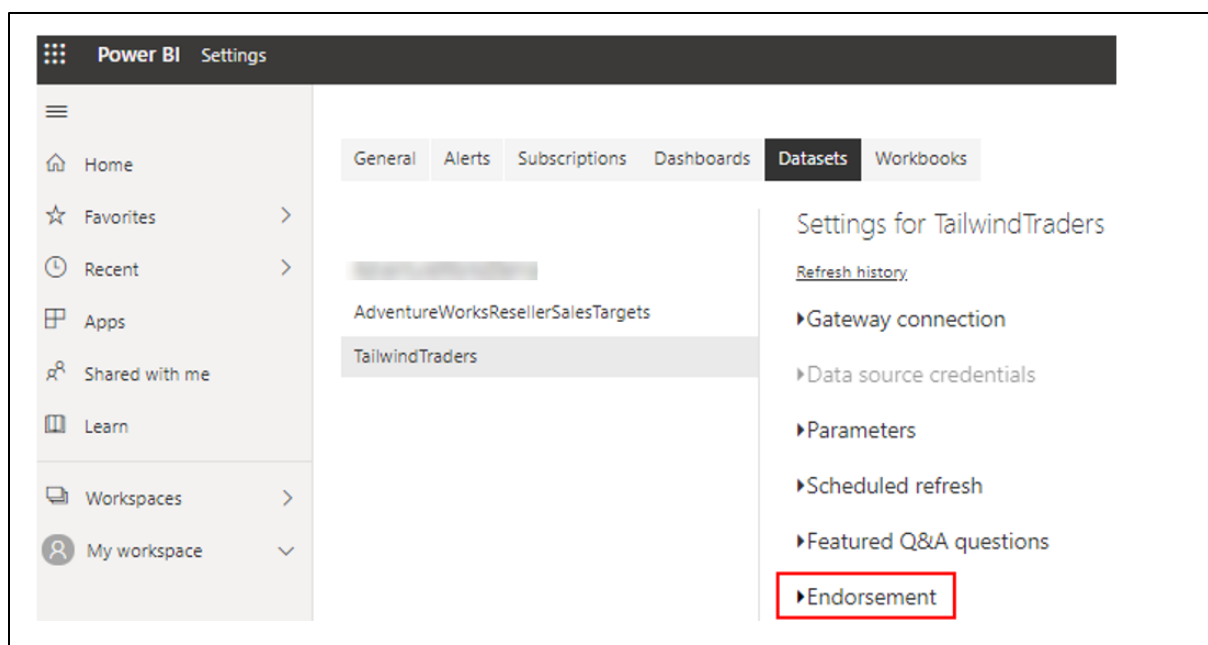
In this example, the certification type of endorsement is best suited for the Sales team because it will require users to have special access before they can view the Sales dashboards. By implementing the certification, you'll lead your users to the most appropriate reports and dashboards, avoiding the inevitable confusion that might arise with building and sharing a diversity of reports.

Though you'll soon learn how to certify the semantic model, you'll first learn how to promote a semantic model, in case you prefer to use that method.

Promote a semantic model

To promote content, you must have write permissions on the workspace where the content you want to promote is located.

To promote a semantic model, go to the workspace in Power BI service, and then open the settings page for the semantic model that you want to promote. In this example, you want to promote the Tailwind Traders semantic model.



Select the **Endorsement** setting.

In the **Endorsement** settings, select the **Promoted** option, and then select **Apply**.

Endorsement and discovery

Help coworkers find your quality content by endorsing this semantic model and making it discoverable. [Learn more](#)

☒ None
This semantic model will appear in search results but isn't endorsed.

☐ Promoted
When you're ready to distribute the semantic model to your coworkers, promote it to let them know.

☐ Certified
Certify your semantic model to show coworkers that it's been reviewed and meets your org's certification criteria. [How do I get my semantic model certified?](#)

☐ Make discoverable
Allow users without access to this semantic model to discover it and request permissions to access the data [Learn more](#)

When you return to your workspace, a badge in the **Endorsement** column for that semantic model will appear, indicating that it's ready for viewing by all of your users.

My workspace

+ New

View Filters Search

All Content Datasets + dataflows

Name	Type	Owner	Refreshed	Next refresh	Endorsement
[Redacted]	Report		6/10/20, 4:03:50 PM	—	—
[Redacted]	Dataset		6/10/20, 4:03:50 PM	N/A	—
AdventureWorksResellerSalesTargets	Report		6/3/20, 8:00:53 PM	—	—
AdventureWorksResellerSalesTargets	Dataset		6/3/20, 8:00:53 PM	N/A	—
TailwindTraders	Report		6/5/20, 7:28:05 PM	—	—
TailwindTraders	Dataset		6/5/20, 7:28:05 PM	N/A	

Certify a semantic model

Content certification is a big responsibility, and only authorized users can certify content. Other users can request content certification.

To request certification for a semantic model, you start the same way as you did to promote the semantic model in **Endorsement** section. If it is greyed out, your admins will provide details in a link titled, *"How do I get my semantic model certified?"* in the **Certified** section.

General Alerts Subscriptions Dashboards **Datasets** Workbooks

AdventureWorksResellerSalesTargets

TailwindTraders

Settings for TailwindTraders

[Refresh history](#)

Gateway connection

Data source credentials

Parameters

Scheduled refresh

Featured Q&A questions

Endorsement

Help your colleagues find, learn about, and connect to your dataset.

☒ Default
This dataset can be searched for and used by others.

☐ Promoted
Promote this dataset with a badge to show it's ready to be used by others.

☐ Certified
Request certification from experts in your org to get a badge that shows it's recommended for use by others. [Learn more](#)

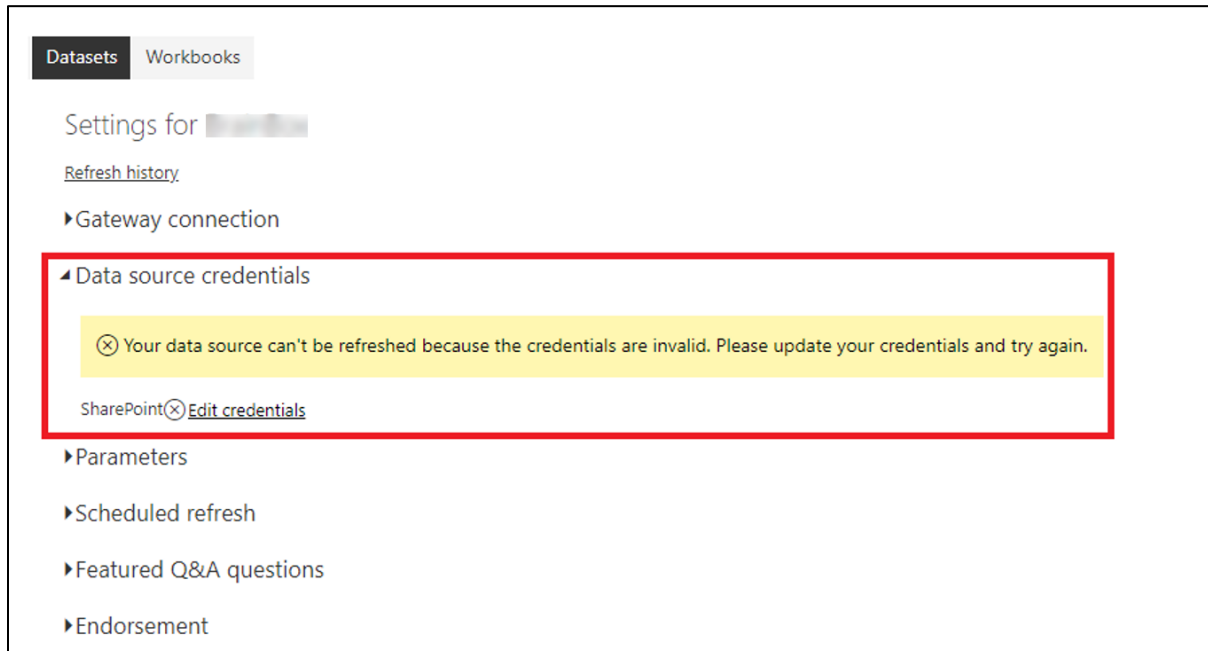
Description

Describe the contents of this dataset.

Troubleshoot Service Connectivity

Cloud services, such as SharePoint, do not require a gateway because the data is already in the cloud. You only need to provide your authorization credentials to set up a data source connection.

If your report fails to refresh, ensure that your data source credentials are up to date.



If your data source credentials are not up to date, you'll need to take further action to investigate and resolve the issue.

Boost Performance with Query Caching (Premium)

With the **Query Caching** feature, you can use the local caching services of Power BI to process query results. Instead of relying on the semantic model to calculate queries, you use cloud resources on your Premium capacities in the Power BI service to load your report.

Query caching ensures constant performance instead of overloading the semantic model resources.

Building on the scenario, you notice that some of your semantic models are causing the reports to load more slowly than before, an issue that is starting to annoy your users.

The Sales team wants to know how they can improve performance and make these reports load faster. You decide to use **Query Caching** to solve this problem.

Query caching

Query Caching is a local caching feature that maintains results on a user and report basis. Query caching reduces load time and increases query speed, especially for semantic models that aren't refreshed often and are accessed frequently.

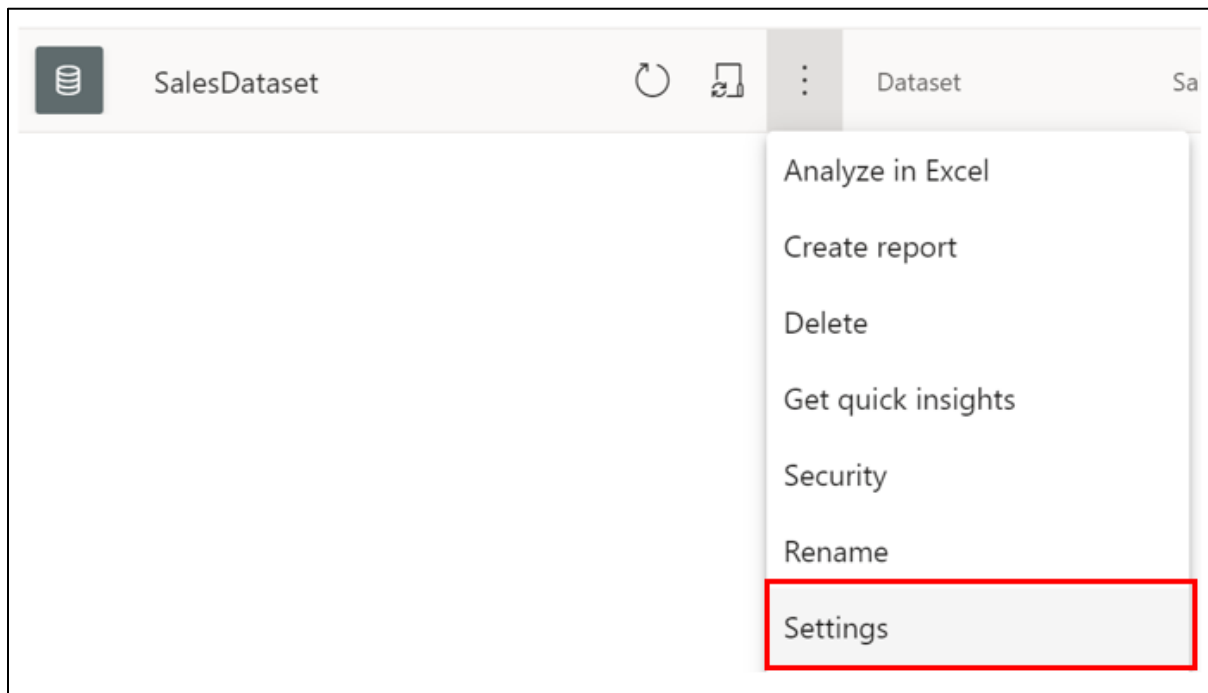
This service is only available to users with Power BI Premium or Power BI Embedded.

Query caching results are user-specific, and apply specific page of a report only. Several benefits to using query caching include:

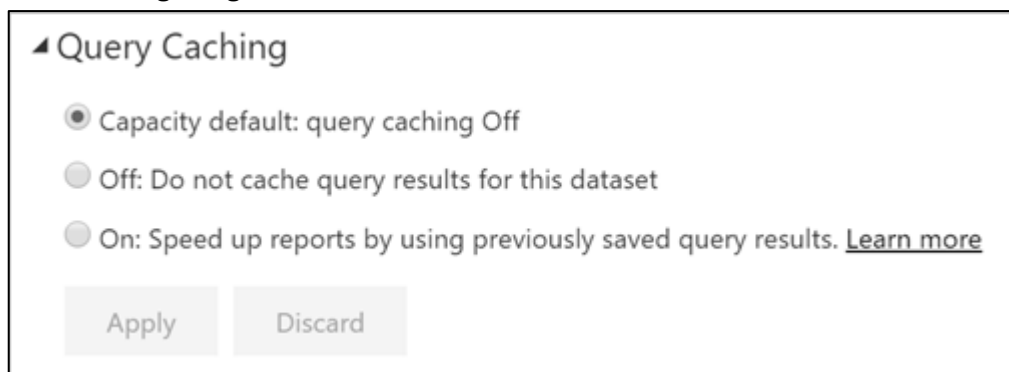
- Improvement of the performance of reports, dashboards, and dashboard tiles.
- It respects bookmarks and default filters.
- Cached query results are specific to the user.
- All security labels are followed.
- It reduces the load on your dedicated capacity.

To access and configure query caching, follow these steps:

1. Go to a semantic model in your workspace and open its **Settings** page. In this example, you enable query caching for **SalesModel**.



2. Select the **semantic models** tab and expand the **Query Caching** options, as shown in the following image.



3. On

the **QueryCaching** page, choose one of the available options. The default option is that query caching is turned off; however, you can also select **Off**, which turns off query caching for the specific semantic model in question. If you select **On**, query caching is turned on for this specific semantic model only. For this example, you'll select **On** for your semantic model because you want to apply query caching to your specific semantic model.

Switching from **On** to **Off** clears all previously saved query results. When turning off query caching (either through the default or the **Off** option), a small delay occurs in query loading

because the report queries are running against the semantic model and it does not have saved queries to fall back on.

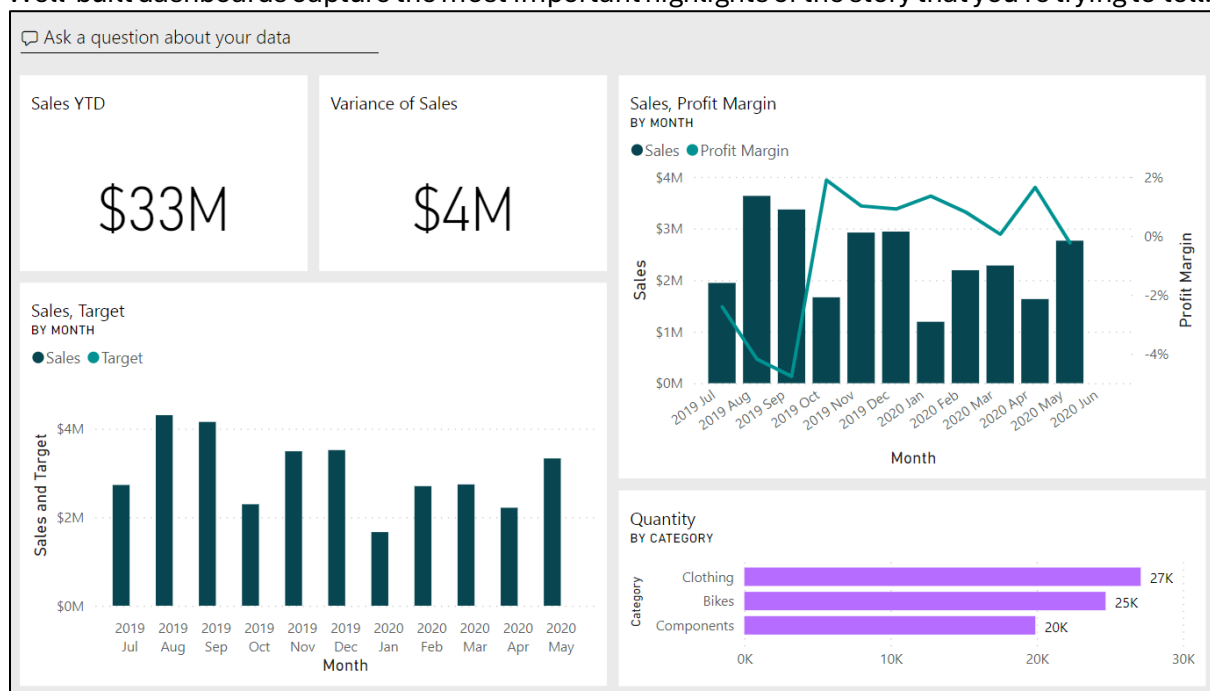
If many semantic models have query caching enabled, and a refresh occurs, a reduction in performance might occur because a large number of queries are being processed at once.

Create Dashboards in Power BI

Introduction to Dashboards

Microsoft Power BI dashboards are different than Power BI reports. Dashboards are made up of visuals from different reports and allow report consumers to create a single page of targeted data insights, with the option to explore more in a report.

Well-built dashboards capture the most important highlights of the story that you're trying to tell.



Dashboards vs. reports

The Power BI dashboard is similar to a dashboard in a car - you want to have the most important information for the consumer to see immediately. For additional insights, consumers can follow through to the corresponding report.

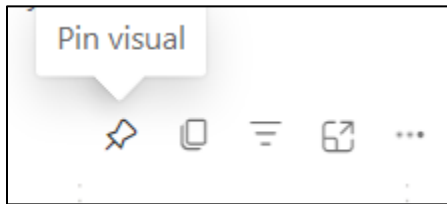
Dashboards are often the next step after building your reports in Power BI Desktop. Consider the following differences when deciding how to create reports and dashboards:

Dashboards	Reports
Create only in service	Create in Power BI Desktop (limited in service)
Single page	Multiple pages
Static tiles	Interactive visuals
Read-only tiles	Filters pane

Both dashboards and reports can be refreshed to show the latest data.

Get started with dashboards

Dashboards are created by *pinning* report visuals, and are then called *tiles* within reports. When you make changes to the visuals in the report, and then republish to Power BI Service, changes are reflected on the dashboard.

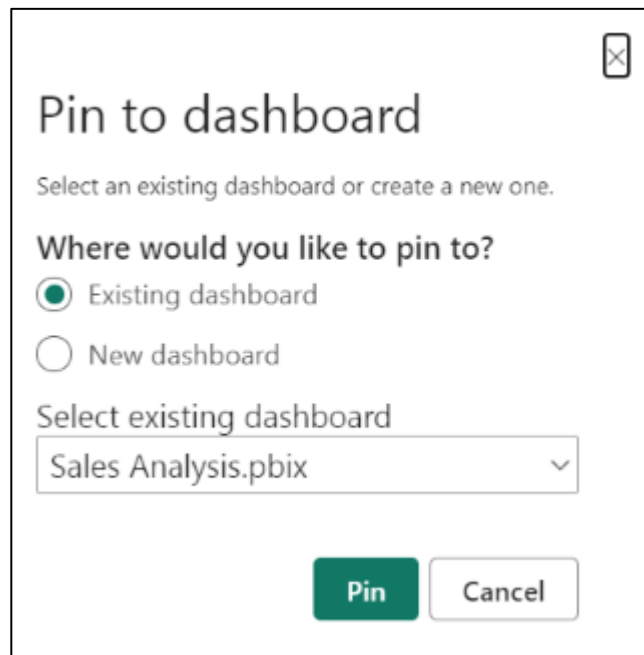


First log into the Power BI service and navigate to a workspace with reports. Open a report and in the visual header, select the **Pin Visual** icon. You can choose to pin this visual to a new or existing dashboard.

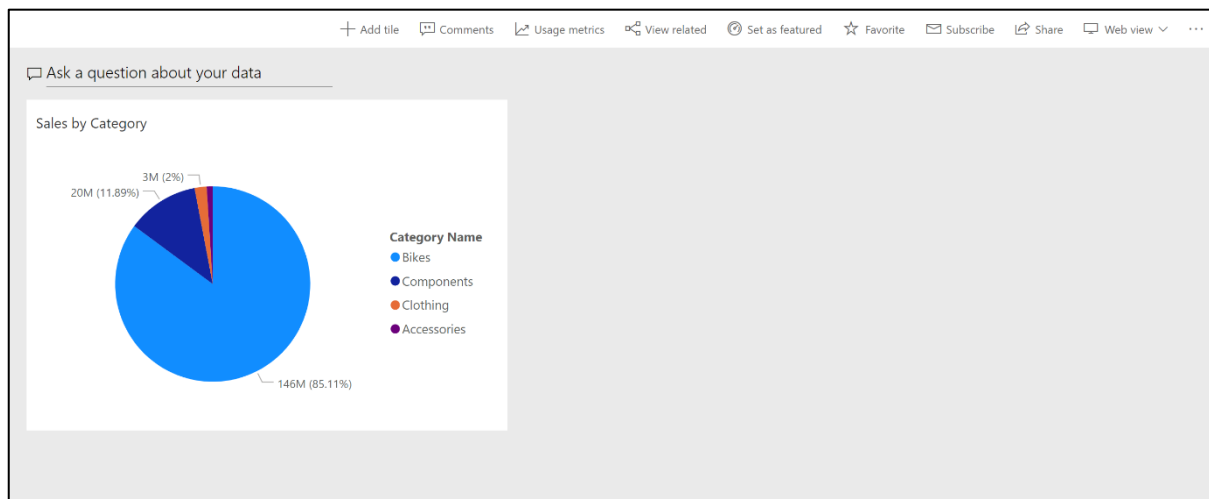
After pinning your visuals, you can resize and move the visuals to suit your needs.

Pin a tile from a different report

One of the biggest benefits of a dashboard is being able to pin a visual that is sourced from a different semantic model.



In the following image, we show two separate tiles pinned from different reports.



Repeat this process until your dashboard has the data you want to show.

Configure Data Alerts

Configuring data alerts is a simple process to complete for a dashboard in Power BI.

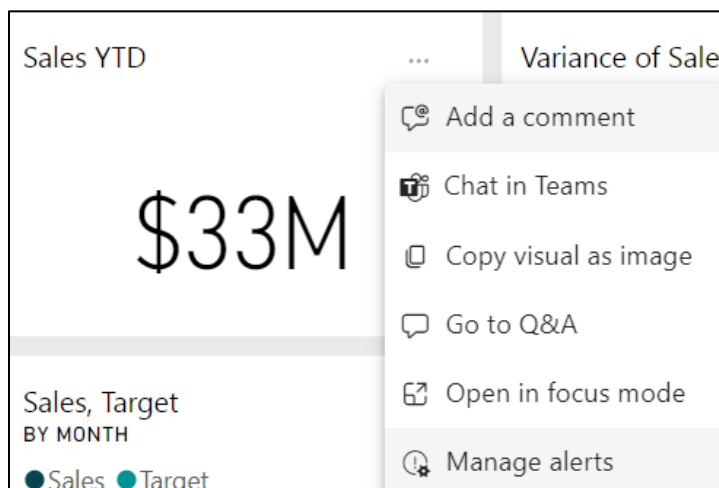
Data alerts can notify you or a user that a specific data point is above, below, or at a specific threshold that you can set.

Configure alerts

These alerts are features that are only available on Power BI service and they're available on such report elements such as KPI visuals, gauges, and cards.

After you have uploaded your reports to Power BI service and have pinned your chosen visuals to a dashboard, select the ellipsis (...) in the corner of the tile you want to set an alert for and then select **Manage Alerts**.

Select **+ Add Alert Rule**, which will add a new alert.



Ensure that the **Active** toggle switch is turned **On**, name the alert, and then set the condition.

Next, you can choose the threshold that you want to create the alert for, which includes options for **Above** or **Below** a specific threshold.

A screenshot of the 'Manage alerts' dialog box in Power BI. The dialog has a title bar 'SALES YTD' and a close button. Below the title is a green button '+ Add alert rule'. The main content area shows settings for an alert on 'Sales YTD'. There's a trash icon to the right of the visual name. The 'Active' toggle switch is turned 'On'. The 'Alert title' field contains 'Sales YTD'. The 'Set alerts rule for' dropdown also shows 'Sales YTD'. Under 'Condition', a dropdown is set to 'Above' and the 'Threshold' field contains the value '33139748.07'. For 'Maximum notification frequency', the radio button for 'At most every 24 hours' is selected. Below this, it says 'Alerts are only sent if your data changes.' At the bottom, there's a note: 'By default, you'll receive notifications on the service in the notification center.' and a checked checkbox for 'Send me email, too'.

Finally, select how frequently you want alerts. These alerts are sent directly to your Notification Centre in Power BI, and you can also configure emails.

This feature is available to all consumers to configure as they so choose.

You can also enable or disable the alert by using the toggle switch.

Explore Data by Asking Questions

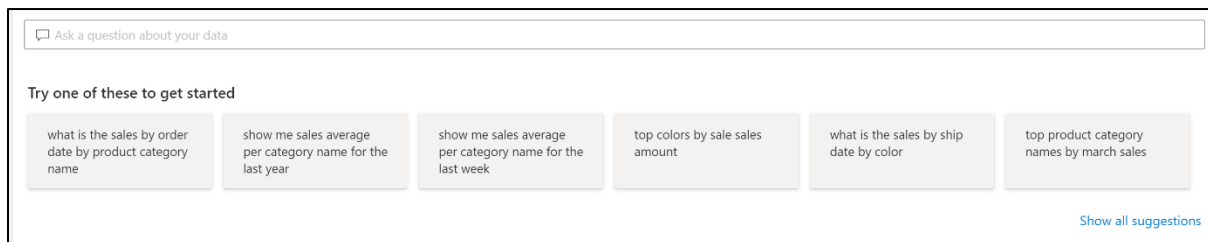
Power BI dashboards are about having a user-friendly experience. When users first interact with data, they often have additional questions and reach out to you. A few questions might be manageable to answer, but this isn't sustainable.

Power BI solves this problem with the Q&A visual. From the dashboard view, users can use the **Ask a question about your data** feature, which increases engagement between users and the dashboard.

Q&A feature

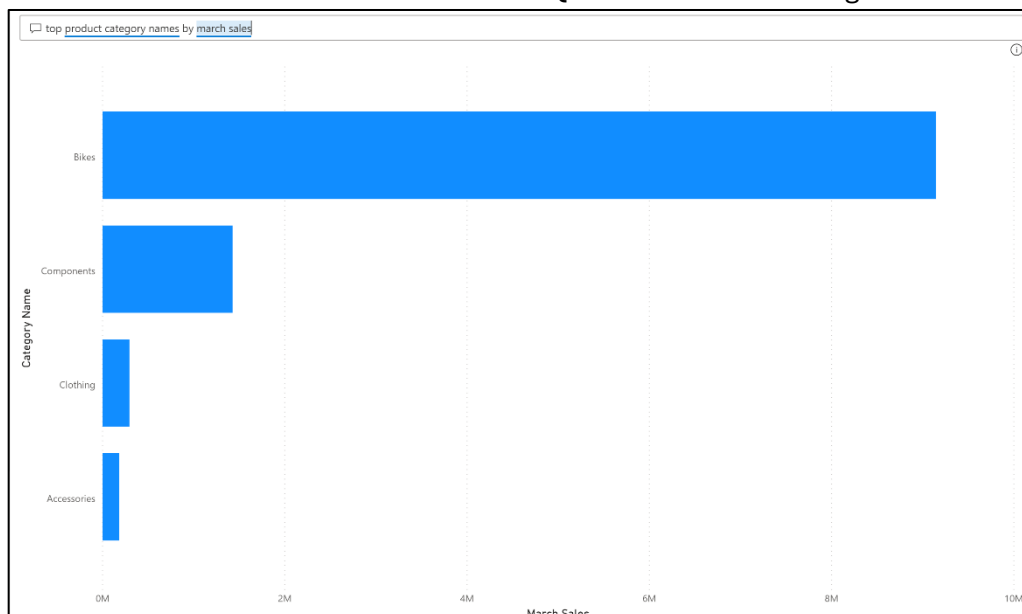
The Q&A feature is a tool within Power BI Desktop that allows you to ask natural-language questions about the data.

To locate the Q&A feature, go to your dashboard in Power BI service. Along the top ribbon is the **Ask a question about your data** search box. Some prompts are also available to help get you started.



The Q&A visual consists of three main elements:

- **Question box:** Use natural language to ask questions about the data. Example: *What were the average sales amount by category?*
- **Pre-populated suggestion tiles:** Questions to help users consider how to phrase their questions. Example, *top product category names by march sales* tile results as a bar chart with category name and March sales.
- **Pin visual:** Pin the visual created with Q&A to a new or existing dashboard.



With the Q&A feature, users can interact directly with the visual to ask their data questions, which will increase their interactions with the visual and save time.

Review Quick Insights

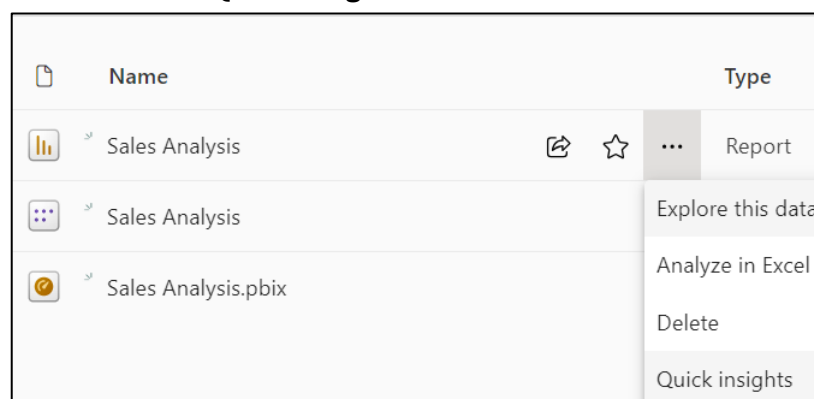
The **Quick insights** feature in Power BI uses machine learning algorithms to go over your entire semantic model and produce insights (results) for you quickly.

This feature is a great way to build dashboards when you don't know where to start. It also helps you find insights you might have missed when building your reports. From the insights that Power BI discovers, you can generate appealing, interactive visualizations.

This feature is available in the Power BI service only. Also, this feature doesn't work with DirectQuery; it only works with data that is imported to Power BI.

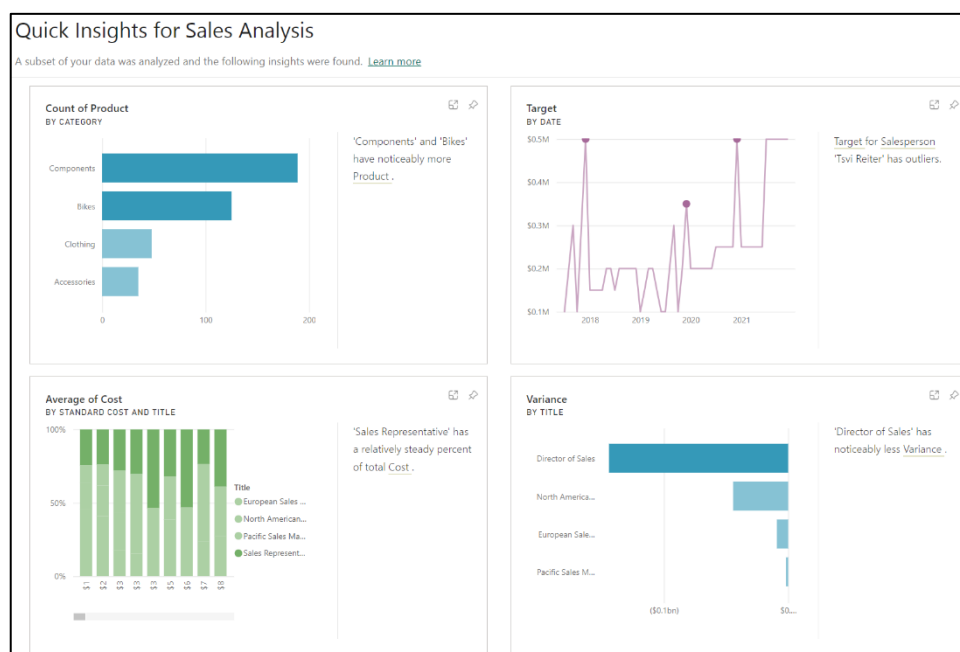
Get quick insights on your semantic model

To get quick insights on your semantic model, open your Power BI service and workspace, locate your report, and select the **... > Quick insights**.



Power BI uses various algorithms to search for trends in your semantic model. This process might take a few seconds, but you receive a pop-up message when the insights are ready.

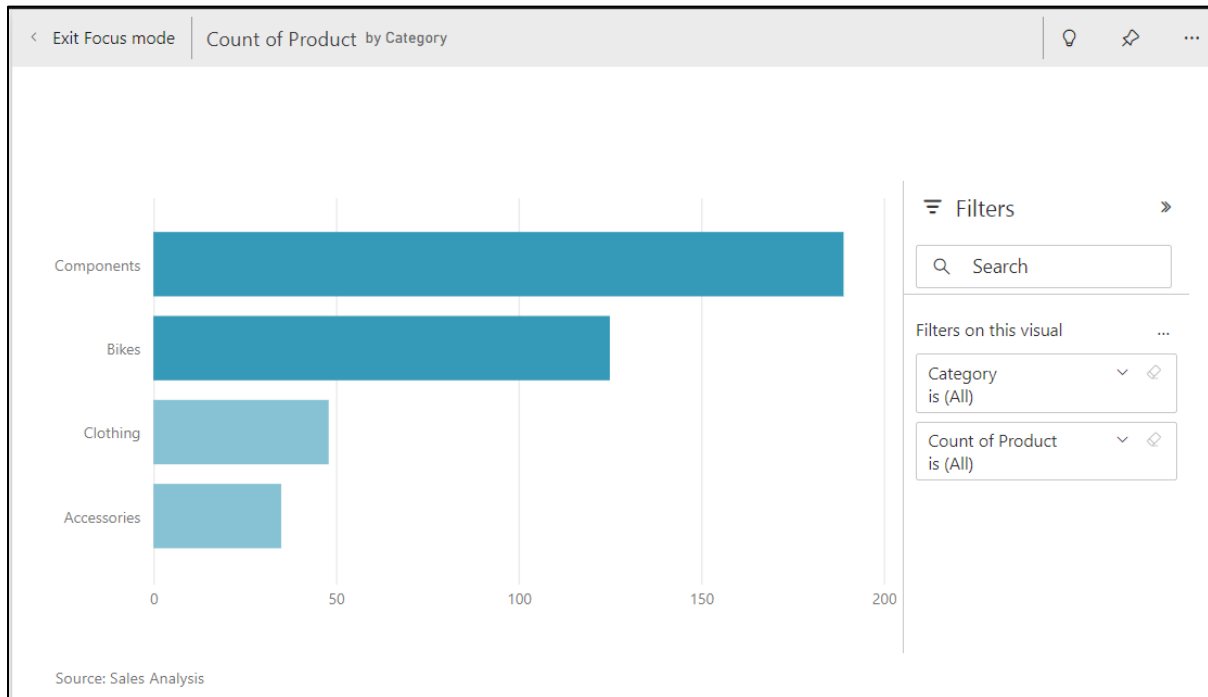
Select **View insights** to open the **Quick Insights** page for the selected semantic model, and then view the insights that Power BI has found for you. The **Quick Insights** page contains up to 32 separate insight cards, and each card has a chart or graph plus a short description.



Interact with the quick insights results

If you see a compelling insight card, you can even pin it to your dashboard.

To take a closer look at a particular insight card on the **Quick Insights** page, select an insight card to open. The insight screen opens in **Focus** mode.



You can then perform the following actions:

- Filter the visualization by using the available options in the **Filters** panel.
- Pin the insight card to a dashboard by selecting **Pin visual**.
- Run insights on the card (scoped insights) by selecting **Get insights** in the upper-right corner. The scoped insights allow you to drill into your data.
- Return to the original insights canvas by selecting **Exit Focus mode** in the upper-left corner.

Add a Dashboard Theme

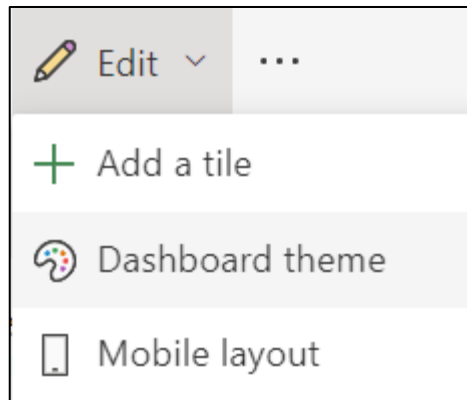
When building dashboards, you can create a cohesive picture by applying the same theme as in your reports.

Or apply a specific theme to reports and dashboards so that all report elements or tiles are uniform.

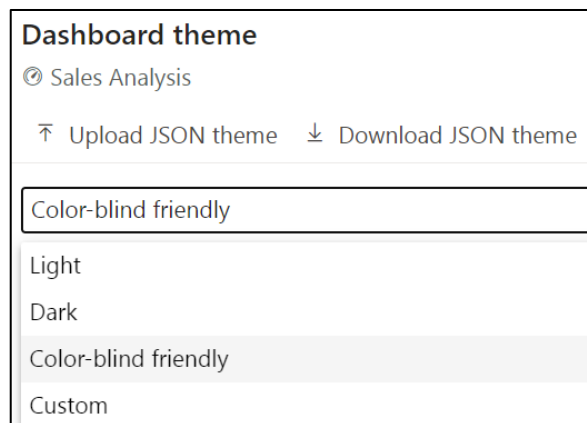
This consideration is important when you're building multiple dashboards. Power BI provides options to apply a theme directly to a dashboard as well as all visuals of a report.

Themes in Power BI

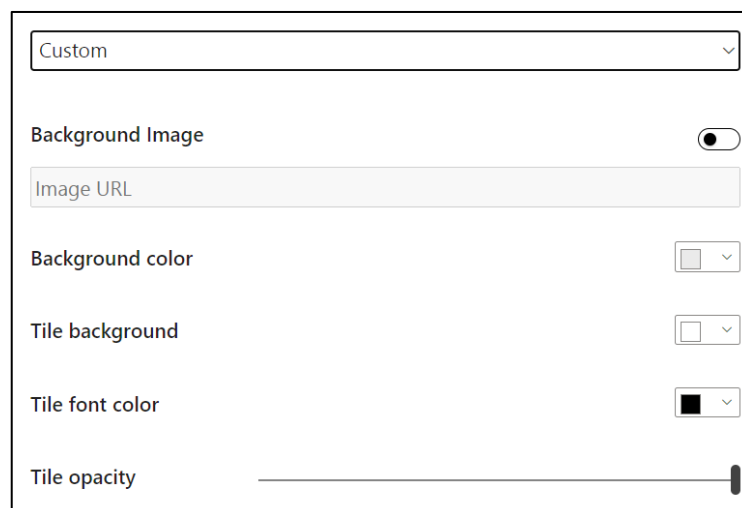
Various themes are available for use in Power BI service. Go to a dashboard, select the edit dropdown arrow, and then select **Dashboard theme**.



This selection will open a window, where you can choose from various themes, including **Light** (the default theme), **Dark**, **Color-blind friendly**, and **Custom**, where you can create your own theme. You can also upload your own JSON theme or download the current theme.



For instance, if you select **Custom**, you can add your own background image, or you can change the background color, tile color, the opacity, or even the font color, as shown in the following figure.



Now, you can customize your report to cater specifically to your needs.

Pin a Live Report Page to a Dashboard

Power BI allows you to pin live report pages to dashboards, ensuring that your visuals reflect real-time changes. The iterative process of building reports and dashboards as business requirements evolve becomes easier with this feature.

Pin a live page

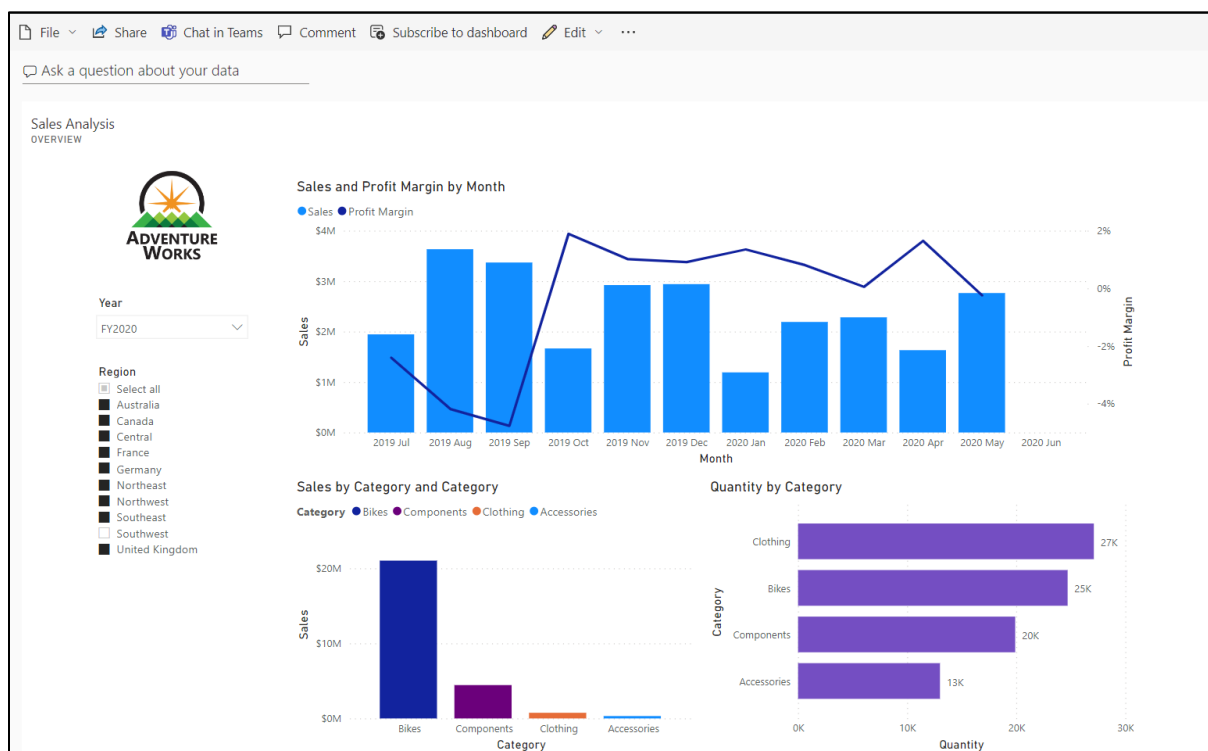
The **Pin a live page** feature reflects real-time changes, ensuring that your reports always stay up-to-date.

You can also pin entire report pages to a dashboard. These visuals are *live*, meaning that any changes you make on the report are immediately reflected on the selected dashboard.

Pinning a report page is a simple way to pin all visuals at once. All slicers and filters still work and that the visuals have the same data as in the report. The visuals appear as a page, so there are no reformatting options.

Any changes that you make to the underlying report automatically show on the dashboard when the page is refreshed.

To pin an entire page, select the ellipsis (...) on the navigation bar of the report and then select **Pin to a dashboard > (Select dashboard) > Pin live**.



Dashboards are intended to be a collection from various sources, not just as a "launching pad" for reports.

We recommend that you pin at the tile level first and foremost, and if needed, the entire report page can also be pinned.

Seeing an entire report page in a dashboard tile can be difficult.

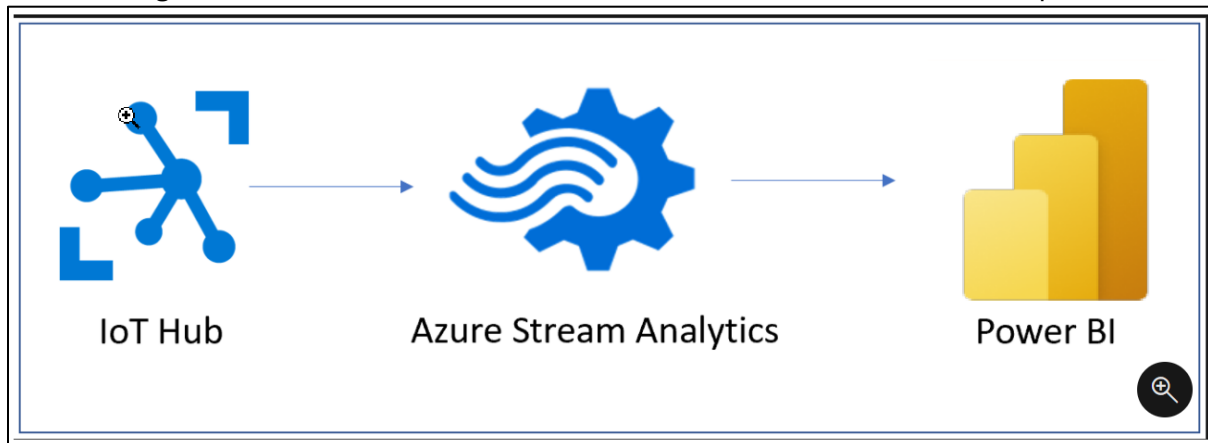
Configure a Real-Time Dashboard

In this data-centric world, it has become increasingly important to be able to view how data changes in real time. Streaming data on dashboards is crucial for telling the story of your data and can greatly benefit your business.

With Power BI's real-time streaming semantic models, you can stream data and update dashboards as soon as the data is logged.

Stream in Power BI

Streaming data can come from various sources, including from social media, factory sensors, service-usage metrics, and other sources that contain a constant stream of data points.



Data that comes from a streaming data source is stored in a temporary cache, not a semantic model.

This means you can't make changes to the semantic model.

The only way to visualize the data from a streaming data source is to create a tile directly on a dashboard and use a custom streaming data source.

These tiles are optimized because no database exists to pull the data from.

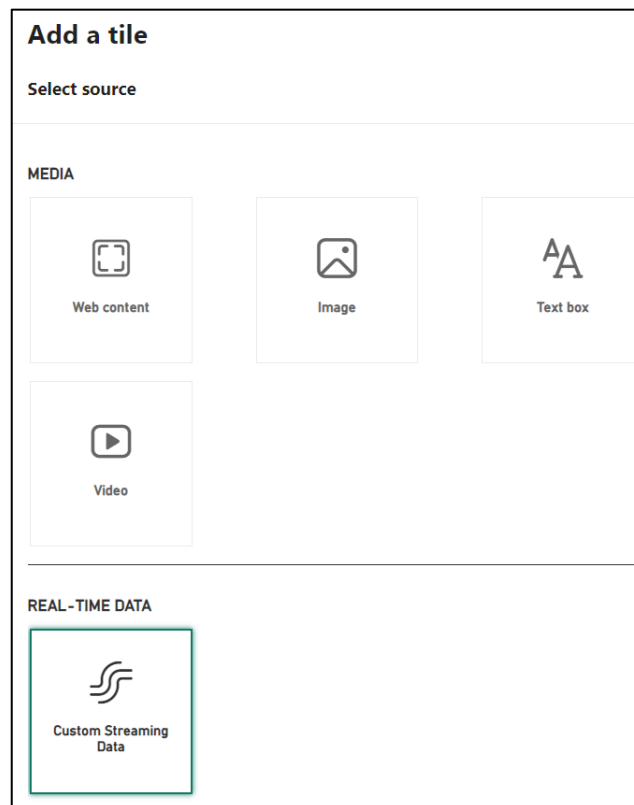
These types of tiles have low latency and are best suited for data that doesn't need additional transformations, such as temperature or humidity.

Visualize real-time data in Power BI

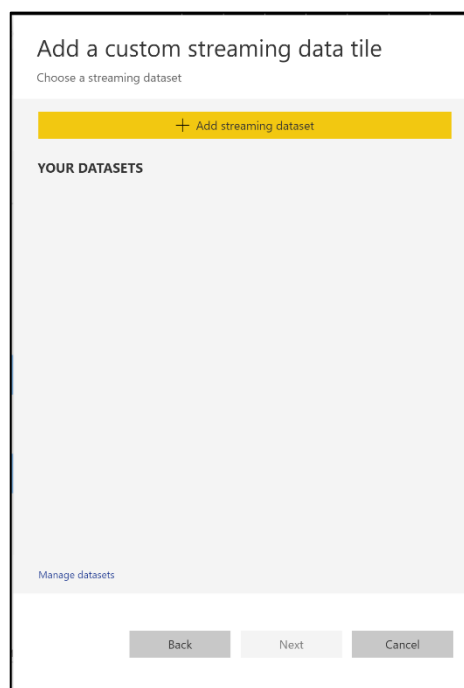
To visualize streaming data, you need to create a new tile directly on an existing or new dashboard.

Select the Edit drop-down arrow and then **Add Tile**.

The following window will appear, where you can select **Custom Streaming semantic models** under **Real-Time Data**.



Select **Next**, which will redirect you to the following window where you can choose an existing streaming semantic model, or get new streaming semantic models, as shown in the following image.

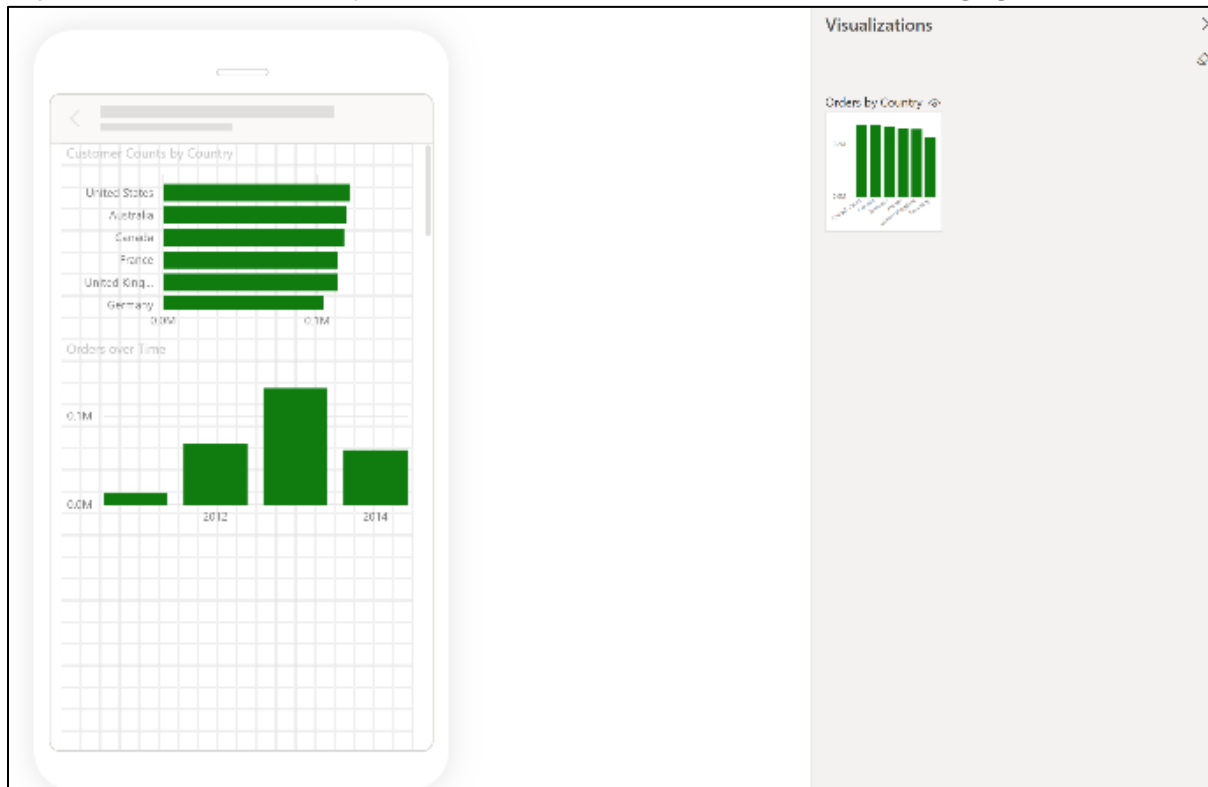


Enter the details for your streaming semantic model and then add a new streaming semantic model tile. Streaming semantic model tiles can be formatted as line charts, stacked bar charts, cards, and gauges, similarly to any other tiles.

Set Mobile View

Power BI reports are built in Power BI Desktop and then deployed to Power BI service, where they can be viewed and shared. However, if you're building dashboards for your organization and need reports and dashboards viewable on mobile devices, Power BI offers a mobile layout configuration to meet your needs.

To navigate to mobile view in Power BI Desktop, select **View** on the ribbon and then select **Mobile Layout**, which will redirect you to the mobile view, as shown in the following figure.



In the mobile view in Power BI Desktop, you're able to accomplish several tasks. The mobile view shows you how it will look to users in the Power BI Mobile app.

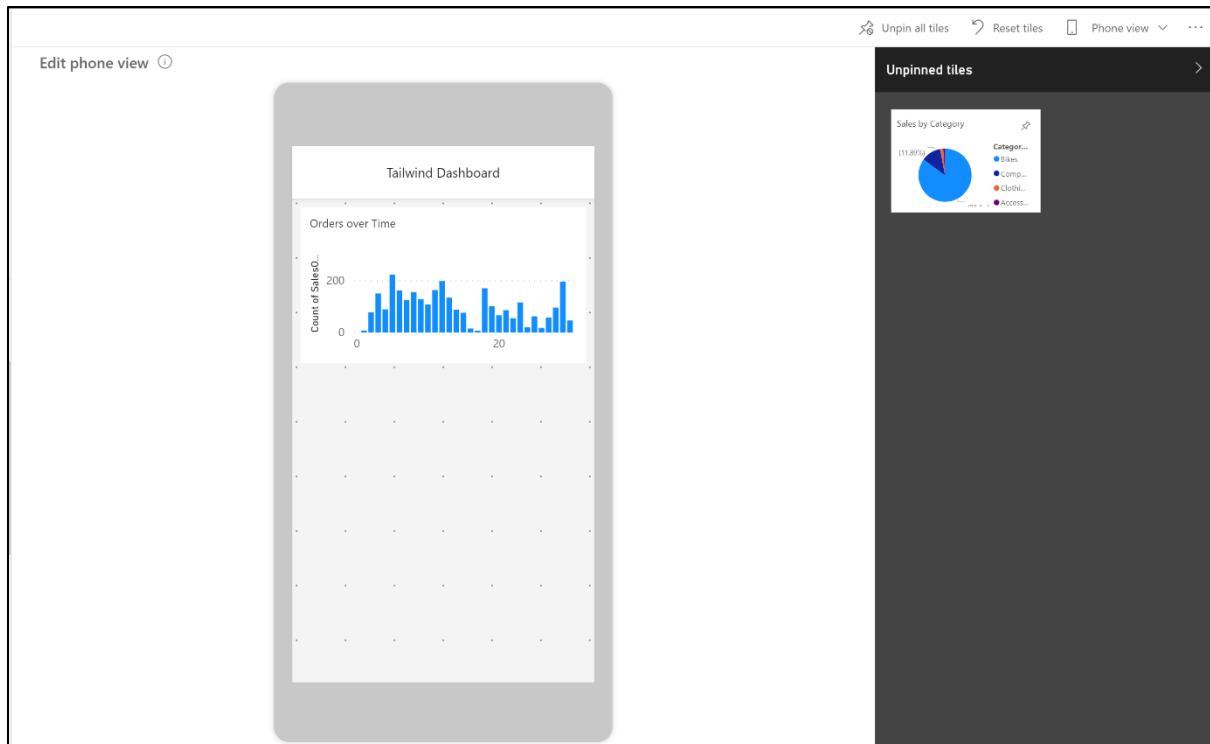
You can add visuals, resize them, and change the formatting.

Use the grid view to orient your visuals with more ease and overlay visuals on top of each other. This feature can be useful if you want to insert a visual on top of an image.

Alternatively, you can also optimize your dashboards for mobile view in Power BI service.

To see a dashboard in mobile view, select the Edit drop down arrow on the home ribbon and select **Mobile layout**.

Then you can choose which tiles that you want to see on the mobile view.



You can also resize and reorient the tiles and visuals in whichever order you want. This phone view is customizable for each person who uses the dashboard; selecting **Phone view** will allow you to create a new view that you can see on your phone when signing in to Power BI service.

Implement Row Level Security

Introduction

Microsoft Power BI can help you secure reports and workspaces by allowing you to share them to active directory users and groups. You can also share a single report but have users see different data according to their job role.

For example, consider a scenario where you work for Tailspin Traders. You use the following table to track sales.

employeeName	emplID	department	product	qty	price	orderAmount
Kelli Hinojos	1	Game	Settlers of Air	1	24.99	24.99
Jeffrey Reiss	5	Sports	Driver - Stiff Shaft	1	399.99	399.99
Roselyn James	7	Clothing	V-Neck T-Shirt	1	19.99	19.99
Lavonna Domingo	5	Sports	Golf Balls - Dual Core	3	32.5	97.5
Hermina Leslie	7	Clothing	Athletic Shorts	4	17.75	71
Jess Dammann	6	Automotive	Tire Guard	1	44.99	44.99
Kitty Hudman	1	Game	Santo Domingo	1	31	31
Sonia Coss	9	Clothing	Leather Sandels	2	111.97	223.94
Becky Pearsall	6	Automotive	True Coat	1	980	980
Echo Lundeen	3	Sports	Frisbee Golf Set	2	98	196
Sheryl Cayton	9	Clothing	Hoodie	1	44.99	44.99
Veronika Lopes	2	Automotive	Window Scrape	4	9.99	39.96
Sally Corliss	8	Game	Lords of Avalon	1	19.99	19.99
Sheldon Allende	3	Sports	Putting Green	1	59.25	59.25
Noma Yoakum	7	Clothing	Bathing Trunks	1	42.99	42.99
Hazel Kapinos	5	Sports	Weighted Bands	2	8.99	17.98
Francisco Ayers	7	Clothing	Hoodie	1	44.99	44.99
Tom Etienne	4	Game	Spider, spider	1	44.5	44.5
Jamel Carol	5	Sports	Boxing gloves	1	99	99
Rosemary Treacy	4	Game	Invest in it All	1	31.99	31.99

You can also use the following table for employee information.

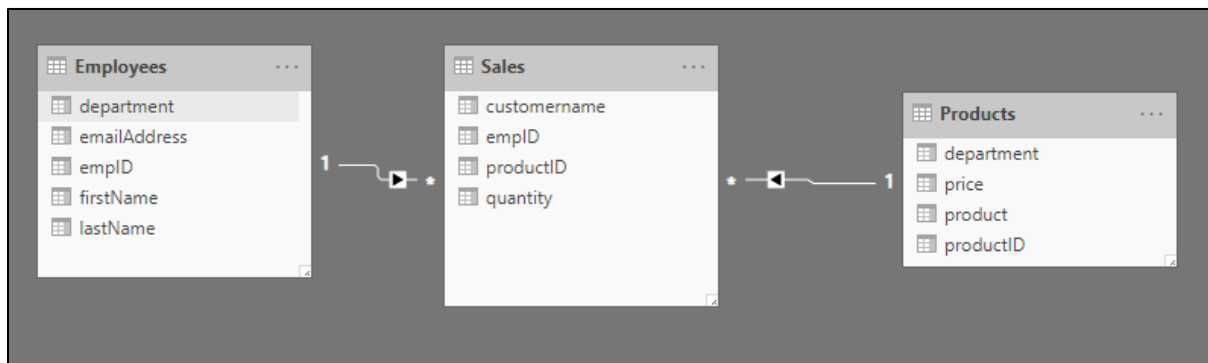
employeeName	empID	productID	quantity
Kelli Hinojos	1	82	1
Jeffrey Reiss	5	24	1
Roselyn James	7	67	1
Lavonna Domingo	5	42	3
Hermine Leslie	7	88	4
Jess Dammann	6	19	1
Kitty Hudman	1	47	1
Sonia Coss	9	98	2
Becky Pearsall	6	73	1
Echo Lundeen	3	3	2
Sheryl Cayton	9	58	1
Veronika Lopes	2	8	4
Sally Corliss	8	61	1
Sheldon Allende	3	91	1
Noma Yoakum	7	16	1
Hazel Kapinos	5	4	2
Francisco Ayers	7	71	1
Tom Etienne	4	36	1
Jamel Carol	5	83	1
Rosemary Treacy	4	65	1

The following table shows the list of products.

productID	department	product	price
3	Sports	Frisbee Golf Set	98
4	Sports	Weighted Bands	8.99
8	Automotive	Window Scrape	9.99
16	Clothing	Bathing Trunks	42.99
19	Automotive	Tire Guard	44.99
24	Sports	Driver - Stiff Shaft	399.99
36	Game	Spider, spider	44.5
42	Sports	Golf Balls - Dual Core	32.5
47	Game	Santo Domingo	31
58	Clothing	Hoodie	44.99
61	Game	Lords of Avalon	19.99
65	Game	Invest in it All	31.99
67	Clothing	V-Neck T-Shirt	19.99
71	Clothing	Hoodie	44.99
73	Automotive	True Coat	980
82	Game	Settlers of Air	24.99
83	Sports	Boxing gloves	99
88	Clothing	Athletic Shorts	17.75
91	Sports	Putting Green	59.25
98	Clothing	Leather Sandels	111.97

You want to make one report where employees in a specific department can only see the sales for that department. For instance, Maria Cameron works in the Game department and should only see the sales from that department, not sales from the Sports, Clothing, or Automotive departments.

This data is organized in a star schema. The Sales table contains all attributes of a fact table, while employees and products are dimension tables. The semantic model is shown in the following screenshot.



Two ways of implementing row-level security in Power BI are the static method and the dynamic method.

Row-level security (RLS) uses a DAX filter as the core logic mechanism. This module will demonstrate how you can implement row-level security in Power BI by using DAX to ensure that only the appropriate person can view the appropriate records.

Configure Row-Level Security with the Static Method

The static method in row-level security (RLS) uses a fixed value in the DAX filter, while the dynamic method uses a DAX function.

RLS involves several configuration steps, which should be completed in the following order:

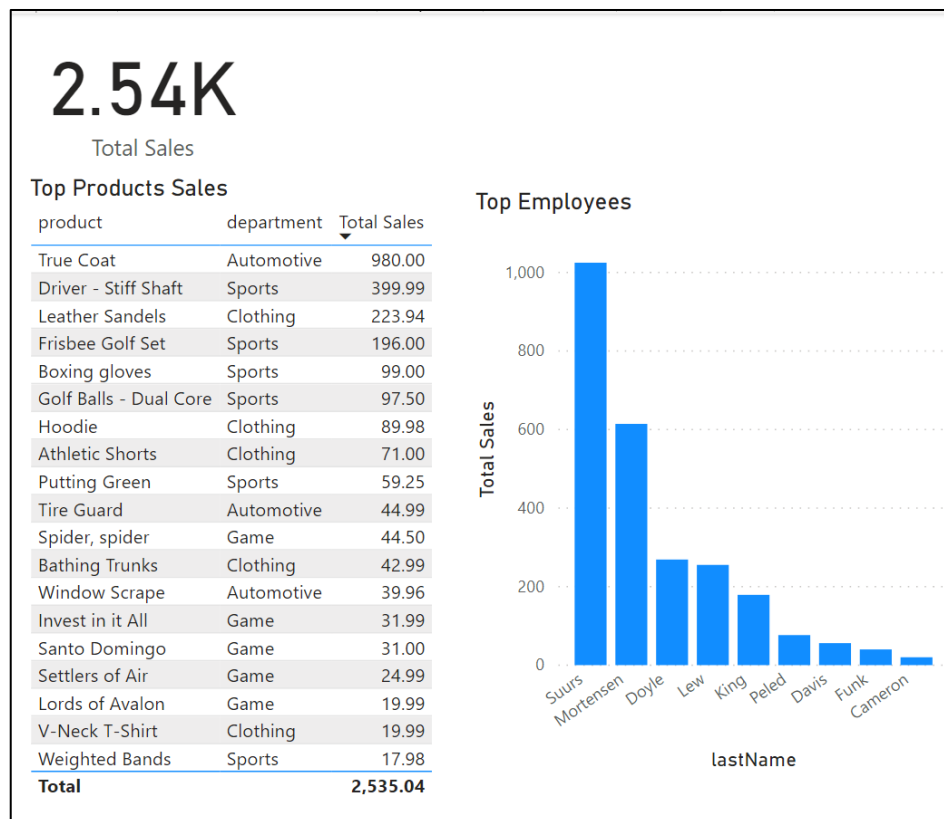
1. Create a report in Microsoft Power BI Desktop.
 - a. Import the data.
 - b. Confirm the semantic model between both tables.
 - c. Create the report visuals.
2. Create RLS roles in Power BI Desktop by using DAX.
3. Test the roles in Power BI Desktop.
4. Deploy the report to Microsoft Power BI service.
5. Add members to the role in Power BI service.
6. Test the roles in Power BI service.

Create a report in Power BI Desktop

Follow the typical steps to create a report in Power BI Desktop. Use Microsoft Power Query to retrieve and clean the data.

Then, confirm that the relationship exists between the two tables by using the **Modeling** tab; it should be a one-to-many relationship on the **emplID** column.

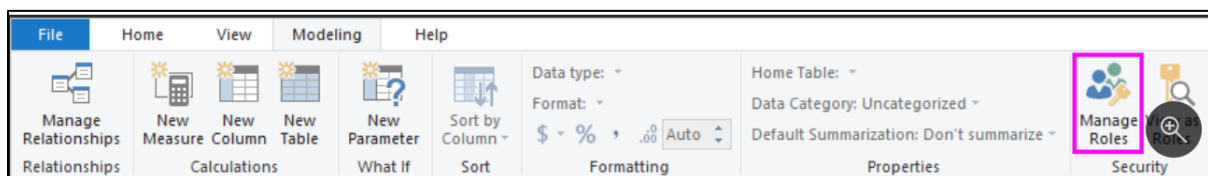
Your next step is to create a Power BI report.



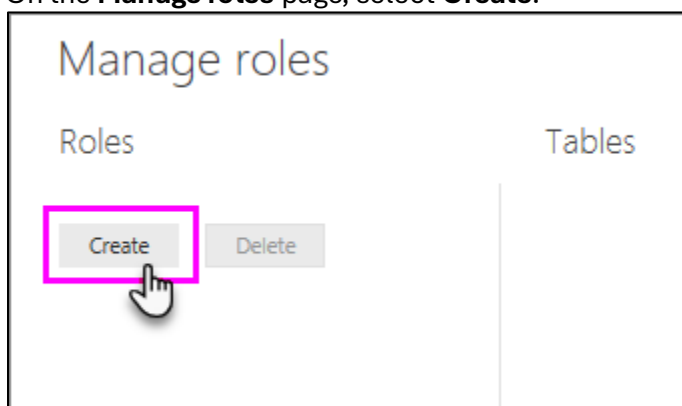
Notice how the preceding table has rows for all sales, including all departments. You will be limiting visibility so that only employees of a specific department can see their own sales.

Create RLS roles in Power BI Desktop

To create RLS roles in Power BI Desktop, select the **Modeling** tab, and then select **Manage Roles**.



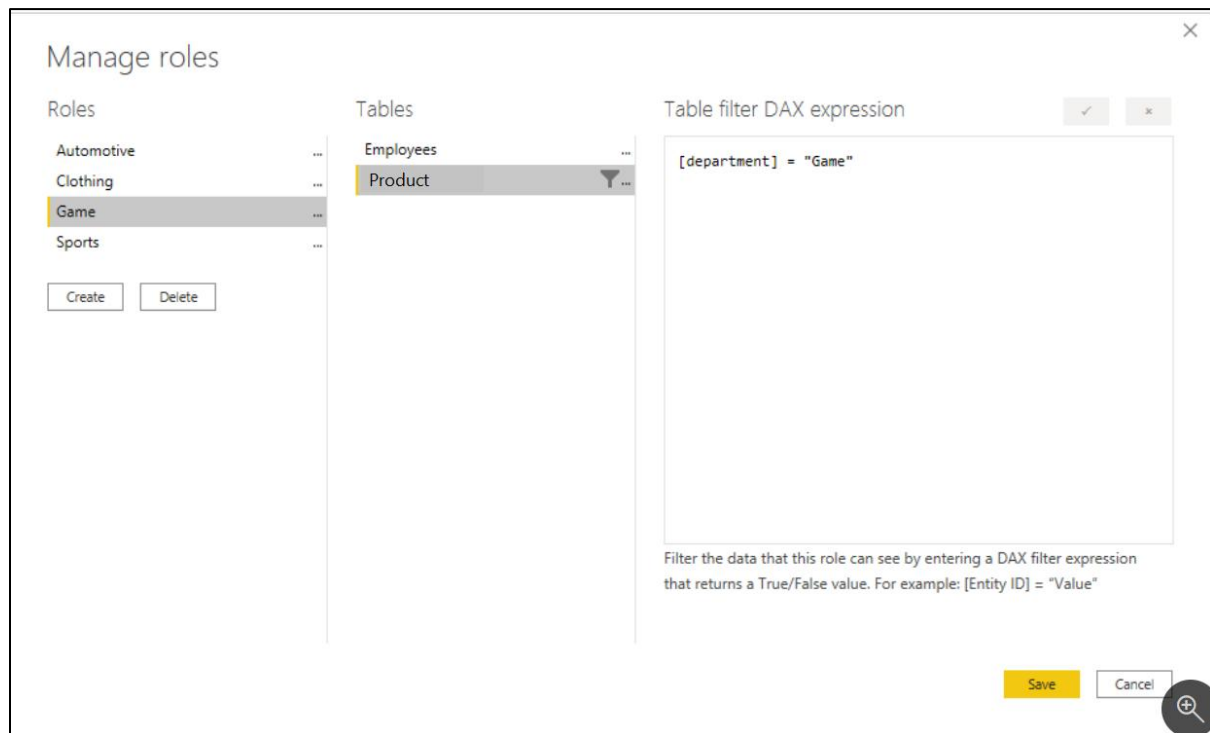
On the **Manage roles** page, select **Create**.



Power BI row-level security (RLS) uses DAX to control who can see which data. Consider it as always adding another filter to the appropriate users, regardless of the filters, slicers, or interactions that the users choose on a Power BI report.

On the **Manage roles** page, create a role for each department and then add a DAX expression to it. For instance, you can create a role called **Game** and then add the DAX expression **[department] = "Game"**. Then, whenever a member of that role interacts with the report, Power BI will add that filter to their interactions, thus limiting what they see.

A fixed value is used in the filter on the right side of the equal sign (in this case, "Game"). The intention is that, if you ever need to add a category, you will need to create a new role with a new value in the DAX expression.

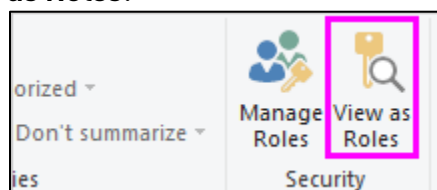


Notice how the DAX filter is applied on the dimension table. Row-level security performs better when the data is organized in a star schema. Apply the DAX filter to a dimension table, as was done with the Products table.

The DAX filter is applied to every interaction, slicer, and filter that the user applies. If you have a DAX filter that performs poorly, the user experience will be negatively impacted. Therefore, keep the DAX filter as simple as possible.

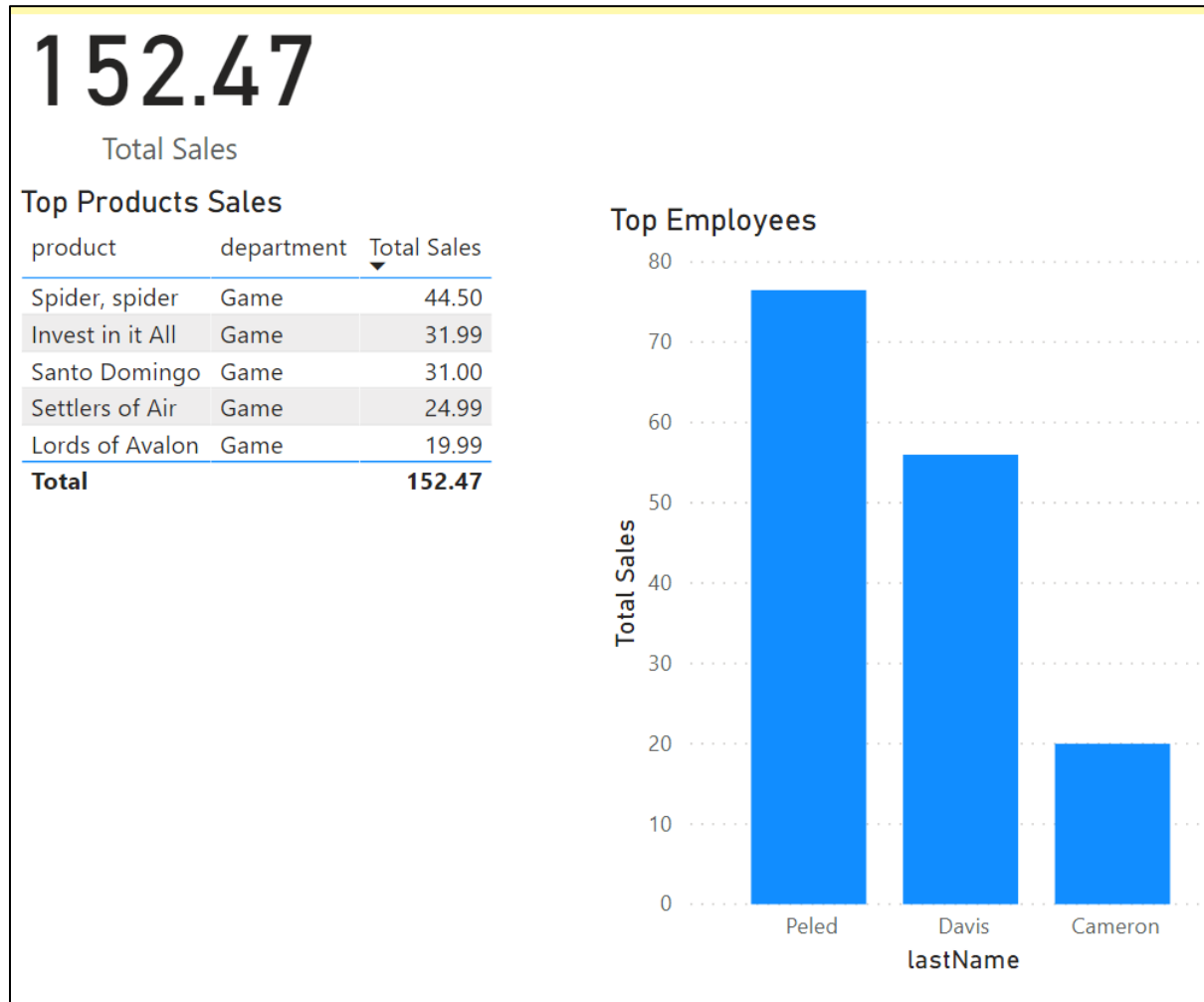
Test the roles in Power BI Desktop

You can validate that the filter is working by selecting the **Modeling** tab and then selecting **View as Roles**.



In the **View as roles** window, select the **Game** role.

The report now renders as if you were in that role, and you will only see the records that are included in the Game department.



You can undo this filter by selecting **View as roles** again and then selecting **None**.

Deploy the report to Power BI service

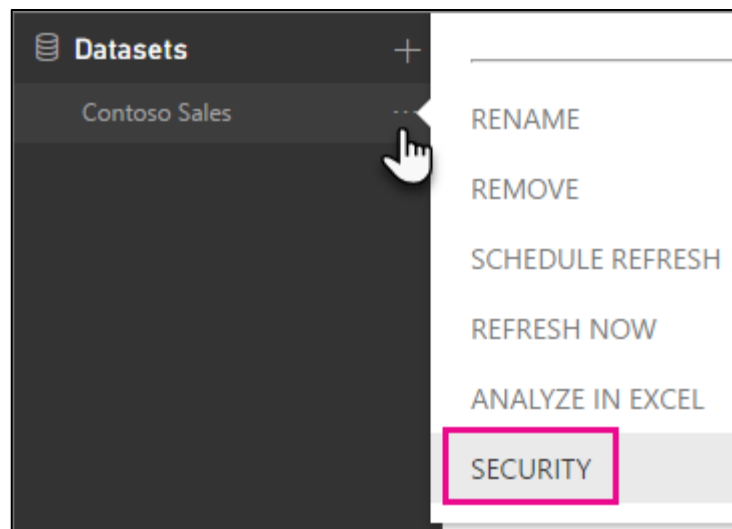
You can deploy the report to Power BI service by selecting the **Publish** button on the **Home** tab and then selecting a workspace.

Add members to the role in Power BI service

To add members to the role in Power BI service, go to your workspace in Power BI service.

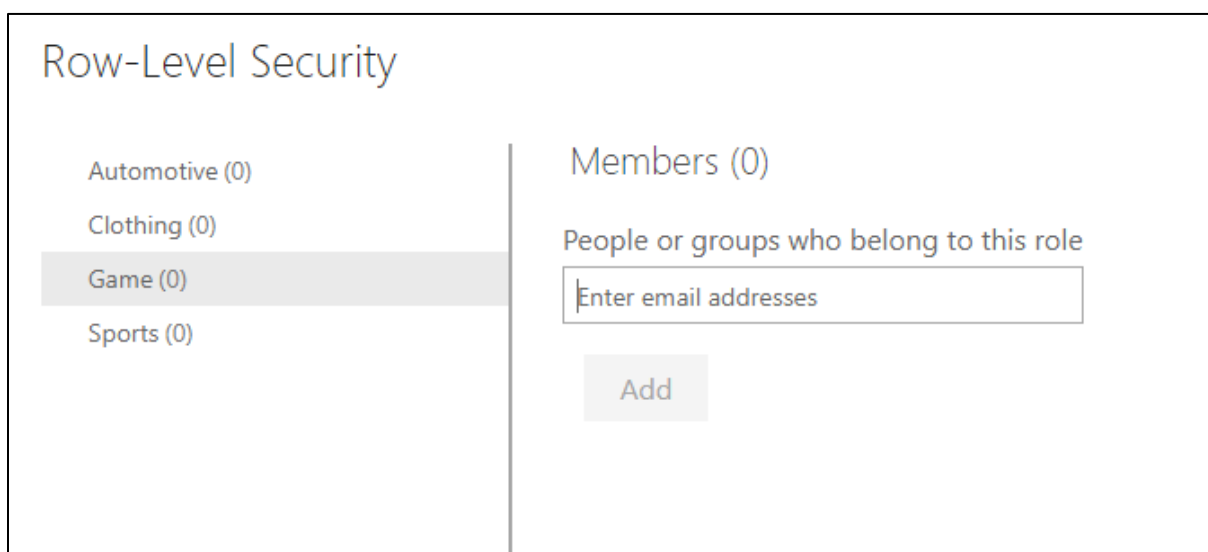
Find the semantic model that you created with the same name as your report.

Select the ellipsis (...) button and then select **Security**.



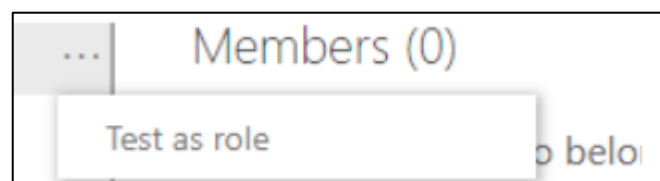
In the **Row-Level Security** screen, you can add Microsoft Entra ID users and security groups to the security role. When members are added to this role, the DAX filter that you previously defined will be applied to them.

If members are not added to the role, but they have access to the report, RLS will not apply to them. You can add the three people in the Game department to the **Game** role. Now, when those members sign in, they will only see the report with data that applies to them.



Test the roles in Power BI service

You can test the roles inside Power BI service by selecting the ellipsis (...) next to the **Game** role on the **Row-Level Security** screen and then selecting **Test as role**.



This selection will display the report as if you were a member of the role in Power BI service. And there you have it! We've successfully implemented row-level security in Power BI.

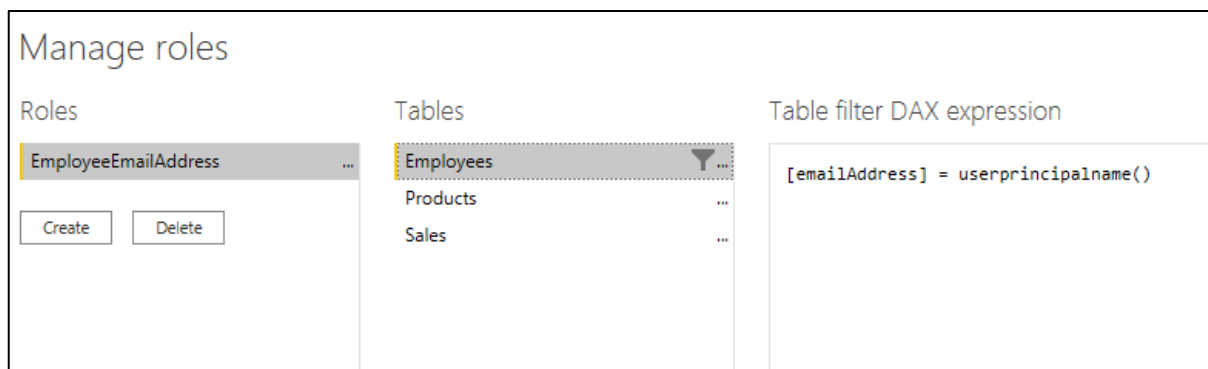
Configure Row Level Security with the Dynamic Method

You can set up row-level security only once, without the need to continue maintaining it dynamically.

As the admin at Tailwind Traders, you want Power BI row-level security to only show sales to the person who made them. In this example, Russel King has made four sales.

When viewing your report, Russel should only see the sales that he's responsible for and no other sales. You can configure row-level security exactly the way you configured it previously, with only a single change.

Instead of creating four roles, you only need to create one role. The DAX filter for that role would look similar to the following image.



Notice that instead of the fixed string, such as **Game** or **Clothing**, this uses a DAX function in the row-level security filter.

The `userprincipalname()` function will compare the email address from the Employees table with the email that the user entered when signing in to Power BI service.

If Russel King uses the email address `russel@tailwindtraders.com` to sign in to Power BI service, the system will compare that value to the email address in the Employees table.

Assuming that a relationship has been created between Employees and Sales, Russel will only see his four sales.